PROJECT TITAN

Complete Specification — All 10 Volumes

Multi-Agent Autonomous Software Engineering Platform

Specification Series — Draft v1.0

2026-07-04

PROJECT TITAN — Complete Specification (All 10 Volumes)

> Multi-Agent Autonomous Software Engineering Platform — full engineering specification series.
> Combined single-file edition. Individual volumes are also available as separate files in this folder.

PROJECT TITAN — Master Overview & Document Index

Series	PROJECT TITAN — Multi-Agent Autonomous Software Engineering Platform
Doc ID	TITAN-VOL-00
Status	Draft v1.0
Date	2026-07-04
Source	Conversation between the user and ChatGPT (preserved in full in `99-APPENDIX-Original-ChatGPT-Conversation.md`)

---

1. What This Folder Is

This folder is the complete specification document set for **PROJECT TITAN** — an idea developed in a conversation with ChatGPT and turned here into real, usable engineering documents.

PROJECT TITAN in one sentence:

> A platform where a user types *"Build me a Netflix clone"* (or a banking system, hospital management system, AI SaaS — anything), and a team of specialized AI agents — organized exactly like a real software company — autonomously plans, designs, builds, tests, reviews, secures, documents, and deploys the entire application. The human acts only as the client.

The core philosophy: **don't build one giant AI — build an organization.** Every AI agent has a role, responsibilities, memory, tools, communication abilities, decision-making authority, and a review process, and the agents collaborate like colleagues at a real engineering firm.

---

2. The Story of the Conversation (What They Were Talking About)

The conversation with ChatGPT went through three stages:

Stage 1 — One giant prompt. ChatGPT first wrote a single massive "mission prompt" instructing Claude to act as a CTO/architect and build the entire platform: 15+ named agents (CEO, Product Manager, Project Manager, System Architect, Backend, Frontend, Database, DevOps, QA, Security, Code Reviewer, Documentation, Research, Memory), structured inter-agent communication, task orchestration, a self-improvement loop where no first draft is ever accepted, a persistent memory system, a plugin architecture, multi-LLM support, a professional dashboard UI, strict engineering standards, and a 13-phase development process.

Stage 2 — "Why not 10 prompts?" The user suggested splitting that one prompt into ten equally detailed prompts, one per topic. ChatGPT agreed and sketched 10 specification documents (~2,000–4,000 words each) covering: vision & requirements, multi-agent architecture, backend & infrastructure, frontend & dashboard, memory & RAG, the autonomous development workflow, the tool ecosystem, engineering standards, deployment & production, and autonomous evolution.

Stage 3 — "Fine, make that document." ChatGPT's final proposal: don't write "prompts" at all — write **professional engineering specification documents**, like the internal design docs of a company with a 50-person engineering team. A 10-volume series (~500+ pages equivalent) in RFC/design-doc format: numbered sections, requirement IDs, acceptance criteria, decision records, risk analysis, trade-offs, ASCII architecture diagrams, folder trees, database schemas, and API docs. The reasoning: that level of structured detail gives a frontier AI model far more context to make consistent architectural decisions than any single prompt could.

This folder is that 10-volume series, actually written.

---

3. Document Index

Read in order for the full picture, or jump to the volume you need:

#	File	Volume	What It Covers
00	`00-README-Master-Overview.md`	Master Overview	This document — the story, the index, how to use the set
01	`01-Volume-1-Foundation-and-Product-Requirements.md`	Foundation & Product Requirements	Vision, personas, functional & non-functional requirements, user stories, success metrics, roadmap, high-level architecture, tech-stack justification, engineering principles, glossary
02	`02-Volume-2-Multi-Agent-Intelligence.md`	Multi-Agent Intelligence	Full spec for every agent (CEO → Memory): role, authority, prompts, tools, KPIs, failure recovery; agent hierarchy; structured message protocol; conflict resolution
03	`03-Volume-3-Backend-Engineering-and-Infrastructure.md`	Backend & Infrastructure	FastAPI service architecture, REST API + WebSocket specs, PostgreSQL schema, Redis usage, task orchestration engine, auth/RBAC, rate limiting, repo structure
04	`04-Volume-4-Frontend-and-Dashboard.md`	Frontend & Dashboard	Every screen with wireframes: projects, kanban, agent chat, logs, architecture viewer, memory explorer, plugin manager; design system, dark mode, accessibility, realtime sync
05	`05-Volume-5-AI-Brain-Memory-Knowledge-RAG.md`	AI Brain — Memory, Knowledge & RAG	Memory taxonomy, vector DB & embeddings, retrieval pipeline, knowledge graph, context-window management, knowledge base, learning from past projects
06	`06-Volume-6-Engineering-Workflow-and-Standards.md`	Engineering Workflow & Standards	The full idea-to-release pipeline with every handoff and approval gate; the self-improvement loop; sprints; plus the coding-standards "constitution" for all generated code
07	`07-Volume-7-Integrations-and-Tool-Ecosystem.md`	Integrations & Tool Ecosystem	Plugin architecture, agent tool layer, GitHub/Slack/Jira/Figma integrations, MCP support, multi-LLM provider gateway (Anthropic/OpenAI/Google/Ollama)
08	`08-Volume-8-Security.md`	Security	Threat model incl. AI-specific threats (prompt injection, malicious generated code), RBAC matrix, sandboxing, secrets, encryption, audit logs, OWASP + OWASP LLM compliance
09	`09-Volume-9-Production-and-Operations.md`	Production & Operations	CI/CD, Docker Compose → Kubernetes, monitoring (Prometheus/Grafana/Loki/OTel), SLOs, backups & disaster recovery, scaling, LLM cost controls, runbooks
10	`10-Volume-10-Future-and-Autonomous-`	Future & Autonomous	Agent self-evaluation, reflection loops, learning from failures, AI debates, automated new-agent creation, plugin marketplace,

#	File	Volume	What It Covers
	`Evolution.md`	Evolution	enterprise features, fine-tuning, roadmap
99	`99-APPENDIX-Original-ChatGPT-Conversation.md`	Appendix	The original ChatGPT conversation, preserved verbatim as source material

---

4. The Canonical Technology Stack

All ten volumes share one fixed set of technology decisions (justified in detail in Volume 1), so the documents never contradict each other:

- **Frontend:** Next.js 15 + React + TypeScript + Tailwind CSS + shadcn/ui
- **Backend / agent orchestrator:** Python 3.12 + FastAPI (async) + SQLAlchemy + Pydantic v2
- **Data:** PostgreSQL 16 (system of record) · Redis 7 (cache, queues, pub/sub) · Qdrant (vector database)
- **Messaging:** Redis Streams at MVP; NATS JetStream as the scale-out option
- **Realtime:** WebSockets via FastAPI
- **LLM access:** provider-agnostic gateway — Anthropic, OpenAI, Google, and **local models via Ollama**; switching providers is configuration-only
- **Dev & deploy:** Docker Compose locally (runs fully on one laptop with free tools) → Kubernetes in production
- **Observability:** OpenTelemetry + Prometheus + Grafana + Loki
- **Auth:** OAuth2/OIDC + JWT, with role-based access control

---

5. How to Use This Document Set

Option A — As a build blueprint. Give these volumes (one or a few at a time, in order) to Claude Code or another AI coding agent as project context, and implement TITAN phase by phase: Volume 1 first (requirements), then Volume 3 (backend skeleton), Volume 2 (agent framework), Volume 5 (memory), Volume 4 (dashboard), and so on. That is exactly the workflow the ChatGPT conversation recommended — spec-first, never everything in one shot.

Option B — As a reading/learning resource. The set doubles as a course in how real companies specify large systems: requirements engineering, architecture decision records, threat modeling, CI/CD design, and multi-agent AI patterns.

Option C — As a starting point to trim down. TITAN as specified is very ambitious. A realistic first milestone is a slice of it: 3–4 agents (Planner, Backend Engineer, Reviewer, QA), one project type, local-only, no Kubernetes. The volumes are written so an MVP path is always identified.

---

6. Honest Notes (Read Before Building)

- **This is a specification, not working software.** Everything here still has to be implemented, and building even the MVP slice is a serious multi-week project.
- **LLM cost is the biggest practical constraint.** A full agent-company run on cloud models can be expensive; the local-first design (Ollama) exists specifically so development and testing can be free.

- **Autonomy has limits.** Fully autonomous, production-quality software generation is a research frontier — expect the human to stay in the loop for approvals far longer than the vision statement implies. The specs include human approval gates for exactly this reason.

PROJECT TITAN — Autonomous Software Engineering Platform

Volume 1 — Foundation & Product Requirements

Field	Value
Series	PROJECT TITAN — Autonomous Software Engineering Platform (10 volumes)
Volume	1 — Foundation & Product Requirements
Doc ID	TITAN-VOL-01
Requirement Prefix	TITAN-FND-NNN
Status	Draft v1.0
Date	2026-07-04
Audience	All TITAN contributors (human and AI agents); readers of Volumes 2–10

Abstract. This volume is the root document of the PROJECT TITAN specification series. It defines what TITAN is (a production-grade, open-source, local-first multi-agent platform that autonomously designs, builds, tests, reviews, and deploys software from a natural-language request), why it exists, who it serves, and what it must do. It contains the numbered functional and non-functional requirements all other volumes trace to, the user stories and success metrics that define "done," the roadmap from MVP to v2, the high-level architecture, and — uniquely in this series — the authoritative justification of the canonical technology stack via trade-off analysis and ADR-style decision records. Volumes 2–10 reference these decisions; they do not re-litigate them.

Table of Contents

1. Introduction
2. Vision, Mission, and Problem Statement
3. User Personas
4. Functional Requirements
5. Non-Functional Requirements
6. User Stories and Acceptance Criteria
7. Success Metrics and KPIs
8. Constraints
9. Product Roadmap
10. High-Level System Architecture
11. Technology Stack Selection and Justification
12. Development Philosophy and Engineering Principles
13. Risks and Open Questions
14. Cross-References
15. Glossary of TITAN Terms

1. Introduction

1.1 Purpose

This document is the product foundation for PROJECT TITAN. Every agent design, service, screen, workflow, and security control in Volumes 2–10 must trace to a requirement here. If a later volume needs new scope, it proposes an amendment to this volume rather than inventing scope silently.

1.2 Scope of This Volume

In scope: vision and mission; problem definition; personas; functional and non-functional requirements; user stories; KPIs; constraints; roadmap; high-level architecture; canonical stack justification; engineering philosophy; series-wide glossary. Out of scope (owned elsewhere): per-agent specs (Vol. 2), backend service design (Vol. 3), UI screen specs (Vol. 4), memory/RAG internals (Vol. 5), workflow state machines (Vol. 6), integrations (Vol. 7), security controls (Vol. 8), production operations (Vol. 9), evolution features (Vol. 10).

1.3 Conventions

- Requirement IDs use the form `TITAN-FND-NNN`; IDs are never reused or renumbered.
- Priorities follow RFC 2119 semantics: **MUST** (release-blocking), **SHOULD** (expected unless an ADR justifies otherwise), **MAY** (optional).
- Every requirement carries acceptance criteria (AC) testable by a human or an AI agent without further interpretation.

---

2. Vision, Mission, and Problem Statement

2.1 Vision

A user types “Build me a Netflix clone,” “Build a Banking System,” “Build a Hospital Management System,” or “Build an AI SaaS” — and receives a production-ready application built by a coordinated organization of specialized AI agents. The human acts only as the client: answering essential clarification questions and approving releases. Everything else — requirements analysis, architecture, task breakdown, implementation, review, testing, security scanning, documentation, deployment — happens autonomously.

2.2 Mission

Build not one giant AI, but **an organization**: 15–20 specialized agents (CEO, Product Manager, Project Manager, System Architect, Backend Engineer, Frontend Engineer, Database Engineer, DevOps, QA, Security, Code Reviewer, Documentation, Research, Memory, and peers) — each with a role, responsibilities, memory, tools, communication abilities, decision authority, and a review process — collaborating like a real software company. TITAN is open source, runs fully on one laptop with free/local tools, and scales to enterprise Kubernetes without code changes.

2.3 The Problem TITAN Solves

#	Problem	Consequence
P1	Single-shot LLM codegen produces demos, not products	No tests, no review, no architecture; code decays immediately
P2	Coding assistants are single-threaded pair programmers	One context window cannot hold requirements, architecture, QA, and ops at once
P3	AI output is not reviewed the way human code is	No critique loop or rejection path; quality equals the first sample
P4	AI tools forget everything between sessions	No project memory, no reusable patterns, no lessons learned
P5	Vendor lock-in to one LLM provider	Cost, availability, and privacy constraints differ per user
P6	Cloud-only tooling excludes students and privacy-sensitive teams	Zero-budget and air-gapped users are shut out

TITAN answers P1–P4 with the **Self-Improvement Loop** (Implementation → Review → Critique → Refactor → Testing → Security Scan → Performance Analysis → Approval → Merge), executed by distinct agents that never accept the first draft; P5 with the LLM Gateway (ADR-007); P6 with the local-first hard constraint (TITAN-FND-060, C-1).

---

3. User Personas

3.1 Persona Summary

Persona	Canonical name	Environment	Primary Goal	Key Constraint
Solo developer / freelancer	**Dana**	Laptop, Docker Compose, one cloud LLM key + Ollama	Ship client projects 5–10× faster	Predictable cost per project
Startup team (2–10 eng.)	**Arjun's team**	Shared server / small K8s, cloud LLMs	Multiply output; keep quality bar	RBAC, shared memory, GitHub flow
Enterprise platform org	**Meridian Corp**	On-prem/VPC Kubernetes, approved model list	Autonomous tooling under compliance	SSO/OIDC, audit logs, data residency
Student / hobbyist	**Priya**	Laptop, 16 GB RAM, Ollama only, zero budget	Learn by watching an AI company work	Must run 100% free and offline

3.2 Persona Narratives

Dana submits a client brief, answers a handful of clarification questions, and reviews a working staging deployment hours later. She reads the Code Reviewer Agent's comments to keep final accountability, and she watches the cost meter: projected and actual LLM spend per project, before and during a run (TITAN-FND-021/024).

Arjun's team uses TITAN as a force multiplier: agents draft services, tests, and infra; humans review PRs in their real GitHub org. They need multi-user auth, per-project RBAC, shared organizational memory, and Slack notifications — and they are the reason the NATS JetStream scale-out path exists (ADR-005).

Meridian Corp runs TITAN inside its VPC. Only approved providers (a private endpoint plus local models via Ollama) may be called — enforced by gateway configuration, not code. Every agent action lands in an immutable audit log (Vol. 8); deployment is Kubernetes with the full observability stack (Vol. 9).

Priya has no credit card. She runs `docker compose up`, points the LLM Gateway at Ollama, and watches the agent organization plan and build a to-do app — reading every inter-agent message in Live Conversations to learn how engineering teams work. If any TITAN feature requires a paid service, TITAN has failed this persona (C-1).

---

4. Functional Requirements

Grouped by capability. Priorities: MUST / SHOULD / MAY. Targets map to Section 9.

4.1 Project Intake and Planning

ID	Requirement	Priority	Target	Acceptance Criteria
TITAN-FND-001	Accept a free-text project brief via UI and API, creating a Project entity.	MUST	MVP	POST `/api/v1/projects` returns 201 with ID; project visible on dashboard ≤ 2 s.
TITAN-FND-002	CEO Agent asks only essential clarification questions (≤ 7 at MVP); skipping them still yields a plan using documented defaults.	MUST	MVP	Standard brief → ≤ 7 questions; skip-all path produces a valid plan.
TITAN-FND-003	Decompose the brief into milestones, epics, and tasks with dependencies (a task DAG).	MUST	MVP	Plan passes cycle check; every task has owner role, estimate, ≥ 1 AC.
TITAN-FND-004	Auto-generate per project: roadmap, Kanban, task graph, dependency graph, sprint plan, release plan, architecture diagram, risk analysis, tech-debt report.	SHOULD	v1 (Kanban/task graph/roadmap: MVP)	Each artifact retrievable via API, rendered in UI, refreshed on plan change.

4.2 Agent Organization and Communication

ID	Requirement	Priority	Target	Acceptance Criteria
TITAN-FND-005	Run a roster of specialized agents (MVP: CEO, PM, Architect, Backend, Frontend, QA, Code Reviewer; v1: full 15–20 roster per Vol. 2).	MUST	MVP	Registry lists agents with role/status/current task; each MVP role completes ≥ 1 task in the E2E demo.
TITAN-FND-006	Agents communicate only via typed structured messages: Task Assignment, Progress Report, Review Request, Architecture Discussion, Bug Report, Decision Log, Meeting Summary, Conflict Resolution.	MUST	MVP	Every message validates against a published Pydantic schema; malformed messages rejected and logged.
TITAN-FND-007	Every communication is persisted; nothing is lost.	MUST	MVP	Kill orchestrator mid-project; on restart, message history is intact in PostgreSQL and the project resumes.

ID	Requirement	Priority	Target	Acceptance Criteria
TITAN-FND-008	Agents have defined decision authority and escalation paths (Architect binds Engineers; ties escalate to CEO Agent).	MUST	v1	Staged disagreement yields a Conflict Resolution message and a logged binding decision within one cycle.

4.3 Task Orchestration

ID	Requirement	Priority	Target	Acceptance Criteria
TITAN-FND-009	Tasks auto-create, assign, prioritize, estimate, review, complete, and reopen when review fails.	MUST	MVP	A task rejected by the Code Reviewer transitions to `reopened` with critique attached and re-executes without human action.
TITAN-FND-010	Independent tasks execute in parallel under a configurable concurrency limit.	MUST	MVP	Concurrency 3 with 5 independent tasks → exactly 3 concurrent; wall time < sequential baseline.
TITAN-FND-011	Task queue on Redis Streams with consumer groups; at-least-once delivery with idempotent handlers.	MUST	MVP	Killing a worker mid-task redelivers to another worker; no task lost or duplicated in effect.

4.4 Code Generation and the Self-Improvement Loop

ID	Requirement	Priority	Target	Acceptance Criteria
TITAN-FND-012	Generate frontend, backend, database, infrastructure, tests, docs, deployment, scripts, and config into a versioned per-project git workspace.	MUST	MVP	Demo project yields a repo that builds and runs via its own `docker compose up`; all code agent-authored.
TITAN-FND-013	Every component passes the Self-Improvement Loop; the first draft is never merged directly.	MUST	MVP (Security Scan + Perf Analysis stages: v1)	Any merged change shows ≥ 1 review cycle with critique and a diff from draft 1.
TITAN-FND-014	Generated projects include unit, integration, and E2E tests; QA Agent gates merges on passing tests.	MUST	MVP (E2E: v1)	Demo project ≥ 60% line coverage; failing tests block `approved`.
TITAN-FND-015	Security Agent scans code and dependencies (SAST + dependency audit) and can block a release.	MUST	v1	Known-vulnerable dependency in a fixture produces a blocking finding.

4.5 Memory and Knowledge

ID	Requirement	Priority	Target	Acceptance Criteria
TITAN-FND-	Short-term (context), project, and organizational memory; all persistent memory survives restart.	MUST	MVP (org learning	Post-restart, an agent answers “what architecture did we choose and why”

ID	Requirement	Priority	Target	Acceptance Criteria
016			loop: v1)	from memory, not chat history.
TITAN-FND-017	Semantic retrieval over memories, code, and docs via Qdrant with a swappable, local-by-default embedding model.	MUST	MVP	"How did we implement auth last time" returns the prior decision record in top-3 on the fixture corpus.
TITAN-FND-018	Maintain a knowledge base: coding standards, architecture patterns, best practices, API references, decision history, reusable templates.	SHOULD	v1	Agents cite entries by ID in Decision Logs; entries are versioned and editable in UI.

4.6 Multi-LLM Support

ID	Requirement	Priority	Target	Acceptance Criteria
TITAN-FND-019	All model calls pass through a provider-agnostic LLM Gateway supporting Anthropic, OpenAI, Google, and local models via Ollama.	MUST	MVP	Same agent task completes on Anthropic and on Ollama with only a config change.
TITAN-FND-020	Provider/model selection is configuration-only and per-agent-role overridable.	MUST	MVP	Changing `agents.architect.model` reroutes that agent's calls (verified in gateway logs).
TITAN-FND-021	Gateway records tokens, latency, and cost per call, aggregated per agent, task, and project.	MUST	MVP	Dashboard spend within 5% of provider-reported usage in a controlled test.

4.7 User Interface

ID	Requirement	Priority	Target	Acceptance Criteria
TITAN-FND-022	Professional, responsive, dark-mode-first dashboard: Organization View, Projects, Tasks/Kanban, Agent Status, Live Conversations, Logs, Architecture Viewer, Deployment Dashboard, Monitoring, Memory Explorer, Plugin Manager, Settings.	MUST (MVP subset: Projects, Kanban, Agent Status, Live Conversations, Logs, Settings)	MVP/v1	Screens exist per Vol. 4; Lighthouse a11y ≥ 90; usable at 375 px width.
TITAN-FND-023	Agent activity streams to the UI in real time over WebSockets.	MUST	MVP	Agent message appears in Live Conversations ≤ 1 s (p95, local).
TITAN-FND-024	Show projected and running LLM cost per project, with a budget cap that pauses the project when exceeded.	MUST	v1 (display: MVP)	$5 cap halts new LLM calls within one cycle of being reached; project enters `paused_budget`.

4.8 Plugins, Integrations, and Deployment of Generated Apps

ID	Requirement	Priority	Target	Acceptance Criteria
TITAN-FND-025	Plugin architecture for integrations (GitHub, GitLab, Docker, Slack, Discord, Jira, Linear, Figma, VS Code, browser automation, MCP servers) with no core changes.	MUST (architecture); SHOULD (specific plugins, Vol. 7)	v1	A sample plugin installs, registers tools, and is invoked without modifying core packages.
TITAN-FND-026	GitHub integration: push generated repos, open PRs, mirror internal review as PR comments.	SHOULD	v1	Demo project appears as a repo with ≥ 1 merged agent-authored PR carrying review comments.
TITAN-FND-027	Deploy generated apps at minimum to local Docker Compose; cloud targets via plugins.	MUST (local); MAY (cloud, v2)	MVP	"Deploy" yields a reachable local URL; deployment logs stream to UI.
TITAN-FND-028	Configurable human approval gates per project (release, budget, destructive actions).	MUST	MVP	With `release_approval: human`, release blocks until Approve; approval recorded with identity and timestamp.

4.9 Platform Administration

ID	Requirement	Priority	Target	Acceptance Criteria
TITAN-FND-029	Multi-user auth via OAuth2/OIDC with JWT sessions; single-user local mode needs no external IdP.	MUST	MVP (local mode); v1 (OIDC)	Local mode works offline with a bootstrap admin; v1 logs in via Keycloak (reference OIDC IdP).
TITAN-FND-030	RBAC with at least Owner, Maintainer, Viewer per organization and project.	MUST	v1	Viewer mutations return 403, enforced server-side.

5. Non-Functional Requirements

ID	Category	Requirement	Priority	Acceptance Criteria
TITAN-FND-050	Performance	API reads p95 < 200 ms; WS delivery p95 < 1 s; scheduling overhead < 250 ms per task transition (excluding LLM latency).	MUST	Load test, 50 concurrent dashboard users on reference hardware, meets targets.
TITAN-FND-051	Scalability	Scale from 1 laptop to K8s running ≥ 100 concurrent agent tasks across ≥ 20 projects by adding workers — config only (queue swap per ADR-005 allowed).	MUST	3-node cluster sustains 100 concurrent tasks, < 5% failure rate.
TITAN-FND-052	Reliability	No committed data loss on crash: PostgreSQL is the system of record; queue redelivery covers in-flight work; projects always resume after full restart.	MUST	Chaos test (kill any one service) loses no acknowledged data; local recovery ≤ 60 s.

ID	Category	Requirement	Priority	Acceptance Criteria
TITAN-FND-053	Reliability	Graceful LLM-provider degradation: retry with backoff, fail over to configured fallback, pause (not corrupt) projects if all providers are down.	MUST	Simulated outage fails over within 3 retries; no fallback → `paused_provider`, clean resume.
TITAN-FND-054	Cost	A CRUD web-app project completes on the local-only profile at $0 LLM spend; cloud-profile cost is visible and capped.	MUST	Fixture completes end-to-end on Ollama with zero outbound paid calls (network capture).
TITAN-FND-060	Local-first	Full platform (UI, orchestrator, datastores, local LLM path) runs on one laptop via `docker compose up`, ≤ 16 GB RAM excluding model, fully offline after pulls.	MUST	Fresh clone → running platform ≤ 15 min on reference hardware, network then disabled.
TITAN-FND-061	Observability	All services emit OTel traces, Prometheus metrics, structured logs to Loki; Grafana health dashboard ships preconfigured.	MUST	One agent task is followable as a single distributed trace: API → queue → agent → gateway → DB.
TITAN-FND-062	Security	Secure defaults: secrets via env/secret store only; TLS in production; audit log of privileged actions; OWASP ASVS L1 at v1, L2 at v2 (Vol. 8).	MUST	Secret-scanning CI gate passes; Vol. 8 checklist signed off before v1.
TITAN-FND-063	Portability	No hard dependency on any cloud vendor or paid service in the core platform.	MUST	Build + E2E pass with no cloud credentials configured.
TITAN-FND-064	Maintainability	Monorepo, strict typing (`tsc --strict`, `mypy --strict`), enforced lint/format, ≥ 80% coverage on core orchestration code.	SHOULD	CI blocks merges violating typing, lint, or coverage gates.

6. User Stories and Acceptance Criteria

US-1 (Priya — local-first run). *As a student with no budget, I want to run all of TITAN offline with a local model, so I can learn how an AI engineering organization works for free.* - **Given** Docker and a pulled Ollama model, **when** I run `docker compose up` and open the dashboard, **then** I can create a project and watch agents build it with zero paid API calls, every inter-agent message visible live. (TITAN-FND-019, -023, -054, -060)

US-2 (Dana — brief to running app). *As a freelancer, I want to submit a brief and receive a reviewed, tested, locally deployed application.* - **Given** a CRUD-app brief, **when** I answer ≤ 7 questions and approve the plan, **then** TITAN produces a git repo with tests and a running local URL, and every merged change shows ≥ 1 review cycle. (TITAN-FND-001, -002, -012, -013, -027)

US-3 (Dana — cost control). *As a solo dev paying per token, I want a per-project budget cap so a runaway loop cannot surprise my bill.* - **Given** a $5 cap, **when** tracked cost reaches $5, **then** the project pauses within one cycle and notifies me; raising the cap resumes it. (TITAN-FND-021, -024)

US-4 (Arjun's team — GitHub-native). *As a startup engineer, I want TITAN's work delivered as GitHub PRs with the agents' review discussion attached.* - **Given** the GitHub plugin, **when** an agent completes a task, **then** a PR opens with the change, the critique thread, and passing CI. (TITAN-FND-025, -026)

US-5 (Arjun's team — access control). *As a team lead, I want RBAC so a contractor can watch progress but not approve releases.* - **Given** a Viewer-role user, **when** they attempt any mutation, **then** the API returns 403 and the attempt is audit-logged. (TITAN-FND-029, -030, -062)

US-6 (Meridian — provider governance). *As an enterprise platform owner, I want to restrict TITAN to approved providers via configuration so no data leaves our boundary.* - **Given** a gateway config allowing only

`ollama` and one private endpoint, **when** any agent requests a model, **then** only allowed providers are called (egress audit), and gateway config edits require Owner role. (TITAN-FND-019, -020, -030)

US-7 (All — resumability). *As any user, I want projects to survive a crash or reboot so hours of agent work are never lost.* - **Given** a project mid-build, **when** the host restarts, **then** the project resumes from its last consistent state with full message history. (TITAN-FND-007, -052)

US-8 (Priya — learning by inspection). *As a learner, I want to open the Memory Explorer and Decision Log to understand why the architecture was chosen.* - **Given** a completed project, **when** I open its Decision Log, **then** each major choice shows context, alternatives, and rationale in ADR form. (TITAN-FND-016, -018)

---

7. Success Metrics and KPIs

KPI	Definition	MVP	v1	v2
Autonomous completion rate	% of the fixed 10-brief benchmark suite producing a running, test-passing app with no human edits	≥ 50%	≥ 75%	≥ 90%
First-draft rejection rate	% of implementation tasks reopened ≥ once by review (proves the loop bites)	≥ 30%	30–60%	30–60%
Time-to-first-deploy	Median wall time, brief → local deployment, CRUD benchmark, cloud profile	≤ 4 h	≤ 1 h	≤ 30 min
Local-only viability	Benchmark briefs completing on the Ollama-only profile	≥ 3/10	≥ 6/10	≥ 8/10
Cost per benchmark project	Median gateway-tracked spend, cloud profile	≤ $15	≤ $8	≤ $5
Generated test coverage	Median line coverage of generated projects	≥ 60%	≥ 75%	≥ 80%
Crash recovery	Orchestrator recovery success in the chaos suite	100%	100%	100%
Community adoption	GitHub stars / external contributors	n/a	2,000 / 10	10,000 / 50
Setup friction	Fresh clone → running platform, reference laptop	≤ 15 min	≤ 10 min	≤ 10 min

Anti-goal: **human intervention count** (clarifications + approvals) is displayed but not driven to zero — approval gates (TITAN-FND-028) are a feature, not friction.

---

8. Constraints

#	Constraint	Type	Consequence
C-1	**Local-first and free-tier-friendly is a hard requirement**: the core platform runs fully on one laptop with free/local tools; no feature may mandate a paid service.	Product (user-imposed, non-negotiable)	Every design in Vols. 2–10 includes a local profile; paid/cloud paths are optional.
C-2	Open-source project (licence selection tracked as Q1).	Legal	No closed-source dependencies in core.
C-3	Reference hardware: 4-core/16 GB laptop; reference cluster: 3 × 4-vCPU/16 GB nodes.	Technical	All NFR targets measured against these.

#	Constraint	Type	Consequence
C-4	LLM non-determinism and context limits are environmental facts.	Technical	Review loop tolerates variable quality; context bounded per call (Vol. 5).
C-5	The canonical stack (Section 11) is fixed series-wide; changes require an ADR amendment here.	Process	Prevents drift among 10 parallel volume authors.
C-6	Cloud deployment targets for generated apps are plugin territory (Vol. 7), not core.	Scope	Keeps core cloud-neutral (TITAN-FND-063).
C-7	OSS contributors have intermittent availability.	Process	Volumes must be implementable without oral tradition.

---

9. Product Roadmap

9.1 MVP — "The Loop Works" (M1)

Prove the thesis end-to-end on one laptop: intake, clarification, and planning (TITAN-FND-001–003); 7-agent roster (TITAN-FND-005); Redis Streams orchestration with parallelism and reopen-on-reject (TITAN-FND-009–011); Self-Improvement Loop minus Security/Perf stages (TITAN-FND-013); git workspaces and local deploy of generated apps (TITAN-FND-012, -027); LLM Gateway with all four providers, per-role routing, cost tracking (TITAN-FND-019–021); project memory + Qdrant retrieval (TITAN-FND-016–017); dashboard subset with WebSocket streaming (TITAN-FND-022–023); local auth and approval gates (TITAN-FND-028–029); observability stack in Compose (TITAN-FND-061).

Exit criteria: MVP KPI column met; the brief "build a task-tracking web app" completes autonomously on both cloud and local profiles.

9.2 v1 — "A Real Organization" (M2)

Full 15–20 agent roster incl. Security, DevOps, Database, Documentation, Research, Memory (Vol. 2); Security Scan + Performance Analysis stages and Security release-blocking (TITAN-FND-013, -015); organizational memory and cited knowledge base (TITAN-FND-016, -018); full dashboard incl. Architecture Viewer, Memory Explorer, Deployment Dashboard, Monitoring, Plugin Manager (TITAN-FND-022); plugin architecture GA with GitHub and Slack plugins (TITAN-FND-025–026); OIDC, RBAC, budget auto-pause (TITAN-FND-024, -029–030); Kubernetes/Helm production deployment (Vol. 9) with NATS JetStream tested as queue scale-out (ADR-005); complete auto-generated artifacts (TITAN-FND-004).

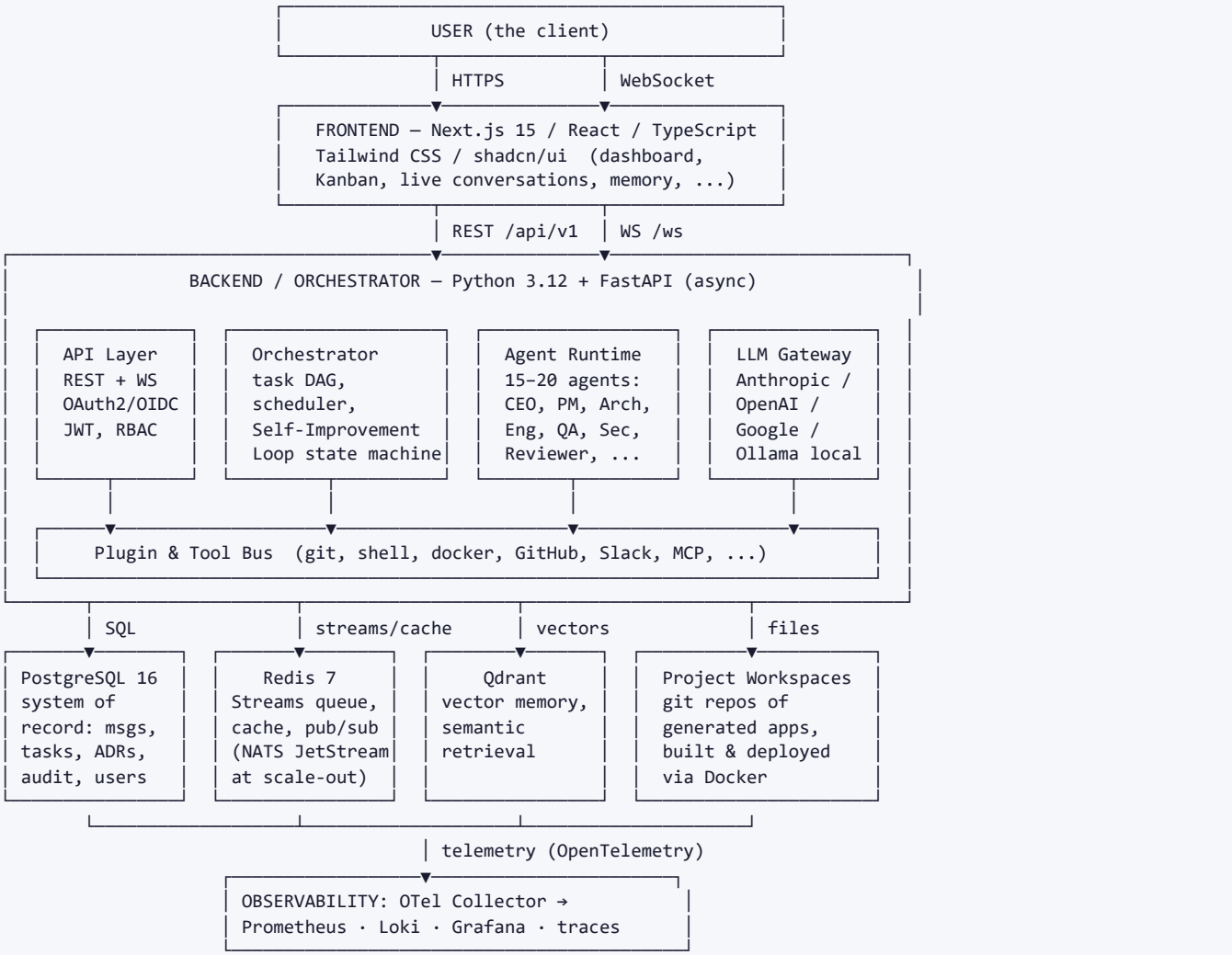
9.3 v2 — "The Platform Learns" (M3)

Cross-project learning and reusable-pattern mining (Vols. 5, 10); agent self-evaluation, reflection loops, AI debates, multi-model collaboration (Vol. 10); plugin marketplace and custom user-defined agents (Vol. 10); cloud deployment plugins for generated apps (TITAN-FND-027 MAY scope); enterprise hardening — ASVS L2, air-gap profile, data residency (Vol. 8); v2 KPI column incl. ≤ 30 min time-to-first-deploy.

---

10. High-Level System Architecture

TITAN is a **modular monolith at MVP** — one FastAPI process hosting API, orchestrator, agent runtime, and gateway as boundary-enforced packages — decomposable into services at scale; the module seams are the service seams (Vol. 3). This is the canonical context view:



Canonical dataflow (detailed in Vol. 6): brief → API → CEO/PM/Architect plan → orchestrator materializes the task DAG in PostgreSQL and enqueues ready tasks on Redis Streams → agent workers consume tasks, call models through the LLM Gateway, act on workspaces via the Tool Bus → every message and state change persists to PostgreSQL and streams to the UI over WebSockets → the Self-Improvement Loop gates every artifact before merge → approved releases deploy via Docker.

---

11. Technology Stack Selection and Justification

This volume **owns** stack justification; other volumes cite these ADRs (e.g., "per ADR-005") without repeating the argument. Changing any decision requires amending this section (C-5).

11.1 Canonical Stack and Trade-off Table

Layer	Canonical Choice	Leading Alternatives	Decisive Trade-off
Repo layout	Monorepo	Polyrepo	Cross-cutting agent changes and shared types outweigh repo-size cost
Frontend	Next.js 15 + React + TypeScript + Tailwind CSS + shadcn/ui	SvelteKit; Vue/Nuxt; Vite SPA	Ecosystem depth; streaming UI; shadcn/ui is owned code, not a dependency
Backend	Python 3.12 + FastAPI (async) + SQLAlchemy + Pydantic v2	NestJS; Go	AI/LLM ecosystem gravity; Pydantic as message-schema backbone
System of record	PostgreSQL 16	MySQL; MongoDB	ACID + JSONB covers relational and document needs; boring and free
Cache/queue /pub-sub	Redis 7 (Streams)	RabbitMQ; Kafka; NATS	One container serves three roles → smallest local footprint
Queue at scale	NATS JetStream (documented option)	Kafka	Far lighter operationally; semantics map from Streams
Vector DB	Qdrant	pgvector; Weaviate; Chroma; Pinecone	OSS, small container, first-class filtering; Pinecone fails C-1
Realtime	WebSockets via FastAPI	SSE; socket.io; Pusher	Duplex needed (approvals travel up); no extra service; Pusher fails C-1
LLM access	Provider-agnostic gateway (Anthropic, OpenAI, Google, Ollama)	Direct SDKs; LangChain coupling	Config-only switching is a product requirement (TITAN-FND-019/020)
Local runtime	Docker Compose	Nix; bare venv/node	One-command onboarding; same containers promote to K8s
Production	Kubernetes (Helm)	ECS; Nomad; Fly.io	Cloud-neutral (TITAN-FND-063); HPA on workers
Observability	OpenTelemetry + Prometheus + Grafana + Loki	Datadog; ELK	Fully OSS; OTel keeps instrumentation vendor-neutral
Auth	OAuth2/OIDC + JWT + RBAC	Sessions only; Auth0/Clerk	OIDC = enterprise SSO path; JWT suits WS auth; local mode needs no IdP

11.2 Decision Records

ADR-001 — Monorepo - **Context:** TITAN spans a TypeScript frontend, Python backend, shared schemas, infra, and docs — edited by humans and by TITAN's own agents, whose changes are frequently cross-cutting. - **Decision:** One monorepo (`apps/web`, `apps/api`, `packages/schemas`, `infra/`, `docs/`); TypeScript types generated from Pydantic/OpenAPI bridge the languages. - **Alternatives:** Polyrepo — rejected: cross-repo atomic changes and version skew are poison for agents that must see the whole system. Bazel-style build tooling — rejected: onboarding cost violates the ≤ 15 min setup NFR. - **Trade-offs:** Accept larger clones and mixed-language CI; gain atomic cross-stack commits, one source of truth for agent code-reading, and simpler onboarding.

ADR-002 — Python 3.12 + FastAPI (async) + SQLAlchemy + Pydantic v2 - **Context:** The backend is I/O-bound orchestration (LLM calls, queue, DB) at the center of a Python-first AI ecosystem. - **Decision:** Python 3.12, fully async FastAPI, SQLAlchemy 2.x async, Pydantic v2 for all models — including the typed agent messages of TITAN-FND-006. - **Alternatives:** NestJS/TypeScript (one language across the stack) — rejected: the LLM tooling ecosystem, from provider SDKs to tokenizers to local-model glue, is Python-first; fighting that daily costs more than a two-language stack. Go — rejected: weakest LLM ecosystem, slower iteration on agent

logic. - **Trade-offs:** Accept Python's per-process ceiling and the GIL (mitigated: async I/O-bound workload, horizontal workers); gain Pydantic v2 (Rust-core) validation as the message backbone, first-class SDKs, and the largest OSS-AI contributor pool.

ADR-003 — Next.js 15 + React + TypeScript + Tailwind CSS + shadcn/ui - **Context:** The dashboard is large (12+ screen areas), real-time, dark-mode-first, and maintained by contributors and by TITAN's own Frontend Agent. - **Decision:** Next.js 15 (App Router, RSC), strict TypeScript, Tailwind, shadcn/ui. - **Alternatives:** SvelteKit — rejected: far smaller component/contributor ecosystem, and LLMs generate React most reliably, which matters when agents maintain the UI. Vite SPA — rejected: loses streaming/RSC and routing conventions that keep a 12-screen app coherent. MUI/Chakra — rejected: shadcn/ui vendors components into the repo, giving full theming control and no upstream breakage, ideal for agent modification. - **Trade-offs:** Accept Next.js framework complexity; gain conventions that constrain generated code, streaming for live agent feeds, and accessible Radix primitives supporting the a11y AC of TITAN-FND-022.

ADR-004 — PostgreSQL 16 as the single system of record - **Context:** Durable, audit-critical, strongly consistent state: projects, task DAGs, every agent message (TITAN-FND-007), decisions, users, audit logs (TITAN-FND-052). - **Decision:** PostgreSQL 16 for all durable state — relational tables for tasks/projects/users, JSONB for message payloads, LISTEN/NOTIFY available for change signals. - **Alternatives:** MongoDB — rejected: the core data is relational (DAG edges, joins for Kanban) and transactional integrity is the crash-recovery backbone. MySQL — rejected: weaker JSONB/extension story. Separate event store — rejected at MVP: a second system without need; event sourcing stays a Vol. 3 option inside Postgres. - **Trade-offs:** Accept a single engine to scale later (replicas/partitioning); gain ACID everywhere, one backup story, and a 256 MB local footprint.

ADR-005 — Redis 7 (Streams) at MVP; NATS JetStream as scale-out option - **Context:** The task/agent queue needs consumer groups, redelivery, and ordering (TITAN-FND-011); the platform also needs a cache and pub/sub for WebSocket fan-out; C-1 demands a minimal footprint. - **Decision:** Redis 7 serves all three roles at MVP — Streams + consumer groups for the queue, key/value cache, pub/sub for realtime fan-out — behind a `TaskQueue` interface. **NATS JetStream** is the canonical scale-out replacement, documented and integration-tested at v1. - **Alternatives:** Kafka — rejected for MVP: disproportionate footprint and ops burden on a laptop. RabbitMQ — rejected: adds a fourth service while Redis is already present. NATS from day one — rejected: Redis is still needed for caching, so MVP would run two systems where one suffices. - **Trade-offs:** Accept weaker delivery guarantees than a dedicated broker (mitigated by idempotent handlers and PostgreSQL as source of truth) and a bounded future migration; gain one ~30 MB container covering three needs — the largest single contributor to the 16 GB local budget.

ADR-006 — Qdrant as the vector database - **Context:** Memory and RAG (TITAN-FND-016/017, Vol. 5) need semantic search over messages, code, decisions, and docs with per-project/type metadata filtering, locally and free. - **Decision:** Qdrant, single container, with a swappable embedding model — local sentence-transformers by default, provider embeddings optional via the gateway. - **Alternatives:** pgvector — rejected as canonical: weaker filtering/ANN operations at organizational-memory scale, and coupling vector load to the system of record threatens the p95 read NFR; retained as the documented "minimal profile" fallback. Chroma — rejected: less mature for concurrent multi-project use. Weaviate — rejected: heavier, more opinionated. Pinecone — rejected outright: paid SaaS violates C-1. - **Trade-offs:** Accept one more container (~150 MB); gain purpose-built ANN performance and the rich payload filtering Vol. 5's retrieval pipeline depends on.

ADR-007 — Provider-agnostic LLM Gateway (Anthropic, OpenAI, Google, Ollama) - **Context:** Config-only provider switching (TITAN-FND-019/020) serves cost (Dana), governance (Meridian), and zero-budget use (Priya); per-role model routing and cost metering (TITAN-FND-021) are product features. - **Decision:** A single in-process gateway module (extractable to a service later): unified chat/tool-call/embedding interface, four provider adapters, per-role routing table in config, retry/failover (TITAN-FND-053), token/cost accounting, full OTel tracing. Agents never import provider SDKs directly — enforced by lint rule. - **Alternatives:** Direct SDK

use per agent — rejected: N×M coupling kills config-only switching. LangChain/LiteLLM as the abstraction — LiteLLM may be used *inside* an adapter, but TITAN's own interface is the contract; external abstractions churn too fast to anchor a spec. - **Trade-offs:** Accept adapter maintenance and a common-denominator interface (provider extras exposed via typed capability flags); gain the product requirement itself — provider swaps in one YAML edit, uniform budget caps, one egress audit point.

ADR-008 — Docker Compose local; Kubernetes (Helm) production - **Context:** C-1 requires the whole platform on one laptop in ≤ 15 minutes; enterprises require declarative, scalable production (TITAN-FND-051). - **Decision:** One `docker-compose.yml` runs web, api, workers, PostgreSQL, Redis, Qdrant, and observability locally. Production ships as Helm charts deploying the *same images* to any Kubernetes, HPA on agent workers. - **Alternatives:** Nix — rejected: contributor onboarding cost. Fly.io/ECS as primary target — rejected: vendor coupling violates TITAN-FND-063. k3d/kind locally — optional profile for chart testing, not the default. - **Trade-offs:** Accept two deployment descriptors (mitigated: same images, env-based config in both); gain dev/prod parity and a zero-cloud local story.

ADR-009 — WebSockets served by FastAPI - **Context:** The dashboard streams agent messages, task transitions, and logs live (TITAN-FND-023) and carries approvals back. - **Decision:** Native FastAPI WebSocket endpoints, JWT-authenticated, fanned out across instances via Redis pub/sub. - **Alternatives:** SSE — rejected: uni-directional; approvals and agent chat need duplex. socket.io — rejected: protocol lock-in on both ends for unneeded features. Pusher/Ably — rejected: violates C-1. - **Trade-offs:** Accept in-house reconnect/connection-state logic (specified in Vol. 4); gain a duplex channel with no extra infrastructure and one auth model.

ADR-010 — OAuth2/OIDC + JWT with RBAC - **Context:** Enterprises need SSO (Meridian); teams need roles (US-5); Priya needs zero external identity dependency (C-1). - **Decision:** OAuth2/OIDC (any compliant IdP; Keycloak as OSS reference), JWT access tokens for API and WebSocket auth, RBAC with Owner/Maintainer/Viewer as the base set (extended in Vol. 8). Local single-user mode issues JWTs from a built-in bootstrap identity — no IdP required. - **Alternatives:** Server-side sessions only — rejected: awkward for WS, API tokens, and horizontal scale. Auth0/Clerk — rejected: C-1. Fully custom auth — rejected: security surface without benefit; OIDC is what enterprises demand. - **Trade-offs:** Accept JWT revocation discipline (short TTL + refresh, Vol. 8); gain standard SSO, stateless service auth, identical mechanism locally and in production.

ADR-011 — OpenTelemetry + Prometheus + Grafana + Loki - **Context:** A system that acts unattended for hours is undebuggable without correlated traces; TITAN-FND-061 demands one distributed trace per agent task, on free tooling. - **Decision:** OTel SDKs in api/workers/gateway → OTel Collector; Prometheus metrics, Loki logs, Grafana dashboards — preconfigured in Compose and Helm. - **Alternatives:** Datadog/New Relic — rejected: paid, C-1. ELK — rejected: Elasticsearch's RAM appetite breaks the 16 GB budget; Loki's label model is cheaper and sufficient. "Just logs" — rejected: cross-service agent workflows require traces. - **Trade-offs:** Accept ~4 observability containers locally (~1.5 GB; a `--profile minimal` Compose profile can disable them on constrained machines); gain vendor-neutral instrumentation and out-of-the-box "follow one task through the system."

---

12. Development Philosophy and Engineering Principles

These principles bind human contributors, TITAN's agents when they modify TITAN, and the code TITAN generates (elaborated as the engineering "constitution" in the Engineering Standards volume).

1. **Build an organization, not a monolith brain.** Capability comes from specialized roles, structured communication, and adversarial review — not one ever-larger prompt.

2. **Never accept the first draft.** Every artifact passes the Self-Improvement Loop; a 0% rejection rate is a defect in the reviewers, not excellence in the authors (see the KPI band).
3. **The human is the client; consent is sacred.** Autonomy operates between explicit, configurable approval gates (TITAN-FND-028) — few questions, but gates are never bypassed.
4. **Local-first is a feature, not a fallback.** Every capability ships with a free/local profile. If it cannot run on Priya's laptop, it is not core.
5. **Everything persists; nothing is lost.** Messages, decisions, critiques, and costs are durable records (TITAN-FND-007); debuggability and organizational learning both depend on total recall.
6. **Boring technology, exciting architecture.** Postgres, Redis, and Docker are deliberately unexciting; the novelty budget is spent on agent intelligence.
7. **Interfaces over implementations.** Queue, vector store, LLM provider, and plugins sit behind typed interfaces (ADR-005/006/007, TITAN-FND-025) so components swap without core changes.
8. **Strict typing end to end.** Pydantic v2 and strict TypeScript are the contract layer; `mypy --strict` and `tsc --strict` gate CI (TITAN-FND-064); untyped agent messages are rejected at runtime (TITAN-FND-006).
9. **No placeholders, no TODOs.** In specs, platform code, and generated code, every phase completes and is reviewed before the next begins — per the source mandate's 13-phase process.
10. **Observable by default.** New code paths must be traceable end-to-end before merge (TITAN-FND-061). SOLID, Clean Architecture, DI, and the repository pattern apply where they serve these goals; DDD and CQRS only where an ADR justifies them (Vol. 3).
11. **Measure the product, not the vibes.** Every release is judged against Section 7's KPI table on the fixed benchmark-brief suite.

---

13. Risks and Open Questions

13.1 Risks

#	Risk	Likelihood	Impact	Mitigation
R1	Small local models (7–13B) cannot complete complex briefs → local-only KPI missed	High	High	Decomposition tuned for small contexts (Vol. 5); staged local-only KPI (3/10 → 8/10); per-role routing lets a strong cloud model backstop hard tasks where allowed
R2	Review ping-pong burns tokens without converging	Medium	High	Loop iteration caps, budget caps (TITAN-FND-024), escalation to human after N rejections (Vol. 6)
R3	Redis Streams delivery edge cases corrupt orchestration state	Low	High	PostgreSQL as sole source of truth; idempotent handlers; chaos suite in CI (TITAN-FND-052)
R4	Provider API churn breaks gateway adapters	Medium	Medium	Nightly contract tests against live + recorded fixtures; gateway interface limits blast radius
R5	Ten parallel volumes drift or contradict	Medium	High	Canonical decisions + C-5 amendment process; cross-reference review before each volume's v1.0
R6	Agent-executed shell/docker tools endanger user machines	Medium	Critical	Sandboxed (containerized) tool runs, allowlisted commands, approval gates for destructive actions (Vol. 8)
R7	OSS project fails to attract maintainers	Medium	Medium	≤ 15 min setup NFR, spec-first docs, benchmark suite as a contribution target

13.2 Open Questions (tracked; none block MVP implementation)

#	Question	Owner	Resolve by
Q 1	Final OSS licence: Apache-2.0 (working assumption series-wide) vs. AGPL-3.0 to deter closed SaaS forks	Vol. 1 amendment	First public release
Q 2	Default local embedding model (quality vs. RAM on 16 GB machines)	Vol. 5	MVP feature freeze
Q 3	LLM Gateway extracted to a standalone service at v1, or in-process until v2	Vol. 3	v1 architecture review
Q 4	Composition of the fixed 10-brief benchmark suite behind Section 7 KPIs	Vol. 6	MVP test-plan sign-off
Q 5	Agents as RBAC principals with scoped permissions: v1 or v2	Vol. 8	v1 security review

14. Cross-References

Volu me	Title	What it takes from Volume 1
Vol. 2	Multi-Agent Intelligence	Agent roster (TITAN-FND-005), communication and authority model (TITAN-FND-006–008)
Vol. 3	Backend Engineering & Infrastructure	Architecture (Sec. 10); ADR-002/004/005/008/009/010; NFR targets (Sec. 5); Q3
Vol. 4	Frontend & Dashboard	Screen inventory (TITAN-FND-022–024); ADR-003/009; personas driving UX
Vol. 5	AI Brain — Memory, Knowledge & RAG	Memory requirements (TITAN-FND-016–018); ADR-006/007; R1, Q2
Vol. 6	Autonomous Engineering Workflow	Self-Improvement Loop (TITAN-FND-013); orchestration (TITAN-FND-009–011); gates (TITAN-FND-028); Q4
Vol. 7	Tool Ecosystem & Integrations	Plugin architecture (TITAN-FND-025–027); Tool Bus (Sec. 10); C-6
Vol. 8	Security & Compliance	TITAN-FND-029/030/062; ADR-010; R6, Q5
Vol. 9	Deployment & Production Operations	ADR-008/011; TITAN-FND-051/052/061; roadmap milestones
Vol. 10	Autonomous Evolution & Future	v2 roadmap (Sec. 9.3); KPI evolution targets; marketplace scope

15. Glossary of TITAN Terms

Term	Definition
Agent	A specialized autonomous worker (e.g., Backend Engineer Agent) with a defined role, tools, memory, decision authority, and message interfaces (Vol. 2).
Agent Runtime	The backend subsystem that instantiates agents, feeds them tasks, and mediates tool/LLM access.

Term	Definition
Approval Gate	A configurable checkpoint where autonomous execution pauses for human consent (TITAN-FND-028).
Benchmark-Brief Suite	The fixed set of 10 project briefs used to measure KPIs across releases (Q4).
Brief	The user's natural-language project request that initiates a Project.
Clarification Round	The bounded set of essential questions (≤ 7 at MVP) asked after intake (TITAN-FND-002).
Decision Log	Persisted ADR-style records of significant choices, citable by ID.
Knowledge Base	Curated organizational content — standards, patterns, templates, references — that agents cite (TITAN-FND-018).
LLM Gateway	The provider-agnostic model-access layer (ADR-007); the only path to any LLM.
Local-Only Profile	Configuration in which every component, including models (Ollama), runs on the user's machine with zero paid calls.
Memory (short-term / project / organizational)	In-context working state; durable knowledge scoped to one project; durable knowledge shared across all projects (Vol. 5).
Message (structured)	A Pydantic-validated inter-agent communication: Task Assignment, Progress Report, Review Request, Architecture Discussion, Bug Report, Decision Log, Meeting Summary, or Conflict Resolution.
Orchestrator	The scheduler that materializes task DAGs, enqueues ready tasks, enforces the Self-Improvement Loop state machine, and handles escalation.
Plugin / Tool Bus	The extension architecture through which integrations and agent tools register without core changes (TITAN-FND-025, Vol. 7).
Project Workspace	The per-project git repository holding all generated artifacts, built and deployed via Docker.
Reference Hardware	The 4-core/16 GB laptop and 3-node cluster against which all NFRs are measured (C-3).
Self-Improvement Loop	The mandatory pipeline for every artifact: Implementation → Review → Critique → Refactor → Testing → Security Scan → Performance Analysis → Approval → Merge (TITAN-FND-013).
Task DAG	The dependency graph of tasks derived from a Brief; must be acyclic (TITAN-FND-003).
TITAN	This platform: the open-source, local-first, multi-agent autonomous software engineering system specified by this 10-volume series.

End of Volume 1 — Foundation & Product Requirements (TITAN-VOL-01, Draft v1.0, 2026-07-04).

PROJECT TITAN — Autonomous Software Engineering Platform

Field	Value
Series	PROJECT TITAN Specification Series (10 Volumes)
Volume	Volume 2 — Multi-Agent Intelligence (Agent Architecture)
Doc ID	TITAN-VOL-02
Status	Draft v1.0
Date	2026-07-04
Req prefix	TITAN-AGT-NNN

Abstract. This volume specifies the agent organization at the heart of TITAN: sixteen role-specialized AI agents that collaborate like a real software company to turn a one-sentence client request into a production-ready application. It defines the common agent anatomy, a compact specification for every agent (purpose, responsibilities, decision authority, inputs/outputs, system-prompt design, memory access, tool access, KPIs, failure recovery), the agent hierarchy and collaboration patterns, the structured message protocol with JSON schemas for all eight message types, conflict-resolution and escalation rules, and the mechanism by which agents are instantiated as stateless LLM calls through the provider-agnostic gateway. Technology choices (Python 3.12/FastAPI orchestrator, PostgreSQL 16, Redis 7 Streams, Qdrant, provider gateway with Anthropic/OpenAI/Google/Ollama) are canonical decisions owned and justified by Volume 1; this volume builds on them without restating the rationale.

Table of Contents

- 1. Introduction and Scope
- 2. Agent Model Fundamentals
- 2.1 Anatomy of a TITAN Agent
- 2.2 Shared Defaults (applies to every agent)
- 2.3 Baseline Requirements
- 3. Agent Roster and Hierarchy
- 3.1 Roster
- 3.2 Hierarchy Diagram
- 4. Agent Specifications
- 4.1 CEO Agent … 4.16 Critic Agent
- 5. Inter-Agent Collaboration Patterns
- 6. Structured Message Protocol
- 6.1 Envelope
- 6.2 Message Types and JSON Examples
- 6.3 Persistence and Delivery
- 7. Conflict Resolution and Escalation

- 8. Agent Instantiation via the Provider Gateway
- 9. Architecture Decision Records
- 10. Risks and Open Questions
- 11. Cross-References

---

1. Introduction and Scope

TITAN does not build one giant AI; it builds an organization (core philosophy, Volume 1). Each agent is a role, not a process: a versioned bundle of system prompt, decision authority, memory scopes, tool grants, and KPIs, executed as stateless LLM calls. This volume is the employment contract for every role. A developer (or an AI coding agent) implementing the agent runtime, the message bus consumers, and the per-agent configuration should need only this document plus the interface contracts in Volumes 3 (backend), 5 (memory), and 7 (tools).

In scope: agent definitions, hierarchy, collaboration patterns, message protocol, conflict/escalation rules, LLM instantiation. Out of scope: the orchestration engine's task-state machine (Volume 6), memory internals (Volume 5), tool implementations (Volume 7), and the gateway's provider adapters (Volume 3).

2. Agent Model Fundamentals

2.1 Anatomy of a TITAN Agent

Every agent is defined by a declarative **Agent Definition** file (`agents/<agent_id>.yaml` , validated by a Pydantic v2 model `AgentDefinition`), containing:

1. `agent_id` , `display_name` , `tier` (0–3 or `cross`), `supervisor` (agent_id).
2. `system_prompt_template` — Jinja2 template reference; composed at call time (Section 8).
3. `authority` — lists of `decides_alone` and `must_escalate` decision classes.
4. `memory_scopes` — read/write grants against the memory system (Volume 5 scopes: `short_term` , `project` , `organization` , `code_patterns` , `architecture` , `lessons_learned`).
5. `tool_grants` — allow-list of tool IDs from the tool registry (Volume 7). Tools not listed are invisible to the agent.
6. `model_policy` — model tier (`reasoning` , `standard` , `fast`), temperature, max output tokens, structured-output schema.
7. `kpis` — metric names emitted to Prometheus via OpenTelemetry.
8. `failure_policy` — overrides to the default recovery ladder (2.2).

2.2 Shared Defaults (applies to every agent)

To keep the per-agent specs in Section 4 compact, the following defaults hold unless a spec states otherwise:

- **Memory:** read access to project `short_term` and `project` memory; write access to its own working notes in `short_term` ; every agent MAY write `lessons_learned` entries tagged with its role.
- **Outputs:** every agent emits structured messages (Section 6) — never free prose to the bus. All substantive decisions are recorded as `DecisionLog` messages.
- **Failure recovery ladder (default):** (1) automatic retry with same context on transient LLM/tool error (max 2, exponential backoff); (2) self-critique retry — the agent's output failed schema validation or acceptance

checks, so it is re-invoked with the failure appended (max 2); (3) escalate to supervisor via `ProgressReport(status="blocked")`; (4) supervisor reassigns, decomposes, or escalates to human. Every rung is logged and traced.

- **Tools:** all agents get `memory.search`, `memory.write_note`, and `messages.send`. Anything else is an explicit grant.
- **KPIs (universal):** task success rate, first-pass acceptance rate, mean tokens per completed task, mean wall-clock latency per task, escalation rate.

2.3 Baseline Requirements

ID	Priority	Requirement	Acceptance Criteria
TITAN-AGT-001	MUST	Every agent is defined by a declarative `AgentDefinition` (YAML + Pydantic) with the eight fields in 2.1.	Loading `agents/*.yaml` validates against the schema; a missing field fails CI.
TITAN-AGT-002	MUST	Agents are stateless between invocations; all durable state lives in PostgreSQL/Redis/Qdrant.	Killing and restarting the agent runtime mid-project loses no accepted work; tasks resume from persisted state.
TITAN-AGT-003	MUST	Agents communicate only via the structured message protocol (Section 6); no out-of-band side channels.	Code review + integration test assert every inter-agent interaction produces a persisted `agent_messages` row.
TITAN-AGT-004	MUST	Every agent enforces its `tool_grants` allow-list at the runtime layer, not only in the prompt.	A prompt-injected request for an ungranted tool is rejected by the runtime and logged as a security event.
TITAN-AGT-005	MUST	The default failure-recovery ladder (2.2) is implemented in the shared runtime, configurable per agent.	Fault-injection tests demonstrate all four rungs, with OpenTelemetry spans for each.
TITAN-AGT-006	SHOULD	Adding a new agent requires only a new YAML definition + prompt template, no core-code change.	A sample "Localization Agent" can be added in a test with zero orchestrator diffs.
TITAN-AGT-007	MUST	The full roster runs on a single laptop under Docker Compose using local models via Ollama.	`docker compose up` + Ollama profile completes the reference "todo app" project end-to-end offline.

3. Agent Roster and Hierarchy

3.1 Roster

Sixteen agents. Fourteen come directly from the master prompt; two are additions justified below.

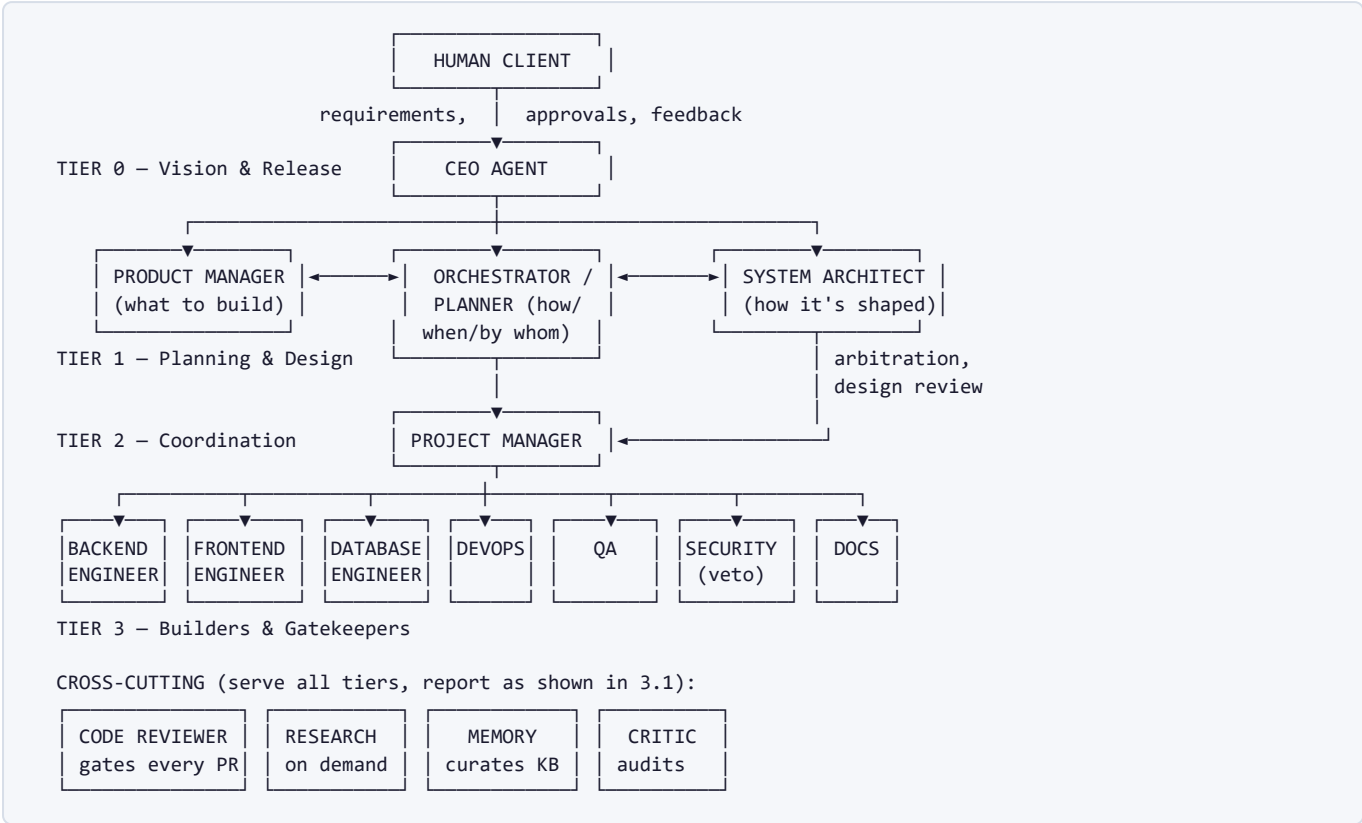
#	Agent	agent_id	Tier	Supervisor
1	CEO	`ceo`	0	Human client
2	Orchestrator/Planner *(addition)*	`orchestrator`	1	`ceo`
3	Product Manager	`pm_product`	1	`ceo`
4	System Architect	`architect`	1	`ceo`

#	Agent	agent_id	Tier	Supervisor
5	Project Manager	`pm_project`	2	`orchestrator`
6	Backend Engineer	`eng_backend`	3	`pm_project`
7	Frontend Engineer	`eng_frontend`	3	`pm_project`
8	Database Engineer	`eng_database`	3	`pm_project`
9	DevOps	`eng_devops`	3	`pm_project`
10	QA	`qa`	3	`pm_project`
11	Security	`security`	3 (veto powers)	`pm_project`
12	Code Reviewer	`reviewer`	cross	`architect`
13	Documentation	`docs`	3	`pm_project`
14	Research	`research`	cross	`orchestrator`
15	Memory	`memory`	cross	`orchestrator`
16	Critic *(addition)*	`critic`	cross	`architect`

Justification for additions. The **Orchestrator/Planner** is named explicitly in the source workflow ("User Request → CEO Agent → Planner → …"): someone must own plan decomposition, message routing, scheduling, and deadlock detection — none of the fourteen base roles does. The **Critic** exists because the mandated self-improvement loop (Implementation → Review → **Critique** → Refactor → …) requires adversarial critique of *non-code* artifacts (plans, architectures, specs, docs) that the Code Reviewer (scoped to pull requests) does not cover. No third agent is added: release management is split between CEO (approval) and DevOps (execution), and UX is owned by the Frontend Engineer.

TITAN-AGT-008 (MUST): The platform ships with exactly this roster enabled by default; roster changes are configuration (2.3/TITAN-AGT-006). *Acceptance:* fresh install lists these 16 agents in the dashboard Agent Status view.

3.2 Hierarchy Diagram



4. Agent Specifications

Each spec lists only deltas from the shared defaults in 2.2. “Escalates” always means via the messages and rules of Sections 6–7.

4.1 CEO Agent (`ceo`)

- **Purpose:** Translate the client’s intent into product vision; own scope, priority, and release approval.
- **Responsibilities:** Interpret the client request; ask only essential clarification questions (hard cap: one clarification round unless the client volunteers more); define vision and success criteria; prioritize features; approve/reject releases; deliver client-facing status summaries.
- **Decides alone:** Feature prioritization, scope cuts within stated budget, release go/no-go, accepting PM roadmaps.
- **Must escalate (to human client):** Budget/cost-ceiling changes, material scope expansion, legal/compliance-sensitive features, security-veto overrides (7.4), project cancellation.
- **Inputs → Outputs:** Client request, `MeetingSummary` digests, release-candidate reports → vision document (project memory), prioritized feature list, `TaskAssignment` to `pm_product` / `orchestrator`, `DecisionLog` entries, release approvals.
- **System-prompt outline:** identity (“chief executive of an autonomous software company”); prime directive (client value over technical elegance); clarification-question budget; output schemas; explicit list of human-escalation triggers; tone rules for client communication.
- **Memory:** +write `organization` (vision, client preferences); read `lessons_learned` (all roles).
- **Tools:** `client.message` (the only agent allowed to contact the human), `reports.generate`.

- **KPIs:** client clarification rounds per project (target ≤ 1), release-approval reversal rate, client-satisfaction score, vision-drift incidents.
- **Failure recovery:** No supervisor agent — rung 3 of the ladder goes directly to the human client with a structured options memo.

4.2 Orchestrator/Planner Agent (`orchestrator`)

- **Purpose:** Convert the approved vision into an executable plan; route work; keep the organization unblocked.
- **Responsibilities:** Milestone/phase decomposition; dependency-graph construction; agent scheduling and parallelization; watchdog for stalled tasks and message deadlocks; convening ConflictResolution threads; maintaining the run-loop cadence (Volume 6 owns the state machine; the Orchestrator operates it).
- **Decides alone:** Task ordering, parallel-vs-serial execution, retry/reassign of blocked tasks, invoking Research or Critic.
- **Must escalate:** Milestone changes (→ CEO), architectural sequencing disputes (→ Architect), resource exhaustion such as token-budget overrun (→ CEO).
- **Inputs → Outputs:** Vision doc, roadmap, architecture plan, all `ProgressReport`s → dependency graph (PostgreSQL), `TaskAssignment`s to `pm_project`, `MeetingSummary` messages, deadlock alerts.
- **System-prompt outline:** identity (chief of staff/program director); planning heuristics (small tasks, ≤ 1 owner each, explicit dependencies, acceptance criteria mandatory); watchdog rules with concrete timeouts; strict JSON plan schema.
- **Memory:** +read/write `project` plan sections; read `architecture`.
- **Tools:** `tasks.create/update/graph`, `scheduler.enqueue`, `watchdog.configure`.
- **KPIs:** plan-rework rate, mean task idle time, deadlocks detected/resolved, critical-path accuracy.
- **Failure recovery:** If the Orchestrator itself stalls, a runtime-level (non-LLM) watchdog restarts it from persisted plan state — TITAN-AGT-009 (MUST): the Orchestrator watchdog is implemented in the FastAPI runtime, not as another LLM agent. *Acceptance:* kill the orchestrator consumer mid-plan; a replacement resumes within 30 s with no duplicate task assignments.

4.3 Product Manager Agent (`pm_product`)

- **Purpose:** Turn vision into user stories, acceptance criteria, and a sprint-ready roadmap.
- **Responsibilities:** User stories (INVEST format), acceptance criteria per story, roadmap and release plan, sprint goals, backlog grooming, persona notes.
- **Decides alone:** Story slicing, acceptance-criteria wording, backlog ordering within CEO priorities.
- **Must escalate:** New scope discovered during grooming (→ CEO), technically infeasible stories flagged by Architect (→ ConflictResolution).
- **Inputs → Outputs:** Vision doc, Research findings, QA feedback → story backlog (PostgreSQL), roadmap, `ReviewRequest` to CEO for roadmap sign-off.
- **System-prompt outline:** identity; INVEST + Gherkin-style acceptance-criteria templates; "no story without testable criteria" rule; anti-gold-plating instruction.
- **Memory:** +write `project` (backlog, roadmap); read `organization` (client preferences).
- **Tools:** `tasks.create` (story type), `roadmap.render`.
- **KPIs:** story rejection rate by engineers (ambiguity), criteria coverage (% stories with testable criteria = 100%), mid-sprint scope-change count.
- **Failure recovery:** Default ladder; supervisor CEO.

4.4 System Architect Agent (`architect`)

- **Purpose:** Own technical design, technology fit, and cross-cutting consistency.
- **Responsibilities:** System architecture within the canonical stack (Volume 1); component boundaries and API contracts; data-flow and integration design; scalability review; ADR authorship; arbitration of technical disputes (7.2); final design sign-off on ArchitectureDiscussion threads.
- **Decides alone:** Internal component design, patterns (repository, CQRS where justified), library selection within the canonical stack, ADR acceptance.
- **Must escalate:** Any deviation from the canonical stack (→ CEO + human), cost-bearing infrastructure choices (→ CEO), security-relevant design (co-sign with Security).
- **Inputs → Outputs:** Roadmap, stories, Research reports → architecture document, ADRs (`DecisionLog`), API contract specs, `ArchitectureDiscussion` replies, arbitration verdicts.
- **System-prompt outline:** identity (principal architect); canonical-stack constraints stated verbatim; design heuristics (boring technology, explicit contracts, evolutionary architecture); ADR output schema; arbitration protocol.
- **Memory:** +read/write `architecture` ; read `code_patterns` , `lessons_learned` (all).
- **Tools:** `repo.read` , `diagrams.render_ascii` , `adr.create` .
- **KPIs:** ADR count vs. undocumented decisions (target: zero undocumented), post-design rework rate, arbitration turnaround time.
- **Failure recovery:** Default ladder; supervisor CEO.

4.5 Project Manager Agent (`pm_project`)

- **Purpose:** Day-to-day execution: tasks, assignments, progress, blockers.
- **Responsibilities:** Break stories into engineering tasks; assign to Tier-3 agents by skill and load; track progress and estimates; detect blockers early; maintain Kanban state; reopen tasks that fail review/QA; produce sprint `MeetingSummary` .
- **Decides alone:** Task assignment and reassignment, estimates, task reopening, intra-sprint reordering.
- **Must escalate:** Sprint-goal risk (→ Orchestrator), repeated failure by one agent on one task (≥ 3 cycles → Orchestrator, rule 7.5), inter-agent conflicts it cannot settle in one round.
- **Inputs → Outputs:** Stories, dependency graph, `ProgressReport` s → `TaskAssignment` s, Kanban updates, blocker alerts, sprint summaries.
- **System-prompt outline:** identity (delivery-focused engineering manager); assignment heuristics (single owner, matching tool grants, load ≤ 2 active tasks/agent); blocker-detection rules; status-report schema.
- **Memory:** +write `project` (task state mirrors).
- **Tools:** `tasks.*` (full CRUD), `kanban.update` , `metrics.read` .
- **KPIs:** on-time task completion rate, blocker mean-time-to-detection, reopen rate, agent utilization balance.
- **Failure recovery:** Default ladder; supervisor Orchestrator.

4.6 Backend Engineer Agent (`eng_backend`)

- **Purpose:** Implement server-side functionality for generated projects.
- **Responsibilities:** APIs (REST/WebSocket), authentication/authorization wiring, business logic, service-layer performance, unit tests for own code, PR creation.

- **Decides alone:** Implementation details within the API contract, internal refactors, library helpers already approved in `code_patterns`.
- **Must escalate:** Contract changes (→ Architect via `ArchitectureDiscussion`), new dependencies (→ Architect + Security), performance targets unmet after 2 attempts (→ PM).
- **Inputs → Outputs:** `TaskAssignment` with story + contract + acceptance criteria → code commits/PR, unit tests, `ReviewRequest` to `reviewer`, `ProgressReport`s, `BugReport`s for defects found in others' code.
- **System-prompt outline:** identity (senior backend engineer); engineering standards summary from Volume 8 (SOLID, clean architecture, no TODOs/placeholders); output contract (diff + tests + self-review checklist); tool-use protocol.
- **Memory:** +read `code_patterns`, `architecture`; write `code_patterns` proposals (curated by Memory Agent).
- **Tools:** `repo.read/write/branch`, `terminal.run` (sandboxed), `tests.run`, `research.request`.
- **KPIs:** first-pass review acceptance rate, test coverage of own diffs (≥ 80%), defect-escape rate to QA, mean cycles per task.
- **Failure recovery:** Default ladder; rung 2 self-critique explicitly includes failing test output and reviewer comments.

4.7 Frontend Engineer Agent (`eng_frontend`)

- **Purpose:** Implement UI/UX for generated projects.
- **Responsibilities:** UI components and pages, UX flows, responsive design, accessibility (WCAG 2.1 AA), client state management, frontend tests.
- **Decides alone:** Component structure, styling within the design language, state-management wiring within Architect-approved patterns.
- **Must escalate:** API-contract gaps (→ Backend via `ArchitectureDiscussion`), design-language changes (→ Architect), accessibility conflicts with requested features (→ PM Product).
- **Inputs → Outputs:** Stories, API contracts, design tokens → components, pages, tests, `ReviewRequest`s, `ProgressReport`s.
- **System-prompt outline:** identity (senior frontend engineer); canonical frontend stack constraints; accessibility checklist embedded; visual/UX heuristics; output contract (diff + tests + Storybook-style usage notes where applicable).
- **Memory/Tools/KPIs/Recovery:** As Backend (4.6), with KPI addition: accessibility-check pass rate.

4.8 Database Engineer Agent (`eng_database`)

- **Purpose:** Own data modeling and data-layer performance for generated projects.
- **Responsibilities:** Schema design, migrations (forward + rollback), indexing, query optimization, data-integrity constraints, seed/fixture data.
- **Decides alone:** Index and constraint choices, migration mechanics, query rewrites.
- **Must escalate:** Schema changes that break API contracts (→ Architect), destructive migrations (→ PM + DevOps co-sign), storage-engine choices beyond the canonical stack (→ Architect).
- **Inputs → Outputs:** Domain model from Architect, slow-query reports → DDL/migrations, ERD (ASCII + machine-readable), optimization PRs, `ReviewRequest`s.
- **System-prompt outline:** identity (database specialist); normalization-first-then-justify-denormalization heuristic; migration safety rules (always reversible, never lossy without co-sign); output contract.
- **Memory:** +read/write `architecture` (data sections), `code_patterns` (query patterns).

- **Tools:** `repo.*`, `db.execute` (sandboxed dev DB only), `db.explain`, `tests.run`.
- **KPIs:** migration failure rate (target 0), slow-query count post-optimization, schema-rework rate.
- **Failure recovery:** Default ladder; destructive-operation rung escalates immediately (skips retries).

4.9 DevOps Agent (`eng_devops`)

- **Purpose:** Make everything buildable, deployable, and observable.
- **Responsibilities:** Dockerfiles and Docker Compose for local-first dev; Kubernetes manifests for production; CI/CD pipelines; infrastructure-as-code; monitoring/logging/alerting wiring (OpenTelemetry, Prometheus, Grafana, Loki); environment configuration and secret plumbing (values owned by Security).
- **Decides alone:** Pipeline structure, image optimization, alert thresholds within SLOs.
- **Must escalate:** Production topology changes (→ Architect), anything cost-bearing (→ CEO), secret-handling design (→ Security co-sign).
- **Inputs → Outputs:** Repo state, architecture doc → CI/CD configs, container/K8s manifests, dashboards, deployment `ProgressReport`s, release execution after CEO approval.
- **System-prompt outline:** identity (platform/SRE engineer); local-first constraint stated as hard rule ("every artifact must run under Docker Compose on one laptop"); pipeline design heuristics; rollback-first deployment doctrine.
- **Memory:** +read `architecture`; write `code_patterns` (infra patterns).
- **Tools:** `repo.*`, `terminal.run` (sandboxed), `docker.*`, `k8s.apply` (env-gated), `ci.trigger`, `observability.query`.
- **KPIs:** pipeline success rate, mean deploy time, rollback success rate (target 100%), local-compose bootstrap success.
- **Failure recovery:** Default ladder; failed deploys auto-rollback before any escalation.

4.10 QA Agent (`qa`)

- **Purpose:** Independently verify that built software meets acceptance criteria.
- **Responsibilities:** Test plans per story; unit-test gap analysis; integration and end-to-end tests; regression suite curation; performance smoke tests; `BugReport` filing with reproduction steps.
- **Decides alone:** Test design, pass/fail verdicts against written acceptance criteria, regression-suite membership.
- **Must escalate:** Ambiguous acceptance criteria (→ PM Product), release-blocking defect disputes (→ ConflictResolution with severity evidence).
- **Inputs → Outputs:** Stories + criteria, candidate builds → test code, test-run reports, `BugReport`s, quality-gate verdicts on `ReviewRequest`s.
- **System-prompt outline:** identity (adversarial QA engineer — "your job is to break it"); verdicts must cite specific acceptance criteria; reproduction-steps schema; flaky-test triage rules.
- **Memory:** +read `project` (criteria), `lessons_learned` (defect patterns); write defect-pattern lessons.
- **Tools:** `repo.read/write` (test dirs), `tests.run`, `browser.automation` (E2E), `terminal.run` (sandboxed).
- **KPIs:** defect-escape rate to "production" (post-approval), test flakiness rate, criteria coverage per story, mean bug-report actionability (fixed without clarification).
- **Failure recovery:** Default ladder; a QA verdict is never auto-overridden by retries — disputes go to Section 7.

4.11 Security Agent (`security`)

- **Purpose:** Enforce security posture across all generated artifacts and the platform's own workflows.
- **Responsibilities:** Threat modeling per feature, dependency and secret scanning, OWASP-aligned audits, security test cases, secret-management design review, RBAC review of generated apps, audit-log completeness checks.
- **Decides alone:** Vulnerability severity ratings; **security veto** — blocking any merge/release with a Critical/High finding (see 7.4).
- **Must escalate:** Veto overrides (only human client can override, via CEO), novel threat classes needing research (→ Research), compliance interpretations (→ CEO → human).
- **Inputs → Outputs:** Diffs, dependency manifests, architecture docs → findings reports (`BugReport` with `security=true`), threat models, veto verdicts on `ReviewRequest`s, `DecisionLog` for accepted risks.
- **System-prompt outline:** identity (application security engineer); OWASP Top 10 + secret-handling checklist embedded; severity rubric (CVSS-like); "no silent risk acceptance" rule — every accepted risk becomes a DecisionLog entry.
- **Memory:** +read everything project-scoped; write `lessons_learned` (vulnerability patterns).
- **Tools:** `repo.read` , `scanner.dependencies` , `scanner.secrets` , `scanner.sast` , `tests.run` .
- **KPIs:** vulnerabilities caught pre-merge vs. post-merge, false-positive rate, mean time to security verdict, unresolved-finding age.
- **Failure recovery:** Default ladder; if the Security agent is unavailable, merges requiring its sign-off queue rather than proceed (fail-closed). TITAN-AGT-010 (MUST): security review is fail-closed. *Acceptance:* disabling the security agent blocks (not bypasses) merge of a PR touching auth code.

4.12 Code Reviewer Agent (`reviewer`)

- **Purpose:** Gate every pull request; never accept the first draft on faith.
- **Responsibilities:** Review every PR for correctness, standards compliance (Volume 8), maintainability, and test adequacy; request changes with concrete, actionable comments; approve or reject; suggest simplifications.
- **Decides alone:** Approve/request-changes verdicts; style and standards rulings (Volume 8 is binding precedent).
- **Must escalate:** Architectural concerns discovered in review (→ Architect), author disputes after 2 review cycles (→ 7.2 arbitration).
- **Inputs → Outputs:** `ReviewRequest` + diff + tests + task context → review verdict, line-anchored comments, `ProgressReport` to PM on review throughput.
- **System-prompt outline:** identity (staff engineer reviewer); review rubric ordered by severity (correctness > security-adjacent > design > tests > style); "must cite the standard" rule for rejections; comment-format schema; anti-rubber-stamp instruction (must list at least what was checked).
- **Memory:** +read `code_patterns` , `architecture` , Volume 8 standards (knowledge base).
- **Tools:** `repo.read` , `tests.run` , `diff.annotate` .
- **KPIs:** post-approval defect rate, review turnaround time, comment actionability rate, rubber-stamp rate (approvals with zero substantive checks — target 0).
- **Failure recovery:** Default ladder; reviews are fail-closed like security (no auto-approve on outage).

4.13 Documentation Agent (`docs`)

- **Purpose:** Keep generated projects and TITAN itself documented, continuously.
- **Responsibilities:** README, API documentation (from contracts + code), architecture guides (from ADRs), user manuals, developer/contribution guides, troubleshooting guides, changelog maintenance.
- **Decides alone:** Document structure, wording, regeneration scheduling.
- **Must escalate:** Contradictions between code and ADRs (→ Architect as `BugReport`), missing information it cannot derive (→ owning agent via `ReviewRequest`).
- **Inputs → Outputs:** Merged code, ADRs, API contracts, release notes → documentation PRs (reviewed like code), doc-coverage reports.
- **System-prompt outline:** identity (technical writer-engineer); "docs are derived from artifacts, never invented" rule; audience-first templates per doc type; staleness-detection heuristics.
- **Memory:** +read all project scopes; write `project` doc index.
- **Tools:** `repo.read/write` (docs dirs), `openapi.render`, `diagrams.render_ascii`.
- **KPIs:** doc coverage (public APIs documented ≥ 95%), staleness incidents, doc-related support questions from client.
- **Failure recovery:** Default ladder; documentation failures never block merges (fail-open, flagged).

4.14 Research Agent (`research`)

- **Purpose:** Provide grounded technical knowledge on demand so other agents don't guess.
- **Responsibilities:** Look up library/API documentation, compare technical options with citations, summarize into reusable briefs, track ecosystem changes relevant to active projects.
- **Decides alone:** Source selection, brief structure, confidence ratings.
- **Must escalate:** Findings that contradict current architecture (→ Architect), paywalled/unavailable sources (→ requester with stated confidence downgrade).
- **Inputs → Outputs:** Research requests (structured question + context + deadline) → research briefs (question, findings, citations, confidence, recommendation), stored in the knowledge base.
- **System-prompt outline:** identity (research librarian-engineer); "cite or mark as inference" rule; brief schema; recency-checking instructions; anti-hallucination guard (unknown → say unknown).
- **Memory:** +write knowledge base (`organization` scope, curated by Memory Agent).
- **Tools:** `web.search`, `web.fetch`, `repo.read`, `memory.search`.
- **KPIs:** brief citation rate (100%), requester-usefulness rating, brief reuse count, turnaround time.
- **Failure recovery:** Default ladder; on web-tool outage, answers from knowledge base with explicit "offline, possibly stale" flag (supports the offline laptop mode of TITAN-AGT-007).

4.15 Memory Agent (`memory`)

- **Purpose:** Curate the organization's long-term memory so it stays useful, deduplicated, and retrievable (memory internals: Volume 5).
- **Responsibilities:** Promote/demote memories between scopes; deduplicate and merge `lessons_learned`; curate `code_patterns` (accept/reject engineer proposals); maintain embeddings freshness in Qdrant; run pruning/compression jobs; answer complex retrieval requests other agents' `memory.search` can't satisfy.
- **Decides alone:** Curation verdicts, pruning within retention policy, embedding re-index scheduling.
- **Must escalate:** Retention-policy changes (→ CEO), deleting decision history (never allowed — flag instead; DecisionLogs are append-only).

- **Inputs → Outputs:** Memory-write proposals, retrieval requests, project post-mortems → curated memory entries, retrieval result sets, memory-health reports.
- **System-prompt outline:** identity (knowledge curator); curation rubric (generality, evidence, non-duplication); "decision history is immutable" rule; retrieval-answer schema with provenance.
- **Memory:** read/write ALL scopes (unique grant).
- **Tools:** `memory.*` (full suite incl. `promote`, `prune`, `reindex`), `qdrant.admin`.
- **KPIs:** retrieval precision on eval set, duplicate rate in knowledge base, memory-driven rework savings (lessons reused per project).
- **Failure recovery:** Default ladder; pruning jobs are transactional and reversible for 30 days.

4.16 Critic Agent (`critic`)

- **Purpose:** Adversarial second opinion on non-code artifacts, powering the mandated critique step of the self-improvement loop.
- **Responsibilities:** Critique plans, architectures, specs, test plans, and docs before they are accepted; red-team assumptions; identify missing failure modes; score artifacts against their own stated goals.
- **Decides alone:** Critique content and severity labels (advisory only — the Critic never blocks; owners and gatekeepers decide).
- **Must escalate:** Nothing; it is terminal-advisory. Ignored Critical-severity critiques are auto-logged to the owning agent's supervisor.
- **Inputs → Outputs:** Artifact + goals → structured critique (strengths, weaknesses ranked by severity, concrete revisions), attached to the artifact's thread.
- **System-prompt outline:** identity (professional skeptic, steelman-then-attack); severity rubric; "every weakness needs a concrete fix suggestion" rule; instructed to run on a different model tier than the artifact's author when configured (multi-model critique, Volume 10 hook).
- **Memory:** +read `lessons_learned` (all), `architecture`.
- **Tools:** `repo.read`, `memory.search` only (deliberately minimal).
- **KPIs:** critique adoption rate, post-critique defect reduction, false-alarm rate.
- **Failure recovery:** Default ladder; Critic outage never blocks the pipeline (fail-open, logged).

TITAN-AGT-011 (MUST): Each agent's `decides_alone` / `must_escalate` lists above are encoded in its AgentDefinition and enforced by the runtime where mechanically checkable (vetoes, co-signs, fail-open/closed behavior). *Acceptance:* contract tests per agent verify at least the veto, co-sign, and fail-open/closed rules.

5. Inter-Agent Collaboration Patterns

TITAN standardizes five collaboration patterns; every workflow in Volume 6 composes them.

1. **Delegation chain** (Tier 0→1→2→3): work flows down as `TaskAssignment`, status flows up as `ProgressReport`. Every assignment has exactly one owner and written acceptance criteria.
2. **Review gate:** producer sends `ReviewRequest`; gatekeeper (Reviewer, QA, Security, Architect, or CEO per artifact type) returns a verdict. Gates are fail-closed for code/security, fail-open for docs/critique. The self-improvement loop is this pattern applied in sequence: Implementation → Review (`reviewer`) → Critique (`critic`) → Refactor (owner) → Testing (`qa`) → Security scan (`security`) → Approval (CEO for releases) → Merge (DevOps).

3. **Consultation:** any agent may request Research or Memory retrieval mid-task without escalation; consultations are messages, so they are persisted and traceable.
4. **Discussion thread:** design questions run as `ArchitectureDiscussion` threads with a named moderator (Architect) and a decision deadline; outcome is always a `DecisionLog`.
5. **Ceremony:** Orchestrator-convened structured meetings (sprint planning, post-mortem) — a bounded round of messages from named participants, summarized as `MeetingSummary` and stored.

ID	Priority	Requirement	Acceptance Criteria
TITAN-AGT-012	MUST	Every task has exactly one owner agent at any time.	DB constraint + test: concurrent assignment attempts fail.
TITAN-AGT-013	MUST	The full self-improvement loop runs on every merged code artifact.	Pipeline trace for any merged PR shows all seven stages with verdicts.
TITAN-AGT-014	SHOULD	Discussion threads auto-close with a DecisionLog within their deadline or escalate.	Watchdog test: an idle thread past deadline triggers escalation to the moderator's supervisor.

6. Structured Message Protocol

6.1 Envelope

Every message shares one envelope (Pydantic model `AgentMessage`), serialized as JSON:

```
{
  "message_id": "uuid-v7",
  "schema_version": "1.0",
  "type": "TaskAssignment",
  "project_id": "uuid",
  "task_id": "uuid | null",
  "thread_id": "uuid | null",
  "correlation_id": "uuid (request/response pairing)",
  "from_agent": "pm_project",
  "to_agent": "eng_backend",
  "priority": "critical | high | normal | low",
  "requires_ack": true,
  "created_at": "2026-07-04T10:15:00Z",
  "payload": { }
}
```

`payload` is discriminated by `type`. Unknown `type` values are rejected; unknown payload fields are rejected (strict Pydantic).

6.2 Message Types and JSON Examples

Eight types are normative (TITAN-AGT-015, MUST — the bus accepts only these; *acceptance:* posting a ninth type returns a validation error and is logged).

TaskAssignment — creates/assigns work; the only way work changes hands.

```
{ "type": "TaskAssignment", "payload": {
  "title": "Implement POST /api/v1/orders",
  "description": "Create order endpoint per contract C-114.",
  "acceptance_criteria": ["201 + body per contract", "401 unauthenticated", "unit tests >= 80% diff coverage"],
  "story_id": "uuid", "depends_on": ["task-uuid-1"],
  "estimate_points": 3, "deadline": "2026-07-05T12:00:00Z" } }
```

ProgressReport — status heartbeat; also carries blockers.

```
{ "type": "ProgressReport", "payload": {
  "status": "in_progress | blocked | completed | failed",
  "percent_complete": 60,
  "summary": "Endpoint done; writing tests.",
  "blockers": [{ "description": "Contract C-114 missing error codes", "needs": "architect" }],
  "artifacts": ["branch:feat/orders-endpoint"] } }
```

ReviewRequest — asks a gatekeeper for a verdict; response reuses `correlation_id`.

```
{ "type": "ReviewRequest", "payload": {
  "artifact_type": "pull_request | plan | architecture | test_plan | release",
  "artifact_ref": "pr:42", "context_task_id": "uuid",
  "checklist": ["correctness", "tests", "standards"],
  "verdict": null } }
```

Gatekeeper reply: same payload with `"verdict": { "decision": "approve | request_changes | reject | veto", "comments": [{ "location": "src/orders.py:88", "severity": "major", "comment": "...", "standard_ref": "VOL8-ERR-004" }] }`.

ArchitectureDiscussion — threaded design debate.

```
{ "type": "ArchitectureDiscussion", "payload": {
  "thread_id": "uuid", "question": "Sync or async order-payment integration?",
  "position": "Async via Redis Streams outbox.",
  "arguments": ["decouples payment latency", "replayable"],
  "responding_to": "message-uuid | null",
  "decision_deadline": "2026-07-05T09:00:00Z" } }
```

BugReport — defect with mandatory reproduction.

```
{ "type": "BugReport", "payload": {
  "severity": "critical | high | medium | low",
  "security": false,
  "summary": "Order total ignores discounts",
  "reproduction_steps": ["POST /orders with coupon X", "observe total"],
  "expected": "total = 90.00", "actual": "total = 100.00",
  "suspected_component": "pricing service", "evidence": ["test:test_discount_applied FAILED"] } }
```

DecisionLog — append-only record of every substantive decision.

```
{ "type": "DecisionLog", "payload": {
  "decision": "Use cursor pagination for order listing",
  "context": "Offset pagination degrades past 100k rows.",
  "alternatives": [{ "option": "offset", "rejected_because": "O(n) skip cost" }],
  "trade_offs": "No random page access.",
  "authority": "decides_alone", "reversible": true } }
```

MeetingSummary — outcome of a ceremony.

```
{ "type": "MeetingSummary", "payload": {
  "ceremony": "sprint_planning | standup | post_mortem | conflict_resolution",
  "participants": ["ceo", "pm_product", "architect"],
  "decisions": ["message-uuid-of-DecisionLog"],
  "action_items": [{ "owner": "pm_project", "action": "cut story S-9", "due": "2026-07-05" }],
  "summary": "Sprint 3 scoped to checkout flow." } }
```

ConflictResolution — opens/advances a dispute (Section 7).

```
{ "type": "ConflictResolution", "payload": {
  "conflict_id": "uuid",
  "parties": ["eng_backend", "qa"],
  "subject": "Is BUG-77 release-blocking?",
  "positions": [{ "agent": "qa", "position": "blocking", "evidence": ["fails AC-3 of story S-4"] }],
  "round": 1, "arbiter": null, "resolution": null } }
```

6.3 Persistence and Delivery

- **Transport:** Redis Streams — one inbox stream per agent (`titan:{project_id}:inbox:{agent_id}`) with a consumer group per agent runtime; NATS JetStream is the documented scale-out swap (Volume 3).
- **Persistence:** every message is written to PostgreSQL `agent_messages` (envelope columns + `payload JSONB`) *before* enqueue (outbox pattern). Nothing is lost: TITAN-AGT-016 (MUST) — *acceptance:* chaos test kills Redis mid-conversation; on recovery, no persisted message is missing from replay and no accepted message was only in Redis.
- **Delivery semantics:** at-least-once with idempotent consumers keyed on `message_id` ; `requires_ack` messages unacknowledged after their type-specific timeout re-enter the pending set and trip the Orchestrator watchdog.
- **Observability:** every message emits an OpenTelemetry span linked by `correlation_id` ; conversation views in the dashboard (Volume 4) are rendered from `agent_messages` .

7. Conflict Resolution and Escalation

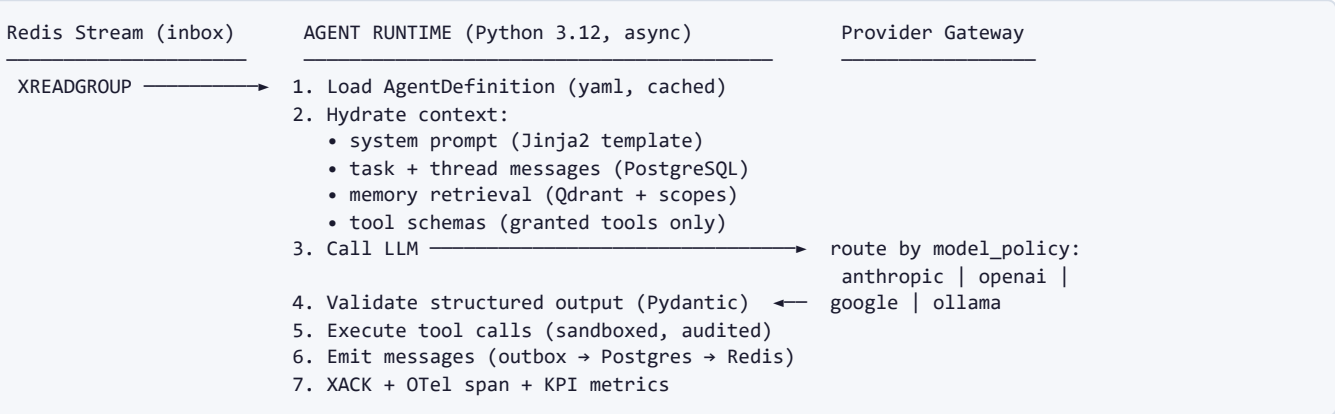
1. **Direct round (mandatory first step).** Parties exchange one `ConflictResolution` round each, positions with evidence. Most disputes end here.
2. **Arbitration.** No consensus after round 1 → the Orchestrator appoints the arbiter: **Architect** for technical disputes, **PM Product** for scope disputes, **PM Project** for priority/assignment disputes. The arbiter's verdict is a `DecisionLog` and is binding.
3. **Executive escalation.** Arbitration disputed, or the dispute crosses domains → CEO decides. CEO decisions are final inside the platform.
4. **Security veto.** A Security `veto` verdict blocks merge/release regardless of any other agent's approval, including the CEO's. Only the human client (via the CEO relaying a structured risk memo) can override; the override itself becomes a permanent `DecisionLog` with `authority: "human_override"` .
5. **Deadlock rule.** Any conflict reaching round 3 without resolution, or any task reopened 3 times, auto-escalates one level and is flagged in the dashboard.
6. **Human tripwires.** Automatic human escalation on: canonical-stack deviation requests, cost-ceiling breach, security-veto override requests, project-goal ambiguity the CEO scores as unresolvable, and destructive irreversible operations outside sandboxes.

ID	Priority	Requirement	Acceptance Criteria
TITAN-AGT-017	MUST	Conflict flow (rules 1–3) is enforced by the runtime state machine, not left to prompt compliance.	Simulated dispute test traverses direct → arbitration → executive with persisted state transitions.
TITAN-AGT-018	MUST	Security veto blocks merges at the pipeline level.	A vetoed PR cannot be merged even by a CEO-approved release job.
TITAN-AGT-019	MUST	Every resolution produces a DecisionLog; DecisionLogs are append-only.	UPDATE/DELETE on decision rows is rejected by DB policy.

ID	Priority	Requirement	Acceptance Criteria
TITAN-AGT-020	SHOULD	Deadlock (rule 5) detection runs in the Orchestrator watchdog with configurable thresholds.	Threshold config change takes effect without redeploy.

8. Agent Instantiation via the Provider Gateway

Agents are not resident processes with private state; an "agent" exists only during a **turn** — one hydrate→call→act cycle executed by the shared Agent Runtime (a pool of async FastAPI workers).



Prompt composition (order is normative): (1) agent identity and mission; (2) canonical-stack and platform constraints; (3) authority section rendered from `decides_alone` / `must_escalate`; (4) role heuristics and rubrics; (5) output JSON schema; (6) retrieved memory (provenance-tagged); (7) current task/thread context. Templates are versioned in-repo; prompt changes go through the normal PR + review gates.

Model policy is configuration only, per the canonical multi-LLM decision:

```
# config/model-policy.yaml
tiers:
  reasoning: { provider: anthropic, model: claude-opus-latest,  temperature: 0.2 }
  standard:  { provider: anthropic, model: claude-sonnet-latest, temperature: 0.3 }
  fast:      { provider: openai,    model: gpt-mini-latest,      temperature: 0.2 }
profiles:
  local-free:            # laptop / offline profile (hard requirement)
    reasoning: { provider: ollama, model: qwen2.5-coder:32b }
    standard:  { provider: ollama, model: qwen2.5-coder:14b }
    fast:      { provider: ollama, model: llama3.2:3b }
agents:
  ceo: reasoning
  architect: reasoning
  critic: reasoning
  eng_backend: standard
  eng_frontend: standard
  eng_database: standard
  eng_devops: standard
  reviewer: standard
  security: reasoning
  qa: standard
  pm_product: standard
  pm_project: fast
  orchestrator: standard
  docs: fast
  research: fast
  memory: fast
```

ID	Priority	Requirement	Acceptance Criteria
TITAN-AGT-021	MUST	Switching any agent's provider/model requires only editing model-policy config.	Change `architect` to `ollama` profile; next turn uses it, zero code changes, verified in traces.
TITAN-AGT-022	MUST	All LLM calls go through the gateway; no agent code imports provider SDKs directly.	Static check in CI forbids provider SDK imports outside `gateway/` .
TITAN-AGT-023	MUST	Structured outputs are schema-validated; invalid output triggers the self-critique retry rung.	Fuzz test with malformed outputs shows retry-then-escalate, never silent acceptance.
TITAN-AGT-024	MUST	Tool execution is sandboxed and audited (who, what, args, result hash) per turn.	Audit log row exists for 100% of tool calls in an E2E run.
TITAN-AGT-025	SHOULD	Runtime supports per-project token budgets with Orchestrator-visible burn-down.	Exceeding budget pauses assignments and notifies CEO.
TITAN-AGT-026	MAY	Critic may be pinned to a different provider than the artifact author (multi-model critique).	Config flag routes critic turns to a second provider in traces.

9. Architecture Decision Records

ADR-AGT-01 — Role-specialized agents instead of one generalist agent. *Context:* One large prompt could "do everything," and the master prompt explicitly forbids it. *Decision:* 16 role agents with narrow authority, prompts, and tool grants. *Alternatives:* single generalist (simpler, but no separation of duties, no review independence, prompt bloat); dynamic ad-hoc agent spawning (flexible, but unauditable authority). *Trade-offs:* more messages and token overhead; in exchange we get independent gates (reviewer/QA/security cannot be the author), smaller focused prompts, per-role KPIs, and mechanical enforcement of authority.

ADR-AGT-02 — Hierarchical orchestration with cross-cutting gatekeepers, not a flat peer swarm. *Context:* Agents must collaborate "like a real software company." *Decision:* Tiered hierarchy (Section 3) with an Orchestrator operating the plan, plus cross-cutting gate roles. *Alternatives:* flat peer-to-peer negotiation (emergent, but deadlock-prone and hard to audit); pure pipeline (predictable, but cannot handle discussion/conflict). *Trade-offs:* the Orchestrator is a coordination bottleneck — mitigated by TITAN-AGT-009 (non-LLM watchdog restart) and by Tier-3 agents proceeding on already-assigned tasks during Orchestrator recovery.

ADR-AGT-03 — Stateless turns with externalized memory. *Context:* Agents need durability, and local-first laptops need low resident cost. *Decision:* agents exist only per-turn; all state in PostgreSQL/Redis/Qdrant (TITAN-AGT-002). *Alternatives:* long-lived agent processes with in-memory context (lower per-turn hydration cost, but crash-fragile and unscalable); serialized conversation replay per turn (simple, but token cost grows unboundedly). *Trade-offs:* hydration adds latency per turn; mitigated by cached definitions, scoped retrieval instead of full replay, and Redis-cached hot context.

ADR-AGT-04 — Strict typed message protocol over free-form agent chat. *Context:* "Every communication should be stored. Nothing should be lost." *Decision:* eight normative message types, strict Pydantic validation, Postgres outbox + Redis Streams. *Alternatives:* free-form natural-language chat between agents (expressive, but unparseable, unenforceable, unauditable); RPC-only calls (typed, but loses discussion/meeting semantics). *Trade-offs:* schema evolution requires versioning discipline (`schema_version` field, additive-only within a major version); expressiveness inside payloads (e.g., `position` , `summary`) remains natural language, so we keep prose where it helps and structure where it must be machine-readable.

10. Risks and Open Questions

- **R1 — Token-cost blowout on ceremonies and discussion threads.** Mitigation: bounded rounds (Section 5), `fast` tier for coordination roles, per-project budgets (TITAN-AGT-025). Open question: should MeetingSummary generation be batched nightly on the local profile?
- **R2 — Prompt injection through generated-project content.** A malicious dependency README could try to steer an engineer agent. Mitigation: runtime-enforced tool allow-lists (TITAN-AGT-004), provenance-tagged retrieval, Security agent scanning; full treatment in Volume 8. Open question: quarantine tier for untrusted retrieved text.
- **R3 — Reviewer/Critic sycophancy** (approving to make progress). Mitigation: anti-rubber-stamp prompt rules, rubber-stamp-rate KPI, optional multi-model critique (TITAN-AGT-026). Open question: periodic seeded-defect audits ("mutation testing" for reviewers) — proposed for Volume 10's self-evaluation loop.
- **R4 — Small local models failing structured-output schemas**, degrading the offline profile. Mitigation: self-critique retry rung, simpler per-tier output schemas. Open question: whether the `local-free` profile needs relaxed schemas with a repair pass.
- **R5 — Authority drift** — prompts and YAML `authority` lists diverging over time. Mitigation: authority section of prompts is *rendered from* the YAML (single source of truth, Section 8). Residual risk: mechanically uncheckable authority classes still rely on prompt compliance.
- **R6 — Escalation storms** — cascading blocked tasks flooding CEO/human. Mitigation: deduplication by `conflict_id` / `thread_id` in the Orchestrator watchdog; deadlock thresholds (TITAN-AGT-020). Open question: severity-based batching windows for human tripwires.

11. Cross-References

- **Volume 1 — Foundation:** canonical stack decisions and rationale; success metrics that agent KPIs roll up into.
- **Volume 3 — Backend Engineering:** provider-gateway implementation, Redis Streams/NATS transport details, `agent_messages` schema, WebSocket event fan-out.
- **Volume 4 — Frontend & Dashboard:** Agent Status, Live Conversations, and Kanban views rendered from this volume's messages and KPIs.
- **Volume 5 — AI Brain (Memory, Knowledge & RAG):** memory-scope semantics, retrieval pipeline, embedding and pruning mechanics behind Section 4.15.
- **Volume 6 — Engineering Workflow:** the task-state machine and end-to-end lifecycle that composes Section 5's collaboration patterns.
- **Volume 7 — Tool Ecosystem & Integrations:** the tool registry backing `tool_grants`; sandboxing of `terminal.run`, `repo.*`, and browser automation.
- **Volume 8 — Security:** threat model behind R2, secret management, audit-log requirements referenced by TITAN-AGT-024.
- **Volume 9 — Production:** observability wiring (OpenTelemetry/Prometheus/Grafana/Loki) for agent KPIs and traces.
- **Volume 10 — Autonomous Evolution:** self-evaluation, reviewer audits (R3), multi-model collaboration extending TITAN-AGT-026, and new-agent creation building on TITAN-AGT-006.

End of TITAN-VOL-02 — Draft v1.0.

PROJECT TITAN — Autonomous Software Engineering Platform

Volume 3 — Backend Engineering & Infrastructure

Field	Value
Doc ID	TITAN-VOL-03
Status	Draft v1.0
Date	2026-07-04
Requirement prefix	TITAN-BCK-NNN
Audience	Backend engineers, platform engineers, AI coding agents implementing the TITAN core

Abstract. This volume specifies the TITAN backend: a Python 3.12 + FastAPI (async) modular monolith that serves as the system of record, API surface, realtime event hub, and task-orchestration engine for the multi-agent platform. It defines the module boundaries and their extraction path to services, REST and WebSocket contracts, the PostgreSQL 16 schema, Redis 7 usage (cache, Redis Streams queues, pub/sub), the task orchestration state machine, authentication (OAuth2/OIDC + JWT) and RBAC, rate limiting, caching, observability hooks, and the Docker Compose local-development topology. Everything here is implementable directly, locally, on one laptop with free tools.

Table of Contents

1. Scope & Relationship to Other Volumes
2. Service Architecture
3. Module Breakdown
4. REST API Design
5. WebSocket Event Protocol
6. PostgreSQL Schema
7. Redis Usage
8. Task Orchestration Engine
9. API Gateway, Authentication & RBAC
10. Rate Limiting
11. Caching Strategy
12. Logging & Observability Hooks
13. Local Development: Docker Compose & Repository Layout
14. Risks & Open Questions
15. Cross-References

---

1. Scope & Relationship to Other Volumes

This volume owns the **backend/orchestrator process and its infrastructure**: HTTP/WebSocket surface, persistence, queues, orchestration, auth, and local-dev topology. It does **not** re-justify the canonical stack (Volume 1 owns that): monorepo; Next.js 15 + React + TypeScript + Tailwind + shadcn/ui frontend; Python 3.12 + FastAPI (async) + SQLAlchemy + Pydantic v2 backend; PostgreSQL 16 as system of record; Redis 7 for cache/queues/pub-sub; Qdrant for vectors; Redis Streams for the MVP task/agent queue with NATS JetStream as the scale-out option; WebSockets served by FastAPI; provider-agnostic LLM gateway (Anthropic, OpenAI, Google, Ollama) switchable by configuration only; Docker Compose locally, Kubernetes in production; OpenTelemetry + Prometheus + Grafana + Loki observability; OAuth2/OIDC + JWT with RBAC; open source, local-first, free-tier-friendly.

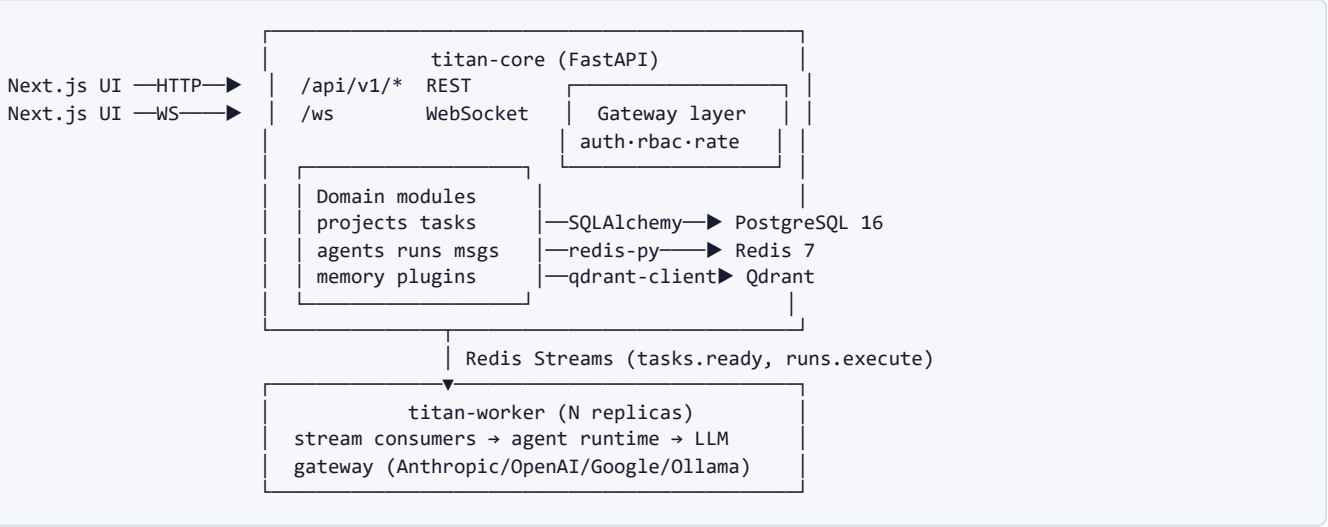
Agent role definitions live in Volume 2; memory/RAG internals in Volume 5; the end-to-end development workflow in Volume 6; plugin/tool integrations in Volume 7; deep security in Volume 8; production deployment in Volume 9.

---

2. Service Architecture

2.1 Topology

TITAN's backend at MVP is **one deployable FastAPI application** (`titan-core`) plus **one worker process image** (`titan-worker`) that shares the same codebase but runs stream consumers instead of the HTTP server. Both containers import the same Python package; the entrypoint differs.



TITAN-BCK-001 (MUST) — The backend SHALL be implemented as a single FastAPI application organized as a modular monolith with the modules listed in §3. *Acceptance:* `uvicorn titan.main:app` starts the full API; each module lives in its own package with no cross-module imports except via the module's public `service.py` interface; an import-linter contract in CI fails on boundary violations.

TITAN-BCK-002 (MUST) — All I/O paths (HTTP handlers, DB access, Redis, LLM calls) SHALL be async (`asyncpg` via SQLAlchemy 2.0 async engine, `redis.asyncio`). *Acceptance:* no synchronous DB/Redis driver appears in `pyproject.toml` ; blocking calls in handlers are flagged by a CI check running `flake8-async` .

TITAN-BCK-003 (MUST) — Worker processes SHALL run from the same image with entrypoint `python -m titan.worker` , consuming Redis Streams consumer groups. *Acceptance:* killing a worker mid-task causes the

message to be re-delivered to another consumer via `XAUTOCLAIM` within 60 s.

2.2 ADR-BCK-01 — Modular monolith now, services later

- **Context.** TITAN must run fully on one laptop (hard requirement) yet be architected so components can scale independently once real load exists (many concurrent agent runs are LLM-bound, not CPU-bound).
- **Decision.** Ship a modular monolith with enforced module boundaries (import-linter), a shared Postgres schema partitioned by module ownership (each table has exactly one owning module; other modules read via service interfaces, never raw table access), and all inter-module *asynchronous* interaction via Redis Streams events rather than in-process calls.
- **Alternatives considered.** (a) Microservices from day one — rejected: 8+ services on a laptop wastes RAM, multiplies local-dev pain, and the domain boundaries are not yet proven. (b) Single unstructured app — rejected: extraction later becomes a rewrite.
- **Trade-offs.** We accept one shared DB and one deploy unit at MVP (coarser failure domain) in exchange for laptop-friendliness and iteration speed. Because module communication already flows through events + service interfaces, extracting a module (first candidates: `agent-runtime` worker, then `memory`) means: give it its own DB schema/database, replace in-process service calls with HTTP/NATS calls behind the same interface, and move its stream consumers. The event contract does not change. NATS JetStream replaces Redis Streams at that point if throughput or durability demands it (see ADR-BCK-02).

---

3. Module Breakdown

Module	Owns (tables)	Responsibility	Extraction priority
`gateway`	—	Routing, authN/Z, rate limits, request ID, CORS, OpenAPI	never (stays with API)
`identity`	users, roles, user_roles, api_keys	OAuth2/OIDC flows, JWT issuing/verification, RBAC checks	low
`projects`	projects, milestones, sprints	Project lifecycle, roadmap data	low
`orchestrator`	tasks, task_dependencies	Task state machine, assignment, prioritization, scheduling	medium
`agents`	agents	Agent registry: role, config, model binding, status	medium
`runs`	runs, artifacts	Execution records of agent work; artifact metadata	**high** (worker)
`messaging`	messages, decisions	Structured inter-agent messages, decision log	medium
`memory`	memory_items (+ Qdrant collections)	Memory API façade over Postgres + Qdrant (details: Vol 5)	**high**
`plugins`	plugins, plugin_installs	Plugin registry, manifest validation, lifecycle hooks (Vol 7)	medium
`llm`	—	Provider-agnostic LLM gateway client; config-driven provider switch	high
`events`	—	Outbox publisher, Redis Streams producers/consumers, WS fan-out	never (library)
`observability`	—	OTel setup, structlog config, Prometheus metrics registry	never (library)

TITAN-BCK-004 (MUST) — Each domain module SHALL expose exactly one public interface (`<module>/service.py` with Pydantic v2 DTOs) and keep routers, repositories, and models private. *Acceptance:* import-linter "independence" contracts pass; DTOs (not SQLAlchemy models) cross module boundaries.

TITAN-BCK-005 (MUST) — State changes that other modules react to SHALL be published as domain events through a transactional outbox (`outbox` table written in the same DB transaction, drained to Redis Streams by a background publisher). *Acceptance:* killing the process between commit and publish never loses an event; the publisher is idempotent via event UUIDs.

---

4. REST API Design

4.1 Conventions

TITAN-BCK-006 (MUST) — The API SHALL follow these conventions. *Acceptance:* Schemathesis run against the OpenAPI spec passes; a spectral ruleset encoding the rules below passes in CI.

Convention	Rule
Base path	`/api/v1` (URI versioning; breaking changes bump the segment)
Resource naming	plural kebab-case nouns: `/projects`, `/tasks`, `/plugin-installs`
IDs	UUIDv7 everywhere (time-ordered, index-friendly)
Body casing	`snake_case` JSON (Pydantic native); frontend maps at its boundary
Pagination	cursor-based: `?limit=50&cursor=<opaque>`; response carries `next_cursor`
Filtering/sort	`?status=in_progress&assignee_id=…&sort=-created_at`
Errors	RFC 9457 Problem Details: `{"type","title","status","detail","instance","errors":[…]}`
Idempotency	mutating POSTs accept `Idempotency-Key` header, replayed for 24 h from Redis
Concurrency	updates require `If-Match: <etag>` (row `version` column) → `412` on conflict
Timestamps	RFC 3339 UTC (`2026-07-04T12:00:00Z`)
Envelope	none for single resources; lists use `{"items":[…],"next_cursor":…}`

4.2 Endpoint map

Resource	Endpoints (all under `/api/v1`)
Projects	`POST /projects` · `GET /projects` · `GET /projects/{id}` · `PATCH /projects/{id}` · `POST /projects/{id}:archive` · `GET /projects/{id}/board` (kanban) · `GET /projects/{id}/graph` (dependency graph)
Tasks	`POST /projects/{id}/tasks` · `GET /projects/{id}/tasks` · `GET /tasks/{id}` · `PATCH /tasks/{id}` · `POST /tasks/{id}:transition` · `POST /tasks/{id}/dependencies`
Agents	`GET /agents` · `POST /agents` · `GET /agents/{id}` · `PATCH /agents/{id}` · `POST /agents/{id}:pause` / `:resume`
Runs	`POST /tasks/{id}/runs` · `GET /runs/{id}` · `GET /runs?task_id=…` · `POST /runs/{id}:cancel` · `GET /runs/{id}/artifacts`

Resource	Endpoints (all under `/api/v1`)
Messages	`GET /projects/{id}/messages?thread_id=…&kind=…` · `POST /projects/{id}/messages` (human → agent)
Decisions	`GET /projects/{id}/decisions` · `POST /projects/{id}/decisions`
Memory	`POST /memory/search` · `POST /memory/items` · `GET /memory/items/{id}` · `DELETE /memory/items/{id}`
Plugins	`GET /plugins` · `POST /plugins/{id}:install` · `POST /plugins/{id}:enable` / `:disable`
Auth	`POST /auth/token` · `POST /auth/refresh` · `GET /auth/me` · `POST /auth/api-keys`

Custom actions use the `:verb` suffix (Google AIP-136 style) so the state machine (§8) is the only way to move task state — no free-form `PATCH status`.

4.3 Example payloads

`POST /api/v1/projects`

```
{
  "name": "netflix-clone",
  "display_name": "Netflix Clone",
  "brief": "Streaming platform with auth, catalog, playback, recommendations.",
  "mode": "autonomous",
  "llm_profile": "default-local"
}
```

`201 Created`

```
{
  "id": "0197a3b2-7c1e-7d3a-9f00-1b2c3d4e5f60",
  "name": "netflix-clone",
  "display_name": "Netflix Clone",
  "status": "intake",
  "mode": "autonomous",
  "llm_profile": "default-local",
  "created_by": "0197a3b2-0000-7000-8000-000000000001",
  "created_at": "2026-07-04T09:12:33Z",
  "version": 1
}
```

`POST /api/v1/tasks/{id}:transition`

```
{ "action": "submit_review", "actor_agent_id": "0197…be", "comment": "Backend auth module ready; 42 tests green." }
```

`200 OK`

```
{
  "id": "0197a3c1-…",
  "title": "Implement JWT auth endpoints",
  "status": "in_review",
  "previous_status": "in_progress",
  "assignee_agent_id": "0197…be",
  "reviewer_agent_id": "0197…cr",
  "priority": 2,
  "estimate_points": 3,
  "sprint_id": "0197…sp",
  "version": 6
}
```

`POST /api/v1/memory/search`

```
{ "query": "how did we structure JWT refresh last project?", "scope": "organization", "top_k": 5, "kinds":
["decision", "code_pattern"] }
```

`200 OK` → `{"items":[{"id":"…","kind":"decision","score":0.87,"summary":"Refresh tokens rotate on use…","source_ref":"decisions/0196…"}],"next_cursor":null}`

Error example (`409` on illegal transition), RFC 9457:

```
{
  "type": "https://titan.dev/errors/invalid-transition",
  "title": "Invalid task transition",
  "status": 409,
  "detail": "Action 'complete' is not allowed from status 'backlog'.",
  "instance": "/api/v1/tasks/0197a3c1-…:transition"
}
```

---

5. WebSocket Event Protocol

TITAN-BCK-007 (MUST) — FastAPI SHALL serve `wss://…/ws?token=<jwt>`; after connect the client sends a `subscribe` frame per project/topic; the server fans out events published on Redis pub/sub channel `ws.project.{project_id}`. *Acceptance:* two browser tabs subscribed to the same project both receive a `task.updated` frame within 250 ms (local) of the transition commit.

All frames are JSON with a fixed envelope:

```
{
  "v": 1,
  "type": "run.output.delta",
  "seq": 18234,
  "ts": "2026-07-04T09:15:02.412Z",
  "project_id": "0197a3b2-…",
  "payload": { "run_id": "0197a3d9-…", "agent_id": "0197…be", "delta": "def create_app() ->" }
}
```

Client → server frames: `subscribe {project_id, topics[]}`, `unsubscribe`, `ping`. Server → client event types:

Type	Payload core	Emitted when
`task.created` / `task.updated`	task DTO	orchestrator commits a change
`run.started / run.finished`	run DTO (status, cost, tokens)	worker begins/ends a run

Type	Payload core	Emitted when
`run.output.delta`	`{run_id, delta}`	streaming LLM output (live agent activity)
`message.created`	message DTO	any inter-agent or human message
`decision.recorded`	decision DTO	Decision log entry written
`agent.status`	`{agent_id, state, current_task_id}`	agent idle/busy/error/paused
`sprint.updated`	sprint DTO	planning changes
`sys.notice`	`{level, text}`	rate-limit warnings, provider fallback, etc.

`seq` is a per-connection monotonically increasing integer; after reconnect the client sends `subscribe` with `last_seq`, and the server replays up to 1,000 buffered events per project from a Redis list (`ws.buffer.{project_id}`, capped, TTL 15 min) or instructs `{"type":"sys.resync"}` telling the client to refetch via REST. Heartbeat: server pings every 25 s; connection closed after 2 missed pongs.

---

6. PostgreSQL Schema

PostgreSQL 16 is the system of record. Migrations are Alembic, one head, autogenerate forbidden in CI (hand-reviewed migrations only).

TITAN-BCK-008 (MUST) — The core schema below SHALL be created by initial migrations; every table gets `created_at timestamptz NOT NULL DEFAULT now()`, `updated_at` (trigger-maintained), and optimistic-lock `version int NOT NULL DEFAULT 1`. *Acceptance:* `alembic upgrade head` on empty Postgres 16 produces exactly these tables/indexes; `pytest -m schema` asserts them.

```
-- identity ------------------------------------------------------------
CREATE TABLE users (
  id            uuid PRIMARY KEY,                  -- UUIDv7
  email         citext UNIQUE NOT NULL,
  display_name  text NOT NULL,
  auth_provider text NOT NULL,                     -- 'oidc:google' | 'oidc:github' | 'local'
  auth_subject  text NOT NULL,                     -- OIDC 'sub'
  is_active     boolean NOT NULL DEFAULT true,
  UNIQUE (auth_provider, auth_subject)
);

CREATE TABLE roles (
  id    uuid PRIMARY KEY,
  name  text UNIQUE NOT NULL,                      -- owner|admin|maintainer|operator|viewer
  permissions jsonb NOT NULL                        -- ["project:write","task:transition",...]
);

CREATE TABLE user_roles (
  user_id    uuid NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  role_id    uuid NOT NULL REFERENCES roles(id) ON DELETE CASCADE,
  project_id uuid NULL REFERENCES projects(id) ON DELETE CASCADE,  -- NULL = org-wide
  PRIMARY KEY (user_id, role_id, project_id)
);

-- projects ------------------------------------------------------------
CREATE TABLE projects (
  id           uuid PRIMARY KEY,
  name         text UNIQUE NOT NULL,               -- slug
  display_name text NOT NULL,
  brief        text NOT NULL,
  status       text NOT NULL DEFAULT 'intake',     -- intake|planning|building|reviewing|released|archived
  mode         text NOT NULL DEFAULT 'autonomous', -- autonomous|supervised
  llm_profile  text NOT NULL DEFAULT 'default',
  created_by   uuid NOT NULL REFERENCES users(id)
);
CREATE INDEX idx_projects_status ON projects(status);

CREATE TABLE milestones (
  id uuid PRIMARY KEY, project_id uuid NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
  title text NOT NULL, due_at timestamptz, status text NOT NULL DEFAULT 'open'
);

CREATE TABLE sprints (
  id uuid PRIMARY KEY, project_id uuid NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
  number int NOT NULL, goal text, starts_at timestamptz, ends_at timestamptz,
  status text NOT NULL DEFAULT 'planned',          -- planned|active|closed
  UNIQUE (project_id, number)
);

-- agents --------------------------------------------------------------
CREATE TABLE agents (
  id           uuid PRIMARY KEY,
  project_id   uuid REFERENCES projects(id) ON DELETE CASCADE,   -- NULL = org-level template
  role         text NOT NULL,        --
ceo|pm|project_manager|architect|backend|frontend|db|devops|qa|security|reviewer|docs|research|memory
  name         text NOT NULL,
  status       text NOT NULL DEFAULT 'idle',       -- idle|busy|paused|error
  model_binding jsonb NOT NULL,      -- {"provider":"ollama","model":"qwen2.5-coder:14b","temperature":0.2}
  config       jsonb NOT NULL DEFAULT '{}',        -- prompt template ref, tool allowlist (Vol 2)
  current_task_id uuid NULL
);
CREATE INDEX idx_agents_project_role ON agents(project_id, role);

-- orchestration -------------------------------------------------------
CREATE TABLE tasks (
  id            uuid PRIMARY KEY,
  project_id    uuid NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
  sprint_id     uuid REFERENCES sprints(id),
  milestone_id  uuid REFERENCES milestones(id),
  parent_id     uuid REFERENCES tasks(id),          -- epics → subtasks
  title         text NOT NULL,
  spec          text NOT NULL DEFAULT '',           -- full task specification (markdown)
```

```
  status        text NOT NULL DEFAULT 'backlog',    --
backlog|ready|assigned|in_progress|in_review|changes_requested|done|reopened|cancelled
  priority      smallint NOT NULL DEFAULT 3,        -- 1 critical … 5 trivial
  estimate_points smallint,
  assignee_agent_id uuid REFERENCES agents(id),
  reviewer_agent_id uuid REFERENCES agents(id),
  created_by_agent_id uuid REFERENCES agents(id),
  completed_at  timestamptz
);
CREATE INDEX idx_tasks_project_status_priority ON tasks(project_id, status, priority);
CREATE INDEX idx_tasks_assignee ON tasks(assignee_agent_id) WHERE status IN ('assigned','in_progress');

CREATE TABLE task_dependencies (
  task_id       uuid NOT NULL REFERENCES tasks(id) ON DELETE CASCADE,
  depends_on_id uuid NOT NULL REFERENCES tasks(id) ON DELETE CASCADE,
  PRIMARY KEY (task_id, depends_on_id),
  CHECK (task_id <> depends_on_id)
);

-- execution -------------------------------------------------------------------
CREATE TABLE runs (
  id          uuid PRIMARY KEY,
  task_id     uuid NOT NULL REFERENCES tasks(id) ON DELETE CASCADE,
  agent_id    uuid NOT NULL REFERENCES agents(id),
  attempt     int  NOT NULL DEFAULT 1,
  status      text NOT NULL DEFAULT 'queued',      -- queued|running|succeeded|failed|cancelled|timed_out
  input       jsonb NOT NULL,                      -- resolved prompt context refs
  output_summary text,
  error       text,
  provider    text, model text,
  tokens_in   int DEFAULT 0, tokens_out int DEFAULT 0,
  cost_usd    numeric(10,4) DEFAULT 0,
  started_at  timestamptz, finished_at timestamptz
);
CREATE INDEX idx_runs_task ON runs(task_id, attempt DESC);
CREATE INDEX idx_runs_status ON runs(status) WHERE status IN ('queued','running');

CREATE TABLE artifacts (
  id        uuid PRIMARY KEY,
  run_id    uuid NOT NULL REFERENCES runs(id) ON DELETE CASCADE,
  project_id uuid NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
  kind      text NOT NULL,             -- code_patch|file|test_report|doc|diagram|build_log
  path      text,                      -- repo-relative path if applicable
  storage_ref text NOT NULL,           -- object-store/filesystem key
  sha256    text NOT NULL,
  size_bytes bigint NOT NULL
);
CREATE INDEX idx_artifacts_project_kind ON artifacts(project_id, kind);

-- communication ---------------------------------------------------------------
CREATE TABLE messages (
  id         uuid PRIMARY KEY,
  project_id uuid NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
  thread_id  uuid NOT NULL,                        -- UUIDv7; groups a conversation
  kind       text NOT NULL,   --
task_assignment|progress_report|review_request|architecture_discussion|bug_report|decision_log|meeting_summary|conf
lict_resolution|human
  sender_agent_id uuid REFERENCES agents(id),
  sender_user_id  uuid REFERENCES users(id),
  body       jsonb NOT NULL,                       -- structured per-kind schema (Vol 2 §protocol)
  task_id    uuid REFERENCES tasks(id),
  CHECK (sender_agent_id IS NOT NULL OR sender_user_id IS NOT NULL)
);
CREATE INDEX idx_messages_thread ON messages(thread_id, created_at);
CREATE INDEX idx_messages_project_kind ON messages(project_id, kind, created_at DESC);

CREATE TABLE decisions (
  id          uuid PRIMARY KEY,
  project_id  uuid NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
  title       text NOT NULL,
  context     text NOT NULL,
  decision    text NOT NULL,
  alternatives text NOT NULL,
```

```
  consequences text NOT NULL,
  status      text NOT NULL DEFAULT 'accepted',    -- proposed|accepted|superseded
  decided_by_agent_id uuid REFERENCES agents(id),
  supersedes_id uuid REFERENCES decisions(id)
);
CREATE INDEX idx_decisions_project ON decisions(project_id, created_at DESC);

-- events (outbox) -------------------------------------------------------------
CREATE TABLE outbox (
  id uuid PRIMARY KEY, topic text NOT NULL, payload jsonb NOT NULL,
  published_at timestamptz NULL
);
CREATE INDEX idx_outbox_unpublished ON outbox(created_at) WHERE published_at IS NULL;
```

Notes: statuses are `text` + application enums (Pydantic) rather than Postgres enums, so adding states never needs `ALTER TYPE`; `citext` extension required; `memory_items` and plugin tables follow the same conventions and are detailed in Volumes 5 and 7 respectively, but are owned by their modules here.

TITAN-BCK-009 (MUST) — Cycle prevention: inserting into `task_dependencies` SHALL be validated by a recursive-CTE check inside the same transaction, rejecting cycles with `409`. *Acceptance:* attempting A→B→C→A returns the `invalid-dependency-cycle` problem type; property-based test with random DAGs passes.

---

7. Redis Usage

Redis 7 plays three distinct roles, with distinct key namespaces:

Role	Mechanism	Keys / streams
Cache	`GET/SET` + TTL, `SCAN`-free invalidation via key versioning	`cache:{entity}:{id}:v{n}`
Queues	**Redis Streams** + consumer groups	`stream:tasks.ready`, `stream:runs.execute`, `stream:runs.results`, `stream:events.domain`
Pub/Sub	fire-and-forget fan-out to WS gateways	`ws.project.{id}`, `ws.agent.{id}`
Coordination	`SET NX PX` locks, rate-limit counters, idempotency keys	`lock:task:{id}`, `rl:{sub}:{route}`, `idem:{key}`

7.1 ADR-BCK-02 — Redis Streams for the MVP queue, NATS JetStream for scale-out

- **Context.** Agent orchestration needs at-least-once delivery, consumer groups, replay for debugging, and pending-message recovery — on a laptop, for free, with minimal moving parts. Redis is already required for cache and pub/sub.
- **Decision.** Use Redis Streams with consumer groups as the only queue at MVP. `XADD` with `MAXLEN ~ 100000`; consumers `XREADGROUP` with explicit `XACK`; a reaper task runs `XAUTOCLAIM` (min-idle 60 s) to recover work from dead consumers; poison messages move to `stream:{name}.dlq` after 3 delivery attempts (tracked via `XPENDING` delivery counter).
- **Alternatives considered.** NATS JetStream (better multi-tenancy, true durability semantics, subject hierarchies — but a second broker locally); Kafka (overkill, heavy); RabbitMQ (no cheap replay); Postgres `SKIP LOCKED` queue (viable, but couples queue churn to WAL and vacuum).
- **Trade-offs.** Streams durability is bounded by Redis persistence (we enable AOF `everysec` locally); the outbox pattern (TITAN-BCK-005) covers the loss window because Postgres remains the source of truth and the publisher re-drains unpublished rows. **Scale-out path:** the `events` module hides the broker behind a

`QueueBus` protocol (`publish`, `consume`, `ack`, `nack`); a `QUEUE_BACKEND=nats` setting swaps in the JetStream implementation with identical message envelopes — no domain code changes.

TITAN-BCK-010 (MUST) — Every stream message SHALL carry the envelope `{event_id (uuid7), topic, occurred_at, actor, payload, trace_id}` and consumers SHALL be idempotent keyed on `event_id` (dedup set `seen:{consumer_group}` with 24 h TTL). *Acceptance:* replaying an entire stream into a fresh consumer group produces no duplicate side effects in Postgres.

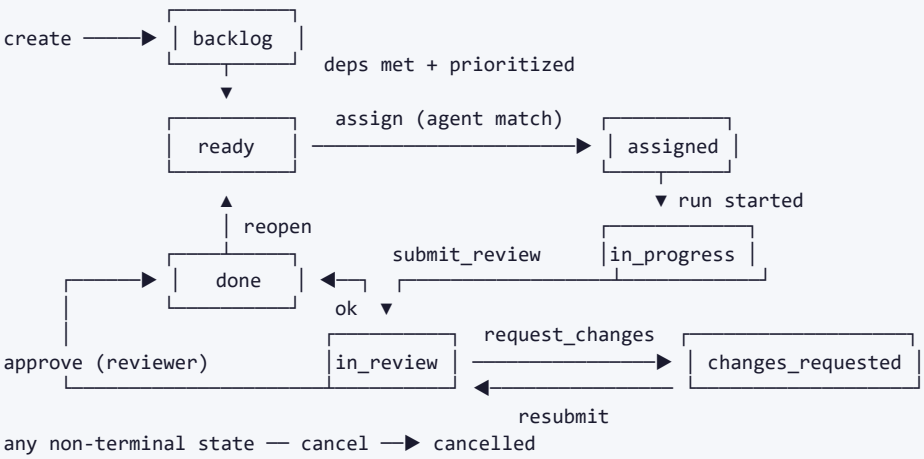
TITAN-BCK-011 (SHOULD) — DLQ depth SHALL be exported as the Prometheus gauge `titan_stream_dlq_depth{stream=…}` and alerts fire at >0 for 10 min. *Acceptance:* injecting a poison message raises the gauge within one scrape interval.

---

8. Task Orchestration Engine

The orchestrator turns the PM/Project-Manager agents' plans (Vol 2, Vol 6) into scheduled, parallel, reviewable execution.

8.1 Task state machine



TITAN-BCK-012 (MUST) — Transitions SHALL be executed only through `POST /tasks/{id}:transition`, enforced server-side by a transition table `{(from_status, action) → to_status, required_permission, required_actor_role}`; each transition writes a `messages` row (kind matching the action) and an outbox event `task.{action}`. *Acceptance:* exhaustive unit tests cover every (state, action) pair; illegal pairs return 409.

TITAN-BCK-013 (MUST) — *Create/estimate/prioritize.* Tasks are created by agents (or humans) with `title`, `spec`, `priority` (1–5), and `estimate_points` (Fibonacci 1–13, set by the assignee-role agent during grooming). The scheduler orders the `ready` pool by `(priority ASC, blocked_dependents_count DESC, created_at ASC)` — critical work first, unblockers next, FIFO ties. *Acceptance:* given a seeded fixture project, the scheduler's dequeue order matches the documented sort in an integration test.

TITAN-BCK-014 (MUST) — *Dependencies.* A task enters `ready` only when all `depends_on` tasks are `done`. Completing a task SHALL re-evaluate its dependents in the same transaction and `XADD` newly-ready tasks to `stream:tasks.ready`. *Acceptance:* completing the last blocker makes the dependent visible to a worker within 1 s locally.

TITAN-BCK-015 (MUST) — *Assignment.* The scheduler matches `ready` tasks to agents by role capability (task labels ↔ agent `role`), current load (max 1 concurrent run per agent at MVP; `config.max_parallel` later), and

acquires `lock:task:{id}` (`SET NX PX 30000`, heartbeat-renewed) before transitioning to `assigned`. *Acceptance:* two workers racing for one task result in exactly one `assigned` transition (verified by a 100-iteration race test).

TITAN-BCK-016 (MUST) — *Review & reopen.* Every task SHALL have a reviewer distinct from its assignee (Code Reviewer agent by default). `approve` → `done`; `request_changes` → `changes_requested` with a structured review message; `reopen` (from `done`) requires `task:reopen` permission and creates a new run attempt with `attempt = max+1`. *Acceptance:* a task cannot reach `done` without a review-approval message row from a distinct agent id.

TITAN-BCK-017 (MUST) — *Parallel execution.* Independent `ready` tasks SHALL execute concurrently across worker replicas and across distinct agents within one worker (asyncio task group, bounded by `WORKER_CONCURRENCY`, default 4). *Acceptance:* with 2 workers × concurrency 4, eight independent tasks show overlapping `started_at` / `finished_at` windows.

TITAN-BCK-018 (SHOULD) — *Milestones & sprints.* Sprints are planning containers (Vol 6 owns ceremony logic): the orchestrator SHALL only auto-schedule `ready` tasks belonging to the **active** sprint (or sprintless hotfix tasks with priority 1), and a milestone auto-closes when all its tasks are `done`. *Acceptance:* moving a task out of the active sprint removes it from the dequeue pool without changing its status.

8.2 Run lifecycle

A run is one agent attempt at a task: worker pops from `stream:runs.execute`, sets run `running`, resolves context (task spec + memory search + relevant artifacts), streams LLM output (deltas → `ws.project.{id}` pub/sub + append to artifact buffer), executes tools within the plugin sandbox (Vol 7), then writes artifacts, token/cost accounting, and a result to `stream:runs.results`, which the orchestrator consumes to drive the state machine (success → `submit_review`). Timeouts: soft 15 min (configurable) → `timed_out`, retriable twice with exponential backoff before DLQ.

---

9. API Gateway, Authentication & RBAC

9.1 Gateway concerns

At MVP the "gateway" is a FastAPI middleware stack inside `titan-core` (no separate gateway process to keep the laptop footprint small); in production, Kubernetes ingress (NGINX/Traefik) fronts it for TLS termination and coarse limits (Vol 9). Middleware order (outermost first): request-ID injection (`X-Request-ID`, UUIDv7, also the OTel trace correlation attribute) → CORS (allowlist, `localhost:3000` in dev) → authentication → RBAC → rate limiting → idempotency replay → body-size cap (2 MiB JSON; artifacts upload via presigned object-storage URLs, never through the API) → compression → router.

TITAN-BCK-019 (MUST) — Security headers (`X-Content-Type-Options: nosniff`, `Referrer-Policy`, HSTS in prod) SHALL be set by middleware; OpenAPI docs (`/docs`) require auth outside dev mode. *Acceptance:* header assertions in the API test suite; `/docs` returns 401 when `TITAN_ENV=production` without a token.

9.2 Authentication

TITAN-BCK-020 (MUST) — AuthN SHALL be OAuth2/OIDC (authorization-code + PKCE) against a pluggable IdP — **Keycloak in the local Compose stack** (free, self-hosted), any OIDC provider (Google/GitHub/Auth0) via configuration — plus a `local` password mode for laptop-only use. TITAN issues its **own** JWTs after IdP exchange: RS256 access tokens (15 min TTL; claims `sub`, `email`, `roles`, `scope`, `jti`) and rotating refresh

tokens (7 days, revocable via `jti` denylist in Redis). *Acceptance:* full login round-trip against dockerized Keycloak in the integration suite; a revoked refresh token is rejected in < 1 s.

Agents and CI use **API keys** (`POST /auth/api-keys` , hash-at-rest with SHA-256, prefix-identifiable `titan_ak_…`), exchanged internally for short-lived service JWTs carrying an `agent_id` claim so audit trails always resolve to a principal.

9.3 RBAC model

Roles are permission bundles stored in `roles.permissions` ; assignments in `user_roles` are org-wide (`project_id NULL`) or per-project. Permission strings are `resource:action` .

Role	Key permissions
`owner`	everything, incl. `org:manage` , `plugin:install` , role assignment
`admin`	all project CRUD, `agent:configure` , `task:reopen` , `run:cancel`
`maintainer`	`project:write` , `task:*` except reopen, `decision:write`
`operator`	`task:transition` , `run:create` , `message:write`
`viewer`	`*:read` only

TITAN-BCK-021 (MUST) — Every route SHALL declare its required permission via a dependency (`Requires("task:transition")`); checks resolve role → permissions with a per-request cache; agents' service JWTs map to the `operator` bundle scoped to their project. *Acceptance:* a route without a `Requires` dependency fails a CI AST lint; permission-matrix tests cover all roles × representative endpoints.

---

10. Rate Limiting

TITAN-BCK-022 (MUST) — Rate limiting SHALL be enforced in Redis using a token-bucket Lua script (atomic `EVALSHA`), keyed per principal and route class, returning `429` with `Retry-After` and `X-RateLimit-{Limit,Remaining,Reset}` headers. *Acceptance:* k6 burst test shows hard cutoff at configured limits; headers present on both 2xx and 429.

Route class	Default (per principal)	Notes
Read (`GET`)	300/min	burst 60
Write (POST/PATCH)	60/min	burst 20
`:transition` , run creation	30/min	protects orchestrator
Auth endpoints	10/min per IP	brute-force guard
WS connections	5 concurrent per user	connect-time check
LLM-triggering ops	project-level budget: `llm_profile.max_runs_per_hour`	cost control; `sys.notice` WS warning at 80 %

Internal agent traffic bypasses per-route limits but is governed by the LLM budget and `WORKER_CONCURRENCY` — the platform must never rate-limit itself into deadlock.

---

11. Caching Strategy

Postgres is always authoritative; caches are read-through and invalidated by version bump (no wildcard deletes).

Layer	What	TTL	Invalidation
Redis `cache:project:{id}:v{n}`	project DTO + board aggregates	5 min	version key bump on write (outbox consumer)
Redis `cache:agents:{project}`	agent registry list	60 s	bump on agent update
Redis `idem:{key}`	idempotency response replay	24 h	TTL only
In-process (per replica)	JWKS keys, role→permission map, settings	5 min	TTL only (small, safe to be stale)
HTTP	`ETag` on all GETs (from `version`)	—	client revalidates; `304` saves payloads

TITAN-BCK-023 (MUST) — Hot read endpoints (`GET /projects/{id}`, `/board`, `/agents`) SHALL be served read-through from Redis; cache hit ratio exported as `titan_cache_hits_total` / `titan_cache_misses_total`. *Acceptance:* repeated board fetches hit Redis (observed via metric delta); a task transition changes the board response within one poll cycle (< 1 s locally).

Deliberately **not** cached: task lists (change constantly; cursor pagination is cheap with the `(project_id,status,priority)` index), messages, runs.

---

12. Logging & Observability Hooks

TITAN-BCK-024 (MUST) — All logs SHALL be structured JSON via `structlog` with mandatory fields `ts, level, event, request_id, trace_id, span_id, module, principal, project_id` (where applicable); stdout only (collected by Loki via promtail/docker driver); no secrets, tokens, or full LLM prompts at INFO (prompt/response bodies go to run artifacts, DEBUG-gated). *Acceptance:* log-schema test parses 100 % of lines emitted during the integration suite; a secret-pattern scanner over captured logs finds zero hits.

TITAN-BCK-025 (MUST) — OpenTelemetry SHALL instrument FastAPI, SQLAlchemy, redis-py, httpx, and the LLM gateway (spans: `llm.call` with attributes `provider, model, tokens_in, tokens_out, cost_usd`; `queue.consume` with `stream, group, delivery_count`), exported OTLP to the local collector; traces span API → outbox → stream → worker → LLM via propagated `trace_id` in the message envelope (TITAN-BCK-010). *Acceptance:* one task's life (create → run → review) renders as a single connected trace in Grafana Tempo/Jaeger locally.

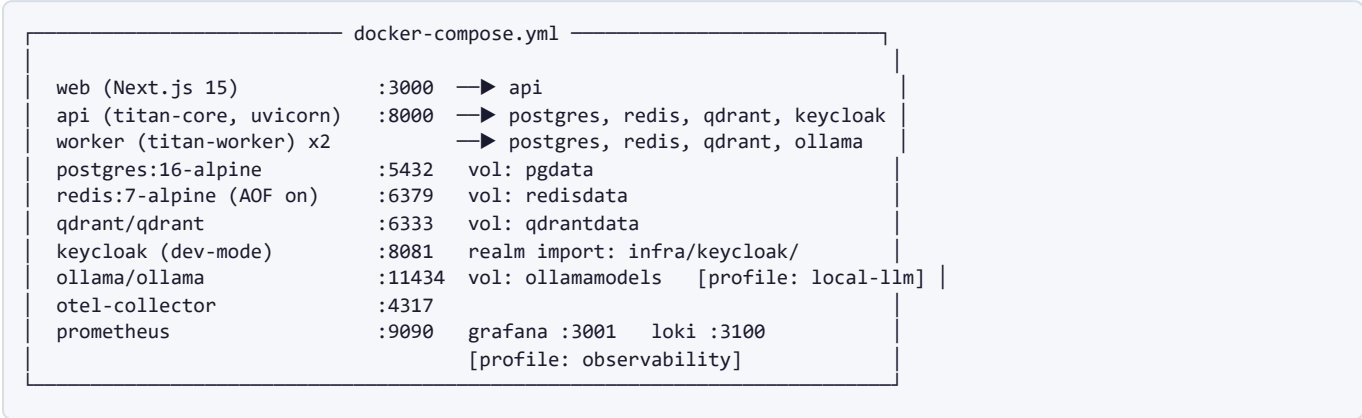
Prometheus metrics (via `/metrics`): RED metrics per route (`titan_http_request_duration_seconds` histogram), `titan_stream_depth`, `titan_stream_dlq_depth`, `titan_runs_active`, `titan_llm_tokens_total{provider,model,direction}`, `titan_llm_cost_usd_total`, `titan_ws_connections`. Health endpoints: `/healthz` (liveness, no deps), `/readyz` (checks Postgres, Redis, Qdrant with 500 ms budget). Dashboards and alert rules ship in `infra/grafana/` (Vol 9 extends for production).

---

13. Local Development: Docker Compose & Repository Layout

13.1 Compose topology

TITAN-BCK-026 (MUST) — `docker compose up` from the repo root SHALL start the full platform on one laptop with no paid dependencies. *Acceptance:* fresh clone → `cp .env.example .env` → `docker compose up` → UI at `localhost:3000` creates a project end-to-end against Ollama.



Compose **profiles** keep the default footprint small: core profile = web/api/worker/postgres/redis/qdrant/keycloak (~2.5 GB RAM); `--profile observability` and `--profile local-llm` are opt-in. All service config flows through environment variables with a checked-in `.env.example`; switching LLM provider is exactly one variable set (`LLM_PROVIDER=anthropic|openai|google|ollama` plus the provider key), per the canonical configuration-only rule.

13.2 Repository layout (monorepo)

```
titan/
├── docker-compose.yml
├── .env.example
├── Makefile                      # make dev / test / lint / migrate / seed
├── apps/
│   ├── web/                      # Next.js 15 + TS + Tailwind + shadcn/ui (Vol 4)
│   └── api/                      # Python 3.12 backend (this volume)
│       ├── pyproject.toml        # uv-managed; ruff, mypy --strict, pytest
│       ├── alembic/              # migrations (hand-written)
│       └── src/titan/
│           ├── main.py           # FastAPI app factory, middleware stack
│           ├── worker.py         # stream-consumer entrypoint
│           ├── settings.py       # pydantic-settings, env-driven
│           ├── gateway/          # authn/z deps, rate limit, idempotency, errors
│           ├── identity/         # routers/ service.py repo.py models.py schemas.py
│           ├── projects/         #   (same internal layout per module)
│           ├── orchestrator/     #   + state_machine.py scheduler.py
│           ├── agents/
│           ├── runs/
│           ├── messaging/
│           ├── memory/           # façade; internals per Vol 5
│           ├── plugins/          # registry; sandbox per Vol 7
│           ├── llm/              # provider gateway: anthropic|openai|google|ollama
│           ├── events/           # QueueBus, outbox publisher, WS fan-out
│           └── observability/    # otel.py logging.py metrics.py
├── packages/
│   └── shared-types/             # OpenAPI-generated TS client for web
├── infra/
│   ├── keycloak/realm.json
│   ├── grafana/  prometheus/  loki/  otel/
│   └── k8s/                      # production manifests (Vol 9)
└── docs/                         # this specification series
```

TITAN-BCK-027 (SHOULD) — The TypeScript API client in `packages/shared-types` SHALL be generated from the FastAPI OpenAPI spec in CI, failing the build on uncommitted drift, so frontend and backend contracts never diverge silently. *Acceptance:* changing a Pydantic schema without regenerating the client fails CI.

---

14. Risks & Open Questions

#	Risk / question	Impact	Mitigation / owner
R1	Redis Streams durability gap (AOF `everysec`) could lose ≤1 s of queue messages on crash	Lost work signals	Outbox re-drain (TITAN-BCK-005) makes Postgres authoritative; accepted at MVP; JetStream at scale-out
R2	Single shared Postgres becomes a hotspot with many concurrent runs	Latency, lock contention	Owning-module schema discipline enables per-module DB split; runs/artifacts extract first
R3	LLM streaming over WS at high fan-out may saturate one API replica	UI lag	Pub/sub fan-out already decouples producers; add dedicated WS gateway replicas behind sticky LB when needed
R4	Optimistic locking (`version` + `If-Match`) may frustrate agents retrying stale writes	Retry storms	Transitions are the only contended write path and are serialized via `lock:task:{id}`; ETags mainly protect human edits
R5	Keycloak adds ~700 MB RAM to the core profile	Laptop footprint	`AUTH_MODE=local` (password + JWT, no IdP) documented for constrained machines; same JWT code path

#	Risk / question	Impact	Mitigation / owner
Q 1	Should run artifacts live in local MinIO at MVP or plain bind-mounted filesystem?	Storage abstraction	Current spec: filesystem behind a `Storage` protocol; decide before beta whether MinIO joins a Compose profile (Vol 9 input needed)
Q 2	Per-agent token budgets vs per-project budgets — which is primary for cost control?	Cost governance	Volume 10 metrics work will inform; both counters are already recorded on `runs`
Q 3	Multi-tenancy (multiple orgs per deployment) is out of MVP scope — confirm in Volume 1 v1.1	Schema (org_id columns)	If confirmed in, add `org_id` before GA to avoid painful backfill

---

15. Cross-References

- **Volume 1 — Foundation**: canonical stack rationale, non-functional requirements this volume implements.
- **Volume 2 — Multi-Agent Intelligence**: agent roles bound to `agents.role`, structured message `body` schemas per `messages.kind`.
- **Volume 4 — Frontend & Dashboard**: consumes the REST/WS contracts in §4–§5 (board, live activity, logs).
- **Volume 5 — AI Brain (Memory, Knowledge & RAG)**: internals behind the `memory` module façade and Qdrant collections.
- **Volume 6 — Engineering Workflow**: sprint/review ceremonies that drive the orchestration state machine in §8.
- **Volume 7 — Integrations & Tool Ecosystem**: plugin registry tables, sandboxing, MCP; LLM gateway providers.
- **Volume 8 — Security**: threat model, secrets management, audit depth beyond §9–§10 baselines.
- **Volume 9 — Production & Deployment**: Kubernetes topology, scaling the extraction path, backup/DR for Postgres/Redis/Qdrant.
- **Volume 10 — Autonomous Evolution**: run metrics (`tokens`, `cost_usd`, attempts) feeding agent self-evaluation.

End of Volume 3 — TITAN-VOL-03, Draft v1.0.

PROJECT TITAN — Autonomous Software Engineering Platform

Volume 4 — Frontend & Dashboard (UI/UX Specification)

Field	Value
Series	PROJECT TITAN — Autonomous Software Engineering Platform
Volume	4 of 10 — Frontend & Dashboard
Doc ID	TITAN-VOL-04
Status	Draft v1.0
Date	2026-07-04
Requirement prefix	TITAN-FRT-NNN

Abstract. This volume is the implementation-ready UI/UX specification for the TITAN web dashboard — the interface through which a human client observes and steers an autonomous AI software organization. It defines the information architecture, the design system (Tailwind CSS + shadcn/ui, dark mode default), the state-management model (Next.js 15 server components + React Query + a multiplexed WebSocket sync layer), and screen-by-screen specifications with ASCII wireframes for thirteen surfaces: Organization Overview, Projects, Kanban Board, Task/Dependency Graph, Agent Status Grid, Live Agent Conversations, Logs Viewer, Architecture Viewer, Deployment Dashboard, Monitoring, Memory Explorer, Plugin Manager, and Settings — each with purpose, layout, key components, loading/empty/error states, interactions, and realtime behavior. Responsive rules, WCAG 2.1 AA accessibility, keyboard shortcuts, notifications, and the frontend folder structure complete the volume. Canonical stack decisions are justified in Volume 1; this volume consumes them.

Table of Contents

1. Introduction & Scope
2. Information Architecture & Navigation
3. Design System
4. State Management & Data Flow
5. Realtime: WebSocket Synchronization
6. Screen Specifications
7. Notifications System
8. Responsive Behavior
9. Accessibility (WCAG 2.1 AA)
10. Keyboard Shortcuts
11. Frontend Folder Structure
12. Decision Records
13. Risks & Open Questions
14. Cross-References

1. Introduction & Scope

1.1 Purpose

TITAN's dashboard is the client's window into an autonomous engineering company: the human types a goal ("Build me a Netflix clone"), then watches agents plan, build, review, and deploy. The UI therefore optimizes for **observability of autonomous work**: live status, live conversations, live logs, and clear intervention points (approve, pause, redirect) are first-class; forms and CRUD are secondary.

1.2 In scope / out of scope

In scope: all dashboard screens, navigation, design system, realtime sync, notifications, responsiveness, accessibility, keyboard shortcuts, state management, folder structure. **Out of scope:** REST/WS payload schemas (Volume 3), agent behavior (Volume 2), memory internals (Volume 5), deployment pipelines (Volume 9). Where this volume names an endpoint or event, the authoritative schema is Volume 3's.

1.3 Canonical stack (consumed, not re-decided)

Next.js 15 (App Router) + React + TypeScript (strict) + Tailwind CSS + shadcn/ui, inside the monorepo under `apps/web`. Realtime via WebSockets served by FastAPI. Auth via OAuth2/OIDC + JWT with RBAC. Everything must run locally under Docker Compose with free/local tools (local-first is a hard requirement); production targets Kubernetes.

1.4 Global requirements

ID	Priority	Requirement	Acceptance criteria
TITAN-FRT-001	MUST	The dashboard SHALL be a Next.js 15 App Router application in TypeScript strict mode with zero `any` in app code.	`tsc --noEmit` passes with `"strict": true`; ESLint rule `@typescript-eslint/no-explicit-any` = error; CI blocks merge on violation.
TITAN-FRT-002	MUST	Every screen SHALL define and render distinct loading, empty, error, and populated states.	Storybook story exists per state per screen; visual regression suite snapshots all four states.
TITAN-FRT-003	MUST	Any data visible on screen that changes server-side SHALL update in the UI within 2 s of the server event, without page reload.	E2E test: mutate via API, assert DOM update ≤ 2 s on Kanban, Agent Grid, Conversations, Logs, Deployments.
TITAN-FRT-004	MUST	The dashboard SHALL function fully against a local Docker Compose backend with no external network access (local LLMs via Ollama).	Playwright suite passes with machine offline except loopback.
TITAN-FRT-005	MUST	First Load JS for the Organization Overview route SHALL be ≤ 300 kB gzipped; heavy visualizations (graphs, charts, terminals) SHALL be dynamically imported.	`next build` budget check in CI fails the build when exceeded.

2. Information Architecture & Navigation

2.1 Hierarchy

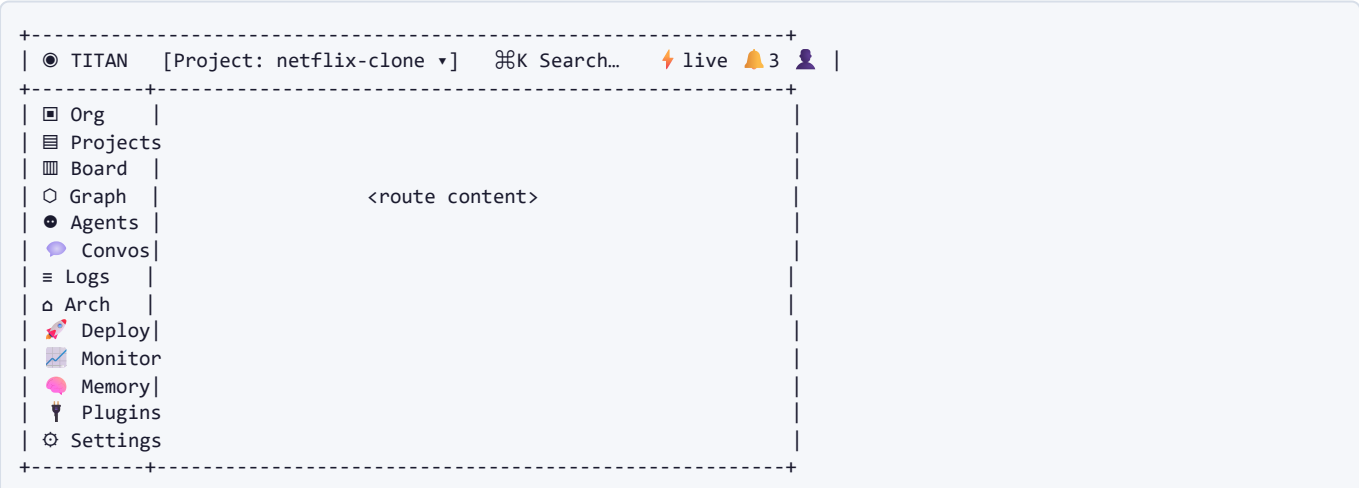
Data hierarchy is **Organization → Projects → (Tasks, Agents, Conversations, Deployments, Artifacts)**. Agents, Memory, Plugins, and Settings are organization-scoped; Board, Graph, and Architecture are project-scoped. The URL is the single source of navigational truth — selected project, filter, tab, and selected node are path- or query-encoded, so links are shareable and refresh-safe.

2.2 Route map

```
/                                → redirect → /org
/org                             Organization Overview
/projects                        Projects list
/projects/[pid]                  Project detail (Overview tab)
/projects/[pid]/board            Kanban board
/projects/[pid]/graph            Task / Dependency graph
/projects/[pid]/architecture     Architecture viewer
/projects/[pid]/deployments      Project deployments
/agents                          Agent status grid
/agents/[agentId]                Agent detail drawer route (intercepted)
/conversations                   Live agent conversations (channel list + feed)
/conversations/[channelId]       Specific channel
/logs                            Logs viewer (org-wide; ?project= filters)
/deployments                     Deployment dashboard (org-wide)
/monitoring                      Monitoring
/memory                          Memory explorer
/plugins                         Plugin manager
/settings/{profile|appearance|providers|members|api-keys|notifications}
```

2.3 App shell

Persistent shell: a 240 px left sidebar (collapsible to 64 px icon rail, state persisted in `localStorage`), a 56 px top bar, and a content region. The top bar contains: project switcher (`Ctrl/⌘+P`), global command palette trigger (`Ctrl/⌘+K`), connection-status indicator (WS live/reconnecting/offline), notification bell with unread badge, and user menu.



ID	Priority	Requirement	Acceptance criteria
TITAN-FRT-006	MUST	All navigational state (selected project, tab, filters, selected graph node) SHALL be URL-encoded and restorable on reload.	Deep-link E2E test per screen restores identical view.
TITAN-FRT-007	MUST	A command palette (`Ctrl/⌘+K`) SHALL navigate to any screen, project, agent, or task by fuzzy search.	Palette opens ≤ 100 ms; typing "kan net" surfaces the netflix-clone board.
TITAN-FRT-008	MUST	Navigation items SHALL be filtered by RBAC role from the JWT (e.g., `viewer` sees no Settings→Members, no destructive actions).	Role-matrix test renders sidebar per role and asserts visibility.
TITAN-FRT-009	SHOULD	The sidebar SHALL show per-item live badges (running agents count, failing deployments) via the notification store.	Badge updates within 2 s of WS event.

---

3. Design System

3.1 Foundations

Tailwind CSS is the styling layer; shadcn/ui provides the vendored component primitives (Button, Dialog, DropdownMenu, Table, Tabs, Toast, Command, Sheet, Tooltip, Badge, Card, Skeleton…). All colors are expressed as CSS custom properties consumed by Tailwind semantic tokens — components never hardcode hex values.

3.2 Color tokens

Dark mode is the **default**; light mode is a supported user preference (`class` strategy: `<html class="dark">` by default, toggled in Settings → Appearance, persisted per user and pre-hydrated via a blocking inline script to prevent flash).

Token	Dark (default)	Light	Usage
`--background`	`hsl(224 20% 7%)`	`hsl(0 0% 100%)`	App background
`--card`	`hsl(224 18% 10%)`	`hsl(0 0% 98%)`	Cards, panels
`--foreground`	`hsl(215 20% 92%)`	`hsl(224 20% 10%)`	Primary text
`--muted-foreground`	`hsl(217 12% 62%)`	`hsl(220 10% 40%)`	Secondary text
`--primary`	`hsl(210 100% 60%)`	`hsl(212 92% 45%)`	Actions, links, focus
`--destructive`	`hsl(0 72% 56%)`	`hsl(0 74% 46%)`	Errors, dangerous actions
`--success`	`hsl(150 62% 45%)`	`hsl(152 60% 34%)`	Healthy, passed
`--warning`	`hsl(38 92% 55%)`	`hsl(35 92% 40%)`	Degraded, pending
`--border`	`hsl(224 14% 18%)`	`hsl(220 12% 88%)`	Borders, dividers
`--ring`	`hsl(210 100% 60%)`	`hsl(212 92% 45%)`	Focus rings

Status colors map 1:1 to domain states everywhere (agent state, task state, deployment state): `success` =running/done/healthy, `warning` =waiting/review/degraded, `destructive` =failed/blocked, `muted` =idle/draft. Status is **never** conveyed by color alone — always paired with an icon or label (see §9).

3.3 Typography, spacing, elevation

- Fonts: `Inter` (UI) and `JetBrains Mono` (code, logs, IDs), self-hosted via `next/font` (no external font CDN — offline requirement).
- Type scale: 12 / 13 / 14 (body default) / 16 / 18 / 22 / 28 px, line-height 1.5.
- Spacing: Tailwind 4 px grid; density is compact — tables at 36 px row height, 44 px touch targets on mobile.
- Elevation: borders over shadows in dark mode; radius `--radius: 8px`.
- Motion: 120–200 ms ease-out for micro-interactions; all animation respects `prefers-reduced-motion` (§9).

ID	Priority	Requirement	Acceptance criteria
TITAN-FRT-010	MUST	All components SHALL consume semantic CSS-variable tokens; raw palette values in app code are forbidden.	Lint rule (custom Tailwind plugin) rejects arbitrary hex/rgb values in `apps/web`.
TITAN-FRT-011	MUST	Dark theme is default; theme choice SHALL persist per user and apply before first paint (no flash of wrong theme).	Lighthouse trace shows no theme flip; new anonymous session renders dark.
TITAN-FRT-012	MUST	Both themes SHALL meet WCAG 2.1 AA contrast (≥ 4.5:1 body text, ≥ 3:1 large text/UI components).	axe-core CI run reports zero contrast violations in both themes.
TITAN-FRT-013	SHOULD	shadcn/ui components SHALL be vendored under `components/ui` and modified in-place rather than wrapped ad hoc.	No second styling wrapper layer exists; changes reviewed like normal code.

---

4. State Management & Data Flow

4.1 Model

Three complementary layers, chosen per data class (ADR-FRT-01):

1. **React Server Components (RSC)** render the first paint of every route from FastAPI REST endpoints (server-side fetch with the user's JWT); read-only content (settings pages, metadata headers) ships no client JS.
2. **TanStack React Query v5** owns all client server-state: lists, boards, grids. RSC data is passed as `initialData` via the Hydration API so the cache starts warm. Structured keys: `['project', pid, 'tasks', filters]`.
3. **WebSocket sync layer** (§5) invalidates or patches React Query caches on server events; `refetchInterval` is only a 60 s safety net on critical screens.

Pure client UI state (sidebar collapsed, palette open, drawer selection) lives in **Zustand** stores. Forms use `react-hook-form` + `zod`, with API types generated from the FastAPI OpenAPI spec (`openapi-typescript` in `packages/api-client`).

4.2 Mutations

All mutations use React Query `useMutation`: optimistic update where the outcome is locally predictable (Kanban drag, toggle), rollback on error with a toast, and cache invalidation driven by the WS confirmation event rather than the HTTP response — the WS event is the canonical "it happened" signal, deduplicated by `seq`.

ID	Priority	Requirement	Acceptance criteria
TITAN-FRT-014	MUST	Server state SHALL be managed exclusively by React Query; no bespoke fetch-in-`useEffect` patterns.	Lint rule bans `fetch` outside `packages/api-client` and route handlers; code review checklist item.
TITAN-FRT-015	MUST	Optimistic mutations SHALL roll back on failure and surface a toast with a retry action.	E2E test: kill API mid-drag on Kanban → card returns to origin column, toast shown.
TITAN-FRT-016	MUST	API types SHALL be generated from the backend OpenAPI schema; hand-written duplicates are forbidden.	CI job regenerates types and fails on uncommitted diff.
TITAN-FRT-017	SHOULD	RSC SHALL provide `initialData` hydration so no visible refetch flash occurs on route entry.	No loading skeleton on warm navigation to Projects list in E2E trace.

---

5. Realtime: WebSocket Synchronization

5.1 Connection model

One multiplexed WebSocket per browser tab to FastAPI at `wss://<host>/ws?token=<jwt>` (ADR-FRT-03). The client subscribes/unsubscribes to **topics** as routes mount/unmount:

```
Client                                  FastAPI /ws
  | connect (JWT)                           |
  |---------------------------------------->|  auth, register session
  | {op:"sub", topics:["project:42:tasks", |
  |   "agents:status","notifications:me"]} |
  |---------------------------------------->|
  |        {topic, event, seq, ts, payload}|
  |<----------------------------------------|  fan-out from Redis pub/sub
  | {op:"ping"} / {op:"pong"} every 25s    |
```

Envelope: `{ topic: string, event: string, seq: number, ts: string, payload: T }`. `seq` is monotonic per topic; a gap triggers a targeted React Query refetch — the same reconciliation path used after reconnect.

5.2 Core topics and events

Topic	Events	Consumed by
`org:{oid}:activity`	`activity.created`	Organization Overview feed
`project:{pid}:tasks`	`task.created/updated/moved/completed`	Kanban, Graph, Project detail
`agents:status`	`agent.status_changed`, `agent.heartbeat`	Agent grid, sidebar badges
`channel:{cid}:messages`	`message.created`, `message.chunk` (token stream)	Conversations
`logs:{stream}`	`log.appended` (batched ≤ 100/frame)	Logs viewer
`project:{pid}:deployments`	`deployment.status_changed`, `deployment.step`	Deployment dashboard
`monitoring:summary`	`metric.sample` (1 Hz aggregate)	Monitoring, Overview KPIs
`notifications:{userId}`	`notification.created`	Bell, toasts

5.3 Degradation

Reconnect uses exponential backoff (1 s → 30 s cap, jitter). While disconnected the top bar shows an amber "reconnecting" pill; after 60 s a banner offers manual retry and React Query falls back to 30 s polling on the active screen, keeping the app usable behind hostile proxies.

ID	Priority	Requirement	Acceptance criteria
TITAN-FRT-018	MUST	The client SHALL maintain exactly one WS connection per tab, with topic-scoped subscribe/unsubscribe on route change.	Network inspector shows 1 socket across full navigation; server session log shows sub/unsub pairs.
TITAN-FRT-019	MUST	On reconnect or `seq` gap, affected caches SHALL be re-fetched; no stale or duplicated items may remain.	Chaos test: drop socket during 50 task mutations → board converges to server state, no dupes.
TITAN-FRT-020	MUST	Connection state SHALL be always visible (live / reconnecting / offline) in the top bar.	Visual states verified in Storybook + E2E toggling network.
TITAN-FRT-021	SHOULD	High-frequency events (logs, token streams, metrics) SHALL be batched and applied via `requestAnimationFrame` to keep interaction latency < 100 ms.	Performance test: 1,000 log lines/s keeps INP < 200 ms.

---

6. Screen Specifications

Conventions for this section: every screen lists Purpose, Layout (wireframe), Key components, States, Interactions, and Realtime. All list screens use skeleton loaders shaped like their content (not spinners); all error states show the failed scope inline (not a blank page) with a Retry button and error reference ID; all empty states include a one-line explanation and a primary action.

6.1 Organization Overview (`/org`)

Purpose. The landing screen: answers "what is my AI company doing right now?" in under five seconds.



- **Key components:** four KPI stat cards; active-project cards (progress, sprint, agent count, CI status); virtualized live-activity feed; "Needs Your Attention" queue of human-approval items (release approvals, blocked decisions, budget overruns).

- **States:** loading = skeleton cards + shimmer feed; empty = onboarding hero "Create your first project — describe what you want built" with prompt input; error = per-panel inline error, KPIs degrade independently.
- **Interactions:** KPI cards deep-link to filtered screens (e.g., "5 blocked" → Kanban filtered `state=blocked`); attention items open an approval dialog with context and Approve/Reject/Comment; activity rows link to their entity.
- **Realtime:** subscribes `org:{oid}:activity`, `agents:status`, `monitoring:summary`. Feed prepends items with a subtle 200 ms highlight; KPIs patch in place.

TITAN-FRT-022 (MUST). Human-approval items SHALL appear on the Overview within 2 s of creation and be actionable in ≤ 2 clicks. *AC:* E2E — backend requests release approval; approve via Overview dialog; workflow proceeds.

6.2 Projects List (`/projects`) & Project Detail (`/projects/[pid]`)

Purpose. Portfolio management: create, find, and inspect projects.

```
/projects
+-------------------------------------------------------------------+
| Projects   [Search…] [Status ▾] [Sort: Activity ▾]  [+ New]       |
+-------------------------------------------------------------------+
| ● netflix-clone   building   ▓▓▓▓ 71%  9 agents  CI ✓  2m ago     |
| ● banking-sys     planning   ▓▓▒▒ 34%  4 agents  CI –  1h ago     |
| ○ hospital-mgmt   paused     ▓▓▓▒ 55%  0 agents  CI ✓  3d ago     |
+-------------------------------------------------------------------+

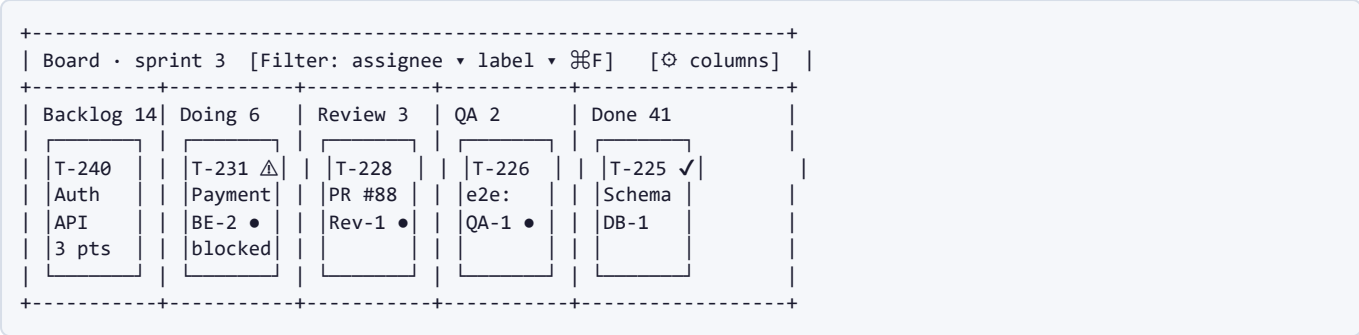
/projects/[pid]   (tabs: Overview | Board | Graph | Architecture |
                        Deployments | Files | Settings)
+-------------------------------------------------------------------+
| netflix-clone  · building · sprint 3/6        [⏸ Pause] [⋯]       |
| Milestones: [M1 ✓]–[M2 ✓]–[M3 ▓▒ 64%]–[M4]–[Release]              |
| Requirements summary | Team (agents) | Recent decisions (ADRs)    |
| Risk register        | Tech-debt report | Sprint burndown         |
+-------------------------------------------------------------------+
```

- **Key components:** filterable table/card list; "New Project" wizard (a chat-style intake where the CEO Agent asks its essential clarification questions before the roadmap is generated); detail tabs; milestone timeline; auto-generated artifacts (roadmap, risk analysis, tech-debt report) rendered from backend Markdown.
- **States:** loading skeleton rows; empty = same onboarding hero as §6.1; error inline. Detail tab errors isolate to the tab.
- **Interactions:** row click → detail; Pause/Resume gates all agent work on the project (confirmation dialog, RBAC `admin|owner`); wizard streams the CEO Agent's questions token-by-token.
- **Realtime:** `project:{pid}:tasks` and `org:{oid}:activity` keep progress bars, CI badges, and milestone bars current.

TITAN-FRT-023 (MUST). Project creation SHALL be a conversational intake that yields a visible roadmap + milestone plan without a page reload. *AC:* E2E from prompt to rendered roadmap on detail Overview tab.

6.3 Kanban Board (`/projects/[pid]/board`)

Purpose. The canonical view of task flow through the autonomous pipeline.

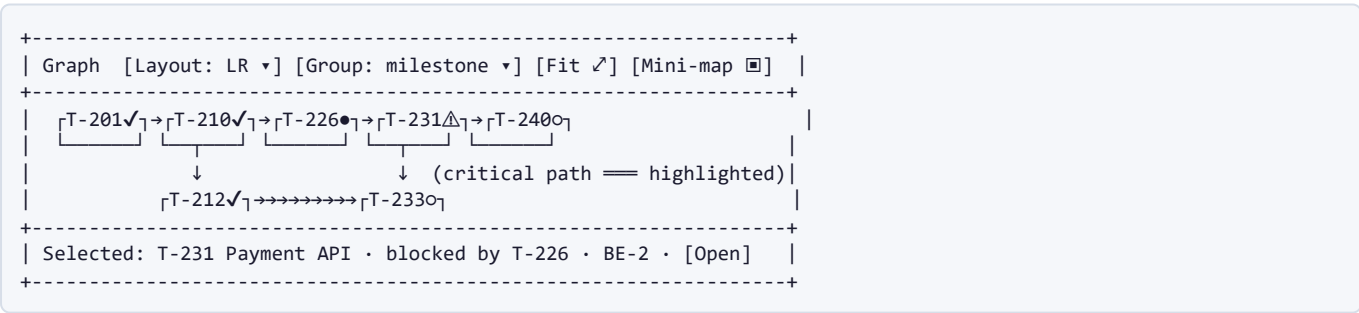


- **Key components:** columns mapped to workflow states (configurable per project from the Volume 6 workflow definition); task cards (ID, title, agent avatar, points, labels, blocked badge); WIP-limit indicators; task detail Sheet (right slide-over) with description, acceptance criteria, dependency links, linked PR/conversation, and full event history.
- **States:** loading = 5 skeleton columns × 3 skeleton cards; empty = "No tasks yet — the Project Manager Agent creates tasks after planning completes" with link to project status; error = board-level retry panel; a column stream failure shows a stale-data badge on that column.
- **Interactions:** drag-and-drop between columns (`@dnd-kit` , optimistic, RBAC-gated — moving a card is a human override that notifies the PM Agent); `⌘F` filter; card click opens Sheet (route `?task=T-231` for deep-linking); keyboard DnD per §10.
- **Realtime:** `task.moved/updated/created` patch cards in place; a card moved by an agent animates (300 ms translate) to its new column with the acting agent's avatar pulsing; conflicting local drag resolves server-wins with toast.

TITAN-FRT-024 (MUST). Board SHALL stay consistent under concurrent agent + human moves (server-wins, no ghost cards). *AC:* chaos E2E with 20 concurrent moves converges to API state. **TITAN-FRT-025 (MUST).** Drag-and-drop SHALL be fully operable by keyboard (§10) and announced to screen readers. *AC:* axe + manual NVDA script pass.

6.4 Task Graph / Dependency Graph (`/projects/[pid]/graph`)

Purpose. Visualize the DAG of tasks and dependencies; spot the critical path and blockers instantly.



- **Key components:** React Flow (`@xyflow/react`) canvas with `dagre` auto-layout (ADR-FRT-04); custom task nodes (status icon, ID, title, agent avatar); mini-map; critical-path toggle (longest incomplete chain, computed client-side); selection inspector bar; blocked edges rendered dashed-red.
- **States:** loading = centered skeleton canvas + "computing layout"; empty = same as board empty; error = retry panel; > 500 nodes triggers clustered milestone-group rendering with drill-down (performance guard).
- **Interactions:** pan/zoom (wheel, pinch, `+/-`), click node → inspector + `?node=` in URL, double-click → task Sheet, hover → highlight upstream/downstream chains, `Shift+click` multi-select for bulk label ops. Layout is read-only — dependencies are edited by agents; humans request changes via the task Sheet.

- **Realtime:** node status patches recolor in place; new tasks enter with fade-in and incremental re-layout limited to the affected subgraph (full re-layout only on explicit "Re-layout" click, to avoid disorienting jumps).

TITAN-FRT-026 (MUST). Graph SHALL render 300 nodes / 600 edges at ≥ 30 fps pan/zoom on a mid-range laptop. *AC:* perf test in CI against seeded fixture.

6.5 Agent Status Grid (`/agents`)

Purpose. The "org chart in motion" — one card per agent, its state, and its current work.

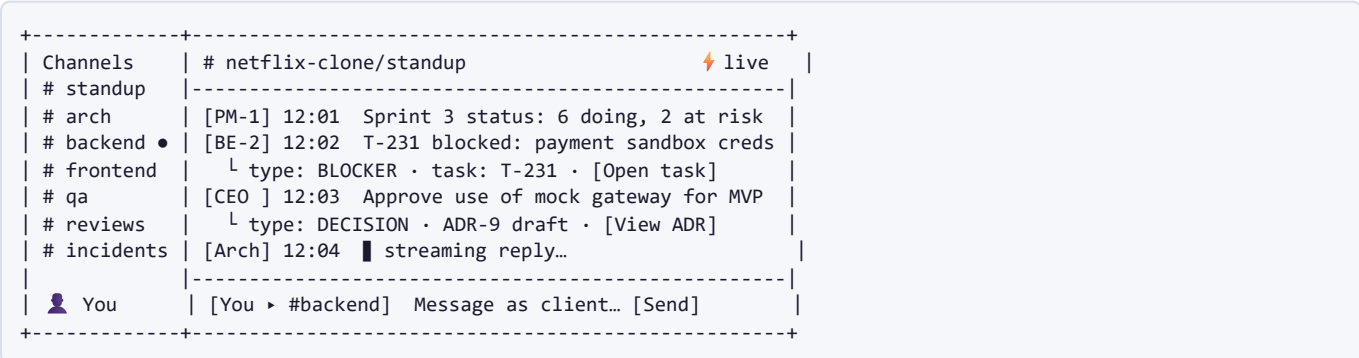


- **Key components:** responsive card grid grouped by hierarchy tier (executive / management / engineering / quality); each card shows role, state (working / idle / blocked / error / paused), current task link, model+provider in use, token throughput, and elapsed time; header roll-up counts; agent detail drawer (`/agents/[id]` , intercepted route) with KPIs, recent messages, memory usage, and action buttons (Pause, Restart, Reassign — RBAC `admin+`).
- **States:** loading skeleton grid; empty = "No agents provisioned — start a project to spawn the organization"; error per-card (an unreachable agent renders its card in error style with last-seen time — the grid never blanks).
- **Interactions:** card click → drawer; Pause/Restart with confirmation; filter by role/state; clicking the model chip opens provider settings (RBAC-gated).
- **Realtime:** `agents:status` heartbeats (5 s cadence) drive state chips; missed 3 heartbeats → card degrades to "stale" with timestamp; error events raise a toast + bell notification.

TITAN-FRT-027 (MUST). An agent entering `error` SHALL be visible in the grid and the notification bell within 2 s, with a one-click path to its logs. *AC:* fault-injection E2E.

6.6 Live Agent Conversations (`/conversations`)

Purpose. The company's Slack: watch structured agent-to-agent communication live; intervene when needed.

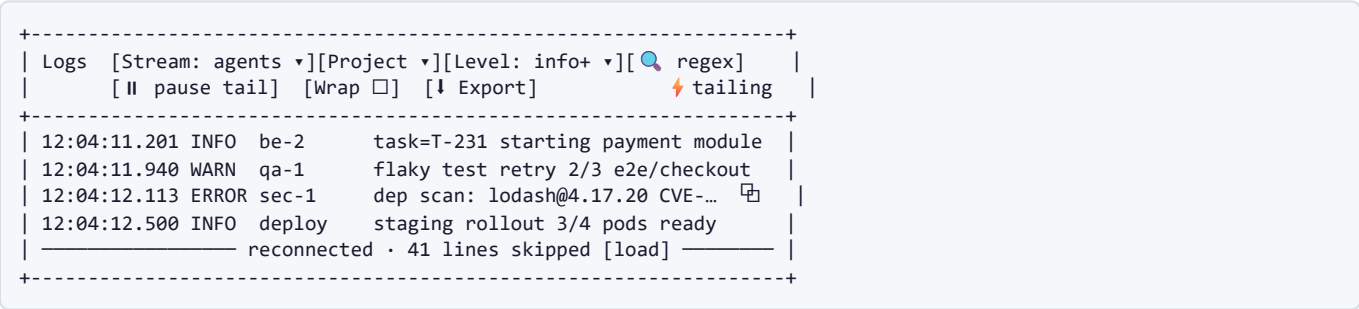


- **Key components:** channel list (per-project + org channels, unread badges); virtualized message feed; **structured message chips** — every message carries its Volume 2 type (TASK_ASSIGNMENT, PROGRESS_REPORT, REVIEW_REQUEST, BUG_REPORT, DECISION, BLOCKER, MEETING_SUMMARY, CONFLICT_RESOLUTION) rendered as a typed chip with deep links to the referenced entity; token-streaming bubbles for in-flight LLM output; human input box (posts as role `client`; agents treat client messages as top-priority input); search across history.
- **States:** loading skeleton bubbles; empty channel = "No messages yet"; error = feed keeps history, shows "stream interrupted — reconnecting" divider; history pagination on scroll-up (cursor-based).
- **Interactions:** jump-to-entity links; filter by message type; "🔕 mute channel"; copy message permalink; `Ctrl/⌘+Enter` send.
- **Realtime:** `message.created` appends; `message.chunk` streams tokens into a growing bubble with a typing caret; auto-scroll only when the user is at bottom (a "↓ 12 new" pill otherwise); unread counts update across channels.

TITAN-FRT-028 (MUST). Streaming agent output SHALL render incrementally (< 500 ms first-token-to-paint after receipt) without layout thrash. *AC:* perf trace during a 2,000-token streamed message keeps CLS ≈ 0.
TITAN-FRT-029 (MUST). Every structured message SHALL deep-link to its referenced entity (task, PR, ADR, deployment). *AC:* link-integrity E2E over all eight message types.

6.7 Logs Viewer (`/logs`)

Purpose. Unified, greppable view of agent logs, system logs, and generated-app build/test output.

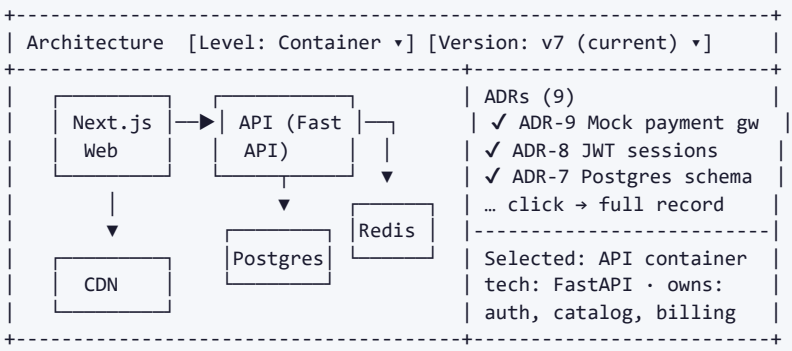


- **Key components:** virtualized monospace list (TanStack Virtual; 100k+ lines, ADR-FRT-05); filter bar (stream, project, agent, level, time range, regex — all URL-encoded); follow-tail toggle (auto-pauses on scroll-up); line expansion for structured JSON payloads; copy-line and export (filtered NDJSON); level color-coding + icons.
- **States:** loading = header instantly + "connecting to stream…"; empty = "No log lines match filters" with one-click filter reset; error = banner with retry while retaining the buffer; gap markers after reconnect with lazy "load skipped" fill from the REST history endpoint (Loki-backed, per Volume 9).
- **Interactions:** click ERROR line → context drawer (±50 lines + link to owning agent/task); `⏸` / `Space` pauses tail; regex highlights matches.
- **Realtime:** `log.appended` batches (≤ 100 lines/frame, rAF-applied per TITAN-FRT-021); client ring buffer caps at 50k lines (older lines fetched on demand).

TITAN-FRT-030 (MUST). Tail mode SHALL sustain 1,000 lines/s without dropping frames below 30 fps or freezing filters. *AC:* load-test fixture in CI perf suite.

6.8 Architecture Viewer (`/projects/[pid]/architecture`)

Purpose. Render the System Architect Agent's living architecture: C4-style diagrams + ADR history for the *generated* project.

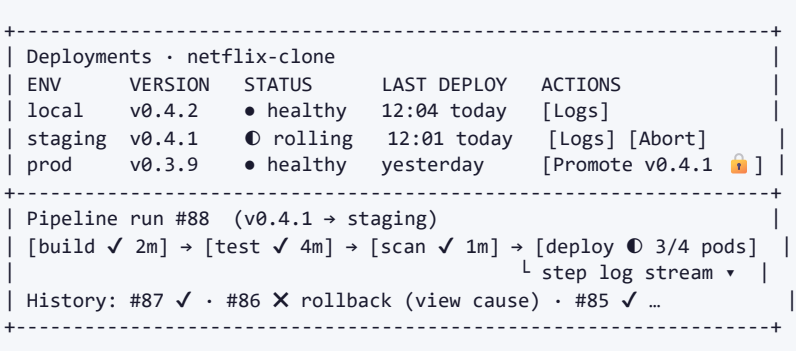


- **Key components:** React Flow canvas reusing the §6.4 graph engine with architecture node types (system/container/component/datastore/external); level switcher (Context → Container → Component, C4); **version timeline** — every architecture revision the Architect Agent commits is selectable, with a visual diff mode (added=green outline, removed=red ghost, changed=amber); ADR side panel (Context/Decision/Alternatives/Trade-offs rendered from backend records); node inspector.
- **States:** loading skeleton canvas; empty = "Architecture not yet designed — appears after the Architecture phase" with phase status link; error retry; oversized component views cluster like §6.4.
- **Interactions:** drill-down (double-click container → component level scoped to it); diff toggle between any two versions; export current view as SVG/PNG (client-side, offline-safe); ADR click opens full record dialog.
- **Realtime:** `architecture.updated` event shows a non-intrusive banner "New version v8 available — [View]" rather than mutating the canvas under the user.

TITAN-FRT-031 (MUST). Every architecture version SHALL be viewable and diffable against any other version. *AC:* E2E creates 3 revisions, asserts diff highlights match fixture.

6.9 Deployment Dashboard (`/deployments` , `/projects/[pid]/deployments`)

Purpose. Observe and control environment state and pipeline runs for generated applications.



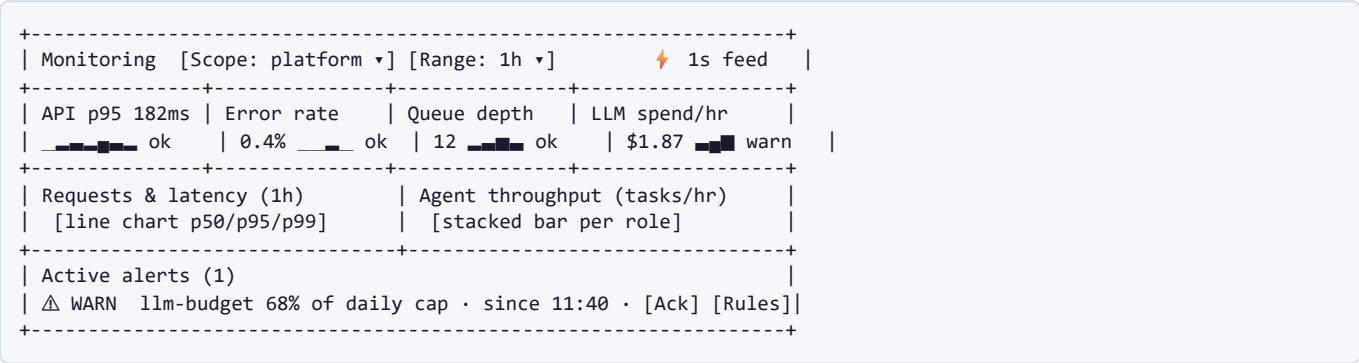
- **Key components:** environment cards (version, health, URL, uptime); pipeline visualization (stage chips with per-stage duration + expandable live step logs reusing the §6.7 log list); deploy history table with rollback causes; **approval-gated actions** — Promote/Rollback/Abort require confirmation dialog restating scope, and `prod` promote requires typed environment name (RBAC `owner|admin` ; 🔒 shown when the release awaits CEO-Agent or human approval per Volume 6).

- **States:** loading skeletons; empty = “No deployments yet — DevOps Agent deploys after the first passing release build”; error inline per environment; an aborted/failed run keeps its stage chips with failure detail.
- **Interactions:** stage chip expands logs; Promote/Rollback/Abort with confirmations; history row → full run detail; env card → live app URL (new tab).
- **Realtime:** `deployment.status_changed` and `deployment.step` animate stage chips; health probes update env cards; a failed prod deploy raises a `critical` notification (§7).

TITAN-FRT-032 (MUST). Destructive deployment actions SHALL require explicit confirmation (typed env name for prod) and be RBAC-gated. *AC:* E2E asserts viewer role sees no actions; admin prod-promote requires typed “prod”.

6.10 Monitoring (`/monitoring`)

Purpose. Health of TITAN itself *and* of generated apps: golden signals, agent throughput, LLM cost.

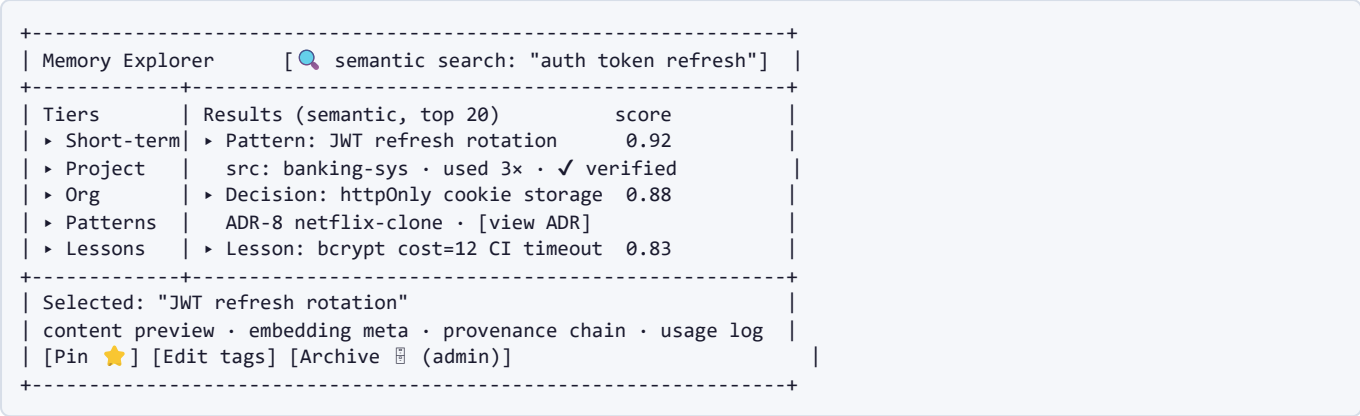


- **Key components:** stat tiles with sparklines and threshold coloring; time-series charts (Recharts, dynamically imported) for latency, error rate, queue depth, token usage and cost per provider/agent; alert list with Ack; scope switcher (platform / per generated app); deep links into Grafana for power users (“Open in Grafana”, local Grafana from the Compose stack per Volume 9) — the dashboard embeds nothing from Grafana, keeping the UI offline-consistent.
- **States:** loading skeleton tiles/charts; empty range = “No data in range”; error per-panel with retry; datasource loss shows last-known values flagged “stale @ 12:03”.
- **Interactions:** range picker (15 m / 1 h / 24 h / 7 d); hover crosshair with synced tooltips across charts; tile click → pre-filtered chart; alert Ack (RBAC `admin+`) and link to alert rules (Settings).
- **Realtime:** `metric.sample` (1 Hz aggregate) rolls tiles and appends chart points in the live range; historical ranges are REST-fetched (Prometheus-backed API per Volume 9).

TITAN-FRT-033 (SHOULD). LLM cost SHALL be attributable per agent and per project over any selected range. *AC:* cost chart group-by toggle matches seeded ledger fixture.

6.11 Memory Explorer (`/memory`)

Purpose. Inspect and curate what the organization has learned: memory tiers, embeddings search, decisions, reusable patterns (backing model in Volume 5).

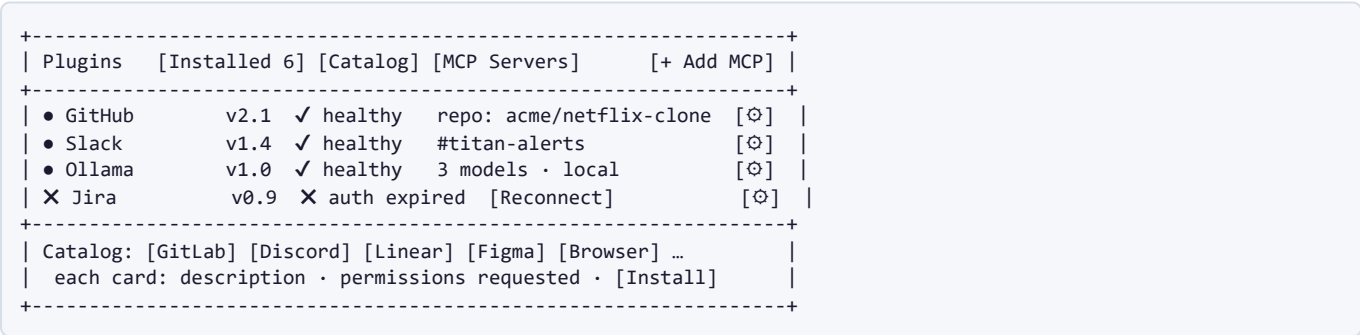


- **Key components:** tier tree (short-term / project / organization / reusable patterns / lessons learned); semantic search box (Qdrant-backed endpoint) with score-ranked results; detail panel with provenance (which agent wrote it, from which task/conversation) and usage history (which retrievals used it); curation actions (pin, tag, archive — archive is soft-delete, RBAC `admin+`); browse mode with faceted filters (type, project, date, agent).
- **States:** loading skeleton list; empty tier = "Nothing stored here yet" with tier explanation; search miss = "No semantic matches ≥ 0.6 — try broader phrasing"; error inline.
- **Interactions:** search (`Enter`), facet filters, result click → detail, provenance links jump to source conversation/task; diff view when a memory entry was revised.
- **Realtime:** `memory.updated` badges the affected tier ("3 new"); the list refreshes on demand, not live-mutating under a reading user.

TITAN-FRT-034 (MUST). Every memory entry SHALL display full provenance (creating agent, source entity, timestamp) and its usage history. *AC:* fixture entry renders complete provenance chain; links resolve.

6.12 Plugin Manager (`/plugins`)

Purpose. Discover, install, configure, and monitor plugins (GitHub, GitLab, Slack, Discord, Jira, Linear, Figma, browser automation, MCP servers, LLM providers — catalog per Volume 7).



- **Key components:** Installed tab (status, version, health check result, config gear); Catalog tab (permission-scoped install cards — each install dialog lists exactly which capabilities/scopes the plugin gets, mirroring the Volume 7 permission model); MCP Servers tab (add by URL/command, tool list preview, enable per-agent); per-plugin config forms generated from the plugin's JSON Schema manifest (rendered with the shared form engine — no hand-built form per plugin); health/activity log per plugin.
- **States:** loading skeletons; empty installed = "No plugins installed — browse the catalog"; catalog fetch failure falls back to the bundled offline catalog (local-first); plugin error state shows last error + Reconnect.
- **Interactions:** Install → permission consent dialog → config form → test-connection step → enabled; Disable/Uninstall with confirmation (uninstall states data retention behavior); secrets entered in config are

write-only (masked after save, per Volume 8).

- **Realtime:** `plugin.status_changed` updates health chips; a plugin failure that blocks an agent raises a notification linking here.

TITAN-FRT-035 (MUST). Plugin configuration forms SHALL be generated from the plugin's JSON Schema manifest; secrets SHALL be write-only in the UI. *AC:* installing fixture plugin renders schema-driven form; saved secret displays as `••••` and never round-trips in any API response.

6.13 Settings (`/settings/*`)

Purpose. Organization and user configuration.

```
+-------------+-----------------------------------------------------+
| Settings    | LLM Providers                                       |
| Profile     |  Default: [Anthropic ▾]  Fallback: [Ollama ▾]       |
| Appearance  |  ┌ Anthropic  key ••••7f2  ✓ verified  [Test]       |
| Providers   |  ├ OpenAI     key —        not set     [Add]        |
| Members     |  ├ Google     key —        not set     [Add]        |
| API Keys    |  └ Ollama     http://ollama:11434  ✓ 3 models       |
| Notifications| Per-role overrides: [Architect→anthropic/opus …]   |
| Budgets     |  Budget: $10/day  hard-stop ☑   alert at 80% ☑      |
+-------------+-----------------------------------------------------+
```

- **Sub-pages: Profile** (name, email, avatar, session list with revoke); **Appearance** (theme dark/light/system, density, reduced motion override, language); **LLM Providers** (provider keys, default + fallback chain, per-agent-role model overrides, budgets with hard-stop — this is the configuration-only provider switching required by the canonical gateway decision); **Members & Roles** (invite, RBAC role assignment owner/admin/developer/viewer, per-project grants); **API Keys** (personal access tokens for the TITAN API, scoped, shown once); **Notifications** (per-category channel matrix, §7); **Budgets & Limits** (token/cost caps per project).
- **States:** each sub-page loads independently (RSC-rendered, minimal JS); form errors inline per field (zod messages); destructive actions (remove member, revoke key) use typed-confirmation dialogs.
- **Interactions:** all forms `react-hook-form` + zod; Save is disabled until dirty; provider "Test" performs a live round-trip and reports latency/model list; role changes take effect on next token refresh and are announced to the affected user via notification.
- **Realtime:** minimal — `member.updated` and `budget.threshold` events refresh relevant panels.

TITAN-FRT-036 (MUST). Switching the default LLM provider (e.g., Anthropic → Ollama) SHALL require only Settings changes and take effect without redeploying. *AC:* E2E switches provider, next agent action logs the new provider. **TITAN-FRT-037 (MUST).** RBAC SHALL be enforced in the UI (hidden/disabled controls) **and** never trusted client-side — server re-checks every mutation. *AC:* forged request from viewer role returns 403 and UI shows permission toast.

---

7. Notifications System

7.1 Channels and taxonomy

Three in-app surfaces plus external channels (via plugins):

1. **Toasts** (shadcn/ui `sonner`, bottom-right, max 3 stacked, auto-dismiss 6 s except `critical` which persists until dismissed) — transient confirmations and failures.

2. **Notification Center** (bell → popover; full page at `/settings/notifications` history) — persistent, filterable, mark-read/mark-all, deep links.
3. **Attention Queue** (Overview §6.1) — items *requiring* human action; these never auto-dismiss and mirror to the bell.

Category	Default severity	Examples	Default channels
`approval`	critical	Release approval, budget hard-stop	Toast + bell + attention queue
`failure`	high	Deploy failed, agent error, CI red	Toast + bell
`progress`	info	Milestone done, sprint closed	Bell
`activity`	low	Task moved, PR merged	Feed only (muted by default)
`security`	critical	Dependency CVE, secret detected	Toast + bell + attention queue

Per-category routing is user-configurable (channel matrix in Settings; Slack/Discord delivery via plugins). Browser (Web Push) notifications are opt-in and used only for `approval` / `critical` while the tab is unfocused.

ID	Priority	Requirement	Acceptance criteria
TITAN-FRT-038	MUST	Notifications SHALL be delivered over `notifications:{userId}` and persisted server-side; the bell badge survives reload.	Reload retains unread count; mark-read syncs across two open tabs ≤ 2 s.
TITAN-FRT-039	MUST	`critical` toasts SHALL persist until explicitly dismissed and never be replaced by lower-severity toasts.	Storybook + E2E stacking test.
TITAN-FRT-040	SHOULD	Toast storms SHALL coalesce (≥ 4 same-category events in 10 s → one grouped toast "12 tasks updated").	Burst fixture produces grouped toast.

---

8. Responsive Behavior

Breakpoints (Tailwind): `sm 640`, `md 768`, `lg 1024`, `xl 1280`, `2xl 1536`. The dashboard is desktop-first (its users watch dense realtime data) but every screen MUST be usable at 375 px width.

Range	Shell	Screen adaptations
≥ 1280	Full sidebar + content	All layouts as wireframed; side panels inline
1024 – 1279	Icon-rail sidebar	Two-panel screens (Conversations, Memory, Architecture) keep both panels; inspectors become overlays
768–1023	Icon rail	Kanban horizontal-scrolls with snap columns; Overview KPI grid 2×2; detail panels → bottom Sheets
< 768	Sidebar → Sheet (hamburger); bottom tab bar with Org/Board/Agents/Convos/More	One panel at a time (channel list ↔ feed); Graph and Architecture render read-only with pinch-zoom; tables → card lists; Logs keeps virtualized list with simplified filters

Rules: no horizontal body scroll ever (wide content scrolls in its own container); touch targets ≥ 44 px under `md`; drag-and-drop gains long-press initiation on touch plus an explicit "Move to…" menu fallback; charts re-render responsively via container queries.

TITAN-FRT-041 (MUST). All 13 screens SHALL pass a 375 px viewport E2E pass with no horizontal body scroll and all primary actions reachable. *AC:* Playwright mobile suite green.

---

9. Accessibility (WCAG 2.1 AA)

ID	Priority	Requirement	Acceptance criteria
TITAN-FRT-042	MUST	Full keyboard operability: every interactive element focusable in logical order; visible focus ring (`--ring` , 2 px offset); no traps; skip-to-content link.	Manual keyboard script per screen + axe automated run in CI (zero serious/critical).
TITAN-FRT-043	MUST	Correct semantics/ARIA: landmarks (`nav` , `main`), tables as tables, live regions — activity feed and toasts `aria-live="polite"` , critical alerts `aria-live="assertive"` ; Kanban DnD announces pick-up/over/drop via `aria-live` (dnd-kit announcements).	NVDA + VoiceOver scripted pass for Board, Conversations, Overview.
TITAN-FRT-044	MUST	Status is never color-only: every colored state pairs an icon/text (✓ ⚠ ✕ ◐ shapes as in wireframes).	Design review checklist + grayscale screenshot audit.
TITAN-FRT-045	MUST	`prefers-reduced-motion` disables non-essential animation (card fly, pulses, streaming caret blink); realtime data still updates instantly.	E2E with emulated preference asserts no CSS animations run.
TITAN-FRT-046	MUST	Contrast per TITAN-FRT-012 in both themes; text resizable to 200 % without loss of function.	Browser zoom 200 % E2E on all screens.
TITAN-FRT-047	SHOULD	Graph/Architecture canvases provide an accessible alternative: a structured list view of nodes/edges toggleable via “List view”.	Toggle exists; list exposes same data and actions.

---

10. Keyboard Shortcuts

Global (never override browser/system defaults; disabled inside text inputs except `Esc` / `⌘Enter` ; discoverable via `?` overlay):

Shortcut	Action
`Ctrl/⌘ + K`	Command palette
`Ctrl/⌘ + P`	Project switcher
`?`	Shortcut help overlay
`g` then `o` / `p` / `b` / `g` / `a` / `c` / `l` / `d` / `m`	Go to Org / Projects / Board / Graph / Agents / Conversations / Logs / Deployments / Monitoring (Gmail-style sequences)

Shortcut	Action
`Esc`	Close dialog/sheet/palette (top-most first)
`Ctrl/⌘ + Enter`	Send message / submit form
`n`	New (contextual: project, message, MCP server)
`/`	Focus the screen's primary search/filter

Screen-scoped: Board — arrows navigate cards, `Enter` opens, `Space` lifts card then arrows move / `Space` drops / `Esc` cancels (dnd-kit keyboard sensor); Graph — `+` / `-` zoom, `0` fit, arrows pan, `Tab` cycles nodes; Logs — `Space` pause tail, `e` jump to next ERROR; Conversations — `Alt+↑/↓` switch channels.

TITAN-FRT-048 (MUST). All listed shortcuts SHALL work and appear in the `?` overlay; sequences time out after 1.5 s. *AC:* E2E per shortcut; overlay snapshot matches table.

---

11. Frontend Folder Structure

Located at `apps/web` in the monorepo (structure canonical per Volume 1; shared packages under `packages/`).

```
apps/web/
├── app/                              # Next.js 15 App Router
│   ├── layout.tsx                    # Root: theme script, providers, shell
│   ├── (dashboard)/                  # Auth-gated group with app shell
│   │   ├── org/page.tsx
│   │   ├── projects/
│   │   │   ├── page.tsx
│   │   │   └── [pid]/
│   │   │       ├── page.tsx          # Overview tab
│   │   │       ├── board/page.tsx
│   │   │       ├── graph/page.tsx
│   │   │       ├── architecture/page.tsx
│   │   │       └── deployments/page.tsx
│   │   ├── agents/
│   │   │   ├── page.tsx
│   │   │   └── @drawer/(.)agents/[agentId]/page.tsx   # intercepted drawer
│   │   ├── conversations/[[...channelId]]/page.tsx
│   │   ├── logs/page.tsx
│   │   ├── deployments/page.tsx
│   │   ├── monitoring/page.tsx
│   │   ├── memory/page.tsx
│   │   ├── plugins/page.tsx
│   │   └── settings/{profile,appearance,providers,members,api-keys,notifications,budgets}/page.tsx
│   ├── (auth)/login/page.tsx         # OIDC redirect flow
│   └── api/                          # BFF-only route handlers (auth callback, WS ticket)
├── components/
│   ├── ui/                           # vendored shadcn/ui primitives
│   ├── shell/                        # Sidebar, TopBar, CommandPalette, ConnectionPill
│   ├── kanban/                       # Board, Column, TaskCard, TaskSheet
│   ├── graph/                        # FlowCanvas, TaskNode, ArchNode, MiniMap, ListViewAlt
│   ├── agents/                       # AgentCard, AgentGrid, AgentDrawer
│   ├── conversations/                # ChannelList, MessageFeed, TypedChip, StreamBubble
│   ├── logs/                         # LogList, LogFilterBar, LogLine
│   ├── deployments/                  # EnvCard, PipelineRun, StageChip
│   ├── monitoring/                   # StatTile, TimeSeries, AlertList
│   ├── memory/                       # TierTree, ResultList, ProvenancePanel
│   ├── plugins/                      # PluginCard, SchemaForm, McpServerForm
│   └── notifications/                # Bell, Center, ToastHost, AttentionQueue
├── lib/
│   ├── ws/                           # socket client, topic registry, seq reconciliation
│   ├── query/                        # QueryClient, key factory, hydration helpers
│   ├── auth/                         # JWT/session helpers, RBAC guards (useCan)
│   ├── shortcuts/                    # sequence engine, registry, ?-overlay data
│   └── utils/
├── hooks/                            # useTopic, useAgentStatus, useTailBuffer, …
├── stores/                           # Zustand: shell.ts, palette.ts, toasts.ts
├── styles/globals.css                # token variables (dark default + light)
├── e2e/                              # Playwright suites incl. a11y + perf budgets
├── tailwind.config.ts
└── next.config.ts

packages/
├── api-client/                       # OpenAPI-generated types + typed fetch
└── ws-protocol/                      # shared envelope/topic/event TS types
```

TITAN-FRT-049 (MUST). Feature code SHALL live in its feature directory; cross-feature imports go through `lib/` or `packages/` (enforced by ESLint import boundaries). *AC:* boundary lint passes; violation fails CI.

---

12. Decision Records

ADR-FRT-01 — RSC-first pages with React Query for interactive state

Context. Screens mix static-ish content (settings, project metadata) with highly dynamic content (boards, feeds). **Decision.** Render first paint via Server Components; hydrate interactive islands with React Query using

RSC-provided `initialData`; WS layer keeps caches fresh. **Alternatives.** (a) Full client SPA + React Query only — slower first paint, larger bundles, weaker SEO-less-relevant but worse TTI; (b) RSC + server actions only — no mature story for realtime cache patching; (c) Redux Toolkit Query — more boilerplate, weaker cache-invalidation ergonomics for our event-driven model. **Trade-offs.** Two data paths (RSC fetch + client query) must share the typed client to avoid drift; mitigated by `packages/api-client` and hydration helpers.

ADR-FRT-02 — Vendored shadcn/ui over a packaged component library

Context. Need accessible primitives, deep theming, and long-term control in an open-source repo. **Decision.** Vendor shadcn/ui (Radix-based) into `components/ui` and treat as first-party code. **Alternatives.** MUI (heavy runtime, harder deep theming), Chakra (runtime CSS-in-JS cost), Mantine (own token system conflicts with Tailwind), headless-only Radix (rebuild everything shadcn already gives). **Trade-offs.** We own upgrades/bugfixes for vendored code; accepted — components are reviewed like normal source, and upstream diffs are applied deliberately.

ADR-FRT-03 — Single multiplexed WebSocket with topic subscriptions

Context. A dozen surfaces need realtime data; browsers and the FastAPI gateway both suffer under per-widget sockets. **Decision.** One socket per tab; JSON envelope with `topic/event/seq`; subscribe on mount, unsubscribe on unmount; gap-triggered refetch for reconciliation. **Alternatives.** Socket per feature (connection storms, N×auth); SSE (one-way only — we need client sends for subs and chat); polling (violates TITAN-FRT-003 latency and wastes local-LLM-laptop resources). **Trade-offs.** Client-side router/registry complexity and head-of-line risk on one socket; mitigated by batching (TITAN-FRT-021) and keeping bulk history on REST.

ADR-FRT-04 — React Flow (@xyflow/react) + dagre for both graphs

Context. Task DAG and Architecture Viewer both need interactive node-edge canvases with custom nodes. **Decision.** One shared graph engine (`components/graph`) on React Flow with dagre layout; custom node types per domain. **Alternatives.** D3 from scratch (months of interaction work), Cytoscape.js (styling model fights React), Mermaid render (static, no interaction), commercial libs (violates open-source/free requirement). **Trade-offs.** React Flow renders nodes as DOM (not canvas), capping practical size — addressed by clustering beyond 500 nodes (TITAN-FRT-026 guard).

ADR-FRT-05 — Virtualized ring-buffer log viewer, not pagination

Context. Agents emit bursty, high-volume logs; users need grep-like tailing. **Decision.** TanStack Virtual list over an in-memory ring buffer (50k lines) with rAF-batched appends; history via REST (Loki) on demand. **Alternatives.** Paginated table (kills tailing UX), xterm.js terminal (great for raw streams, poor for structured filtering/expansion), server-rendered chunks (latency). **Trade-offs.** Client memory bounded but filters only search the buffer; full-history search delegates to the backend search endpoint, which the UI exposes when a filter is active with tail paused.

ADR-FRT-06 — Dark default via CSS variables + `class` strategy

Context. Product requirement: dark mode default, professional; light mode supported; no theme flash. **Decision.** HSL CSS variables per semantic token; `dark` class on `<html>` by default; blocking inline script applies persisted choice pre-paint; both themes AA-validated. **Alternatives.** `prefers-color-scheme` only

(violates "dark default" for light-OS users), duplicate Tailwind palettes (drift), runtime theming lib (unneeded weight). **Trade-offs.** Inline script is a tiny render-blocking cost (< 1 ms) — accepted to guarantee no flash.

---

13. Risks & Open Questions

#	Risk / Question	Impact	Mitigation / owner
R-1	WS fan-out volume (logs + token streams + metrics) may saturate a single laptop in local mode.	UI jank, dropped events	Server-side per-topic rate caps and batch frames (Volume 3); client rAF batching (TITAN-FRT-021); load fixtures in CI.
R-2	React Flow DOM rendering may not scale for very large generated systems (> 500 nodes).	Graph unusable on big projects	Clustering + drill-down is specified; if insufficient, evaluate canvas renderer behind the same `components/graph` API.
R-3	Conversational intake (§6.2) quality depends on CEO Agent latency; slow local models could make first-run UX feel broken.	Onboarding abandonment	Streaming UI with progress narration; skeleton roadmap placeholder; document expected local-model latency in onboarding copy.
R-4	Optimistic Kanban updates vs. agent-initiated moves can conflict in bursts.	Flicker, mistrust	Server-wins + seq reconciliation specified (TITAN-FRT-024); monitor conflict rate metric; consider column-level locks if > 1 % of moves conflict.
R-5	Open question: should human free-text messages in Conversations be able to *directly* reassign tasks, or only signal the PM Agent?	Authority-model ambiguity	Volume 2 (agent authority) + Volume 6 (workflow) must rule; UI ships signal-only until then, which is forward-compatible.
R-6	Open question: multi-organization tenancy in the UI (org switcher) — MVP assumes a single org per deployment.	IA rework if multi-tenant later	Route map already roots data under org IDs server-side; sidebar reserves a slot for an org switcher; decision deferred to Volume 10 roadmap.
R-7	Web Push for approvals requires a push service; strictly offline deployments cannot use it.	Missed approvals when tab closed	Push is opt-in enhancement only; attention queue + external channel plugins (Slack) remain primary; document limitation.

---

14. Cross-References

- **Volume 1 — Foundation:** canonical stack decisions, monorepo layout, personas, success metrics this UI serves.
- **Volume 2 — Multi-Agent Intelligence:** agent roles/states shown in the Agent Grid; structured message types rendered in Conversations; agent authority for R-5.
- **Volume 3 — Backend Engineering:** REST + WebSocket contracts, topic/event schemas, auth (OAuth2/OIDC + JWT) and RBAC roles consumed by `useCan`.
- **Volume 5 — AI Brain (Memory, Knowledge & RAG):** memory tiers, provenance, and semantic search behind the Memory Explorer.
- **Volume 6 — Engineering Workflow:** Kanban column semantics, approval gates surfaced in the Attention Queue and Deployment Dashboard.
- **Volume 7 — Integrations & Tool Ecosystem:** plugin catalog, permission model, and MCP server integration behind the Plugin Manager.

- **Volume 8 — Security:** secret handling (write-only fields), audit-logged UI actions, session management in Settings → Profile.
- **Volume 9 — Deployment & Production:** Prometheus/Grafana/Loki datasources behind Monitoring and Logs; pipeline stages shown in the Deployment Dashboard.
- **Volume 10 — Autonomous Evolution:** plugin marketplace evolution, custom-agent creation UI, and multi-org tenancy (R-6) roadmap.

End of Volume 4 — TITAN-VOL-04, Draft v1.0.

PROJECT TITAN — Autonomous Software Engineering Platform

Volume 5 — AI Brain: Memory, Knowledge & RAG

Field	Value
Series	PROJECT TITAN — Autonomous Software Engineering Platform Specification
Volume	5 of 10 — AI Brain: Memory, Knowledge & RAG
Doc ID	TITAN-VOL-05
Status	Draft v1.0
Date	2026-07-04
Requirement prefix	TITAN-BRN-NNN

Abstract. This volume specifies the cognitive architecture of TITAN: the memory subsystem that lets an organization of AI agents remember, learn, and improve across tasks and projects. It defines the memory taxonomy (short-term/working, long-term, episodic, semantic, procedural; project vs organization scope), the exact mapping of each memory class onto PostgreSQL 16, Qdrant, and Redis 7, the embedding and retrieval pipelines (chunking, hybrid search, reranking, context assembly), a knowledge graph of entities and how it is maintained, context-window budget management with compression and summarization, pruning and decay policies, the Knowledge Base of coding standards and reusable patterns, cross-project learning, and the reflection loops that write experience back into memory. All designs run fully local via Docker Compose with free/open-source components (local embeddings via Ollama), consistent with the canonical stack fixed in Volume 1.

Table of Contents

- 1. Introduction & Scope
- 2. Cognitive Architecture Overview
- 3. Memory Taxonomy
- 4. Storage Mapping: PostgreSQL vs Qdrant vs Redis
- 5. Embedding Pipeline
- 6. Retrieval Pipeline
- 7. Knowledge Graph of Entities
- 8. Context-Window Budget Management & Compression
- 9. Memory Pruning & Decay Policies
- 10. The Knowledge Base
- 11. Learning From Previous Projects
- 12. Reflection Loops Feeding Memory
- 13. Memory Flow Around One Agent Invocation

- 14. Memory Service API Surface
- 15. Risks & Open Questions
- 16. Cross-References

---

1. Introduction & Scope

1.1 Purpose

TITAN is an organization of agents, not one giant model. An organization without institutional memory re-solves the same problems forever. This volume defines the **Memory Service** — a single backend module (Python 3.12 / FastAPI / SQLAlchemy / Pydantic v2, per the canonical stack) that owns all durable cognition: what agents remember inside a task, across a project, and across the whole organization. The Memory Agent defined in Volume 2 is the sole steward of this service; every other agent reads and writes memory only through the APIs specified here.

1.2 In scope

Memory taxonomy and semantics; physical storage mapping; embedding and retrieval (RAG) pipelines; the knowledge graph; context budgeting, compression, and summarization; pruning and decay; the Knowledge Base; cross-project learning; reflection-driven memory writes; data schemas and flow diagrams.

1.3 Out of scope (owned elsewhere)

Agent role definitions and prompt templates (Volume 2); the message bus and task orchestration (Volumes 3 and 6); UI for the Memory Explorer (Volume 4); the LLM gateway internals (Volume 3); security controls on stored data (Volume 8); self-evolution metrics that consume memory (Volume 10).

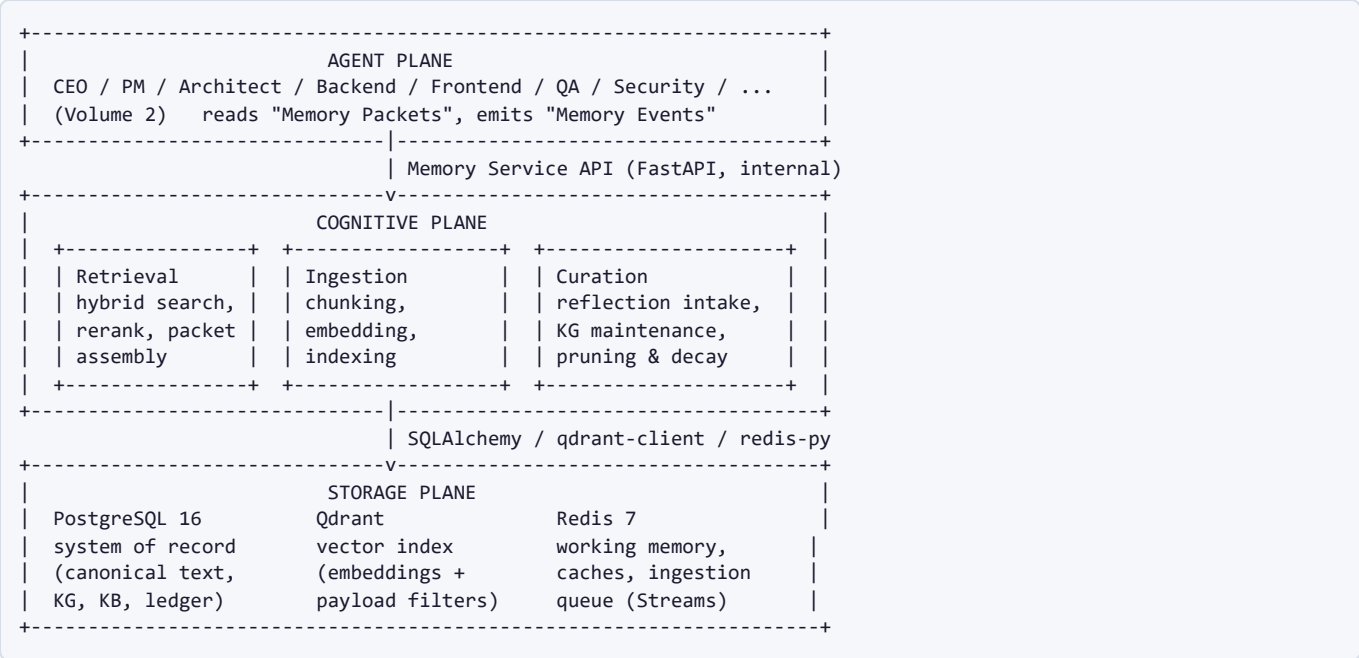
1.4 Canonical constraints (fixed in Volume 1, referenced here)

PostgreSQL 16 is the system of record; Redis 7 provides cache, queues (Redis Streams at MVP), and pub/sub; Qdrant is the vector database; LLM and embedding access go through the provider-agnostic gateway (Anthropic, OpenAI, Google, Ollama), switchable by configuration only; everything must run on one laptop under Docker Compose with free/local tools; production targets Kubernetes; observability via OpenTelemetry/Prometheus/Grafana/Loki.

---

2. Cognitive Architecture Overview

The AI Brain is organized as three planes:



Two invariants govern everything below:

1. **PostgreSQL is canonical; Qdrant and Redis are derived.** Any vector or cache entry can be rebuilt from Postgres. Losing Qdrant or Redis loses no knowledge, only speed (see ADR-05-01).
2. **Agents never touch storage directly.** All reads return a typed *Memory Packet*; all writes are *Memory Events* validated by Pydantic v2 schemas. This keeps memory auditable ("every communication stored, nothing lost" — Volume 1 principle) and lets curation policies evolve without touching agent code.

---

3. Memory Taxonomy

TITAN distinguishes memory by **duration**, **kind**, and **scope**.

3.1 Definitions

Memory class	Contains	Typical lifetime	Example
Short-term / working	The live context of one task attempt: current messages, scratch notes, tool outputs, partial plans	Minutes–hours; ends with the task attempt	Backend Agent's in-flight reasoning while implementing `POST /auth/login`
Long-term	Anything promoted for durable reuse; umbrella for episodic + semantic + procedural below	Months–forever (subject to decay, §9)	"Project Nova uses cursor pagination everywhere"
Episodic	Time-stamped records of *what happened*: task transcripts, decisions taken, review outcomes, incidents, retrospectives	Long, with decay	"2026-07-02: QA rejected PR #41 — flaky test caused by shared fixture"
Semantic	Distilled *facts and knowledge* independent of when learned: entity facts, API knowledge, architecture facts, glossary	Long, versioned	"Qdrant payload filtering supports keyword match and range conditions"
Procedural	*How to do things*: coding standards, runbooks, prompt recipes, workflow checklists, reusable templates	Long, versioned; the Knowledge Base (§10) is its curated core	"Checklist: adding a new Alembic migration safely"

3.2 Scope: project memory vs organization memory

Every long-term record carries a scope:

- **Project memory** (`scope = project:<uuid>`): true only within one project — its requirements, decisions, component inventory, conventions, open risks. Isolated per project so unrelated projects never contaminate each other's retrieval.
- **Organization memory** (`scope = org`): validated across projects — patterns, lessons learned, coding standards, template library, provider quirks. Records enter org scope only via the promotion workflow in §11.3; agents cannot write org memory directly.

Retrieval always searches project scope plus org scope, never other projects' scopes (enforced by mandatory Qdrant payload filters and SQL predicates — see Volume 8 for the security rationale).

3.3 Requirements

TITAN-BRN-001 (MUST) — The Memory Service SHALL classify every stored record with exactly one `kind` ∈ {episodic, semantic, procedural} and one `scope` ∈ {org, project:}, plus lifecycle metadata (created_at, last_accessed_at, access_count, decay_score). *Acceptance:* Inserting a record without kind/scope fails Pydantic validation with HTTP 422; a database CHECK constraint rejects invalid enum values.

TITAN-BRN-002 (MUST) — Working memory SHALL live only in Redis under task-scoped keys and SHALL never be the sole copy of a decision, artifact, or lesson; anything durable must be written through a Memory Event before task completion. *Acceptance:* Flushing Redis between two tasks does not change the outcome of a replayed retrieval-only integration test; the task-completion handler rejects completion if pending Memory Events remain unflushed.

TITAN-BRN-003 (MUST) — Retrieval SHALL be constrained to the requesting task's project scope plus org scope. *Acceptance:* An integration test seeding two projects with distinctive markers shows zero cross-project leakage over 100 varied queries.

---

4. Storage Mapping: PostgreSQL vs Qdrant vs Redis

4.1 Placement matrix

Data	PostgreSQL 16	Qdrant	Redis 7
Canonical memory records (full text, metadata)	✓ source of truth	—	—
Embeddings of chunks	chunk rows (no vectors)	✓ vectors + payload	—
Knowledge graph nodes/edges	✓	node label embeddings (for fuzzy entity lookup)	—
Knowledge Base documents & versions	✓	✓ chunk vectors	hot-doc cache
Working memory (scratchpads, running summaries)	—	—	✓ `wm:{task_id}:*` , TTL
Embedding/ingestion queue	outbox table	—	✓ Stream `titan:memory:ingest`

Data	PostgreSQL 16	Qdrant	Redis 7
Retrieval cache (query → packet)	—	—	✓ TTL 300 s
Access/audit ledger of memory reads & writes	✓ append-only	—	—
Rolling conversation summaries (checkpointed)	✓	—	✓ live copy

4.2 ADR-05-01 — PostgreSQL as canonical store; Qdrant strictly a derived index

- **Context.** Memory must survive years and be auditable; vectors must be re-computable when the embedding model changes; local-first operation requires simple backup/restore.
- **Decision.** All memory text and metadata are written to PostgreSQL first, inside the same transaction as an outbox row; an ingestion worker consumes the outbox via Redis Streams and upserts vectors into Qdrant. Qdrant point IDs equal the Postgres `chunk.id` (UUID), making reconciliation a set difference.
- **Alternatives considered.** (a) Qdrant as primary store for memories — rejected: weak transactional guarantees, no relational joins for the knowledge graph and ledger. (b) pgvector only, no Qdrant — attractive for simplicity, but rejected at MVP scale ceiling: Qdrant's HNSW performance, payload-filtered search, and named-vector support are needed for the hybrid pipeline in §6, and it is fixed canon from Volume 1. (c) Dual-write from the API process — rejected: partial-failure windows; the outbox pattern gives at-least-once delivery with idempotent upserts.
- **Trade-offs.** Slight indexing latency (target p95 < 5 s from write to searchable, TITAN-BRN-007); operational cost of one more container — acceptable, Qdrant runs in ~150 MB RAM locally.

4.3 PostgreSQL schema (memory core)

```
-- schema: brain
CREATE TABLE brain.memory_records (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    kind            TEXT NOT NULL CHECK (kind IN ('episodic','semantic','procedural')),
    scope           TEXT NOT NULL,              -- 'org' | 'project:<uuid>'
    project_id      UUID REFERENCES core.projects(id),
    source_agent    TEXT NOT NULL,              -- e.g. 'qa_agent'
    source_task_id  UUID REFERENCES core.tasks(id),
    title           TEXT NOT NULL,
    body_md         TEXT NOT NULL,              -- canonical content, Markdown
    metadata        JSONB NOT NULL DEFAULT '{}',
    importance      REAL NOT NULL DEFAULT 0.5,  -- 0..1, set at write, updated by reflection
    decay_score     REAL NOT NULL DEFAULT 1.0,  -- recomputed nightly (§9)
    access_count    INTEGER NOT NULL DEFAULT 0,
    last_accessed_at TIMESTAMPTZ,
    status          TEXT NOT NULL DEFAULT 'active'
                    CHECK (status IN ('active','archived','superseded','deleted')),
    superseded_by   UUID REFERENCES brain.memory_records(id),
    created_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
    tsv             TSVECTOR GENERATED ALWAYS AS
                    (to_tsvector('english', title || ' ' || body_md)) STORED
);
CREATE INDEX ON brain.memory_records USING GIN (tsv);          -- lexical leg of hybrid search
CREATE INDEX ON brain.memory_records (scope, kind, status);

CREATE TABLE brain.memory_chunks (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),    -- == Qdrant point id
    record_id   UUID NOT NULL REFERENCES brain.memory_records(id) ON DELETE CASCADE,
    seq         INTEGER NOT NULL,
    text        TEXT NOT NULL,
    token_count INTEGER NOT NULL,
    embed_model TEXT,                                          -- e.g. 'nomic-embed-text-v1.5'
    embedded_at TIMESTAMPTZ,
    UNIQUE (record_id, seq)
);

CREATE TABLE brain.ingest_outbox (
    id          BIGSERIAL PRIMARY KEY,
    chunk_id    UUID NOT NULL REFERENCES brain.memory_chunks(id),
    op          TEXT NOT NULL CHECK (op IN ('upsert','delete')),
    enqueued_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    processed_at TIMESTAMPTZ
);

CREATE TABLE brain.memory_ledger (                             -- append-only audit
    id          BIGSERIAL PRIMARY KEY,
    at          TIMESTAMPTZ NOT NULL DEFAULT now(),
    actor_agent TEXT NOT NULL,
    task_id     UUID,
    action      TEXT NOT NULL CHECK (action IN ('read','write','promote','prune','supersede')),
    record_ids  UUID[] NOT NULL,
    detail      JSONB NOT NULL DEFAULT '{}'
);
```

4.4 Qdrant collections

Collection	Vector	Payload (indexed)	Purpose
`titan_memory`	768-d cosine (named vector `dense`)	scope, kind, project_id, record_id, source_agent, created_at, tags[], decay_score	All long-term memory chunks
`titan_kb`	768-d cosine	doc_type, version, scope, tags[], language	Knowledge Base chunks (§10)
`titan_kg_labels`	768-d cosine	node_type, scope	Fuzzy entity resolution for the knowledge graph (§7)

Collections are created with `on_disk_payload: true` and HNSW `m=16, ef_construct=128` — comfortable defaults for laptop-scale (≤ 5M points) with sub-50 ms search.

4.5 Redis keyspace (working memory & pipeline)

Key pattern	Type	TTL	Content
`wm:{task_id}:scratch`	HASH	24 h	Agent scratchpad fields (plan, notes, tool-state)
`wm:{task_id}:summary`	STRING	24 h	Current rolling summary (also checkpointed to PG)
`wm:{task_id}:events`	LIST	24 h	Pending Memory Events awaiting flush
`titan:memory:ingest`	STREAM	—	Chunk ingestion queue (consumer group `embedders`)
`rc:{scope}:{query_hash}`	STRING	300 s	Cached Memory Packet (JSON)

TITAN-BRN-004 (MUST) — A `rebuild-index` admin command SHALL fully reconstruct all Qdrant collections from PostgreSQL. *Acceptance:* After deleting the Qdrant volume, running `titan memory rebuild-index` restores search results equivalent to pre-deletion on a golden query set (recall@10 ≥ 0.99 vs. baseline).

TITAN-BRN-005 (MUST) — Every memory read and write SHALL append a `memory_ledger` row identifying actor agent, task, action, and affected records. *Acceptance:* The Memory Explorer (Volume 4) can display a complete access trail for any record; a test performing N reads/writes finds exactly N ledger rows.

---

5. Embedding Pipeline

5.1 Flow

```
Memory Event (write) --TX--> memory_records + memory_chunks + ingest_outbox
                                        |
                     outbox relay (async loop, 1 s poll)
                                        v
                      Redis Stream titan:memory:ingest
                                        |
                  consumer group "embedders" (1..N workers)
                                        v
        LLM Gateway /embeddings (Ollama local by default)
                                        v
             Qdrant upsert (point id = chunk id, payload from record)
                                        v
             mark chunk.embedded_at, outbox.processed_at
```

Workers are idempotent (upsert by chunk UUID), acknowledge only after Qdrant confirms, and use the Streams pending-entries list for crash recovery. Failures retry with exponential backoff (max 5, then dead-letter stream `titan:memory:ingest:dlq` with an alert via Prometheus).

5.2 Chunking strategy

Chunking is content-type aware; the writer declares `content_type` in the Memory Event.

Content type	Splitter	Target size	Overlap
Markdown / prose (decisions, lessons, docs)	Heading-aware recursive splitter (split at `##` / `###`, then paragraphs)	512 tokens	64 tokens

Content type	Splitter	Target size	Overlap
Source code	Syntax-aware via tree-sitter: whole functions/classes; oversize bodies split at statement boundaries	400 tokens	0 (unit boundaries)
Conversations / transcripts	Turn-grouped: consecutive turns until budget	512 tokens	1 turn
Tables / schemas	Never split a row/DDL statement; group rows	384 tokens	header repeated per chunk

Every chunk is embedded with a contextual header prefix (`[{scope} | {kind} | {record title} | chunk {seq}/{n}]`) so isolated chunks remain interpretable to the encoder and reranker.

5.3 ADR-05-02 — Embedding model: local-first `nomic-embed-text-v1.5` via Ollama, provider-swappable

- **Context.** Local-first and free-tier-friendly are hard requirements; embeddings are the highest-volume LLM-adjacent workload; changing models later must not require code changes.
- **Decision.** Default embedding model is **nomic-embed-text-v1.5** (768-d, 8192-token context, Apache-2.0) served by Ollama and called through the provider-agnostic gateway's `/embeddings` route. Cloud alternatives (OpenAI `text-embedding-3-small`, Google `gemini-embedding-001`, Voyage via the Anthropic partnership) are selected purely by gateway configuration. The active model name is stamped on every chunk (`embed_model`); collections are dimension-locked, so a model change triggers a background re-embed into a new collection followed by an alias swap.
- **Alternatives considered.** `bge-m3` (multilingual, heavier, 1024-d — SHOULD-level option for non-English orgs); `all-MiniLM-L6-v2` (tiny and fast but 256-token context is too small for code chunks); cloud-only embeddings (rejected: violates local-first).
- **Trade-offs.** Local embedding throughput on a laptop (~200–800 chunks/min CPU) bounds ingestion speed — acceptable because ingestion is asynchronous; slightly lower retrieval quality than the best cloud embedders — mitigated by hybrid search and reranking (§6).

5.4 Requirements

TITAN-BRN-006 (MUST) — Embeddings SHALL be requested only through the LLM gateway; switching embedding provider/model SHALL require configuration changes only. *Acceptance:* Changing `EMBEDDINGS_PROVIDER=ollama` to `openai` in `.env` and running the re-embed job requires zero code edits; grep shows no direct provider SDK imports in the Memory Service.

TITAN-BRN-007 (MUST) — p95 time from Memory Event commit to the chunk being searchable in Qdrant SHALL be ≤ 5 s under nominal local load (≤ 10 events/s). *Acceptance:* Load test asserts the ingestion-lag histogram (exported via OpenTelemetry → Prometheus) meets the target.

TITAN-BRN-008 (SHOULD) — Chunking SHALL be syntax-aware for the languages TITAN generates (TypeScript, Python, SQL at minimum), never splitting inside a function signature or DDL statement. *Acceptance:* Golden-file tests over representative sources show zero mid-signature splits.

---

6. Retrieval Pipeline

6.1 Stages

Every retrieval request (`query` , `scope` , `kinds[]` , `k` , `budget_tokens`) runs:

1. **Query shaping.** The caller's raw need (usually the task description plus the agent's stated information need) is normalized; for multi-part needs the Memory Service issues up to 3 sub-queries.
2. **Hybrid candidate generation** — run in parallel: - *Dense:* Qdrant search on `titan_memory` (and `titan_kb` when `kinds` includes procedural), payload-filtered by scope/kind/status, top-40 per sub-query. - *Lexical:* PostgreSQL full-text search on `memory_records.tsv` (`websearch_to_tsquery` , `ts_rank_cd`), top-40. This leg catches exact identifiers — error codes, function names, ticket IDs — that dense vectors miss.
3. **Fusion.** Reciprocal Rank Fusion: `score(d) = Σ 1/(60 + rank_i(d))` across all lists; deduplicate by `record_id` , keep top-24 chunks.
4. **Reranking.** Cross-encoder **bge-reranker-v2-m3** (open weights) scores (query, chunk) pairs; runs in-process in the Memory Service via `sentence-transformers` (CPU, ONNX int8 — ~30 ms per pair batch locally). Fallback when disabled (`RERANKER=off` for low-RAM laptops): LLM listwise rerank through the gateway using the cheapest configured model. Keep top-N until 85 % of `budget_tokens` is filled.
5. **Freshness/importance adjustment.** Final ordering key: `0.7·rerank_norm + 0.2·decay_score + 0.1·importance` — recent, important memories win ties.
6. **Packet assembly** (§6.2).

6.2 The Memory Packet

Retrieval returns a typed structure the orchestrator splices into the agent prompt (prompt template slots are defined in Volume 2):

```
{
  "packet_id": "9f3c…",
  "assembled_for": {"agent": "backend_agent", "task_id": "…", "call_seq": 3},
  "sections": {
    "identity_context":   "…project one-pager + active conventions (always included)…",
    "procedural":         ["KB: FastAPI service checklist v4 (chunk 2/5)", "…"],
    "semantic":           ["Component map: auth-service owns /auth/*", "…"],
    "episodic":           ["2026-07-02 QA rejection of PR #41: …"],
    "working_summary":    "…rolling summary of this task so far…"
  },
  "citations": [{"record_id": "…", "chunk_id": "…", "score": 0.83}],
  "token_cost": 3480,
  "budget_tokens": 4000
}
```

Rules: (a) every included chunk carries its `record_id` so agents can cite memory in decisions and reviewers can audit provenance; (b) sections are ordered procedural → semantic → episodic because standards must dominate anecdotes when they conflict; (c) `identity_context` (project one-pager, §8.2) is non-evictable.

6.3 ADR-05-03 — Hybrid RRF + cross-encoder rerank instead of dense-only retrieval

- **Context.** Software-engineering memory is dense with exact symbols (`ERR_CONN_RESET` , `useAuthStore` , `TITAN-BRN-007`) where embedding similarity underperforms; local embedding models amplify this gap.
- **Decision.** Always fuse dense (Qdrant) and lexical (Postgres FTS) candidates with RRF, then rerank with a local cross-encoder.

- **Alternatives considered.** Dense-only (simplest, misses exact matches); Qdrant sparse vectors/BM42 (viable, but Postgres FTS is already free with the canonical stack and keeps the lexical leg transactionally fresh); dedicated search engine e.g. OpenSearch (rejected at MVP: one more stateful service on the laptop).
- **Trade-offs.** Two query legs add ~20–40 ms and code complexity; reranker costs ~200–400 MB RAM — mitigated by the `RERANKER=off` fallback.

6.4 Requirements

TITAN-BRN-009 (MUST) — Every agent LLM call SHALL receive its context through a Memory Packet assembled by the retrieval pipeline; ad-hoc raw dumps of history into prompts are prohibited. *Acceptance:* Orchestrator code review + an integration assertion that each `agent_call` span (OpenTelemetry) links to exactly one `packet_id`.

TITAN-BRN-010 (MUST) — Retrieval SHALL meet p95 ≤ 800 ms end-to-end locally (query → packet) with reranker on, ≤ 400 ms with it off. *Acceptance:* Benchmark suite over a 100k-chunk corpus on the reference laptop profile passes both targets.

TITAN-BRN-011 (MUST) — A retrieval evaluation harness with a versioned golden dataset (≥ 200 query→relevant-record pairs) SHALL run in CI, reporting recall@10 and nDCG@10; regressions > 5 % fail the build. *Acceptance:* CI job exists, produces the two metrics, and a deliberately broken fusion weight fails it.

TITAN-BRN-012 (SHOULD) — Identical (scope, query) retrievals within 300 s SHALL be served from the Redis packet cache. *Acceptance:* Second identical call returns in < 20 ms and increments a `cache_hit` counter.

---

7. Knowledge Graph of Entities

7.1 Purpose and model

Vectors answer “what is similar”; the graph answers “what is connected”: which decision constrains which component, which pattern was applied in which projects, which lesson was learned from which incident. TITAN stores the graph relationally in PostgreSQL (ADR-05-04) with typed nodes and edges:

Node types: `project`, `component` (service/module/package), `decision` (ADR), `pattern` (reusable architecture/code pattern), `technology`, `agent`, `person` (human client/reviewer), `incident`, `lesson`, `kb_doc`.

Edge types: `PART_OF`, `DEPENDS_ON`, `DECIDED_FOR` (decision→project/component), `SUPERSEDES`, `APPLIES` (project/component→pattern), `DERIVED_FROM` (lesson→incident/task), `CONSTRAINS` (decision→component), `AUTHORED_BY`, `USES` (component→technology), `DOCUMENTED_IN`.

```
CREATE TABLE brain.kg_nodes (
    id         UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    node_type  TEXT NOT NULL,
    scope      TEXT NOT NULL,                  -- 'org' | 'project:<uuid>'
    name       TEXT NOT NULL,
    properties JSONB NOT NULL DEFAULT '{}',
    record_id  UUID REFERENCES brain.memory_records(id),  -- backing prose, if any
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    UNIQUE (scope, node_type, name)
);
CREATE TABLE brain.kg_edges (
    id         UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    src        UUID NOT NULL REFERENCES brain.kg_nodes(id) ON DELETE CASCADE,
    dst        UUID NOT NULL REFERENCES brain.kg_nodes(id) ON DELETE CASCADE,
    edge_type  TEXT NOT NULL,
    properties JSONB NOT NULL DEFAULT '{}',
    valid_from TIMESTAMPTZ NOT NULL DEFAULT now(),
    valid_to   TIMESTAMPTZ,                    -- non-null = historical edge
    UNIQUE (src, dst, edge_type, valid_from)
);
CREATE INDEX ON brain.kg_edges (src, edge_type);
CREATE INDEX ON brain.kg_edges (dst, edge_type);
```

Traversals use recursive CTEs capped at depth 3 (sufficient for "project → components → decisions → lessons" expansion during packet assembly).

7.2 ADR-05-04 — Relational graph in PostgreSQL, no dedicated graph database at MVP

- **Context.** Graph queries here are shallow (depth ≤ 3), write volume is low, and the laptop constraint penalizes every extra stateful service.
- **Decision.** Model the graph as `kg_nodes` / `kg_edges` in PostgreSQL with recursive CTEs; expose typed traversal endpoints; embed node names in `titan_kg_labels` for fuzzy entity resolution.
- **Alternatives considered.** Neo4j Community (best traversal ergonomics; rejected: JVM footprint, licensing friction for managed scale-out, extra service); Apache AGE extension (promising, MAY be adopted later — the API layer hides the change); storing relations only as JSONB (rejected: no referential integrity, unqueryable).
- **Trade-offs.** Deep analytics (centrality, community detection) are awkward in SQL — deferred to Volume 10's evolution analytics, which can export the graph offline.

7.3 Graph maintenance

The graph is maintained by three mechanisms, all owned by the Memory Agent's curation loop:

1. **Structured writes.** Workflow events (Volume 6) carry entity references: an approved ADR creates a `decision` node plus `DECIDED_FOR` / `CONSTRAINS` edges; scaffolding a service creates a `component` node; a retrospective creates `lesson` nodes with `DERIVED_FROM` edges. These are deterministic — no LLM involved.
2. **Extraction pass.** For unstructured episodic records (long transcripts), a nightly LLM extraction job proposes entities/edges as *candidates* with confidence; candidates ≥ 0.8 confidence that resolve to existing nodes are auto-linked, others queue for Memory Agent review.
3. **Entity resolution & hygiene.** New node names are embedded and searched against `titan_kg_labels`; cosine ≥ 0.92 with same node_type/scope triggers merge-or-confirm. Superseded decisions get `valid_to` stamped and a `SUPERSEDES` edge — the graph keeps history rather than deleting it.

TITAN-BRN-013 (MUST) — Approving an ADR, creating a component, or filing a lesson SHALL synchronously create/update the corresponding graph nodes and edges in the same workflow transaction.

Acceptance: Workflow integration tests assert node/edge existence immediately after each event type.

TITAN-BRN-014 (SHOULD) — Entity resolution SHALL prevent duplicate nodes: importing the same component name twice in a scope yields one node. *Acceptance:* Duplicate-import test results in a single node with merged properties and a ledger entry.

---

8. Context-Window Budget Management & Compression

8.1 Token budgets per agent call

The orchestrator computes a **context budget** for every LLM call from the active model's window (known via the gateway's model registry). Default allocation for a working window `W` (e.g., W = 32k tokens for a local model; percentages hold for larger clouds models with absolute caps):

Slot	Share of W	Cap	Evictable?
System + agent role prompt (Volume 2)	8 %	3k	No
Identity context (project one-pager + conventions)	6 %	2k	No
Task instruction + acceptance criteria	8 %	3k	No
Memory Packet (procedural/semantic/episodic)	30 %	12k	Yes — retrieval trims to fit
Working summary of current task	10 %	4k	Compressed, not dropped
Recent verbatim turns / tool outputs	28 %	12k	Oldest evicted first
Response reserve	10 %	8k	—

8.2 Compression & summarization strategy

Three cooperating mechanisms:

1. **Rolling task summary.** After every agent turn, if verbatim history exceeds its slot, the oldest turns are folded into `wm:{task_id}:summary` by a cheap gateway model using a fixed schema: *Goal / Done so far / Open threads / Constraints discovered / Next step*. The summary is regenerated incrementally (previous summary + evicted turns), checkpointed to PostgreSQL every 5 folds so a Redis loss costs at most 5 turns of nuance.
2. **Hierarchical episode summaries.** On task completion, the transcript is distilled into a task summary (≤ 300 tokens); sprint completion distills task summaries into a sprint summary; project completion distills sprints into the **project one-pager** (≤ 800 tokens) that becomes the permanent `identity_context`. Each layer links down (`metadata.children[]`), so retrieval can zoom from one-pager to raw transcript when needed.
3. **Compression-on-retrieval.** If a reranked chunk set exceeds the packet budget, lowest-scoring chunks are replaced by their parent record's stored summary line rather than dropped silently, and the packet marks them `compressed: true` so the agent knows more detail is one query away.

TITAN-BRN-015 (MUST) — No LLM call SHALL be issued whose assembled prompt exceeds the model's context window minus the response reserve; the orchestrator SHALL compress or evict per §8.1 rather than fail. *Acceptance:* Fuzz test with adversarially long tool outputs produces zero context-overflow errors from the gateway across 1,000 calls.

TITAN-BRN-016 (MUST) — Rolling summaries SHALL follow the fixed five-field schema and be checkpointed to PostgreSQL at least every 5 folds. *Acceptance:* Schema-validation test on generated summaries; kill-Redis test loses ≤ 5 turns of summary detail.

TITAN-BRN-017 (SHOULD) — Summarization SHALL route to the cheapest configured gateway model class (`tier: utility`), never the premium reasoning tier. *Acceptance:* Gateway logs over a test run show zero utility-class summarization requests on premium models.

---

9. Memory Pruning & Decay Policies

9.1 Decay scoring

A nightly curation job recomputes for every active record:

```
decay_score = w_r · exp(-Δdays / H(kind)) + w_u · min(1, access_count / 20) + w_i · importance
   w_r = 0.5, w_u = 0.3, w_i = 0.2
   Half-life H: episodic = 30 days, semantic = 120 days, procedural = 365 days
   Δdays = days since last_accessed_at (or created_at if never accessed)
```

Reads refresh `last_accessed_at` and increment `access_count`; reflection loops (§12) may raise `importance`. Decay influences retrieval ordering (§6.1 step 5) long before anything is removed.

9.2 Pruning tiers

Tier	Trigger	Action	Reversible?
Demote	decay_score < 0.35	Excluded from default retrieval (`status` stays active, payload flag `demoted`) unless the query explicitly requests deep search	Yes, automatic on access
Archive	decay_score < 0.15 for 30 consecutive days	`status='archived'`; vectors deleted from Qdrant; text retained in PostgreSQL; excluded from all retrieval except Memory Explorer	Yes (re-embed on restore)
Compact	Archived episodic records > 180 days	Body replaced by its stored summary; full body moved to cold table `brain.memory_bodies_cold`	Partially
Delete	Only via retention policy (Volume 8) or explicit human/admin action	Hard delete + ledger entry	No

Protected classes are never auto-pruned: decision records, lessons learned, anything referenced by an active KG edge from an active project, and all Knowledge Base documents. Prune runs are dry-run-first: the job writes a plan to the ledger, and destructive tiers (Compact/Delete) require Memory Agent approval above 100 records per run.

TITAN-BRN-018 (MUST) — The nightly curation job SHALL recompute decay scores and apply Demote/Archive tiers automatically, logging every transition to the ledger. *Acceptance:* Seeding records with back-dated timestamps and running the job produces the expected tier transitions and ledger rows.

TITAN-BRN-019 (MUST) — Decision records, lessons, KB documents, and KG-referenced records SHALL be exempt from automatic pruning. *Acceptance:* Curation-job unit tests show protected records untouched at decay_score = 0.

TITAN-BRN-020 (SHOULD) — Accessing a demoted or archived record SHALL automatically restore it (re-embed if needed) and reset its decay clock. *Acceptance:* Deep-search access to an archived record flips status to active and its chunks reappear in Qdrant within the TITAN-BRN-007 latency budget.

---

10. The Knowledge Base

10.1 Content types

The Knowledge Base (KB) is the curated, versioned core of procedural + semantic org memory:

doc_type	Content	Primary writers	Primary readers
`coding_standard`	Language/framework rules (the "constitution", Volume 8 of the series governs content)	Human maintainers, Code Reviewer Agent proposals	Every engineer agent; Code Reviewer enforces
`architecture_pattern`	Named patterns with context/solution/consequences + reference implementation links	System Architect Agent	Architect, Backend, Frontend agents
`best_practice`	Focused guidance (e.g., "idempotent consumers on Redis Streams")	Any agent via proposal	All agents
`template`	Reusable scaffolds: service skeletons, Dockerfiles, CI configs, prompt fragments	DevOps/Backend/Frontend agents	Scaffolding steps in Volume 6 workflow
`decision_record`	Org-level ADRs and their history	Architect + CEO approval	All agents
`api_reference_note`	Distilled third-party API knowledge from the Research Agent	Research Agent	All agents

10.2 Storage & versioning

```
CREATE TABLE brain.kb_documents (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    doc_type    TEXT NOT NULL,
    slug        TEXT NOT NULL UNIQUE,          -- 'python-service-standard'
    title       TEXT NOT NULL,
    current_version INTEGER NOT NULL DEFAULT 1,
    status      TEXT NOT NULL DEFAULT 'active' CHECK (status IN ('active','deprecated')),
    tags        TEXT[] NOT NULL DEFAULT '{}'
);
CREATE TABLE brain.kb_versions (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    document_id UUID NOT NULL REFERENCES brain.kb_documents(id),
    version     INTEGER NOT NULL,
    body_md     TEXT NOT NULL,
    change_note TEXT NOT NULL,
    proposed_by TEXT NOT NULL,                 -- agent or human id
    approved_by TEXT,                          -- required before active
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
    UNIQUE (document_id, version)
);
```

Only the approved current version is chunked into `titan_kb` for retrieval; older versions remain queryable by the Memory Explorer.

10.3 How agents read and write the KB

Read path. KB retrieval is part of the standard pipeline (§6): any packet request with `kinds` including `procedural` searches `titan_kb`. Additionally, workflow steps can pin documents deterministically — e.g., the code-review step (Volume 6) always pins the relevant `coding_standard` docs by slug, bypassing similarity search, because standards must be present even when the query doesn't resemble them.

Write path — proposal workflow. Agents never mutate the KB directly:

1. An agent submits a *KB proposal* (new doc or new version) with a diff and rationale, typically emitted by a reflection loop (§12).
2. The Memory Agent validates format, checks for conflicts with existing docs (semantic search for near-duplicates), and routes it: `coding_standard` / `decision_record` require Code Reviewer Agent approval plus human sign-off at MVP; other types require Code Reviewer Agent approval only.
3. On approval, a new `kb_versions` row becomes current, chunks are re-embedded, and a KG node/edge update records provenance (`kb_doc` node, `AUTHORED_BY`, `DERIVED_FROM` lesson if applicable).

TITAN-BRN-021 (MUST) — KB mutations SHALL occur only through the proposal→approval workflow, with every version retaining proposer, approver, and change note. *Acceptance:* Direct-write API attempts return 403; the version history for any doc reconstructs its full provenance chain.

TITAN-BRN-022 (MUST) — Workflow steps SHALL be able to pin KB documents by slug into any Memory Packet, independent of similarity ranking. *Acceptance:* A packet requested with `pins: ["python-service-standard"]` always contains that doc's chunks, verified by test.

---

11. Learning From Previous Projects

11.1 Pattern extraction

At each milestone completion, the Memory Agent runs the **pattern miner**:

1. Collect the project's decision records, component graph, and code-review outcomes since the last run.
2. LLM pass proposes candidate patterns: recurring structures ("JWT refresh handled via Redis allow-list"), recurring fixes, recurring rejections.
3. Candidates are checked against org memory for novelty (semantic search, threshold 0.85); novel ones become `pattern` records in *project* scope with links to their evidence (KG edges `APPLIES`, `DERIVED_FROM`).

11.2 Lessons-learned records

Every incident, rejected review with systemic cause, failed deployment, or retrospective finding produces a structured lesson:

```
{
  "kind": "episodic", "record_subtype": "lesson",
  "title": "Shared pytest fixtures caused cross-test flakiness",
  "context": "Project Nova, sprint 3, QA regression run",
  "observation": "3 intermittent failures traced to a module-scoped DB fixture",
  "root_cause": "State leakage between tests via shared fixture",
  "recommendation": "Default to function-scoped fixtures; module scope requires justification",
  "evidence": ["task:…", "pr:41"],
  "applicability": "org-candidate",
  "importance": 0.8
}
```

11.3 Promotion: project memory → organization memory

A lesson or pattern is promoted to org scope when **any** of: (a) observed independently in ≥ 2 projects (the miner detects near-duplicates across project scopes); (b) flagged `org-candidate` and approved by the Memory Agent's weekly curation review; (c) a human marks it via the Memory Explorer. Promotion clones the record into org scope, links both via `SUPERSEDES`-style provenance edges, and — for procedural content — usually triggers a KB proposal (§10.3) so the lesson hardens into a standard or best practice. This is the pipeline by which TITAN's organization gets measurably better with every project.

TITAN-BRN-023 (MUST) — Milestone completion SHALL trigger pattern mining, and incidents/failed deployments SHALL each produce a structured lesson record within the same workflow run. *Acceptance:* Workflow tests assert a lesson exists after a simulated failed deployment; miner runs are visible in the ledger.

TITAN-BRN-024 (MUST) — Promotion to org scope SHALL require the multi-signal rule above; agents SHALL NOT write org-scope records directly. *Acceptance:* API rejects direct org-scope writes from agent credentials; a two-project duplicate lesson is auto-flagged for promotion in tests.

---

12. Reflection Loops Feeding Memory

Reflection is where the self-improvement loop (Implementation → Review → Critique → … , Volume 6) touches the brain. Three loops, increasing in scope:

1. **Task reflection (every task).** After the final review verdict, the acting agent answers a fixed rubric — *What was the goal? What worked? What surprised me? What would I do differently? What should the org remember?* — in ≤ 250 tokens. Output becomes an episodic record; the last field may spawn a lesson or KB proposal. Cost-bounded: utility model tier, one call.
2. **Sprint retrospective (every sprint).** The Project Manager Agent aggregates task reflections, review statistics, and blocker history; produces the sprint summary (§8.2), updated risk notes, and 0–3 lessons.
3. **Curation review (weekly, org level).** The Memory Agent reviews promotion candidates, KG extraction queue, prune plans, and retrieval-quality metrics (cache hit rate, packet citation rate — how often agents actually cite retrieved records in their outputs). Low-citation record classes get importance down-weighted; high-value classes up-weighted. This closes the loop: *retrieval quality itself is a learned quantity.*

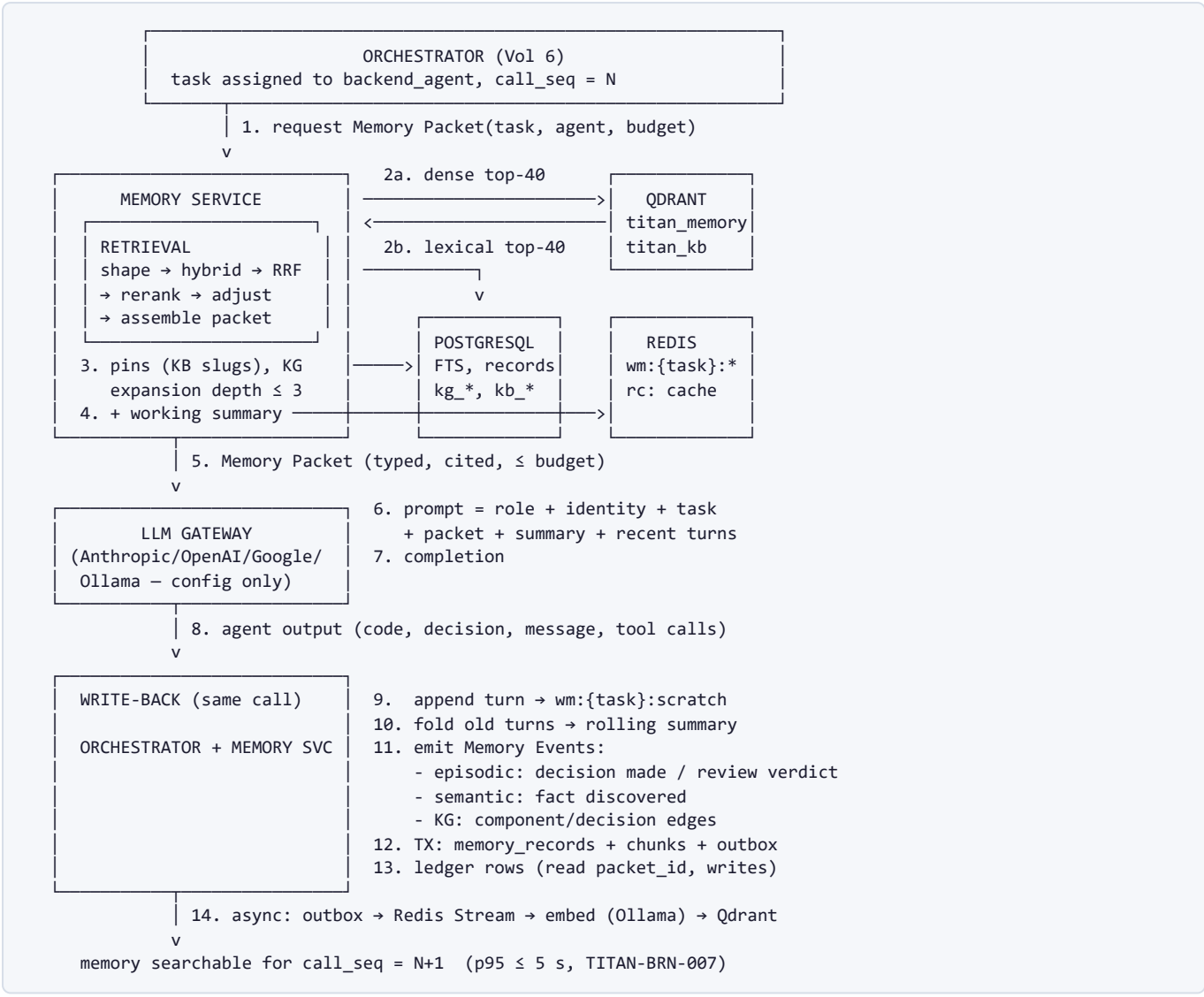
TITAN-BRN-025 (MUST) — Every completed task SHALL produce a rubric-conformant reflection record before the task is marked done. *Acceptance:* Task state machine (Volume 6) blocks the `done` transition without a linked reflection record; schema validated.

TITAN-BRN-026 (SHOULD) — Packet citation rate per record class SHALL be computed weekly and fed into importance weighting. *Acceptance:* Metrics table `brain.retrieval_feedback` populates weekly; importance of a never-cited test class demonstrably decreases over 3 simulated weeks.

---

13. Memory Flow Around One Agent Invocation

The complete read/write cycle for a single agent call (e.g., Backend Agent implementing a task):



Steps 1–8 are synchronous within the agent call; 9–13 complete before the call returns; 14 is asynchronous but bounded by TITAN-BRN-007 so the *next* agent in a handoff can retrieve what this one just learned.

---

14. Memory Service API Surface

Internal REST endpoints (FastAPI, JWT service-auth per Volume 8; full OpenAPI in the repo):

Method & path	Purpose
`POST /memory/packets`	Assemble a Memory Packet (query, scope, kinds, pins, budget)
`POST /memory/events`	Write Memory Events (batch, transactional)
`GET /memory/records/{id}`	Fetch canonical record + chunks + ledger trail
`POST /memory/kg/query`	Typed graph traversal (start nodes, edge types, depth ≤ 3)
`POST /kb/proposals` / `POST /kb/proposals/{id}/approve`	KB write workflow (§10.3)
`POST /memory/admin/rebuild-index`	Rebuild Qdrant from PostgreSQL (TITAN-BRN-004)
`POST /memory/admin/curation/run`	Trigger decay/prune/promotion cycle (also nightly cron)

Method & path	Purpose
`GET /memory/metrics`	Retrieval-quality and ingestion-lag metrics (also exported to Prometheus)

All endpoints emit OpenTelemetry spans; packet assembly spans carry `packet_id`, candidate counts per leg, and rerank latency for Grafana dashboards.

---

15. Risks & Open Questions

#	Risk / question	Impact	Mitigation / owner
R1	Local embedding quality caps retrieval precision on niche code vocabulary	Agents miss relevant memories, redo work	Hybrid lexical leg (§6), reranker, eval harness TITAN-BRN-011 tracks it; config-only upgrade path to cloud embedders
R2	Summarization drift: compounding rolling summaries lose critical constraints	Agent violates a forgotten constraint	Fixed five-field schema, constraints copied verbatim not paraphrased, checkpoint audits sampled in weekly curation
R3	Memory poisoning: a wrong "fact" gets written and reinforces itself via retrieval	Systematic bad decisions	Provenance on every record, citation-required decisions, `supersede` workflow, reviewer agents can flag records; security review in Volume 8
R4	Reranker RAM (~400 MB) too heavy for minimal laptops	Local dev friction	`RERANKER=off` LLM-rerank fallback is a first-class tested mode
R5	Nightly LLM extraction/curation cost at scale	Cost creep against free-tier goal	All curation on utility tier / local models; batch sizes capped; cost surfaced in metrics
Q1	Should episodic half-life (30 d) differ per project phase (e.g., slower decay during active development)?	Tuning	Revisit after first longitudinal eval; parameterized per-scope already
Q2	Adopt Qdrant sparse vectors (BM42) to replace the Postgres FTS leg once stable?	Simplification	Benchmark against golden set in a spike; decision deferred to v1.1
Q3	When multi-tenant SaaS arrives (Volume 10), does org memory need per-tenant partitioning beyond `scope`?	Architecture	Coordinate with Volume 8 tenancy model before GA

---

16. Cross-References

- **Volume 1 — Foundation:** canonical stack decisions (PostgreSQL/Redis/Qdrant, gateway, local-first) that this volume builds on.
- **Volume 2 — Multi-Agent Intelligence:** the Memory Agent's role charter; per-agent prompt template slots that consume Memory Packets.
- **Volume 3 — Backend Engineering:** the LLM gateway, Redis Streams infrastructure, service auth, and deployment of the Memory Service.
- **Volume 4 — Frontend & Dashboard:** the Memory Explorer UI that renders records, the knowledge graph, ledger trails, and KB version history.

- **Volume 6 — Engineering Workflow:** task/sprint state machines that trigger reflections, pattern mining, and the write-back steps in §13.
- **Volume 8 — Security:** retention/deletion policy, scope-isolation guarantees, and threat treatment of memory poisoning (R3).
- **Volume 9 — Production:** backup/restore of PostgreSQL and Qdrant volumes; scaling embedder workers on Kubernetes.
- **Volume 10 — Autonomous Evolution:** self-evaluation metrics and cross-model debates that consume the episodic memory and lessons specified here.

End of Volume 5 — TITAN-VOL-05, Draft v1.0.

PROJECT TITAN — Autonomous Software Engineering Platform

Volume 6 — Autonomous Engineering Workflow & Standards

Field	Value
Series	PROJECT TITAN — Autonomous Software Engineering Platform (10 volumes)
Volume	6 — Autonomous Engineering Workflow & Standards
Doc ID	TITAN-VOL-06
Requirement Prefix	TITAN-WFL-NNN
Status	Draft v1.0
Date	2026-07-04

Abstract. This volume specifies how TITAN turns a one-sentence user request into deployed, documented, release-approved software. It defines the full delivery pipeline — User Request → CEO Agent → Product Manager → Project Manager → Architect → Backend/Frontend/Database Engineers → QA → Security → Code Reviewer → DevOps → Documentation → Release Approval — with every handoff specified as a contract (trigger, artifact, acceptance gate, failure path); the per-component self-improvement loop (Implementation → Review → Critique → Refactor → Testing → Security Scan → Performance Analysis → Approval → Merge) with bounded iteration limits; sprint and milestone mechanics; the auto-generated project artifacts; and the engineering-standards constitution all TITAN-generated code must obey. Everything runs on the canonical stack fixed in Volume 1 (Python 3.12 + FastAPI orchestrator, PostgreSQL 16, Redis 7 Streams, Qdrant, Next.js 15 dashboard), enforced by the orchestrator of Volumes 2–3.

Table of Contents

- 1. Introduction & Scope
- 2. The Delivery Pipeline
- 2.1 Pipeline Overview
- 2.2 The Handoff Contract
- 2.3 Handoff Specifications (H1–H12)
- 2.4 CEO Clarification Policy
- 2.5 Task State Machine
- 3. The Self-Improvement Loop
- 3.1 Loop Overview
- 3.2 Stage Specifications
- 3.3 Iteration Bounds & Escalation
- 4. Sprint Planning & Milestone Mechanics
- 4.1 Sprint Lifecycle
- 4.2 Milestones & Release Trains

- 4.3 Blocker Detection & Recovery
- 5. Auto-Generated Project Artifacts
- 6. The Engineering Standards Constitution
- 6.1 Architecture Standards
- 6.2 Naming Conventions
- 6.3 Error Handling
- 6.4 Logging
- 6.5 Testing Strategy & Coverage Gates
- 6.6 Code Review Rules & Rejection Criteria
- 6.7 Documentation Requirements
- 7. Decision Records
- 8. Risks & Open Questions
- 9. Cross-References

---

1. Introduction & Scope

TITAN is an organization of agents, not a single model. This volume is that organization's **operating manual** — the fixed process every project moves through — and its **constitution** — the non-negotiable quality rules every generated line of code must satisfy.

Two design principles govern everything below:

1. **Every transition is a contract.** Work moves between agents only through schema-validated artifacts passing explicit acceptance gates. Every handoff is a persisted, replayable event on the Redis Streams task queue (NATS JetStream at scale-out, per Volume 1), with the artifact stored in PostgreSQL and indexed in Qdrant.
2. **First drafts are never accepted.** Every component passes the bounded self-improvement loop (Section 3) before merge. Autonomy without gates produces slop; gates without bounds produce livelock. This volume fixes both bounds.

In scope: pipeline stages, handoff contracts, the review loop, sprint/milestone mechanics, auto-generated artifacts, and the code-quality constitution. **Out of scope:** agent prompt design and internal decision trees (Volume 2), orchestrator implementation (Volume 3), UI for the workflow (Volume 4), memory/RAG internals (Volume 5), external tool bindings (Volume 7), threat model (Volume 8), production CI/CD internals (Volume 9).

1.1 Requirement Conventions

Requirements use IDs `TITAN-WFL-NNN` with priority **MUST** (release-blocking), **SHOULD** (deviation requires a logged ADR), or **MAY** (optional). Each carries acceptance criteria testable against the orchestrator.

---

2. The Delivery Pipeline

2.1 Pipeline Overview



TITAN-WFL-001 (MUST) — Pipeline as persisted state machine. The pipeline SHALL be implemented as an explicit state machine per project, persisted in PostgreSQL 16, with each stage transition emitted as an event on Redis Streams and mirrored to the dashboard over WebSockets. *Acceptance:* killing and restarting the orchestrator mid-pipeline resumes the project at the exact stage and iteration count; the event log reconstructs the full history.

TITAN-WFL-002 (MUST) — No stage skipping. No stage may be skipped or reordered; a stage may only be marked `NOT_APPLICABLE` by the CEO Agent with a logged justification (e.g., a docs-only change skipping DB engineering). *Acceptance:* attempting an out-of-order transition raises a rejected-transition event and leaves state unchanged.

2.2 The Handoff Contract

Every handoff Hn is a 4-tuple. This is the universal protocol; Section 2.3 instantiates it twelve times.

Element	Definition	Enforcement
Trigger	The event that starts the handoff (producer marks its stage `COMPLETE`)	Redis Streams message `pipeline.<project>.stage_complete`
Artifact	A versioned, Pydantic-v2-validated JSON/Markdown document	Stored in PostgreSQL (`artifacts` table), embedded into Qdrant
Acceptance gate	Machine-checkable predicate + (where needed) consumer-agent review	Gate evaluator in orchestrator; result persisted with reasons
Failure path	Where work routes on gate failure, with retry budget	Bounded retries (default 3) then human escalation

TITAN-WFL-003 (MUST) — Schema-validated artifacts. Every handoff artifact SHALL validate against a versioned Pydantic v2 schema before the gate is evaluated; schema validation failure counts as gate failure. *Acceptance:* an artifact missing a required field is rejected with a field-level error report attached to the failure event.

TITAN-WFL-004 (MUST) — Auditable gates. Every gate evaluation SHALL be persisted with verdict, evaluator identity (agent + model + prompt version), reasons, and timestamps, and SHALL be visible in the dashboard audit trail. *Acceptance:* for any release, a complete gate ledger can be exported showing every pass/fail with reasons.

TITAN-WFL-005 (MUST) — Bounded failure paths. Every gate SHALL have a retry budget (default 3 per stage, configurable per project). On budget exhaustion the orchestrator SHALL pause the pipeline, mark the project `NEEDS_HUMAN`, and notify the user with a structured escalation report (what failed, all attempts, agent hypotheses). *Acceptance:* a stage forced to fail 4 times produces exactly one escalation and zero further LLM calls for that stage.

2.3 Handoff Specifications (H1–H12)

H1 — User → CEO Agent. *Trigger:* user submits a request via dashboard or API. *Artifact:* `UserRequest` (raw text, attachments, org context). *Gate G1:* request is non-empty and not a duplicate of an active project. *Failure path:* malformed requests are rejected at the API layer; ambiguity is not failure — it triggers the clarification policy (2.4).

H2 — CEO → Product Manager. *Trigger:* CEO marks vision analysis complete. *Artifact:* `ProjectBrief` — product vision (≤300 words), target users, prioritized feature list (MoSCoW), explicit non-goals, constraints (budget/local-first/compliance), success criteria. *Gate G2:* every MUST-feature has a measurable success criterion; non-goals section non-empty; constraints include the canonical stack unless the user overrode it. *Failure path:* PM Agent returns the brief with gap annotations; CEO revises (retries count against CEO stage budget).

H3 — PM → Project Manager. *Trigger:* PM marks backlog complete. *Artifact:* `ProductBacklog` — user stories in "As a…, I want…, so that…" form, each with Gherkin-style acceptance criteria (Given/When/Then), priority, and story-point estimate. *Gate G3:* every story satisfies INVEST (checked by rubric prompt + lint rules: has

actor, has outcome, ≤13 points, ≥1 testable AC); every MUST-feature from the brief is covered by ≥1 story (traceability check). *Failure path:* failing stories return to PM with the specific INVEST violations; passing stories proceed (partial acceptance is allowed at G3 only).

H4 — Project Manager → Architect. *Trigger:* Project Manager marks planning complete. *Artifact:* `SprintPlan` + `TaskGraph` — tasks decomposed from stories (each ≤1 ideal agent-day), assignee role, dependencies as a DAG, sprint assignment, milestone mapping. *Gate G4:* task DAG is acyclic; every story maps to ≥1 task; sprint 1 contains a walking-skeleton slice (thin end-to-end path); no sprint exceeds configured capacity. *Failure path:* cycle or capacity violations return to Project Manager with the offending subgraph highlighted.

H5 — Architect → Engineers. *Trigger:* Architect marks design complete. *Artifact:* `ArchitecturePackage` — component diagram (ASCII + structured JSON), module boundaries mapped to Clean Architecture layers, OpenAPI contract for every backend endpoint, database schema draft (tables, indexes, migration order), frontend route map and state-management plan, ADRs for every non-default choice. *Gate G5:* package complies with the constitution (Section 6) — verified by checklist evaluation; every task in the TaskGraph references a component in the package; API contracts validate as OpenAPI 3.1. *Failure path:* Code Reviewer Agent (acting as design reviewer) returns numbered objections; Architect revises.

H6 — Engineers → QA. *Trigger:* all tasks of the current sprint scope reach `MERGED` (each having individually passed the Section 3 loop). *Artifact:* `MergeCandidateManifest` — list of merged PRs, changelog, updated dependency lockfiles, migration scripts. *Gate G6:* CI is green on the integration branch (lint, typecheck, unit + integration suites, coverage gates per 6.5); all migrations apply cleanly to a fresh PostgreSQL 16 and to a snapshot of the previous schema. *Failure path:* failing components reopen as tasks and re-enter the self-improvement loop at the Refactor stage with the CI failure attached as critique input.

H7 — QA → Security. *Trigger:* QA completes its pass on the assembled build. *Artifact:* `QAReport` — executed test matrix (integration, e2e via Playwright, regression, performance baseline), defects classified P0–P3, flake log. *Gate G7:* zero P0/P1 defects; e2e critical-path suite 100% passing; regression suite passing. *Failure path:* P0/P1 defects become tasks routed to the owning engineer role; the pipeline holds at QA and re-runs only affected suites on fix (retry budget applies to full QA cycles, not individual tests).

H8 — Security → Code Reviewer. *Trigger:* Security Agent completes audit. *Artifact:* `SecurityClearance` — SAST results, dependency audit (lockfile CVE scan), secret scan, OWASP Top-10 checklist with per-item evidence, auth/RBAC verification against Volume 8 policy. *Gate G8:* zero high/critical findings; mediums have logged accepted-risk entries approved by CEO Agent or remediation tasks scheduled within the release. *Failure path:* findings become security-labeled tasks with mandatory loop re-entry; a repeated identical finding after fix attempts escalates immediately (does not consume full retry budget).

H9 — Code Reviewer → DevOps. *Trigger:* Security clearance issued. *Artifact:* `ReleaseReview` — release-level review of the full integrated diff against the constitution: architecture-boundary check, duplication scan, public-API consistency, changelog accuracy. *Gate G9:* Reviewer verdict `APPROVED` (rejection criteria in 6.6 apply at release scope too). *Failure path:* `REJECTED` produces itemized rework tickets; `APPROVED_WITH_COMMENTS` proceeds but logs comments as technical-debt entries (Section 5).

H10 — DevOps → Documentation. *Trigger:* release review approved. *Artifact:* `DeploymentRecord` — image digests, environment (staging→production per Volume 9 promotion rules; Docker Compose for local-first deployments), applied migrations, smoke-test results, health-check status, Grafana dashboard links, rollback pointer. *Gate G10:* staging smoke suite green; production health checks green for the soak window (default 15 min); OpenTelemetry traces flowing. *Failure path:* automatic rollback to the rollback pointer, incident event emitted, defect task created; pipeline returns to H6 scope for the fix.

H11 — Documentation → CEO. *Trigger:* deployment stable. *Artifact:* `DocBundle` — README, architecture guide, API reference (generated from OpenAPI + prose), user manual, deployment/runbook, contribution guide, troubleshooting guide, release notes, updated decision-record index. *Gate G11:* documentation

coverage checks pass (6.7): every public endpoint documented, every ADR indexed, README quickstart executes successfully in a clean Docker Compose environment. *Failure path:* Documentation Agent revises with the specific coverage failures; a quickstart that fails to execute is treated as a P1 defect, not a docs issue, and routes to H7.

H12 — CEO → User (Release Approval). *Trigger:* doc bundle accepted. *Artifact:* `ReleaseApproval` — CEO's verification that the release satisfies the original `ProjectBrief` success criteria, with a traceability table (brief feature → stories → tasks → PRs → tests → docs). *Gate G12:* every MUST-feature traced to passing acceptance tests; user notified with release summary and access details. *Failure path:* unmet criteria reopen as backlog items for the next cycle; the CEO MUST NOT silently narrow scope — scope changes require user confirmation.

TITAN-WFL-006 (MUST) — Full traceability. Every release SHALL carry an automatically generated traceability table from brief features through stories, tasks, PRs, tests, and docs. *Acceptance:* selecting any MUST-feature in the dashboard shows its complete chain with no gaps.

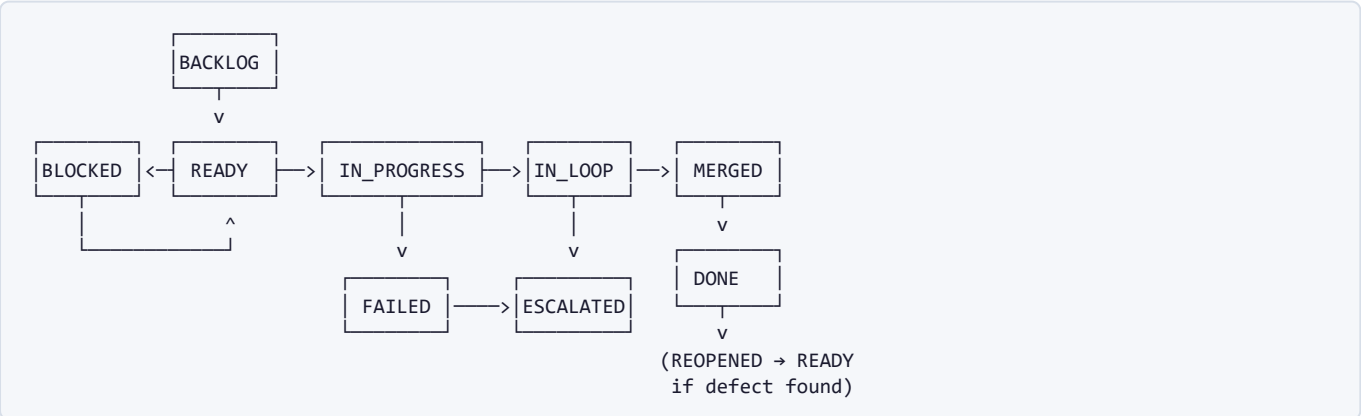
2.4 CEO Clarification Policy

The CEO Agent asks **only essential questions** — those whose answers change architecture, scope, or acceptance criteria. Everything else is decided autonomously using canonical defaults and logged as an assumption.

TITAN-WFL-007 (MUST) — Essential-questions-only. The CEO Agent SHALL ask at most **5 clarification questions per project**, in a single batch, and only when the answer is (a) not inferable from the request or organization memory, (b) not covered by a canonical default, and (c) materially changes the `ProjectBrief`. All other unknowns SHALL be resolved as logged assumptions in the brief's `assumptions` section, each marked user-overridable. *Acceptance:* for the request "Build me a Netflix clone," the CEO produces ≤5 questions (e.g., payment integration real or mocked? content licensing scope?) and ≥10 logged assumptions; asking a question answerable from the canonical stack (e.g., "which database?") is a policy violation flagged by the prompt-regression suite.

TITAN-WFL-008 (MUST) — Non-blocking clarification. If the user does not answer within a configurable window (default 24h), the CEO SHALL proceed with best-judgment answers recorded as high-visibility assumptions, and the release approval (H12) SHALL restate them. *Acceptance:* an unanswered project still reaches release with all assumption-driven decisions flagged in the release summary.

2.5 Task State Machine



TITAN-WFL-009 (MUST) — Task states. Tasks SHALL move only along the edges above; `IN_LOOP` denotes the self-improvement loop; `DONE` requires the parent story's acceptance criteria to pass; any state change

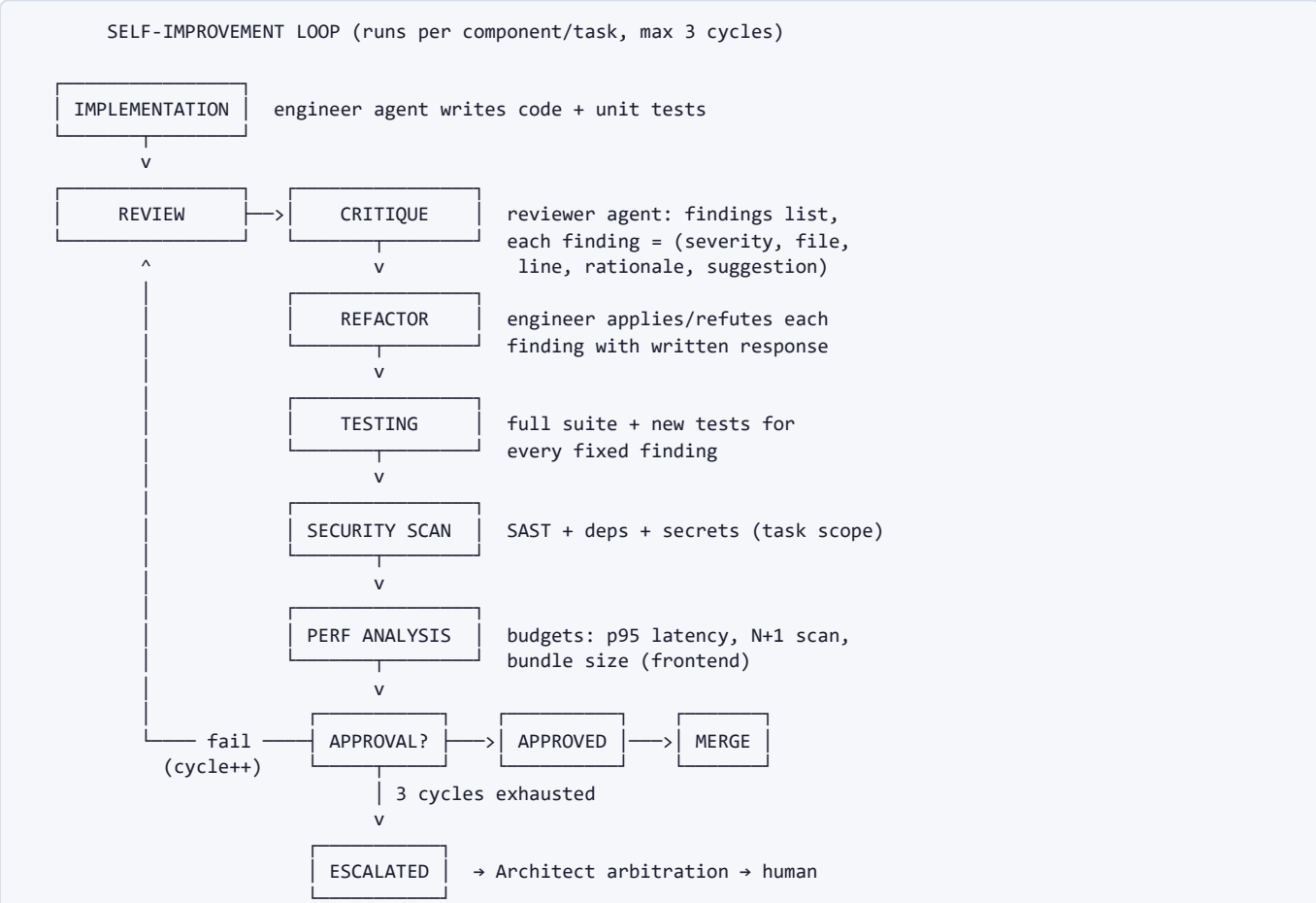
SHALL emit an event consumed by the Kanban artifact (Section 5). *Acceptance:* an illegal transition (e.g., `BACKLOG → MERGED`) is rejected and logged.

---

3. The Self-Improvement Loop

The pipeline in Section 2 is the **macro** flow between agents. The self-improvement loop is the **micro** flow inside every engineering task: no component ships as a first draft.

3.1 Loop Overview



TITAN-WFL-010 (MUST) — Mandatory loop, no first drafts. Every component SHALL complete ≥1 full loop cycle before merge; direct merge of an unreviewed draft SHALL be technically impossible (branch protection + orchestrator gate). *Acceptance:* a PR without an attached loop record cannot reach `MERGED` in the task state machine.

3.2 Stage Specifications

Stage	Actor	Output artifact	Pass condition
Implementation	Owning engineer agent	Branch + PR + unit tests	Compiles; lint/typecheck clean; tests written alongside code
Review	Code Reviewer Agent	`ReviewFindings` (typed list)	N/A (always produces findings; empty list allowed from cycle 2)

Stage	Actor	Output artifact	Pass condition
Critique	Reviewer + QA + Security perspectives	Consolidated critique, deduplicated, severity-ranked	Every finding actionable: file, line, rationale, suggested fix
Refactor	Owning engineer agent	Updated PR + per-finding response (`fixed` / `refuted` + reason)	No finding ignored silently
Testing	QA Agent (task scope)	Test run report	Unit ≥80% line coverage on changed code; all suites green; every `fixed` finding has a covering test
Security Scan	Security Agent (task scope)	Scan report	Zero high/critical in changed code and newly added dependencies
Perf Analysis	Backend/Frontend engineer + QA	Perf report	Within budgets: API p95 ≤ endpoint budget from ArchitecturePackage; no N+1 queries; frontend route JS budget respected
Approval	Code Reviewer Agent	Verdict	`APPROVED` per 6.6 criteria
Merge	DevOps Agent (automation)	Merge commit to trunk	CI green post-merge; task → `MERGED`

TITAN-WFL-011 (MUST) — Adversarial review separation. The reviewing agent instance SHALL NOT share conversation context with the implementing agent instance; the reviewer sees only the PR diff, the task spec, the ArchitecturePackage excerpt, and the constitution. *Acceptance:* review requests contain no implementation-side chain-of-thought; verified by message-log inspection in the audit trail.

TITAN-WFL-012 (MUST) — Refute-or-fix. Every critique finding SHALL receive an explicit `fixed` or `refuted` response; refutations are re-judged by the reviewer, and an upheld finding refuted twice escalates to the Architect for binding arbitration. *Acceptance:* the loop record for any merged PR shows a terminal disposition for 100% of findings.

3.3 Iteration Bounds & Escalation

TITAN-WFL-013 (MUST) — Bounded loops. The loop SHALL run at most **3 full cycles** per task (configurable 2–5). On exhaustion: task → `ESCALATED`, Architect reviews and either (a) re-scopes the task, (b) overrides specific findings with a logged ADR, or (c) escalates to the human user. The loop SHALL also terminate early on convergence: two consecutive cycles with zero new findings of severity ≥ minor. *Acceptance:* no task in the system ever records a 4th cycle; convergence termination is logged distinctly from exhaustion.

TITAN-WFL-014 (SHOULD) — Diminishing-returns detection. The orchestrator SHOULD detect oscillation (a finding re-introduced after being fixed, or reviewer flip-flopping on the same line) and short-circuit to Architect arbitration rather than burning remaining cycles. *Acceptance:* an A→B→A edit sequence on the same hunk triggers arbitration within the same cycle.

---

4. Sprint Planning & Milestone Mechanics

4.1 Sprint Lifecycle

TITAN sprints are logical batches, not calendar rituals — agents work continuously — but the sprint construct bounds work-in-progress, orders dependencies, and gives the user review checkpoints.

TITAN-WFL-015 (MUST) — Auto-planned sprints. The Project Manager Agent SHALL partition the TaskGraph into sprints such that: sprint 1 is a walking skeleton (thin end-to-end slice touching frontend, backend, database, and deploy); each sprint's tasks respect DAG order; sprint size respects configured parallel-agent capacity; each sprint ends with a potentially releasable increment. *Acceptance:* for any generated plan, no task appears in an earlier sprint than any of its dependencies, and sprint 1 contains ≥1 task per engineering role plus a deploy task.

TITAN-WFL-016 (MUST) — Sprint ceremonies as agent events. Each sprint SHALL open with an automated planning record (goals, committed stories, risks) and close with an automated review record (demo links against acceptance criteria, velocity actuals vs. estimates) and a retrospective record (what the agents will change — fed to organization memory per Volume 5). *Acceptance:* every completed sprint has all three records persisted and visible in the dashboard.

TITAN-WFL-017 (SHOULD) — Estimation feedback. Story-point estimates SHOULD be recalibrated per project from actuals (tokens, wall-clock, loop cycles consumed), with the correction factor stored in organization memory. *Acceptance:* estimate error (median absolute percentage) decreases across a project's sprints in benchmark runs.

4.2 Milestones & Release Trains

Milestone	Definition of done	Gate owner
M0 — Inception	ProjectBrief + ProductBacklog accepted (G2, G3)	CEO
M1 — Walking Skeleton	Sprint 1 increment deployed to staging; e2e "hello path" green	DevOps
M2 — Feature Complete	All MUST-stories `DONE`; G6–G7 passed	QA
M3 — Hardened	G8–G9 passed; performance budgets met; debt report reviewed	Security + Reviewer
M4 — Released	G10–G12 passed; docs live; user notified	CEO

TITAN-WFL-018 (MUST) — Milestone gating. Milestones SHALL be strictly ordered; a milestone regression (e.g., new P0 after M2) reverts the project to the earlier milestone and reopens the relevant pipeline stage. *Acceptance:* milestone state in the dashboard always equals the highest milestone whose definition of done currently holds.

4.3 Blocker Detection & Recovery

TITAN-WFL-019 (MUST) — Automatic blocker detection. The Project Manager Agent SHALL flag a task `BLOCKED` when any of: no state change beyond 2× its estimate; its dependency failed; it waits on an unanswered clarification; loop escalation pending. Blockers SHALL trigger: dependency re-check, reassignment to a fresh agent instance with summarized context, or decomposition into smaller tasks — in that order — before human escalation. *Acceptance:* a synthetically stalled task is detected and recovery attempted within one scheduler tick (≤60 s).

---

5. Auto-Generated Project Artifacts

All artifacts are derived views over the same PostgreSQL event log — never hand-maintained, never stale.

Artifact	Source data	Regenerated on	Format
Roadmap	Milestones + sprint plan + brief priorities	Any milestone/sprint change	Timeline JSON + Markdown
Kanban board	Task state machine events	Every task transition	Live board (dashboard, Vol 4)
Dependency graph	TaskGraph DAG + component graph	Task/architecture change	DOT + rendered view
Release plan	Milestones + DeploymentRecords + traceability table	Gate G6–G12 events	Markdown
Risk analysis	Escalations, blocker history, security mediums, assumption log	Sprint close + any escalation	Ranked register (likelihood × impact, owner, mitigation)
Technical debt report	`APPROVED_WITH_COMMENTS` items, deferred findings, coverage deltas, TODO-equivalent debt tickets	Sprint close	Ranked ledger with paydown tasks

TITAN-WFL-020 (MUST) — Derived, versioned artifacts. All six artifacts SHALL regenerate automatically from the event log on their trigger conditions, be versioned per regeneration, and be exportable (Markdown/JSON) via API. *Acceptance:* deleting all artifact rows and replaying the event log reproduces byte-identical current versions.

TITAN-WFL-021 (MUST) — Debt is tracked, never hidden. Any deviation from the constitution accepted at review (6.6) SHALL create a technical-debt ticket with a paydown task proposed into a future sprint; the debt ledger total SHALL appear on the project dashboard. *Acceptance:* every `APPROVED_WITH_COMMENTS` verdict has a corresponding open or closed debt ticket.

---

6. The Engineering Standards Constitution

This section is the **constitution**: every rule here binds all TITAN-generated code, in every project, in every language. Agents receive it verbatim in their system context (retrieved per Volume 5); the orchestrator enforces the machine-checkable subset in CI.

6.1 Architecture Standards

TITAN-WFL-022 (MUST) — Clean Architecture layering. Generated services SHALL follow Clean Architecture: `domain` (entities, invariants — zero framework imports) ← `application` (use cases, ports) ← `infrastructure` (SQLAlchemy repositories, Redis clients, external APIs) ← `interface` (FastAPI routers, CLI, WebSocket handlers). Dependencies point inward only; import-linter (Python) / eslint-boundaries (TypeScript) rules enforce it in CI. *Acceptance:* an import from `domain` into `infrastructure` fails CI.

TITAN-WFL-023 (MUST) — SOLID. All generated code SHALL satisfy SOLID: single responsibility per module/class; extension via interfaces/protocols not modification; substitutable abstractions; narrow role-specific interfaces; dependencies injected against abstractions (FastAPI `Depends` / constructor injection — no service-locator, no module-level singletons holding I/O clients). *Acceptance:* review checklist items for each principle; violations are `major` findings per 6.6.

TITAN-WFL-024 (MUST) — DDD where appropriate. Projects with non-trivial domain logic (more than CRUD over ≤3 entities) SHALL use tactical DDD: ubiquitous language from the ProductBacklog, aggregates

guarding invariants, domain events for cross-aggregate effects, bounded contexts mapped to modules. CRUD-only projects SHALL NOT be forced into DDD ceremony — the Architect records the classification in the ArchitecturePackage. *Acceptance:* the ArchitecturePackage contains an explicit `domain_complexity` classification with rationale.

TITAN-WFL-025 (MUST) — Repository pattern. All persistence access SHALL go through repository interfaces defined in `application` and implemented in `infrastructure` with SQLAlchemy 2.x (async). No ORM queries in routers or use cases. *Acceptance:* grep gate — `sqlalchemy` imports outside `infrastructure` fail CI.

TITAN-WFL-026 (MUST) — CQRS only if justified. CQRS SHALL NOT be the default. It is permitted only when the Architect's ADR demonstrates at least one of: (a) read/write load asymmetry >10×, (b) materially divergent read models, (c) event-sourcing already justified for audit. Absent the ADR, standard use-case + repository flow applies. *Acceptance:* any `commands/` / `queries/` split without a linked ADR is a `blocking` review finding.

6.2 Naming Conventions

Scope	Convention	Example
Python modules/functions/vars	`snake_case`	`create_invoice.py`, `def apply_discount()`
Python classes	`PascalCase`; suffix by role	`InvoiceRepository`, `CreateInvoiceUseCase`
TypeScript files	`kebab-case.ts(x)`; components `PascalCase` export	`invoice-table.tsx` → `InvoiceTable`
TS variables/functions	`camelCase`; hooks `useX`	`useInvoiceFilters`
Constants (both)	`SCREAMING_SNAKE_CASE`	`MAX_RETRY_COUNT`
Database	`snake_case`; plural tables; `id` PK; `<table>_id` FKs; indexes `ix_<table>_<cols>`	`invoices`, `ix_invoices_customer_id`
REST endpoints	plural nouns, kebab-case, versioned prefix	`GET /api/v1/invoices/{invoice_id}`
Events (Redis Streams)	`domain.entity.action` past tense	`billing.invoice.created`
Git branches	`type/task-id-slug`	`feat/TSK-142-invoice-pdf`
Commits	Conventional Commits	`feat(billing): generate invoice PDFs`

TITAN-WFL-027 (MUST) — Machine-enforced naming. The conventions above SHALL be enforced by Ruff, ESLint, a SQL migration linter, and a commit-message hook; violations fail CI. *Acceptance:* a camelCase Python function or a non-conventional commit message fails the pipeline before review.

6.3 Error Handling

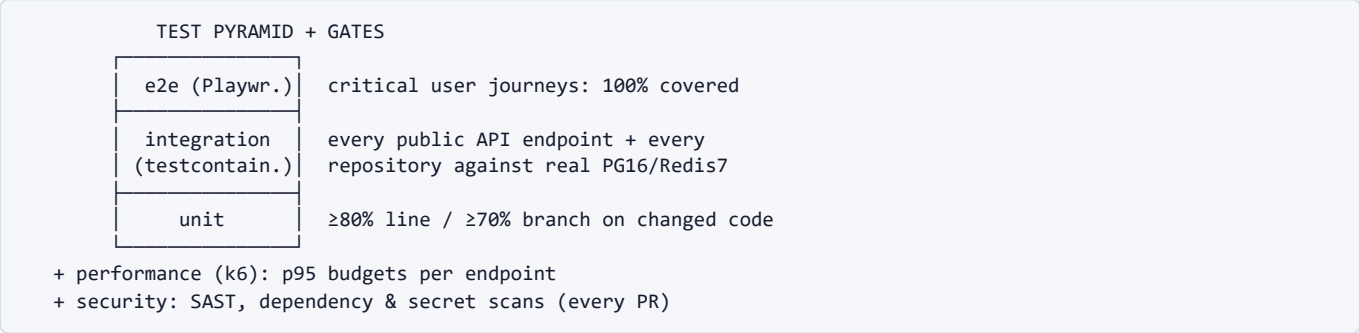
TITAN-WFL-028 (MUST) — Error taxonomy and policy. Every service SHALL define a domain error hierarchy (`DomainError` → e.g., `NotFoundError`, `ConflictError`, `ValidationError`, `AuthorizationError`) raised from domain/application layers and translated exactly once, at the interface layer, into RFC 9457 problem-

details responses with stable machine-readable `type` codes. Rules: no bare `except:`; no swallowed exceptions (catch → log-and-rethrow or handle, never pass); no stack traces or internal identifiers in client responses; all external I/O wrapped with timeouts and typed failure mapping; retries with exponential backoff + jitter only on idempotent operations. *Acceptance:* the error-handling test suite verifies every endpoint returns problem-details on induced failure and that logs contain the stack trace the client response omits.

6.4 Logging

TITAN-WFL-029 (MUST) — Structured, correlated logging. All logs SHALL be structured JSON (via `structlog` / `pino`-equivalent) shipped to Loki, with mandatory fields: `timestamp`, `level`, `service`, `trace_id` / `span_id` (OpenTelemetry-propagated), `project_id`, `agent_id` where applicable, and `event`. Levels: `DEBUG` (dev-only), `INFO` (state changes), `WARNING` (handled anomalies), `ERROR` (failed operations), `CRITICAL` (data-loss/security). Secrets, tokens, and PII SHALL never be logged — enforced by a redaction filter and a CI log-call scanner. *Acceptance:* every request's logs are joinable to its trace in Grafana by `trace_id`; the redaction test suite passes.

6.5 Testing Strategy & Coverage Gates



TITAN-WFL-030 (MUST) — Test layers and gates. Every component SHALL ship with: unit tests (pytest / Vitest) written in the same PR as the code; integration tests running against real PostgreSQL 16 and Redis 7 via testcontainers; e2e tests (Playwright) for every critical user journey named in the ProductBacklog; performance tests (k6) for endpoints with declared budgets; security scans per H8. Coverage gates, enforced in CI on the diff: **≥80% line and ≥70% branch coverage of changed code; 100% of new public endpoints integration-tested; 100% of critical journeys e2e-tested before M2.** Coverage may never decrease project-wide. *Acceptance:* a PR dropping diff coverage below gate values cannot merge; the QA report at M2 shows the journey matrix fully green.

TITAN-WFL-031 (MUST) — Deterministic tests. Tests SHALL be hermetic: no network calls to real external services (use fakes/contract stubs), no time/random flakiness (injected clocks and seeds), parallel-safe. A test failing intermittently is quarantined within one CI run, ticketed as a P1 defect, and counted in the debt ledger. *Acceptance:* the flake log in QAReports trends to zero across benchmark projects; quarantined tests block M3 until resolved.

6.6 Code Review Rules & Rejection Criteria

The Code Reviewer Agent's authority is real: it can and must reject. Verdicts: `APPROVED`, `APPROVED_WITH_COMMENTS` (minor findings → debt tickets), `REJECTED`.

TITAN-WFL-032 (MUST) — Mandatory rejection criteria. A PR SHALL be `REJECTED` if any of the following hold:

#	Rejection criterion	Severity
R1	Violates Clean Architecture boundaries (6.1) or SOLID at module level	blocking
R2	Missing or failing tests; diff coverage below gates (6.5)	blocking
R3	Any high/critical security finding, hardcoded secret, or missing input validation on external input	blocking
R4	Duplicated logic (>10 similar lines) where an existing abstraction applies	blocking
R5	Swallowed errors, bare excepts, or client-visible stack traces (6.3)	blocking
R6	Unjustified pattern use — CQRS/event-sourcing without ADR (6.1)	blocking
R7	Placeholder code: TODO/FIXME, stubbed function bodies, commented-out code	blocking
R8	Public API change without OpenAPI update + versioning per ArchitecturePackage	blocking
R9	Performance budget exceeded or N+1 query introduced	blocking
R10	Naming/lint violations that somehow bypassed CI	major
R11	Missing docstrings/JSDoc on public functions; missing migration rollback	major

Two `major` findings equal one blocking condition. *Acceptance:* the reviewer prompt-regression suite includes fixture PRs for R1–R11 and the reviewer rejects each with the correct code.

TITAN-WFL-033 (MUST) — Review scope discipline. Reviews SHALL address the diff plus its blast radius only; reviewers SHALL NOT demand unrelated refactors (those become debt/backlog proposals). Every finding cites file, line, violated rule ID, and a concrete fix. *Acceptance:* 100% of findings in loop records carry rule citations; findings outside the diff's blast radius are marked `advisory` and never block.

6.7 Documentation Requirements

TITAN-WFL-034 (MUST) — Documentation as a gated deliverable. Every project SHALL ship: README with a copy-paste Docker Compose quickstart verified by execution (G11); architecture guide generated from the ArchitecturePackage; API reference from OpenAPI with prose descriptions on every endpoint, parameter, and error `type`; user manual keyed to the ProductBacklog journeys; deployment runbook (including rollback); contribution guide; troubleshooting guide seeded from defects encountered during the build; ADR index. Code-level: public functions/classes carry docstrings (Google style) / JSDoc; modules carry purpose headers; complex algorithms carry rationale comments — comments explain *why*, never narrate *what*. *Acceptance:* the G11 coverage checker reports 100% on endpoints and ADR indexing, and the quickstart executes cleanly in CI.

---

7. Decision Records

ADR-WFL-01 — Stage-gated pipeline over free-form agent swarm

- **Context:** Multi-agent systems can be orchestrated as an open "swarm" (agents self-select work, converse freely) or as a gated pipeline with typed handoffs.
- **Decision:** Gated pipeline with the twelve handoff contracts of Section 2.3; agent conversation is allowed *within* a stage but stage transitions require artifacts passing gates.
- **Alternatives considered:** (a) Free-form swarm with a coordinator — rejected: non-reproducible, unauditable, unbounded token spend; (b) fully static waterfall without loops — rejected: forfeits the self-improvement requirement.

- **Trade-offs:** Gates add latency and orchestration code; in exchange we get resumability, auditability (TITAN-WFL-004), bounded cost, and testable prompt regression per stage. For an open-source, local-first platform, reproducibility outweighs raw speed.

ADR-WFL-02 — Loop iteration cap of 3 with convergence early-exit

- **Context:** "Never accept the first draft" conflicts with bounded cost on a laptop-class local deployment (Ollama models included).
- **Decision:** Maximum 3 full loop cycles per task, early exit on two consecutive zero-finding cycles, oscillation short-circuit to Architect arbitration (TITAN-WFL-013/014).
- **Alternatives considered:** (a) Unbounded until reviewer approval — rejected: livelock risk, unbounded spend; (b) single mandatory revision — rejected: benchmarking in comparable systems shows second-cycle findings are common; (c) dynamic cap by model quality — deferred to Volume 10's self-tuning.
- **Trade-offs:** Some residual defects will pass at cycle 3 and be caught at QA/Security stages instead — acceptable because macro gates backstop the micro loop; escalation ensures humans see chronic failures.

ADR-WFL-03 — Trunk-based development with short-lived task branches

- **Context:** Agents work in parallel on a monorepo (canonical, Volume 1); merge strategy determines integration pain.
- **Decision:** Trunk-based: one short-lived branch per task (`feat/TSK-nnn-slug`), merged to `main` only via the self-improvement loop + CI; integration branch per sprint scope assembled for H6; no long-lived feature branches.
- **Alternatives considered:** (a) GitFlow — rejected: long-lived branches multiply merge conflicts between parallel agents; (b) direct-to-trunk commits — rejected: violates TITAN-WFL-010 (no unreviewed drafts).
- **Trade-offs:** Requires disciplined task decomposition (≤1 agent-day, TITAN-WFL @ H4) and feature flags for partially complete features (mechanics in Volume 9); in exchange, merge conflicts stay small and the dependency graph stays honest.

ADR-WFL-04 — Standards enforced by machines first, reviewer second

- **Context:** LLM reviewers are strong at judgment, inconsistent at mechanical rule checking.
- **Decision:** Split the constitution into a machine-checkable subset (lint, import boundaries, coverage, naming, secret scan — enforced in CI before review) and a judgment subset (SOLID at design level, DDD appropriateness, duplication semantics — enforced by the Code Reviewer against the R1–R11 table).
- **Alternatives considered:** (a) Reviewer checks everything — rejected: wastes tokens on what Ruff does for free and drifts across model versions; (b) CI checks everything — rejected: architectural judgment isn't mechanically checkable.
- **Trade-offs:** Two enforcement layers to maintain; mitigated by the prompt-regression fixtures (TITAN-WFL-032) keeping the judgment layer calibrated.

---

8. Risks & Open Questions

#	Risk / question	Impact	Mitigation / owner
1	**Reviewer–implementer collusion drift**: over time, prompt updates could make the reviewer lenient, silently degrading quality.	High	Prompt-regression fixture suite for R1–R11 runs on every prompt change; quality metrics (defects escaping to QA per 100 merges) tracked in Volume 10's evaluation harness.
2	**Gate rubric gaming**: agents may optimize for passing gate wording rather than intent (e.g., trivial tests inflating coverage).	High	Mutation-testing spot checks in QA stage (SHOULD); reviewer explicitly checks test meaningfulness (R2 covers assertion-free tests); open question: mutation testing cost on local-only hardware.
3	**Local-model capability floor**: on Ollama-class models, 3 loop cycles may not converge for complex tasks, raising escalation rates.	Medium	Task decomposition pressure at H4 (≤1 agent-day); per-provider iteration-cap configuration; escalation reports quantify the gap to inform model routing (Volume 5/10).
4	**Retry-budget interaction across stages**: a defect oscillating between QA (H7) and engineering could consume budgets in two stages and double-escalate.	Medium	Single project-level escalation queue deduplicates by defect fingerprint; cross-stage oscillation counts as one escalation.
5	**Clarification-policy tension**: 5-question cap may be too tight for genuinely ambiguous enterprise briefs.	Medium	Cap is per-project configuration; assumption logging plus H12 restatement gives the user a correction point. Open question: allow follow-up rounds after M0? Current answer: new user requests enter as backlog change requests.
6	**Artifact regeneration cost**: replaying large event logs to regenerate artifacts (TITAN-WFL-020) may get slow on big projects.	Low	Incremental materialized views with periodic snapshot compaction; replay-from-snapshot rather than genesis.
7	**Open question — human override granularity**: when a human resolves an escalation, do their edits re-enter the loop or merge directly?	Medium	Current draft position: human edits still pass CI + a single review cycle (humans are not exempt from the constitution), but this needs UX validation in Volume 4.

9. Cross-References

- **Volume 1 — Foundation:** canonical tech-stack decisions and their justification; success metrics that gate G12 traces to.
- **Volume 2 — Multi-Agent Intelligence:** per-agent specifications (prompts, decision trees, KPIs) for every actor named in the pipeline; agent failure-recovery internals behind TITAN-WFL-019.
- **Volume 3 — Backend Engineering:** orchestrator implementation of the state machine, Redis Streams topics, Pydantic artifact schemas, WebSocket event fan-out.
- **Volume 4 — Frontend & Dashboard:** Kanban board, pipeline visualization, gate-audit trail, escalation inbox UX for Section 5 artifacts and TITAN-WFL-005 escalations.
- **Volume 5 — Memory, Knowledge & RAG:** constitution retrieval into agent context; retrospective and lessons-learned storage (TITAN-WFL-016/017).
- **Volume 7 — Integrations & Tool Ecosystem:** Git provider bindings for branch protection (TITAN-WFL-010) and PR mechanics; CI runner integration.

- **Volume 8 — Security:** the full policy behind gate G8 and the Security stage of the loop; secret-handling rules referenced in 6.4.
- **Volume 9 — Deployment & Production:** staging→production promotion, rollback mechanics behind G10, feature-flag mechanics for trunk-based development (ADR-WFL-03).
- **Volume 10 — Autonomous Evolution:** quality metrics, prompt-regression harness, and self-tuning of iteration caps referenced in ADR-WFL-02 and Risk 1.

End of Volume 6 — TITAN-VOL-06, Draft v1.0.

PROJECT TITAN — Autonomous Software Engineering Platform

Volume 7 — Integrations, Plugin System & Tool Ecosystem

Field	Value
Series	PROJECT TITAN — Autonomous Software Engineering Platform (10 volumes)
Volume	7 — Integrations, Plugin System & Tool Ecosystem
Doc ID	TITAN-VOL-07
Status	Draft v1.0
Date	2026-07-04
Requirement prefix	TITAN-INT-NNN

Abstract. This volume specifies how TITAN's agents reach the outside world: the plugin architecture that lets third parties add capabilities without touching core code, the tool abstraction layer through which every agent action is declared, sandboxed, and audited, the core tool set (Git, filesystem, terminal, Docker, browser, web research), the concrete integration specifications for GitHub, GitLab, Slack, Discord, Jira, Linear, Figma, VS Code, and cloud providers, first-class support for the Model Context Protocol (MCP) as a plugin transport, and the multi-LLM provider gateway that makes Anthropic, OpenAI, Google, and local Ollama models interchangeable through configuration alone. It is written to be implemented directly: schemas, lifecycle state machines, permission scopes, and wire formats are normative.

Table of Contents

1. Introduction
2. Ecosystem Architecture Overview
3. Plugin System
4. Tool Abstraction Layer
5. Core Tool Set
6. External Integration Specifications
7. MCP — Model Context Protocol Support
8. Multi-LLM Provider Gateway
9. Decision Records
10. Risks & Open Questions
11. Cross-References

1. Introduction

1.1 Purpose

TITAN's agents (Volume 2) are only as capable as the tools they can call. This volume defines the three layers that turn a reasoning loop into an engineering organization that can ship software: (a) a **plugin system** so new integrations require zero core changes, (b) a **tool abstraction layer** that gives every agent a uniform, sandboxed, audited way to act, and (c) a **provider gateway** that decouples agents from any single LLM vendor.

1.2 Scope

In scope: plugin manifest, lifecycle, permissions, and versioning; tool registry and JSON-Schema tool definitions; execution sandboxing and audit logging; core tools (Git/GitHub/GitLab, filesystem, terminal, Docker, browser automation, web research); integration specs for GitHub, Slack/Discord, Jira/Linear, Figma, VS Code, and cloud providers; MCP as a first-class transport; the multi-LLM gateway with per-agent routing, fallbacks, and cost accounting.

Out of scope: agent role definitions (Volume 2), memory/RAG internals (Volume 5), workflow orchestration (Volume 6), platform-wide security policy (Volume 8), and production deployment topology (Volume 9). Canonical stack decisions (Next.js 15 frontend; Python 3.12 + FastAPI backend; PostgreSQL 16 / Redis 7 / Qdrant; Redis Streams; OAuth2/OIDC + JWT + RBAC; Docker Compose locally, Kubernetes in production) are owned and justified by Volume 1 and are treated here as fixed.

1.3 Terminology

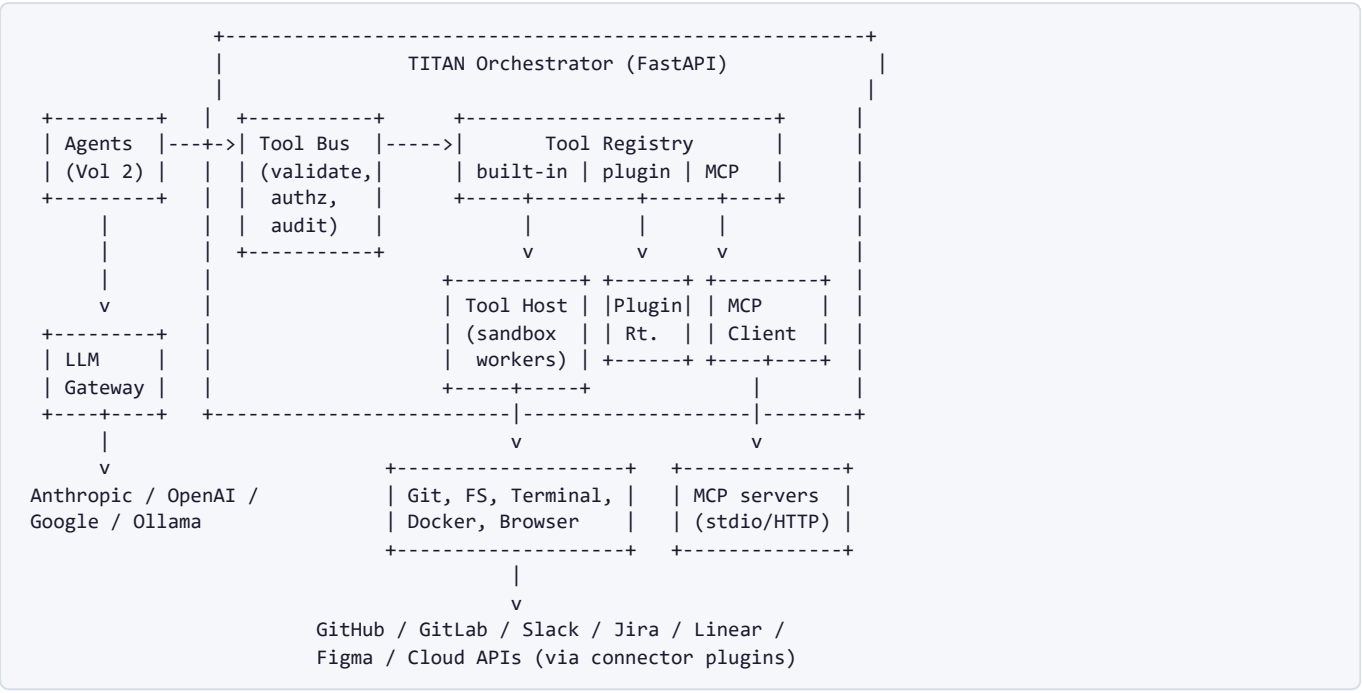
Term	Definition
Tool	A named, JSON-Schema-typed capability an agent can invoke (e.g. `git.commit`).
Plugin	A distributable package that registers tools, agents, UI panels, webhooks, or providers.
Tool Host	The sandboxed process pool that executes tool calls.
Connector	A plugin whose primary purpose is bridging an external SaaS (GitHub, Jira, …).
Gateway	The provider-agnostic LLM access layer.
MCP	Model Context Protocol — an open protocol for exposing tools/resources to LLM apps.

Priorities follow RFC 2119: MUST (mandatory for MVP), SHOULD (expected, deferrable with an ADR), MAY (optional).

---

2. Ecosystem Architecture Overview

All agent side-effects flow through one chokepoint — the Tool Bus — regardless of whether the capability is built-in, a Python plugin, or a remote MCP server. Similarly, all LLM traffic flows through the Gateway. Nothing else in TITAN talks to the network on an agent's behalf.



TITAN-INT-001 (MUST) — Every externally visible side-effect performed by an agent MUST pass through the Tool Bus; no agent code path may open sockets, spawn processes, or write files directly. *Acceptance:* code review gate + a CI lint rule rejecting `subprocess`, `open()` (write mode), and raw HTTP clients inside `titan/agents/**`; runtime egress policy blocks agent workers from all hosts except the orchestrator.

TITAN-INT-002 (MUST) — Every LLM call MUST pass through the Provider Gateway. *Acceptance:* no provider SDK is imported outside `titan/gateway/**`; integration test proves an agent run succeeds with only gateway credentials configured.

---

3. Plugin System

3.1 Design principles

1. **Core stays small.** GitHub, Slack, Jira, etc. are plugins shipped in the monorepo (`plugins/first_party/*`) using the same API as third-party plugins — first-party code gets no private hooks.
2. **Declarative surface, imperative core.** A plugin declares *what* it contributes in a manifest; the runtime decides *where and how* it loads.
3. **Deny-by-default permissions.** A plugin can do nothing it did not declare and the operator did not approve.
4. **Local-first.** Plugins install from a local path or Git URL with no central marketplace required (marketplace is Volume 10 scope).

3.2 Plugin manifest format

Each plugin is a directory (or wheel/archive) containing `titan-plugin.yaml` at its root. The manifest schema is published as JSON Schema at `schemas/plugin-manifest.v1.json` and validated with Pydantic v2 on install.

Example — the first-party Jira connector manifest:

```yaml
manifest_version: 1
id: titan.jira                 # reverse-DNS, globally unique
name: Jira Sync
version: 1.2.0                 # SemVer, required
description: Two-way sync of TITAN tasks with Jira issues.
author: TITAN Core Team
license: Apache-2.0
homepage: https://github.com/titan-platform/titan
titan_api: ">=1.0,<2.0"        # core plugin-API compatibility range

runtime:
  type: python                 # python | mcp
  entrypoint: titan_jira.plugin:JiraPlugin   # subclass of titan.plugins.Plugin
  python: ">=3.12"

permissions:
  network:
    - "https://*.atlassian.net"   # explicit egress allowlist
  secrets:
    - JIRA_API_TOKEN              # names resolved from the secret store
    - JIRA_EMAIL
  events:
    subscribe: [task.created, task.status_changed]
    publish:   [integration.jira.synced]
  scopes: [tasks:read, tasks:write, projects:read]   # TITAN RBAC scopes

contributes:
  tools:
    - name: jira.create_issue
      schema: schemas/create_issue.json
      risk: write                # read | write | admin (see 4.4)
    - name: jira.transition_issue
      schema: schemas/transition_issue.json
      risk: write
  agents: []                     # may contribute agent role definitions (Vol 2 format)
  ui_panels:
    - id: jira-board-link
      title: Jira
      mount: project.sidebar     # named extension slot in the dashboard (Vol 4)
      type: iframe               # iframe | schema-form | markdown
      src: /plugin-ui/titan.jira/panel.html
  webhooks:
    - path: /hooks/jira          # mounted at /api/v1/plugins/titan.jira/hooks/jira
      verify: hmac-sha256
      secret: JIRA_WEBHOOK_SECRET

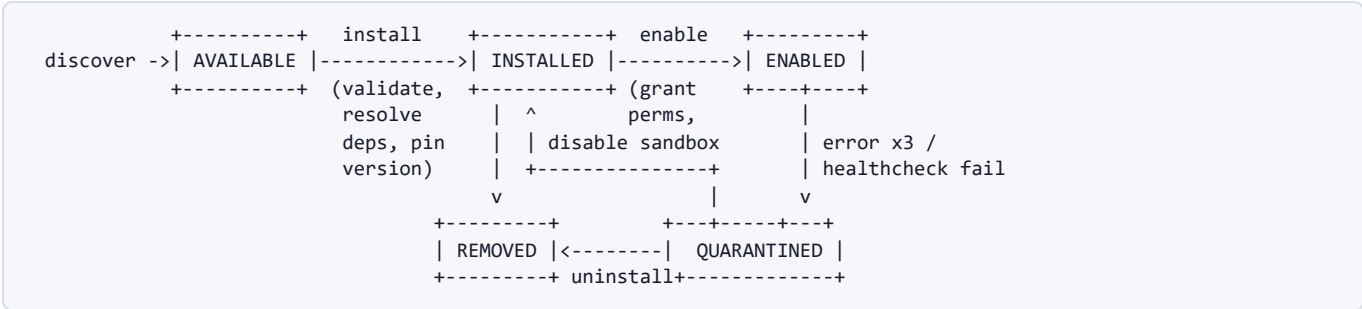
config_schema: schemas/config.json   # JSON Schema rendered as a settings form
```

TITAN-INT-010 (MUST) — Plugins MUST ship a `titan-plugin.yaml` conforming to `plugin-manifest.v1`; installation MUST fail with actionable errors on schema violation, unknown permission scope, or `titan_api` incompatibility. *Acceptance:* installer test suite covers each rejection class; error message names the offending field.

TITAN-INT-011 (MUST) — A plugin MUST be able to contribute tools, agent definitions, UI panels, webhook endpoints, and event subscriptions purely via its manifest and entrypoint, with zero modifications to core packages. *Acceptance:* the first-party GitHub, Slack, and Jira connectors are implemented as plugins; deleting any of their directories leaves core tests green.

TITAN-INT-012 (SHOULD) — Manifests SHOULD be signed (detached Ed25519 signature, `titan-plugin.yaml.sig`); unsigned plugins install only when the operator sets `plugins.allow_unsigned: true`. *Acceptance:* tampered manifest fails verification; unsigned install emits a persistent warning banner in the Plugin Manager UI.

3.3 Plugin lifecycle



Lifecycle stages, each an explicit REST operation on `/api/v1/plugins` and a Plugin Manager UI action (Volume 4):

1. **Discover** — sources: local directory scan (`./plugins`), Git URL, OCI artifact reference. The registry records origin and content hash.
2. **Install** — validate manifest, resolve Python deps into an isolated virtualenv per plugin (never the core environment), persist a `plugin_installs` row (PostgreSQL) with pinned version + hash.
3. **Enable** — operator (RBAC role `plugin:admin`) reviews and grants the declared permissions; runtime loads the entrypoint inside its sandbox; healthcheck (`Plugin.health()`) must return OK within 10 s.
4. **Configure** — settings form generated from `config_schema` ; values stored encrypted (Volume 8); secrets referenced by name only, injected at runtime.
5. **Sandbox (continuous)** — see §3.5; violations move the plugin to QUARANTINED.
6. **Disable / Uninstall** — disable revokes permissions and unloads tools atomically (in-flight tool calls drain, new calls rejected with `TOOL_UNAVAILABLE`); uninstall additionally deletes the virtualenv and config, retaining audit rows.

TITAN-INT-013 (MUST) — Lifecycle transitions MUST be atomic and crash-safe: a failure mid-install/enable leaves the previous state intact. *Acceptance:* chaos test kills the orchestrator during each transition; on restart the registry state matches the last committed transition.

TITAN-INT-014 (MUST) — Three consecutive unhandled exceptions or a failed healthcheck MUST quarantine a plugin automatically, unregistering its tools and notifying operators via the notification bus. *Acceptance:* fault-injection test observes quarantine within 30 s and a `plugin.quarantined` event on Redis Streams.

3.4 Permission model

Plugin permissions are capability grants, checked at two layers: **grant time** (operator approves the manifest's requested set; any later manifest expansion re-triggers approval) and **call time** (Tool Bus verifies the *calling agent's* RBAC scopes intersect the tool's required scopes — a plugin cannot launder an agent into privileges the agent lacks).

Permission class	Granularity	Enforced by
`network`	URL-pattern egress allowlist	Sandbox egress proxy
`secrets`	Named secrets only	Secret store injector
`scopes`	TITAN RBAC scopes (`tasks:write` , …)	Tool Bus authz
`events`	Explicit subscribe/publish topics	Event bus ACL
`filesystem`	Workspace-relative path globs	Tool Host mount policy

TITAN-INT-015 (MUST) — Permissions are deny-by-default; a plugin call touching an undeclared or ungranted capability MUST fail closed and be audit-logged as a `PERMISSION_DENIED` event. *Acceptance:* conformance suite attempts one violation per class; all five are blocked and logged.

3.5 Sandboxing of plugin code

Python plugins run **out-of-process** in a per-plugin worker (local dev: subprocess with seccomp-lite via `python -I`, resource limits via cgroups where available; production: dedicated container with read-only rootfs, non-root UID, egress restricted to the manifest allowlist by a sidecar proxy; Kubernetes NetworkPolicy mirrors the same allowlist). Communication with core is JSON-RPC 2.0 over stdio — deliberately identical in shape to MCP (§7), so the plugin runtime and MCP client share one codebase.

TITAN-INT-016 (MUST) — Plugin code MUST NOT share a process with the orchestrator; a crashing or hanging plugin MUST NOT take down core (call timeout default 60 s, configurable per tool). *Acceptance:* a plugin executing `os._exit(1)` and one sleeping forever both leave the orchestrator serving requests; the offending calls return structured errors.

3.6 Versioning and compatibility

- Plugins use **SemVer**; the core plugin API is versioned independently (`titan_api`, currently `1.x`).
- Core guarantees backward compatibility within a major `titan_api` version; deprecations emit warnings for ≥1 minor release before removal.
- The registry pins exact versions; upgrades are explicit (`POST /plugins/{id}/upgrade`) and produce a diff of manifest permission changes for re-approval.
- Multiple versions MUST NOT be simultaneously active for one plugin id.

TITAN-INT-017 (MUST) — A plugin upgrade that widens permissions MUST re-enter the approval step before activation. *Acceptance:* upgrade test with an added `network` pattern blocks activation until re-approved.

---

4. Tool Abstraction Layer

4.1 Tool registry

The Tool Registry is an in-memory index (rebuilt from PostgreSQL + plugin runtimes on boot) mapping tool names to: JSON-Schema definition, provider (built-in module / plugin id / MCP server id), risk class, required RBAC scopes, timeout, and rate limits. Names are namespaced: `fs.read`, `git.commit`, `github.open_pr`, `mcp.<server>.<tool>`.

Agents receive their tool list per turn, filtered to (agent role's allowed tools) ∩ (enabled providers) ∩ (project-level policy). The filtered definitions are handed to the LLM in the gateway's canonical tool-definition format (§8.1), which is JSON Schema — identical to what Anthropic/OpenAI tool-use APIs consume.

TITAN-INT-020 (MUST) — Every tool MUST be defined by a JSON Schema (draft 2020-12) for inputs and a declared output schema; the Tool Bus MUST validate arguments before dispatch and results before return. *Acceptance:* fuzz test sends malformed args to every registered tool; 100% are rejected pre-execution with schema-pointer error messages that are fed back to the calling agent for self-correction.

Example tool definition (registry entry, abbreviated):

```
{
  "name": "git.commit",
  "provider": "builtin:git",
  "description": "Create a commit in the project workspace repository.",
  "risk": "write",
  "required_scopes": ["workspace:write"],
  "timeout_s": 30,
  "input_schema": {
    "$schema": "https://json-schema.org/draft/2020-12/schema",
    "type": "object",
    "properties": {
      "message":  { "type": "string", "minLength": 10 },
      "paths":    { "type": "array", "items": { "type": "string" } },
      "author_agent": { "type": "string" }
    },
    "required": ["message", "paths"],
    "additionalProperties": false
  },
  "output_schema": {
    "type": "object",
    "properties": { "commit_sha": { "type": "string" }, "files_changed": { "type": "integer" } },
    "required": ["commit_sha"]
  }
}
```

4.2 Tool invocation flow and wire format

```
Agent LLM turn
   |  (tool_use block)
   v
Tool Bus:  1. parse & schema-validate args
           2. authz: agent scopes ∩ tool scopes; project policy; risk gate
           3. write audit row (status=STARTED)
           4. dispatch -> Tool Host worker / plugin runtime / MCP client
           5. enforce timeout, capture stdout/stderr/artifacts
           6. schema-validate result; redact secrets
           7. finalize audit row; emit OTel span; return tool_result
   |
   v
Agent LLM next turn (tool_result block)
```

Example tool-call envelope (as persisted and as sent to the executor):

```
{
  "tool_call_id": "tc_01J9ZK3W8QF4",
  "trace_id": "4bf92f3577b34da6a3ce929d0e0e4736",
  "project_id": "prj_netflix_clone",
  "task_id": "tsk_00412",
  "agent_id": "agent.backend_engineer.2",
  "tool": "git.commit",
  "arguments": {
    "message": "feat(api): add JWT refresh endpoint with rotation",
    "paths": ["services/api/src/auth/refresh.py", "services/api/tests/test_refresh.py"],
    "author_agent": "agent.backend_engineer.2"
  },
  "requested_at": "2026-07-04T10:42:17.031Z"
}
```

Example result envelope:

```
{
  "tool_call_id": "tc_01J9ZK3W8QF4",
  "status": "ok",
  "duration_ms": 412,
  "result": { "commit_sha": "9f2c1ab", "files_changed": 2 },
  "stderr_tail": "",
  "artifacts": []
}
```

Errors use `status: "error"` with a machine-readable `code` (`SCHEMA_INVALID`, `PERMISSION_DENIED`, `TIMEOUT`, `SANDBOX_VIOLATION`, `UPSTREAM_ERROR`, `TOOL_UNAVAILABLE`) plus a human/LLM-readable `message` so agents can recover autonomously (Volume 6 retry policy).

4.3 Execution sandboxing

Tool Host workers execute built-in tools inside per-project workspace containers:

Control	Policy
Filesystem	Workspace mounted at `/workspace`; everything else read-only or absent
Network	Deny-all except per-tool allowlist (e.g. `web.search` egress via research proxy)
Process	Non-root, no privileged flags, PID limit, `no_new_privileges`
Resources	Default 1 vCPU / 1 GiB / 60 s per call; per-tool overrides in registry
Docker access	Only via `docker.*` tools which talk to a rootless DinD/sysbox sidecar, never the host daemon socket

TITAN-INT-021 (MUST) — Terminal and code-execution tools MUST run inside the workspace sandbox with no access to host filesystem, host Docker socket, or platform secrets. *Acceptance:* red-team suite from Volume 8 (`cat /etc/shadow`, socket probing, `docker.sock` mount attempt, env-var secret scraping) fully blocked; violations produce `SANDBOX_VIOLATION` audit events.

4.4 Risk classes and approval gates

Every tool declares `risk: read | write | admin`. Project policy (configurable; defaults below) maps risk to gating:

Risk	Examples	Default gate
`read`	`fs.read`, `github.get_pr`, `web.search`	Auto-approved
`write`	`git.push`, `jira.create_issue`, `docker.build`	Auto-approved within workspace/project scope; rate-limited
`admin`	`github.merge_pr` to protected branch, `cloud.provision`, `k8s.apply`	Requires human approval or CEO-agent sign-off per Volume 6 release policy

TITAN-INT-022 (MUST) — `admin`-risk tool calls MUST block on an approval record (human or policy-delegated) before dispatch. *Acceptance:* attempting `cloud.provision` without approval yields `PERMISSION_DENIED` and a pending-approval item in the dashboard.

4.5 Audit logging

TITAN-INT-023 (MUST) — Every tool call MUST produce an immutable audit record in PostgreSQL (`tool_audit` table: tool_call_id, trace_id, project, task, agent, tool, argument hash + redacted args JSONB, status, duration, result hash, initiator chain) and an OpenTelemetry span; secrets MUST be redacted before persistence. *Acceptance:* audit row exists for 100% of calls in a full autonomous project run; sampled rows show no secret material; Grafana/Loki dashboards can reconstruct any task's tool timeline (Volume 9).

TITAN-INT-024 (SHOULD) — Audit records SHOULD be hash-chained per project (each row stores `prev_hash`) to make tampering evident. *Acceptance:* verification job detects a mutated historical row.

---

5. Core Tool Set

These tools ship with core (as built-in modules under `titan/tools/*`), are always available offline, and require no third-party accounts — satisfying the local-first hard requirement.

Name space	Tools (representative)	Risk	Notes
`fs`	`read`, `write`, `list`, `glob`, `grep`, `patch`	read/ write	Workspace-jailed; `patch` applies unified diffs atomically
`git`	`status`, `diff`, `log`, `branch`, `commit`, `merge`, `rebase`, `push`, `clone`	write	Local repo in workspace; remotes only via connector credentials
`terminal`	`run` (argv + env + cwd + timeout), `run_background`, `kill`	write	§4.3 sandbox; output streamed over WebSocket to the dashboard Logs view
`docker`	`build`, `run`, `compose_up`, `compose_down`, `logs`, `ps`	write	Rootless sidecar daemon per workspace
`browser`	`goto`, `snapshot` (accessibility-tree JSON), `screenshot`, `click`, `type`, `eval_js`	write	Headless Chromium via Playwright in its own container; used by QA agent for E2E and Research agent for docs
`web`	`search`, `fetch` (readability-extracted markdown)	read	Pluggable search backend: SearXNG (default, self-hosted, free) or provider API; egress via caching research proxy
`code`	`run_python`, `run_node`, `lint`, `test`	write	Convenience wrappers over `terminal.run` with structured result parsing (JUnit/coverage)

TITAN-INT-030 (MUST) — Core tool set (`fs`, `git`, `terminal`, `docker`, `browser`, `web`, `code`) MUST be implemented at MVP with full schema definitions and sandbox conformance. *Acceptance:* the Volume 6 end-to-end scenario ("Build me a Netflix clone") completes using only core tools plus the GitHub connector, on a single laptop under Docker Compose.

TITAN-INT-031 (MUST) — `terminal.run` MUST take argv arrays (no shell string interpolation by default), enforce timeouts, cap output (1 MiB per stream, truncation marked), and stream incremental output as `tool.output` events on Redis pub/sub for live dashboard display. *Acceptance:* injection test with hostile filenames executes literally; a `yes`-spam command is truncated and killed at timeout.

TITAN-INT-032 (SHOULD) — `browser.snapshot` SHOULD return a token-efficient accessibility-tree representation (role/name/ref) rather than raw HTML, with `screenshot` available for vision-capable models. *Acceptance:* snapshot of a reference page is <10% of raw HTML token count and sufficient for the QA agent to complete a scripted checkout flow.

---

6. External Integration Specifications

All connectors below are first-party plugins (§3.1). Each stores its config via `config_schema`, its credentials in the secret store, and exposes tools + webhooks per its manifest.

6.1 GitHub (reference connector)

Capabilities: repos (create/clone/fork), branches, PRs (open/update/review/comment/merge), issues (CRUD, labels, milestones), checks/status reads, releases, webhooks.

- **Auth:** GitHub App (preferred; fine-grained installation tokens, higher rate limits) or PAT fallback for local/free-tier use. App private key lives in the secret store.
- **Tools contributed:** `github.create_repo`, `github.open_pr`, `github.update_pr`, `github.review_pr`, `github.merge_pr` (risk `admin` on protected branches), `github.create_issue`, `github.comment`, `github.get_checks`, `github.search_code`.
- **Webhooks:** `/hooks/github` verifying `X-Hub-Signature-256`; events `pull_request`, `issue_comment`, `check_suite`, `push` are translated into TITAN events (`vcs.pr.review_requested`, `vcs.checks.completed`, …) on Redis Streams, which drive the Volume 6 review/deploy workflow.
- **Rate limits:** connector maintains a token bucket from `X-RateLimit-*` headers; at <10% remaining, write tools return `UPSTREAM_ERROR/RATE_LIMITED` with `retry_after`, letting the orchestrator reschedule.

TITAN-INT-040 (MUST) — GitHub connector MUST support the full PR lifecycle (branch → PR → review comments → checks status → merge) and inbound webhooks with signature verification. *Acceptance:* integration test against a scratch org completes the lifecycle end-to-end; webhook with a bad signature is rejected 401 and audited.

6.2 GitLab

Mirror of §6.1 mapped to GitLab semantics (merge requests, pipelines, GitLab webhooks with `X-Gitlab-Token`). Tool names are provider-specific (`gitlab.open_mr`), but the connector also registers the **generic `vcs.*` aliases** (`vcs.open_change_request`, `vcs.merge`) so workflow definitions in Volume 6 remain provider-neutral. Self-hosted GitLab CE is the recommended free/local option. **TITAN-INT-041 (SHOULD)** — GitLab connector at parity with the `vcs.*` alias surface. *Acceptance:* Volume 6 workflow test passes with GitLab as the only VCS connector enabled.

6.3 Slack & Discord notifications

Purpose: outbound notifications (milestone completed, approval requested, build failed, release shipped) and lightweight inbound commands.

- **Slack:** Bot token (`chat:write`, `commands`); tools `slack.post_message`, `slack.post_approval` (Block Kit message with Approve/Reject buttons that resolve a pending `admin`-gate approval via the interactivity webhook). Slash command `/titan status <project>`.
- **Discord:** Bot + webhook URLs; tools `discord.post_message`, `discord.post_embed`; slash commands via Discord interactions endpoint.
- Both subscribe to the notification topics of the event bus; message templates are per-event and configurable.

TITAN-INT-042 (MUST) — Notification connectors MUST deliver event-driven messages with per-project channel routing and MUST support approval resolution from Slack. *Acceptance:* an `admin` tool gate is

approved from a Slack button and the blocked tool call proceeds; delivery failures retry with exponential backoff (max 5) and then surface in the dashboard.

6.4 Jira & Linear sync

Purpose: mirror TITAN's internal task graph (Volume 6) into the customer's tracker so humans can watch progress in their own tools.

- **Direction:** TITAN is the source of truth for agent-created tasks; sync is two-way for status and comments (human comment in Jira → `task.comment_added` event → visible to PM agent).
- **Mapping table (stored per project):** TITAN task ⇄ Jira issue key / Linear issue id; TITAN states (`todo/in_progress/review/done/blocked`) mapped to tracker workflow states via configurable mapping in `config_schema`.
- **Jira:** REST v3 + OAuth 2.0 (3LO) or API token; webhooks for `jira:issue_updated`. **Linear:** GraphQL API + webhook signatures.
- **Conflict rule:** last-writer-wins with audit trail; simultaneous transition conflicts flag the task `sync_conflict` for PM-agent resolution rather than silently overwriting.

TITAN-INT-043 (SHOULD) — Jira and Linear connectors MUST (within this SHOULD-priority feature) keep status parity within 60 s under normal operation. *Acceptance:* soak test of 200 task transitions shows ≥99% mirrored within 60 s, zero lost updates, conflicts flagged not dropped.

6.5 Figma (read-only)

Purpose: give the Frontend Engineer agent design ground truth. Tools: `figma.get_file` (node tree), `figma.get_node_image` (rendered PNG for vision models), `figma.get_variables` (design tokens → Tailwind theme mapping), `figma.get_comments`. Auth: personal access token or OAuth. The connector is strictly read-only at MVP (`risk: read`), aligning with the "Figma read" scope. **TITAN-INT-044 (SHOULD)** — Figma read connector with token/variable extraction usable by the frontend agent to generate Tailwind config. *Acceptance:* given a reference Figma file, the extracted tokens produce a `tailwind.config.ts` whose palette matches the file's variables.

6.6 VS Code extension (concept)

A thin client, not a fork: the extension connects to a TITAN instance over its REST + WebSocket APIs (OAuth2 device flow).

Features (concept scope, post-MVP): project/task tree view; live agent activity feed; inline diff review of agent-proposed changes with Approve/Request-changes (resolves the same approval records as Slack); "hand off to TITAN" command that packages the current repo + a task description into a new TITAN task; workspace file sync via the `fs` tool API rather than direct disk access, preserving the audit chokepoint. **TITAN-INT-045 (MAY)** — VS Code extension implementing task view + diff approval. *Acceptance:* extension approves a pending change and the merge proceeds, with the approval attributed to the human's identity in the audit log.

6.7 Cloud providers

Deployment tools are connector plugins so core stays cloud-agnostic; Volume 9 owns the deployment workflows that call them.

Connector	Tools (representative)	Notes
`titan.aws`	`cloud.provision` (Terraform/OpenTofu plan+apply), `s3.put`, `ecr.push`, `eks.kubeconfig`	All `admin` risk; plan output attached to approval request
`titan.gcp`	analogous (GCS, Artifact Registry, GKE)	
`titan.k8s`	`k8s.apply`, `k8s.rollout_status`, `k8s.logs`	Works against any cluster incl. local kind/k3d
`titan.local`	`deploy.compose`	Default target — free/local deployment via Docker Compose

TITAN-INT-046 (MUST) — The default deploy target MUST be local Docker Compose requiring no cloud account; cloud connectors are optional plugins whose `admin` tools always attach an execution plan (e.g. `terraform plan`) to the approval gate. *Acceptance:* full platform demo deploys its generated app locally with zero cloud credentials configured.

---

7. MCP — Model Context Protocol Support

MCP is a **first-class plugin transport**: any MCP server becomes a TITAN plugin without writing TITAN-specific code.

- **Manifest:** `runtime.type: mcp` with `transport: stdio | http` and either `command: ["npx", "-y", "@modelcontextprotocol/server-filesystem", ...]` or `url: https://...` (Streamable HTTP). All other manifest sections (permissions, scopes, risk overrides) apply identically.
- **Discovery:** on enable, the MCP client performs `initialize`, then `tools/list`; each MCP tool is registered as `mcp.<plugin_id>.<tool_name>` with its advertised JSON Schema. `notifications/tools/list_changed` triggers live re-registration. MCP resources are exposed to the Research/Memory agents as read tools (`mcp.<id>.read_resource`); MCP prompts are surfaced in the prompt library (Volume 2).
- **Sandboxing:** stdio servers run in the same per-plugin container regime as Python plugins (§3.5); HTTP servers are subject to the manifest egress allowlist. MCP tools default to `risk: write` unless the operator downgrades them per tool in configuration — external schemas are not trusted to self-declare risk.
- **Auth:** HTTP transport supports OAuth 2.1 per MCP spec; tokens live in the secret store.

TITAN-INT-050 (MUST) — TITAN MUST connect to spec-compliant MCP servers over stdio and Streamable HTTP, auto-registering their tools into the Tool Registry with full Tool Bus validation, gating, and audit applied. *Acceptance:* the reference `server-filesystem` and one HTTP MCP server are installed via manifest only; their tools appear in the registry, execute under audit, and a `PERMISSION_DENIED` fires when an agent lacking scopes calls one.

TITAN-INT-051 (SHOULD) — TITAN SHOULD also *expose* its own core tools as an MCP server (`titan mcp serve`) so external clients (Claude Code, IDEs) can drive a TITAN workspace. *Acceptance:* an external MCP client lists and invokes `fs.read` against a TITAN workspace with TITAN-side audit rows created.

---

8. Multi-LLM Provider Gateway

8.1 Unified interface

The gateway (`titan/gateway`) exposes one async Python interface and one internal HTTP endpoint; agents never see provider SDKs.

```
ChatRequest  { model_alias, messages[], tools[]?, tool_choice?, temperature?,
               max_tokens, stream?, metadata{agent_id, task_id, project_id} }
ChatResponse { text?, tool_calls[]?, finish_reason, usage{input_tokens,
               output_tokens, cached_tokens}, provider, model_id, cost_usd, latency_ms }
```

Canonical formats: messages follow a provider-neutral schema (system/user/assistant/tool roles; content parts for text, image, tool_use, tool_result); tool definitions are the §4.1 JSON Schemas. Adapters translate to/from each provider: **Anthropic** (Messages API), **OpenAI** (Chat Completions + Responses), **Google** (Gemini API), **Ollama** (local, OpenAI-compatible endpoint). Streaming is normalized to one internal delta-event shape and re-broadcast to the dashboard over WebSockets.

TITAN-INT-060 (MUST) — Adding or switching a provider/model for any agent MUST require configuration changes only — no code changes. *Acceptance:* CI matrix runs the same agent conversation (including tool use and streaming) against Anthropic, OpenAI, Google, and Ollama backends by swapping `models.yaml` alone.

TITAN-INT-061 (MUST) — The gateway MUST normalize tool-calling across providers, including parallel tool calls and provider-specific quirks (e.g. strict-mode schemas), behind the canonical format. *Acceptance:* golden-file tests per adapter verify round-tripping of a multi-tool, multi-turn transcript.

8.2 Configuration and per-agent routing

`config/models.yaml` (hot-reloadable; secrets by env/secret-store reference):

```
providers:
  anthropic: { api_key: ${ANTHROPIC_API_KEY} }
  openai:    { api_key: ${OPENAI_API_KEY} }
  google:    { api_key: ${GOOGLE_API_KEY} }
  ollama:    { base_url: http://ollama:11434 }

model_aliases:
  reasoning-large:  { provider: anthropic, model: claude-opus-4-6 }
  coding-default:   { provider: anthropic, model: claude-sonnet-4-5 }
  fast-cheap:       { provider: google,    model: gemini-2.5-flash }
  local-default:    { provider: ollama,    model: qwen3-coder:30b }

routing:
  default: coding-default
  by_agent:
    agent.ceo:               reasoning-large
    agent.architect:         reasoning-large
    agent.backend_engineer:  coding-default
    agent.frontend_engineer: coding-default
    agent.qa:                fast-cheap
    agent.research:          fast-cheap
  fallbacks:
    coding-default:  [local-default]        # on outage / rate limit / budget cap
    reasoning-large: [coding-default, local-default]

budgets:
  per_project_usd: 25.00
  per_task_usd: 2.00
  action_on_exceeded: fallback_then_pause   # fallback to local, else pause task
```

TITAN-INT-062 (MUST) — Per-agent model routing with alias indirection MUST be supported, and a fully local profile (`routing.default: local-default` with Ollama) MUST run the entire platform with zero paid API keys. *Acceptance:* "local-only" CI profile completes a small end-to-end project using Ollama only.

8.3 Fallbacks and resilience

Fallback triggers: provider 5xx/timeouts (after per-request retries with jittered backoff), 429 rate limits, context-window overflow (falls back only to a larger-context alias), and budget-cap breach. Circuit breaker per provider (open after 5 failures/30 s; half-open probes). Every fallback is recorded on the response metadata and audit trail so evaluation (Volume 10) can attribute quality differences to model substitutions.

TITAN-INT-063 (MUST) — Automatic failover to configured fallback aliases with circuit breaking; agent work continues without human intervention when at least one healthy provider remains. *Acceptance:* chaos test blackholes the primary provider mid-task; task completes on fallback; audit shows the substitution.

8.4 Cost and token accounting

Every `ChatResponse.usage` is written to `llm_usage` (PostgreSQL): timestamp, project, task, agent, provider, model, input/output/cached tokens, computed `cost_usd` from a versioned price table (`config/prices.yaml`, updatable without deploy; Ollama priced at 0 with GPU-seconds tracked instead). Redis maintains rolling counters for budget enforcement at request time. Prometheus exports `titan_llm_tokens_total`, `titan_llm_cost_usd_total`, `titan_llm_latency_seconds` labeled by provider/model/agent; Grafana ships a per-project cost dashboard (Volume 9).

TITAN-INT-064 (MUST) — Token and cost accounting per call, aggregated per agent/task/project, with hard budget caps enforced pre-request. *Acceptance:* a task hitting `per_task_usd` transitions to fallback-or-pause per policy before exceeding the cap by more than one maximal request; dashboard totals reconcile with provider-reported usage within 2%.

---

9. Decision Records

ADR-INT-01 — Out-of-process plugins over in-process imports

- **Context:** Plugins run third-party code adjacent to secrets and workspaces; TITAN must stay stable and local-first.
- **Decision:** All plugin code executes out-of-process (per-plugin venv + subprocess/container) speaking JSON-RPC over stdio, mirroring MCP framing.
- **Alternatives considered:** (a) In-process Python entrypoints — fastest calls, but a crash or dependency conflict kills the orchestrator and permissions become unenforceable; (b) WASM sandbox — strong isolation but immature Python-guest ecosystem and blocks native deps that connectors need.
- **Trade-offs:** ~1–5 ms IPC overhead per call and process-management complexity, accepted in exchange for crash isolation, enforceable egress/permission boundaries, and one shared codepath with MCP.

ADR-INT-02 — MCP as first-class transport rather than a bridge plugin

- **Context:** The ecosystem of MCP servers is large and growing; TITAN needs maximal integrations for minimal maintenance.

- **Decision:** The plugin runtime natively speaks MCP (`runtime.type: mcp`); MCP tools are ordinary registry entries with TITAN gating layered on top.
- **Alternatives considered:** (a) A single "MCP bridge" plugin — hides MCP servers from the permission model, making per-server egress/secret grants impossible; (b) Ignore MCP and hand-write connectors — unbounded maintenance burden.
- **Trade-offs:** TITAN must track MCP spec revisions and cannot trust external schemas' risk semantics (mitigated by write-by-default risk, §7); in exchange, hundreds of existing servers work day one.

ADR-INT-03 — Build a thin in-house gateway instead of adopting LiteLLM/OpenRouter

- **Context:** Canonical decision requires provider-agnostic access to Anthropic, OpenAI, Google, and Ollama with config-only switching, per-agent routing, budgets, and audit metadata tied to TITAN's task graph.
- **Decision:** Implement a thin gateway in `titan/gateway` with four adapters and TITAN-native routing/budget/audit hooks.
- **Alternatives considered:** (a) LiteLLM proxy — broad coverage, but adds a service to the local stack, and per-agent routing/budgets keyed to TITAN entities would still need a wrapper; (b) OpenRouter — hosted-only, conflicts with local-first and free-tier requirements.
- **Trade-offs:** We own adapter maintenance for four providers (bounded, versioned APIs) and forgo LiteLLM's long tail; in exchange we get zero extra runtime dependencies, first-class task/agent attribution, and no data leaving the deployment by default.

ADR-INT-04 — Capability-based deny-by-default permissions with dual-layer checks

- **Context:** Autonomous agents plus third-party plugins is a compound risk: either can be confused or malicious.
- **Decision:** Permissions are granted to plugins at enable time (operator-approved manifest) AND checked against the calling agent's RBAC scopes at call time; risk classes add human gates for `admin` actions.
- **Alternatives considered:** (a) Trust first-party plugins implicitly — creates two security regimes and untested third-party paths; (b) Prompt-level guardrails only — LLM instructions are not a security boundary.
- **Trade-offs:** More approval friction and manifest verbosity; accepted because a single laundering path (low-privilege agent → high-privilege plugin) would defeat the entire Volume 8 threat model.

---

10. Risks & Open Questions

#	Risk / Question	Impact	Mitigation / Owner
R1	**Malicious or compromised plugin** exfiltrates secrets within its granted egress allowlist.	High	Narrow allowlists, per-plugin secrets, egress proxy logging with anomaly alerts (Vol 8); signing (TITAN-INT-012).
R2	**Prompt injection via tool results** (web pages, Jira comments, MCP resources) steering agents into harmful tool calls.	High	Result sanitization markers, risk gates on `write` / `admin` , injection red-team suite shared with Volume 8.
R3	**MCP spec drift** (auth, transport revisions) breaking auto-registered servers.	Medium	Pin supported spec versions per release; conformance tests against reference servers in CI.

#	Risk / Question	Impact	Mitigation / Owner
R4	**Provider API divergence** (new tool-call semantics, deprecations) silently degrading agents.	Medium	Golden-file adapter tests (TITAN-INT-061) run nightly against live sandboxes; price table versioning.
R5	**Local model quality** insufficient for `local-default` fallback, causing silent quality collapse when budgets trip.	Medium	Fallback substitutions are audited and surfaced in dashboards; Volume 10 evaluation harness scores fallback runs.
R6	**Rate-limit starvation** on shared GitHub App installations across many parallel agents.	Medium	Central token buckets per connector; orchestrator backpressure via `retry_after`.
Q1	Should plugin UI panels support a sandboxed module-federation path (richer than iframes) in dashboard v2?	—	Decide with Volume 4 owners after MVP telemetry on panel usage.
Q2	Do we allow plugins to contribute *workflow steps* (Volume 6) in `titan_api` 1.x, or defer to 2.0?	—	Deferred to 2.0 unless two first-party connectors need it; tracked in the decision log.
Q3	Marketplace distribution/signing authority model (central index vs. federated).	—	Volume 10 scope; §3.2 signing format designed to be forward-compatible either way.

---

11. Cross-References

- **Volume 1 — Foundation:** owns and justifies all canonical stack decisions referenced here (FastAPI, PostgreSQL/Redis/Qdrant, Redis Streams, OAuth2/RBAC, local-first mandate).
- **Volume 2 — Multi-Agent Intelligence:** agent roles, per-agent tool allowlists, and prompt templates that consume the Tool Registry definitions of §4.
- **Volume 4 — Frontend & Dashboard:** Plugin Manager screens, extension slots for `ui_panels`, live tool-output Logs view, cost dashboards.
- **Volume 5 — Memory, Knowledge & RAG:** Research agent's use of `web.*` and MCP resources; storage of tool-derived knowledge in Qdrant.
- **Volume 6 — Autonomous Development Workflow:** task graph events that connectors mirror (Jira/Linear), VCS webhook-driven review/merge flow, retry policy on tool errors, approval gates.
- **Volume 8 — Security:** threat model behind the sandbox and permission requirements; secret store; injection red-teaming; audit retention policy.
- **Volume 9 — Deployment & Production:** OTel/Prometheus/Grafana wiring for tool spans and LLM cost metrics; cloud connector usage in CI/CD; scaling the Tool Host on Kubernetes.
- **Volume 10 — Autonomous Evolution:** plugin marketplace, model performance benchmarking that consumes §8.4 accounting data, and agent self-evaluation of fallback runs.

End of Volume 7 — TITAN-VOL-07, Draft v1.0.

PROJECT TITAN — Autonomous Software Engineering Platform

Volume 8 — Security Specification

Field	Value
Series	PROJECT TITAN — Autonomous Software Engineering Platform (10 volumes)
Volume	8 — Security
Doc ID	TITAN-VOL-08
Requirement prefix	TITAN-SEC-NNN
Status	Draft v1.0
Date	2026-07-04
Audience	Platform engineers, security engineers, AI coding agents implementing TITAN

Abstract. TITAN autonomously writes, reviews, executes, and deploys software, which makes it both a high-value target and an unusual attack surface: it runs LLM-generated code, grants agents tool access, and holds credentials for external systems. This volume specifies TITAN's security architecture end to end: a STRIDE threat model extended with AI-specific threats; OAuth2/OIDC + JWT authentication; RBAC with a full permission matrix; sandboxing for agent code execution; local-first secret management; encryption; validation and output encoding; rate limiting; immutable audit logging; CI supply-chain scanning; OWASP Top 10 and OWASP LLM Top 10 mappings; an incident response runbook; security testing; and a secure-defaults checklist. All requirements carry TITAN-SEC-NNN identifiers with MUST/SHOULD priorities and acceptance criteria. Stack choices are canonical per Volume 1; this volume specifies how they are secured.

Table of Contents

1. Introduction, Scope, and Security Principles
2. Threat Model (STRIDE + AI-Specific)
3. Authentication and Authorization Architecture
4. RBAC Roles and Permission Matrix
5. Agent Execution Sandboxing
6. Secret Management
7. Encryption at Rest and in Transit
8. Input Validation and Output Encoding
9. Rate Limiting and Abuse Prevention
10. Audit Logging
11. Supply Chain Security and CI Scanning
12. OWASP Compliance Mapping
13. Incident Response Runbook
14. Security Testing
15. Secure Defaults Checklist

16. Risks and Open Questions
17. Cross-References

---

1. Introduction, Scope, and Security Principles

1.1 Scope

This volume governs the security of the TITAN platform itself: the FastAPI orchestrator, agent runtime, LLM gateway, Next.js dashboard, data stores (PostgreSQL 16, Redis 7, Qdrant), CI pipelines, and both deployment targets (Docker Compose locally, Kubernetes in production). Security gates for software TITAN *generates* are wired into the workflow in Volume 6 and governed here in §11.

1.2 Security Principles

1. **Assume the model is compromised.** Any LLM output — text, code, tool calls — may be attacker-influenced. LLM output is untrusted input, always.
2. **Least privilege.** Users, agents, services, and containers get the minimum permissions their role requires, scoped per project and per task.
3. **Defense in depth.** Prompt hardening is never the only barrier; every AI-specific threat has at least one non-LLM enforcement layer.
4. **Local-first without security debt.** The single-laptop Docker Compose deployment must be secure by default with free tooling.
5. **Secure by default.** Dangerous capabilities (sandbox egress, auto-merge, auto-deploy) ship disabled and require explicit opt-in.
6. **Auditability.** Every security-relevant action is attributable and recorded immutably.

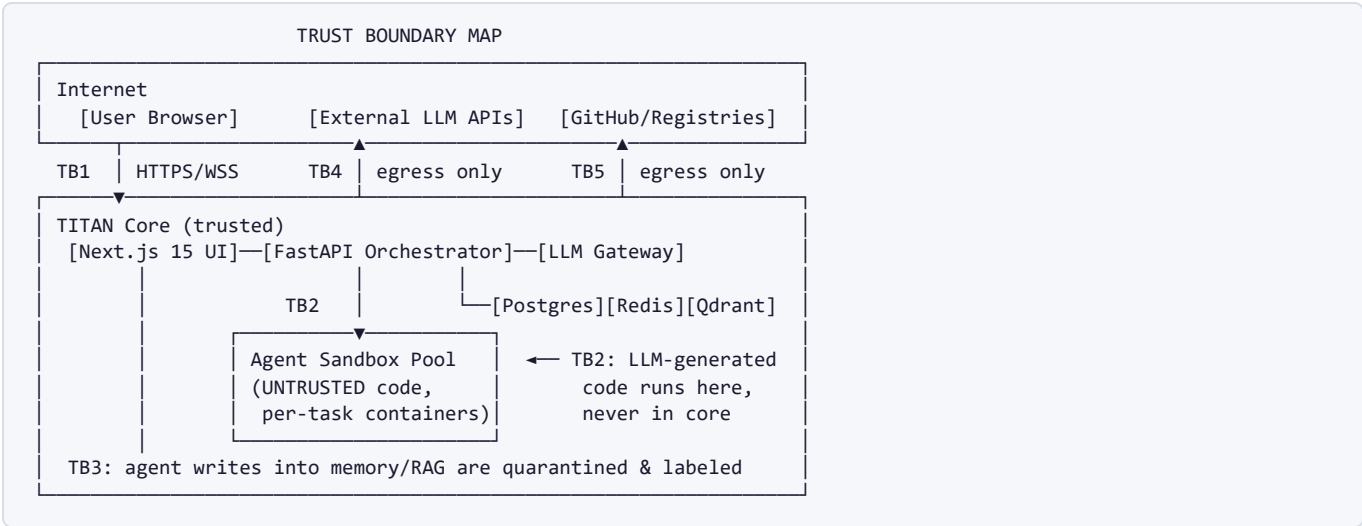
- **TITAN-SEC-001 (MUST):** Every LLM response (text or tool call) MUST pass validation (§8) and authorization (§4) before producing any side effect. *Acceptance:* no code path executes an LLM-proposed action without the tool-authorization layer; a hostile mock LLM's unauthorized tool calls are rejected in integration tests.
- **TITAN-SEC-002 (MUST):** All controls in this volume MUST function in local Docker Compose using only free/open-source components. *Acceptance:* clean-laptop `docker compose up` passes the §15 checklist with zero paid services.

---

2. Threat Model (STRIDE + AI-Specific)

2.1 Assets and Trust Boundaries

Assets: (A1) user and generated source code; (A2) credentials — LLM API keys, GitHub tokens, deploy keys; (A3) the orchestrator's authority to execute code and call tools; (A4) memory/RAG stores feeding future agent context; (A5) the audit trail; (A6) availability and LLM budget.



Boundaries: **TB1** browser↔API; **TB2** orchestrator↔sandbox (most critical); **TB3** agent outputs↔persistent memory (poisoning vector); **TB4** gateway↔LLM providers (context leaves our control); **TB5** platform↔registries and Git hosts.

2.2 STRIDE Analysis

STRIDE class	Representative TITAN threats	Mitigations
Spoofing	Stolen JWT replay; agent impersonation on Redis Streams; forged webhooks	Short-lived rotating tokens (§3); signed message envelopes (§3.5); webhook HMAC (§8)
Tampering	Queue payload modification; altering code between review and merge; poisoning Qdrant memory	Envelope signing (§3.5); artifact content hashes (§11); memory provenance tiers (§2.3)
Repudiation	Agent or user denies destructive action; no LLM-output-to-side-effect linkage	Immutable audit log with actor + task + trace ID (§10)
Information disclosure	Secrets in prompts/logs/providers; cross-project RAG bleed; verbose errors	Redaction pipeline (§6.4); per-project memory namespaces; generic errors (§8); encryption (§7)
Denial of service	Agent loops burning LLM budget; sandbox fork bombs; WSS floods	Budget caps + loop detection (§9); cgroup limits (§5); rate limiting (§9)
Elevation of privilege	Sandbox escape; agent calling tools beyond role; SQLi via generated migrations; IDOR	Hardened runtime (§5); server-side tool allowlists (§4.3); parameterized SQL (§8); object-level authz (§4)

2.3 AI-Specific Threats

Each maps to OWASP LLM Top 10 in §12.2.

T-AI-1 — Prompt injection (direct/indirect). A project brief, fetched README, issue comment, or dependency doc may carry hostile instructions; because agents chain outputs, one injected document can steer the organization. *Mitigations:* retrieved/external content is tagged `untrusted` in prompt assembly (soft control); **hard controls** — model-independent tool authorization (§4.3), egress-deny sandboxes (§5), redaction so context holds nothing worth exfiltrating (§6.4), human approval gates for irreversible actions (Volume 6).

T-AI-2 — Malicious or vulnerable generated code. Backdoors, unsafe patterns, logic bombs. *Mitigations:* generated code executes only in sandboxes (§5); mandatory SAST + Code Reviewer and Security Agent passes before merge (§11, §14); never runs in the orchestrator process.

T-AI-3 — Tool-call abuse. A confused or injected agent invokes `git push --force`, `delete_project`, or shell tools outside its mandate. *Mitigations:* per-role server-side allowlists with parameter validation — the gateway rejects out-of-list calls before execution; destructive tools require an approval token (§4.4); all calls audited.

T-AI-4 — Secret leakage into LLM context. Repo files or logs containing secrets get embedded in prompts and cross TB4. *Mitigations:* secrets never enter agent context by construction (§6.3); pre-send redaction on every gateway request (§6.4); Ollama routing for sensitive projects.

T-AI-5 — Supply-chain risk in generated dependencies. Typosquatted, hallucinated ("slopsquatted"), or vulnerable packages proposed by agents. *Mitigations:* dependency gate — registry existence, age/popularity thresholds, vulnerability scan, pin + hash lock before install (§11.5).

T-AI-6 — Memory/RAG poisoning. Malicious "lessons learned" written into organizational memory influence all future projects. *Mitigations:* provenance metadata on every memory record (author, task, project, trust tier); cross-project retrieval only from human-promoted `curated` tier; per-project namespaces by default (Volume 5).

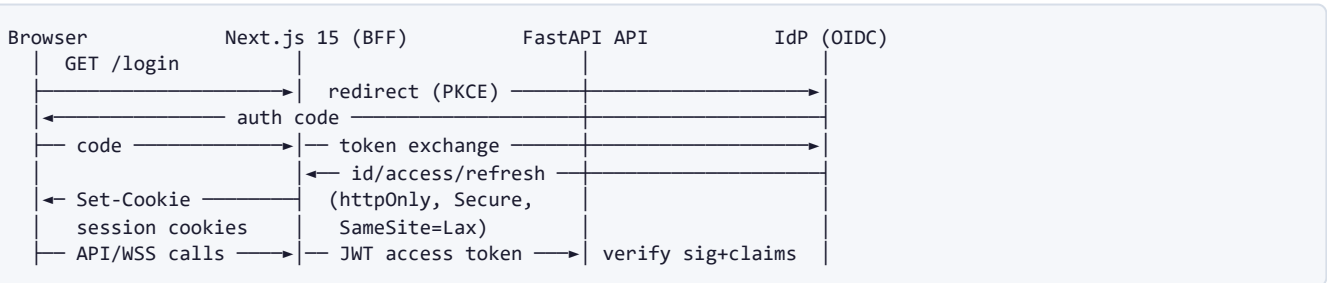
- **TITAN-SEC-003 (MUST):** The threat model MUST be re-reviewed at every minor release and whenever a new tool, agent type, or trust boundary is added. *Acceptance:* release checklist item; doc history shows updates.
- **TITAN-SEC-004 (MUST):** Every T-AI threat MUST have at least one mitigation that does not depend on LLM behavior. *Acceptance:* traceability table T-AI-1..6 → non-LLM controls; red-team tests (§14.4) verify each.

---

3. Authentication and Authorization Architecture

3.1 Overview

Humans authenticate via OAuth2/OIDC; sessions use JWT bearer tokens (canonical, Volume 1). Locally TITAN ships an embedded OIDC-compatible auth service (FastAPI + authlib, username/password + TOTP); production federates to any OIDC IdP (Keycloak as the free reference, or Google/GitHub/Okta).



3.2 Requirements — Authentication

- **TITAN-SEC-010 (MUST):** Login uses OAuth2 Authorization Code + PKCE; implicit and password grants disabled. *Acceptance:* implicit-flow attempt returns error.
- **TITAN-SEC-011 (MUST):** Access tokens are JWTs signed RS256 or EdDSA, lifetime ≤ 15 min, with `sub, org, roles, iat, exp, aud, iss, jti`; verifiers reject `alg=none` and HS256. *Acceptance:* unit tests reject none/HS256, expired, wrong `aud` / `iss`.
- **TITAN-SEC-012 (MUST):** Refresh tokens are opaque, single-use, rotated with reuse detection: presenting a consumed token revokes the family. Lifetime ≤ 14 days. *Acceptance:* replay test triggers family revocation + audit event.

- **TITAN-SEC-013 (MUST):** Browser tokens live only in `httpOnly, Secure, SameSite=Lax` cookies via the Next.js BFF — never in `localStorage` or client JS. *Acceptance:* frontend audit; cookie flags verified.
- **TITAN-SEC-014 (MUST):** Revocation takes effect ≤ 60 s via a Redis denylist of revoked `jti`/families checked per request. *Acceptance:* revoke-then-call fails within 60 s.
- **TITAN-SEC-015 (SHOULD):** TOTP MFA available in the embedded IdP; MUST be requireable per-org for Admin/Owner. *Acceptance:* policy toggle enforces MFA at next privileged login.
- **TITAN-SEC-016 (MUST):** WebSockets authenticate with the same JWT at handshake and close ≤ 60 s after expiry/revocation unless re-authenticated in-band. *Acceptance:* expired-token WSS refused; live connection closes on revocation.
- **TITAN-SEC-017 (MUST):** Failed logins throttled (per-account and per-IP backoff after 5 failures); embedded-IdP passwords hashed with Argon2id. *Acceptance:* brute-force test triggers backoff; DB shows Argon2id.

3.3 Service and Agent Identity

- **TITAN-SEC-018 (MUST):** Each agent instance runs with a service token scoped to `(agent_role, project_id, task_id)`, minted at task assignment, lifetime ≤ task timeout, never reused. *Acceptance:* a Backend-Engineer-scoped token calling a deploy endpoint is rejected.
- **TITAN-SEC-019 (MUST):** Inter-service auth (orchestrator↔gateway↔sandbox controller) uses mTLS in Kubernetes; local Compose MAY use isolated Docker networks + service tokens. *Acceptance:* plaintext inter-service call refused in prod profile.

3.4 Authorization Model

One enforcement point: a FastAPI dependency `require_permission(resource, action)` backed by the RBAC engine (§4), evaluated from JWT claims plus object ownership. The UI hides what you cannot do; the API enforces it.

3.5 Message Integrity on the Queue

- **TITAN-SEC-020 (MUST):** Every Redis Streams (and NATS JetStream at scale-out) message carries a signed envelope `{msg_id, sender_principal, task_id, ts, payload_sha256, hmac}` keyed per-service from the vault; consumers drop and audit invalid signatures. *Acceptance:* tampered message rejected + audit event.

ADR-SEC-01 — Token strategy: short-lived JWT + rotating refresh via BFF cookies

- **Context:** TITAN's dashboard renders LLM-derived content, raising XSS exposure; tokens in `localStorage` would be fatal under XSS.
- **Decision:** Next.js acts as Backend-for-Frontend holding tokens in httpOnly cookies; access JWTs ≤ 15 min; opaque single-use refresh tokens with family revocation; Redis denylist for immediate revocation.
- **Alternatives:** (a) long-lived stateless JWT — no revocation, rejected; (b) server-side sessions only — poor fit for WSS + service calls and horizontal scaling; (c) DPoP sender-constrained tokens — strongest, deferred pending library maturity.
- **Trade-offs:** per-request Redis lookup (sub-ms, already in stack) and one extra BFF hop; accepted for real revocation and XSS containment.

---

4. RBAC Roles and Permission Matrix

4.1 Roles

Human roles (org-scoped, project-assignable): **Owner, Admin, Maintainer, Developer, Viewer**. Machine role class: **Agent**, parameterized by agent type (§4.3). Roles are additive-only — no deny rules — keeping evaluation auditable.

4.2 Permission Matrix (humans)

Legend: ✓ allowed · A = allowed with approval workflow · — denied.

Resource / Action	Owner	Admin	Maintainer	Developer	Viewer
Org: delete org, billing/plan	✓	—	—	—	—
Org: manage members & roles	✓	✓	—	—	—
Org: security policy (MFA, egress, retention)	✓	✓	—	—	—
Project: create / delete	✓	✓	✓ / —	—	—
Project: configure agents & LLM providers	✓	✓	✓	—	—
Tasks: create, edit, reprioritize	✓	✓	✓	✓	—
Read code, dashboards, logs	✓	✓	✓	✓	✓
Approve merge of agent PRs	✓	✓	✓	A	—
Trigger production deploy	✓	✓	A	—	—
Secrets: create/rotate (write-only)	✓	✓	✓	—	—
Secrets: read plaintext	—	—	—	—	—
Audit log: read	✓	✓	—	—	—
Audit log: modify/delete	—	—	—	—	—
Plugins: install / configure	✓	✓	A	—	—
Sandbox policy: relax (e.g., egress)	✓	✓	A	—	—
Memory: promote to org-wide tier	✓	✓	✓	A	—

Deliberate rows: **no role reads secret plaintext** (write-only; only sandbox injection reads values, §6.3) and **no role alters the audit log**.

4.3 Agent Tool Allowlists

The tool gateway rejects out-of-list calls regardless of what the model asks for.

Agent type	Allowed tool classes	Explicitly denied
CEO / PM / Project Manager	task CRUD, messaging, docs read	shell, git write, deploy, secrets
Architect / Research	repo read, web fetch (allowlisted domains), docs write	shell, git push, deploy
Backend / Frontend / DB Engineer	sandbox shell, repo read/write on task branch, test run	protected-branch push, deploy, plugins

Agent type	Allowed tool classes	Explicitly denied
QA	sandbox shell, test run, repo read, bug report	git write outside test dirs, deploy
Security	scanners, repo read, report write	deploy, secret read
Code Reviewer	repo read, PR review API	any code write, deploy
DevOps	IaC repo write, deploy tool (approval token required)	secret plaintext read
Memory / Documentation	memory & docs read/write in namespace	shell, git, deploy

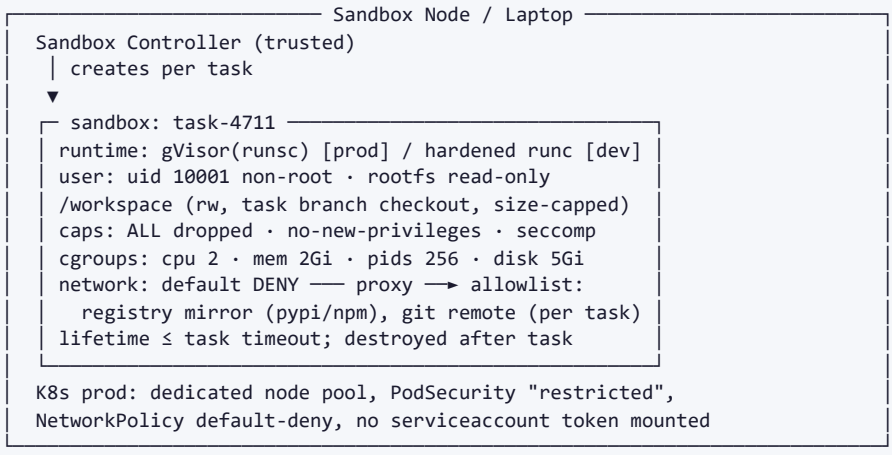
4.4 Requirements

- **TITAN-SEC-030 (MUST):** RBAC enforced server-side on every route and tool invocation via a single middleware; UI checks are advisory. *Acceptance:* automated suite calls every route with every role against a matrix-as-code fixture.
- **TITAN-SEC-031 (MUST):** Agent tool calls validated against allowlist AND task project scope AND parameter constraints (e.g., git writes only to `titan/task-*` branches). *Acceptance:* out-of-scope fuzz calls 100% rejected + audited.
- **TITAN-SEC-032 (MUST):** Object-level authorization on every ID reference — no IDOR. *Acceptance:* org-A token against org-B resources returns 404.
- **TITAN-SEC-033 (MUST):** Irreversible actions (prod deploy, project delete, force-push, sandbox policy relaxation) require a single-use approval token (≤ 15 min TTL) minted by the human approval flow. *Acceptance:* action without token fails; replay fails.
- **TITAN-SEC-034 (SHOULD):** Quarterly access review via an auto-generated report (members, roles, last activity). *Acceptance:* report endpoint exists.

---

5. Agent Execution Sandboxing

All LLM-generated code — builds, tests, scripts, migrations — executes in ephemeral per-task sandboxes. Nothing generated ever runs in the orchestrator process or directly on the host.



- **TITAN-SEC-040 (MUST):** Generated code executes only inside per-task sandboxes created/destroyed by the Sandbox Controller; never shared across tasks or projects. *Acceptance:* two sequential tasks show distinct container IDs and clean workspaces; orchestrator has no exec path for generated code.

- **TITAN-SEC-041 (MUST):** Sandboxes run non-root, all capabilities dropped, `no-new-privileges`, read-only rootfs (writable `/workspace`, `/tmp` only), seccomp at least runtime-default strict. *Acceptance:* in-sandbox probe fails privilege escalation and rootfs writes.
- **TITAN-SEC-042 (MUST):** Per-sandbox limits: CPU ≤ 2, memory ≤ 2 GiB, PIDs ≤ 256, disk ≤ 5 GiB, wall-clock ≤ task timeout (default 30 min); overridable only by Admin policy. *Acceptance:* fork-bomb and memory-hog tests killed without host impact.
- **TITAN-SEC-043 (MUST):** Sandbox network defaults to deny-all egress; only the package-registry mirror and the task's git remote are reachable, via an authenticating, logging forward proxy. Direct egress requires Admin opt-in per project (SEC-033 approval). *Acceptance:* in-sandbox `curl https://example.com` fails; registry install succeeds via proxy with log entries.
- **TITAN-SEC-044 (MUST):** Sandboxes never mount the Docker socket, Kubernetes service-account tokens, or any credentials beyond task-scoped injected secrets (§6.3). *Acceptance:* in-sandbox filesystem scan finds neither.
- **TITAN-SEC-045 (MUST):** In Kubernetes, sandboxes run in a dedicated namespace/node pool with PodSecurity `restricted`, default-deny NetworkPolicies, and gVisor RuntimeClass where supported. *Acceptance:* non-compliant pod rejected by admission.
- **TITAN-SEC-046 (SHOULD):** Sandbox stdout/stderr streams through the redaction filter (§6.4) before logs, UI, or LLM context. *Acceptance:* planted secret in sandbox output appears redacted at all sinks.

ADR-SEC-02 — Sandbox runtime: containers, gVisor in production, hardened runc locally

- **Context:** TITAN must run untrusted generated code on a single laptop (free) and in production Kubernetes.
- **Decision:** Hardened Docker containers everywhere (SEC-041/042/043); production adds gVisor (`runsc`) for syscall interposition; local Compose uses hardened runc (gVisor optional).
- **Alternatives:** (a) Firecracker microVMs — strongest isolation but KVM-dependent and laptop-hostile; documented as scale-out option; (b) WASM/WASI — cannot run arbitrary toolchains (venvs, node_modules, compilers); (c) plain containers only — insufficient against kernel exploits for a platform whose core feature is running attacker-influenceable code.
- **Trade-offs:** gVisor costs ~10–30% on I/O-heavy builds with occasional compatibility gaps — accepted in prod. Local dev accepts container-grade isolation (the laptop user is the only victim), compensated by egress-deny networking enforced at the proxy.

---

6. Secret Management

6.1 Secret Classes

(1) Platform: DB/Redis/Qdrant credentials, JWT signing keys, HMAC keys. (2) Integration: LLM provider keys, GitHub tokens, registry credentials, deploy keys. (3) User project secrets: env vars for generated apps.

6.2 Vault Architecture (local-first)

- **TITAN-SEC-050 (MUST):** All secrets live in a dedicated store — never in code, git, plain config, Redis, or Postgres application tables. Local default: **SOPS + age** encrypted files unlocked at boot into an in-memory secrets service; production: HashiCorp Vault / OpenBao or cloud managers behind the same interface.

Acceptance: gitleaks finds nothing in repo or image layers; only the secrets service holds plaintext, in memory.

- **TITAN-SEC-051 (MUST):** Write-only semantics for humans and APIs: values can be created, rotated, deleted — never read back via any API/UI. *Acceptance:* read attempts return 405/404.
- **TITAN-SEC-052 (MUST):** Every secret has metadata: owner, scope (org/project/task), timestamps, rotation period (default 90 days) with dashboard expiry warnings. *Acceptance:* overdue secrets raise alerts.

6.3 Delivery to Workloads

- **TITAN-SEC-053 (MUST):** Injection at execution time only, scoped to the consumer: platform services get env at boot; sandboxes get only the secrets their task's tool grants require, as tmpfs files destroyed with the sandbox. The LLM gateway alone holds provider keys — agents never see them. *Acceptance:* sandbox env/filesystem dump for a docs task contains zero integration secrets; only the gateway has provider-key env vars.

6.4 Keeping Secrets Out of Prompts and Logs

Enforcement layer for T-AI-4 and the "never in prompts/logs" rule.

- **TITAN-SEC-054 (MUST):** A redaction pipeline runs on (a) every context bundle before the gateway sends it to any provider, (b) every log line before Loki ingestion, (c) every sandbox output stream. It combines exact-match against the live secret inventory (hashed lookup), high-entropy detection, and provider-key patterns (`sk-ant-*` , `ghp_*` , `AKIA*` , …), replacing matches with `[REDACTED:<class>:<hash8>]` . *Acceptance:* planted secrets of each class appear redacted at every sink, including captured gateway payloads.
- **TITAN-SEC-055 (MUST):** Secret-bearing file patterns (`.env*` , `*.pem` , `*_key*` , `id_rsa*` , cloud credential paths) are excluded from context assembly and RAG ingestion by default. *Acceptance:* repo with `.env` shows the file absent from prompts and Qdrant.
- **TITAN-SEC-056 (SHOULD):** Orgs can pin sensitive projects to local models (Ollama) via gateway routing policy so context never leaves the machine. *Acceptance:* policy set → gateway log shows only local endpoints for that project.

ADR-SEC-03 — Local-first vault: SOPS+age default, Vault/OpenBao behind one interface

- **Context:** "Local-first and free" is a hard requirement; Vault is heavy for a laptop, but production needs dynamic secrets and access audit.
- **Decision:** A thin `SecretsProvider` interface (get-by-reference, rotate, list-metadata). Default driver: SOPS+age encrypted YAML (zero infrastructure, git-storable ciphertext). Production driver: Vault/OpenBao. Switching is configuration-only, mirroring the LLM gateway philosophy.
- **Alternatives:** (a) Vault everywhere — too heavy locally; (b) plain `.env` locally — unencrypted at rest, normalizes bad practice; (c) OS keychains — inconsistent across platforms and headless Docker.
- **Trade-offs:** SOPS lacks dynamic secrets and fine-grained audit — acceptable with a single local operator; the interface seam makes the production upgrade invisible to call sites.

---

7. Encryption at Rest and in Transit

- **TITAN-SEC-060 (MUST):** All external traffic (browser, WSS, webhooks, LLM providers, git) uses TLS 1.2+ (prefer 1.3) with HSTS (`max-age` ≥ 15552000) on public endpoints; plain HTTP redirects or is disabled. *Acceptance:* `testssl.sh` grade A; no non-loopback cleartext listeners outside the dev profile.
- **TITAN-SEC-061 (MUST):** Production intra-cluster traffic is mTLS (SEC-019); Postgres, Redis, and Qdrant require TLS + auth in production profiles. *Acceptance:* non-TLS connection to prod Postgres refused.
- **TITAN-SEC-062 (MUST):** At rest: production data volumes and backups use storage-layer encryption (LUKS/cloud volume encryption); backups are age/GPG-encrypted before leaving the host. *Acceptance:* backup artifact is ciphertext.
- **TITAN-SEC-063 (MUST):** Field-level AES-256-GCM (key from vault) for highest-sensitivity columns: stored OAuth tokens, webhook signing secrets. Key rotation via key-version tagging. *Acceptance:* DB dump shows ciphertext; rotation test decrypts old rows with old key version.
- **TITAN-SEC-064 (SHOULD):** JWT signing keys rotate ≤ every 90 days with JWKS publishing. *Acceptance:* staging rotation without auth downtime.

---

8. Input Validation and Output Encoding

- **TITAN-SEC-070 (MUST):** Every API request and queue message is validated by Pydantic v2 models with `extra="forbid"` , explicit types, length bounds, and enum/pattern constraints; failures return 422 without echoing raw input. *Acceptance:* hypothesis-based fuzz on public endpoints yields no 500s.
- **TITAN-SEC-071 (MUST):** All database access uses SQLAlchemy parameterized queries; string-built SQL is banned (including generated migrations), enforced by a semgrep CI rule. *Acceptance:* rule active; sqlmap against the API finds nothing.
- **TITAN-SEC-072 (MUST):** All LLM-derived content in the dashboard is untrusted: React default escaping, no `dangerouslySetInnerHTML` for model content, markdown via a sanitizing pipeline (rehype-sanitize allowlist), CSP `default-src 'self'` without `unsafe-inline` scripts. *Acceptance:* stored-XSS payloads in agent output render inert; ESLint rule bans `dangerouslySetInnerHTML` .
- **TITAN-SEC-073 (MUST):** LLM tool-call arguments are schema-validated and constraint-checked before execution: paths canonicalized and confined to `/workspace` ; shell arguments passed as argv arrays, never `sh -c` ; URLs checked against allowlists. *Acceptance:* red-team suite with `../../etc/passwd` , `; rm -rf /` , and internal/metadata URLs fully rejected.
- **TITAN-SEC-074 (MUST):** Outbound agent fetches (Research Agent, webhook callbacks) pass an SSRF guard: resolved IPs checked against private/link-local/metadata ranges, redirects re-validated, http(s) only. *Acceptance:* SSRF corpus blocked and audited.
- **TITAN-SEC-075 (MUST):** Inbound webhooks verify HMAC signatures; unsigned or stale (> 5 min skew) deliveries rejected. *Acceptance:* unsigned webhook → 401; replay rejected.
- **TITAN-SEC-076 (MUST):** Error responses are generic (correlation ID only); stack traces never reach clients. *Acceptance:* forced exception returns sanitized body; detail only in server logs.

---

9. Rate Limiting and Abuse Prevention

- **TITAN-SEC-080 (MUST):** Redis-backed token-bucket limits, layered: per-IP pre-auth (60 req/min), per-user (600 req/min), per-org, plus stricter per-route limits on auth (10/min), project creation (10/hour), and

LLM-triggering endpoints. 429 with `Retry-After`. *Acceptance:* load test observes limits ±5%; configurable per org.

- **TITAN-SEC-081 (MUST):** WebSocket controls: ≤ 10 concurrent connections per user, ≤ 20 msg/s per connection, backpressure then disconnect on sustained violation. *Acceptance:* flood test → disconnect + audit event.
- **TITAN-SEC-082 (MUST):** LLM budget caps per task, per project/day, per org/month (local models time-capped instead); breach pauses the task and notifies a Maintainer rather than silently truncating. *Acceptance:* runaway-loop simulation halts at cap with alert.
- **TITAN-SEC-083 (MUST):** Loop detection: ≥ N (default 3) near-identical consecutive tool calls, or step budget exceeded, pauses the task for review. *Acceptance:* seeded loop halts within N+1 iterations.
- **TITAN-SEC-084 (SHOULD):** Sandbox egress-proxy traffic rate-limited (default 100 requests/task) to blunt exfiltration. *Acceptance:* over-limit requests blocked and logged.

---

10. Audit Logging

10.1 Design

Append-only PostgreSQL table `audit_events` with INSERT-only grants for the application role (no UPDATE/DELETE at the DB privilege level), plus a hash chain (`event_hash = sha256(prev_hash || canonical_event_json)`) for tamper evidence. Events also stream to Loki for search and to object storage as daily sealed, age-encrypted segments.

10.2 Recorded Events

Authentication (success/failure, MFA, refresh/revocation, reuse detection); authorization denials; role/member changes; secret lifecycle (create/rotate/delete/inject — never values); agent task lifecycle; every tool call (agent, tool, parameter digest, allow/deny); approval-token mint/consume; sandbox create/destroy/policy exception; deploys; plugin install/config; memory-tier promotions; data export/deletion; scan results; redaction hits; rate-limit and SSRF blocks; incident-mode toggles.

- **TITAN-SEC-090 (MUST):** Each event records `{event_id, ts, actor, org, project, task, action, resource, outcome, source_ip, trace_id, event_hash}`; `trace_id` links to OpenTelemetry traces. *Acceptance:* catalog test exercises every category and asserts schema.
- **TITAN-SEC-091 (MUST):** The audit store is immutable to all application and human roles; a daily hash-chain verification job alerts on breaks. *Acceptance:* UPDATE/DELETE as app role fails at DB layer; corrupted row breaks verification and alerts.
- **TITAN-SEC-092 (MUST):** Audit events never contain secret values, full prompts, or code bodies — only references/digests; redaction (§6.4) applies. *Acceptance:* planted-secret action yields an event without the value.
- **TITAN-SEC-093 (MUST):** Retention defaults: 400 days online + 2 years sealed archive; org-configurable upward, downward only by Owner; legal-hold override. *Acceptance:* retention job tested on synthetic aged data.
- **TITAN-SEC-094 (SHOULD):** Admin-facing audit search (actor, action, resource, time) in the dashboard. *Acceptance:* seeded events found < 2 s at 1M rows.

---

11. Supply Chain Security and CI Scanning

11.1 Platform CI Gates

Every platform PR runs, in order: secret scan (gitleaks) → SAST (semgrep + bandit; ESLint security for TS) → dependency audit (pip-audit, npm audit, osv-scanner) → tests → container build → image scan (Trivy: CVEs, misconfig, licenses) → SBOM (Syft, CycloneDX) → image signing (cosign, keyless via CI OIDC).

- **TITAN-SEC-100 (MUST):** CI blocks merge on any detected secret, high+ SAST findings, or high/critical vulnerabilities without a recorded, expiring, Admin-approved exception. *Acceptance:* seeded vulnerable dep and planted secret each fail CI.
- **TITAN-SEC-101 (MUST):** Images built from digest-pinned bases, carry SBOMs, cosign-signed; production Kubernetes verifies signatures at admission (Kyverno/policy-controller). *Acceptance:* unsigned image rejected in prod cluster.
- **TITAN-SEC-102 (MUST):** All manifests use lockfiles with integrity hashes; CI installs are frozen-from-lockfile. *Acceptance:* lockfile drift fails CI.
- **TITAN-SEC-103 (SHOULD):** Weekly scheduled full-tree scans against fresh vulnerability feeds auto-open tracked issues. *Acceptance:* synthetic new CVE produces an issue.

11.2 Generated-Code Pipeline Gates

Generated project code passes the same gate classes inside its sandboxed pipeline (secret scan, SAST, dependency audit, tests) before Code Reviewer and Security Agent review, and before human merge approval (sequence wired in Volume 6).

11.3 Generated-Dependency Policy (T-AI-5)

- **TITAN-SEC-104 (MUST):** Before any agent-proposed dependency installs, the gate verifies: (a) package exists on the official registry (kills hallucinated packages); (b) age ≥ 30 days AND (weekly downloads ≥ 1000 OR Maintainer-allowlisted); (c) no high/critical vulnerabilities (osv-scanner); (d) name is not within Levenshtein distance 1 of a top-1000 package unless it is that package; (e) version pinned and hash-locked. Failures block install and open a review task. *Acceptance:* corpus of hallucinated, typosquatted, and vulnerable packages fully blocked; legitimate packages pass.
- **TITAN-SEC-105 (MUST):** Sandbox installs go only through the caching registry mirror (§5 proxy) — one enforcement and audit point. *Acceptance:* direct-registry install fails; mirror logs show installs.

---

12. OWASP Compliance Mapping

12.1 OWASP Top 10 (2021)

OWASP	TITAN controls	Requirements
A01 Broken Access Control	Central RBAC middleware, object-level checks, agent allowlists, approval tokens	SEC-030..034
A02 Cryptographic Failures	TLS everywhere, storage + field encryption, Argon2id, key rotation	SEC-017, 060..064

OWASP	TITAN controls	Requirements
A03 Injection	Pydantic validation, parameterized SQL, argv-only shell, sanitized rendering, CSP	SEC-070..073
A04 Insecure Design	Living STRIDE + AI threat model, ADRs, defense in depth	SEC-003, 004
A05 Security Misconfiguration	Secure-defaults checklist, PodSecurity restricted, CSP/HSTS	SEC-045, 060, §15
A06 Vulnerable Components	CI dep/image scanning, lockfiles, weekly rescans, dependency gate	SEC-100..105
A07 Identification & Auth Failures	OIDC+PKCE, MFA, rotation + reuse detection, throttling, WSS auth	SEC-010..017
A08 Software & Data Integrity	Signed images + admission verification, signed envelopes, hash-chained audit	SEC-020, 091, 101
A09 Logging & Monitoring Failures	Full audit catalog, Loki + OTel correlation, tamper evidence	SEC-090..094
A10 SSRF	Egress-deny sandboxes, SSRF guard, metadata-IP blocklist	SEC-043, 074

12.2 OWASP LLM Top 10 (2025)

OWASP LLM	TITAN controls	Requirements
LLM01 Prompt Injection	Untrusted-content tagging, model-independent tool authz, approval gates, egress deny	SEC-001, 031, 033, 043
LLM02 Sensitive Information Disclosure	Redaction pipeline, secret-file exclusion, local-model routing	SEC-050..056
LLM03 Supply Chain	Dependency gate, registry mirror, SBOM + signing	SEC-100..105
LLM04 Data & Model Poisoning	Memory provenance tiers, namespace isolation, promotion review	§2.3; Vol. 5
LLM05 Improper Output Handling	Output sanitization, tool-arg validation, sandbox-only execution	SEC-072, 073, 040
LLM06 Excessive Agency	Per-agent allowlists, task-scoped tokens, approval tokens	SEC-018, 031, 033
LLM07 System Prompt Leakage	No secrets in prompts by construction; prompts treated as non-secret config	SEC-053, 054
LLM08 Vector & Embedding Weaknesses	Per-project Qdrant namespaces, authz on retrieval, provenance filtering	§2.3; Vol. 5
LLM09 Misinformation	Reviewer/QA/Security agent pipeline, tests before merge, human approval	Vol. 6; SEC-033
LLM10 Unbounded Consumption	Token budgets, loop detection, rate limits, sandbox caps	SEC-042, 080..084

- **TITAN-SEC-110 (MUST):** These mappings are kept as machine-readable traceability (`security/compliance-map.yaml`) checked in CI. *Acceptance:* removing a mapped test fails a CI consistency check.

---

13. Incident Response Runbook

13.1 Severity Levels

Sev	Definition	Examples	Target
SEV-1	Active compromise or secret exposure	Provider key leaked; sandbox escape; audit chain broken	Contain ≤ 1 h
SEV-2	Exploitable weakness, no confirmed abuse	Auth bypass; injection reaching tool layer	Mitigate ≤ 24 h
SEV-3	Control failure without direct exposure	Scanner outage; redaction false-negative class	Fix ≤ 7 d

13.2 Phases

1. **Detect & report:** alerts from audit anomalies and Prometheus rules (auth-failure spikes, egress-proxy denials, budget breaches, chain-verification failure); `SECURITY.md` intake for external reporters with 90-day coordinated disclosure.
2. **Triage:** on-call Maintainer assigns severity, opens a timestamped, immutable incident record.
3. **Contain:** one-command lockdown — `titan incident lockdown` pauses all agent execution, freezes deploys, revokes agent service tokens, forces privileged re-auth. Targeted: revoke token family, kill sandbox, disable plugin, rotate secret.
4. **Eradicate & recover:** rotate affected credentials, patch, redeploy from signed images, verify audit chain, restore from encrypted backups if integrity is in doubt.
5. **Post-incident:** blameless review within 7 days; outcomes become new requirements and regression tests; user notification per disclosure policy.

13.3 AI-Specific Playbooks

- **P1 Prompt-injection incident:** quarantine offending content; replay the task trace (trace_id) to map influenced tool calls; revert downstream artifacts; add exploit to red-team corpus.
- **P2 Malicious generated code merged:** locate via artifact digests which branches/deployments contain it; auto-open revert PRs; rescan history; review the gate miss.
- **P3 Secret in LLM context:** rotate immediately (assume compromised — provider logs are outside our control); audit which providers received it; strengthen redaction rule; notify org Admins.
- **P4 Memory poisoning:** freeze tier promotions; query memory by provenance for records from the tainted actor/window; purge and rebuild derived embeddings.
- **TITAN-SEC-120 (MUST):** Lockdown and all four playbooks are implemented and rehearsed via game-days twice yearly. *Acceptance:* staging lockdown drill halts agents ≤ 60 s; drill reports archived.
- **TITAN-SEC-121 (MUST):** The repo contains `SECURITY.md` with reporting channel, SLAs, disclosure and supported-version policy. *Acceptance:* file present, linked from README.

---

14. Security Testing

14.1 SAST and Static Controls (every PR)

semgrep with a custom TITAN ruleset (no string SQL, no `sh -c`, no `dangerouslySetInnerHTML`, no unvalidated tool-arg use), bandit, ESLint security + strict TypeScript, gitleaks, IaC scanning (Trivy config/Checkov on K8s manifests and Compose files).

14.2 DAST (nightly, staging)

OWASP ZAP baseline + authenticated full scan against the staging API and dashboard; schemathesis fuzzing from the OpenAPI schema. Findings ≥ high auto-open blocking issues.

14.3 Penetration Test Scope

Annual and pre-1.0: external web/API surface; multi-tenant abuse (cross-org IDOR); WebSocket layer; **sandbox escape from inside an agent sandbox** (explicitly in scope, via a harness task granting agent-equivalent code execution); egress-proxy bypass; queue message forgery; vault interface. Out of scope: third-party LLM provider infrastructure; DoS beyond rate-limit verification.

14.4 LLM Red-Teaming (continuous)

- **TITAN-SEC-130 (MUST):** A red-team corpus of injection and tool-abuse payloads (seeded from public suites such as garak/PyRIT plus TITAN-specific cases: injected README, malicious issue text, poisoned memory record, hostile web page for the Research Agent) runs weekly in CI against a staging orchestrator with a real gateway and mock destructive tools. Pass: zero unauthorized tool executions, zero secret-bearing gateway payloads. *Acceptance:* pipeline publishes a scorecard and blocks release on regression.
- **TITAN-SEC-131 (MUST):** Every incident and pen-test finding becomes a permanent regression test. *Acceptance:* traceability from findings register to test IDs.
- **TITAN-SEC-132 (SHOULD):** Coverage-guided fuzzing on the highest-risk parsers (tool-call argument parser, webhook parser, envelope verifier). *Acceptance:* fuzz targets exist with CI time-box.

---

15. Secure Defaults Checklist

Every fresh install ships with these defaults; a built-in `titan security check` command verifies them and prints a scorecard.

#	Default	Ref
1	Sandbox egress: deny-all (proxy allowlist only)	SEC-043
2	Auto-deploy and auto-merge OFF (human approval required)	SEC-033
3	Services bind loopback/internal networks; only UI/API exposed	SEC-060
4	First admin: forced password change + MFA prompt at first login	SEC-015, 017
5	TLS on all non-loopback listeners; HSTS on public endpoints	SEC-060
6	JWT RS256/EdDSA, 15-min expiry; refresh rotation on	SEC-011, 012

#	Default	Ref
7	SOPS+age vault initialized; secret-file patterns excluded from context/RAG	SEC-050, 055
8	Redaction pipeline on gateway, logs, sandbox output	SEC-054
9	Rate limits and LLM budget caps on, conservative defaults	SEC-080, 082
10	Audit log on: INSERT-only grants, hash chain active	SEC-090, 091
11	Containers non-root, read-only rootfs, caps dropped, limits set	SEC-041, 042
12	Security gates enabled by default in generated projects' pipelines	SEC-100, 104
13	Memory: per-project namespaces; cross-project retrieval off until promotion	§2.3
14	Generic errors + correlation IDs (detail suppression on)	SEC-076
15	Prometheus/Grafana/Loki endpoints auth-protected, not public	SEC-060; Vol. 9

- **TITAN-SEC-140 (MUST):** `titan security check` exists, verifies all rows, exits non-zero on failure; CI runs it against the Compose profile. *Acceptance:* seeded misconfiguration is detected.

---

16. Risks and Open Questions

ID	Risk / Question	Position
R-1	Prompt injection is not fully solvable; soft controls will be bypassed.	Accepted: architecture assumes bypass and leans on hard controls (SEC-001/031/033/043/054). Residual risk: manipulated-but-authorized actions within an agent's legitimate scope; monitored via red-team scorecard trend.
R-2	gVisor overhead/compatibility may frustrate heavy builds.	Per-project runtime-class override (Admin-approved, audited); Firecracker evaluation deferred to Volume 10 timeframe.
R-3	Redaction false negatives (novel key formats, secrets split across chunks).	Entropy + inventory hashing reduce but cannot eliminate; P3 playbook covers escapes. Open: chunk-boundary-aware scanning cost.
R-4	Laptop deployments cannot enforce mTLS/gVisor; isolation is weaker.	Loopback binding, egress-deny proxy, identical authz code paths; documented honestly in install docs.
R-5	Dependency-gate thresholds may block legitimate niche packages.	Maintainer allowlist path; tune thresholds from telemetry. Open: per-language calibration.
R-6	Postgres hash chain is tamper-evident, not tamper-proof, against a DB superuser.	Daily sealed off-store archives bound exposure to ≤ 24 h. Open: external timestamp anchoring — deferred.
R-7	Provider-side prompt retention (TB4) is outside TITAN's control.	Local-model routing (SEC-056) is the strong answer; document per-provider retention settings in gateway drivers.
Q-1	Should approval tokens require WebAuthn step-up?	Leaning yes for production 1.0; SHOULD-level in a future revision.
Q-2	Multi-tenant SaaS hosting needs per-tenant sandbox node isolation review.	Out of MVP scope; flagged for Volume 10 enterprise track.

---

17. Cross-References

- **Volume 1 — Foundation:** canonical stack decisions and non-functional requirements satisfied here.
- **Volume 2 — Multi-Agent Intelligence:** Security Agent role spec; agent authority constrained by §4.3.
- **Volume 3 — Backend Engineering:** FastAPI middleware ordering; Redis Streams topology carrying §3.5 envelopes.
- **Volume 4 — Frontend:** dashboard surfaces for audit search, secret rotation, approvals, security scorecard.
- **Volume 5 — AI Brain:** memory tiers, provenance metadata, and namespace isolation referenced by T-AI-6, LLM04, LLM08.
- **Volume 6 — Engineering Workflow:** placement of review/approval gates (SEC-033, §11.2) in the idea-to-release pipeline.
- **Volume 7 — Integrations:** plugin permission model and webhook handling governed by SEC-075 and §4.2.
- **Volume 9 — Production:** monitoring/alerting feeding §13 detection; encrypted backups (SEC-062); CI/CD hosting §11 gates.
- **Volume 10 — Future:** marketplace agent vetting, enterprise isolation (Q-2), evolution guardrails built on this containment model.

End of Volume 8 — Security Specification (TITAN-VOL-08, Draft v1.0).

PROJECT TITAN — Autonomous Software Engineering Platform

Volume 9 — Production, Deployment & Operations

Field	Value
Series	PROJECT TITAN — Autonomous Software Engineering Platform Specification
Volume	9 of 10 — Production, Deployment & Operations
Doc ID	TITAN-VOL-09
Requirement Prefix	TITAN-OPS-NNN
Status	Draft v1.0
Date	2026-07-04

Abstract. This volume specifies how TITAN is built, shipped, and operated: the GitHub Actions CI/CD pipeline; the three canonical environments (local Docker Compose, staging and production Kubernetes); the Kubernetes deployment architecture with autoscaling and resource governance; blue/green releases with rollback, feature flags, and online database migrations; the observability stack (Prometheus, Grafana, Loki, OpenTelemetry) including agent-level metrics for tokens, cost, and task throughput; SLOs and alerting; high availability and disaster recovery with explicit RPO/RTO targets; the scaling and queue-backpressure model; LLM cost controls with per-project budgets and local-model fallback; a production readiness checklist; and the four core operational runbooks. It is written so that a DevOps engineer — or TITAN's own DevOps Agent — can implement it directly. Canonical stack decisions are owned and justified by Volume 1; this volume applies them.

Table of Contents

- 1. Scope, Principles & Definitions
- 2. Environments
- 2.1 Environment Matrix
- 2.2 Local — Docker Compose
- 2.3 Staging — Kubernetes
- 2.4 Production — Kubernetes
- 3. CI/CD Pipeline Specification
- 3.1 Pipeline Stages
- 3.2 Workflow Definitions
- 3.3 Image Build & Publish
- 3.4 Deployment Gates
- 4. Kubernetes Deployment Architecture
- 4.1 Cluster Topology (ASCII)
- 4.2 Workloads, Services & Ingress

- 4.3 Autoscaling (HPA)
- 4.4 Resource Requests & Limits
- 5. Release Engineering
- 5.1 Blue/Green Deployment
- 5.2 Rollback Strategy
- 5.3 Feature Flags
- 5.4 Database Migrations in Deployment
- 6. Monitoring & Observability
- 6.1 Prometheus Metrics (Platform & Agent-Level)
- 6.2 Grafana Dashboards
- 6.3 Loki Log Aggregation
- 6.4 OpenTelemetry Tracing Across Agent Pipelines
- 7. SLOs & Alerting
- 8. High Availability & Disaster Recovery
- 8.1 HA Design
- 8.2 Backup Schedule
- 8.3 Restore Procedure & RPO/RTO
- 9. Scaling Model & Backpressure
- 10. Cost Optimization & LLM Spend Controls
- 11. Production Readiness Checklist
- 12. Operational Runbooks (Outline)
- 13. Decision Records
- 14. Risks & Open Questions
- 15. Cross-References

---

1. Scope, Principles & Definitions

This volume owns everything between "code merged" and "value delivered reliably in production": CI/CD, environments, Kubernetes architecture, releases, observability, alerting, DR, scaling, and cost. It does not re-litigate stack choices (Volume 1), backend internals (Volume 3), or security controls (Volume 8).

Operating principles:

1. **Local-first parity.** Every production capability must have a free, local equivalent. A contributor on one laptop with Docker Compose and Ollama must be able to run the entire platform, including the observability stack, at zero cost. This is a hard project requirement.
2. **Boring deploys.** Deployments are frequent, automated, reversible, and observable. A deploy that cannot be rolled back in under 5 minutes is a defect.
3. **Agents are workloads.** TITAN's AI agents are ordinary Kubernetes workloads with metrics, budgets, and autoscaling — not magic. Token spend is a first-class operational metric alongside CPU and memory.
4. **Everything as code.** Kubernetes manifests, dashboards, alert rules, and runbooks live in the monorepo under `infra/` and `ops/`; changes flow through the same PR + review process as application code.

Definitions. *Deploy* — replacing running workload versions. *Release* — exposing behavior to users (decoupled from deploy via feature flags). *RPO* — maximum tolerable data loss window. *RTO* — maximum tolerable restoration time. *Blue/green* — two identical production stacks where traffic switches atomically between them.

> **TITAN-OPS-001 (MUST)** — The complete platform, including observability, MUST run on a single developer laptop via `docker compose up` using only free/local tools (Ollama for models). *Acceptance:* Fresh clone → `cp .env.example .env` → `docker compose up` yields a working dashboard, one completed agent task via Ollama, and Grafana at `localhost:3001` — with no paid credentials.
>
> **TITAN-OPS-002 (MUST)** — All infrastructure (K8s manifests, Compose files, dashboards, alert rules) MUST be version-controlled in the monorepo and changed only via pull request. *Acceptance:* `kubectl diff -k infra/k8s/overlays/production` against the live cluster shows no drift after each deploy; CI fails on manifest lint errors.

---

2. Environments

2.1 Environment Matrix

Property	Local	Staging	Production
Platform	Docker Compose	Kubernetes (small cluster or k3s)	Kubernetes
Namespace	n/a	`titan-staging`	`titan-prod`
LLM provider	Ollama (default)	Cheap cloud tier + Ollama	Configured per org; gateway-routed
Data	Seeded fixtures	Anonymized synthetic data	Real data
TLS	None (localhost)	Let's Encrypt staging CA	Let's Encrypt / org CA
Deploy trigger	Manual	Auto on merge to `main`	Manual approval on tagged release
Replicas	1 each	1–2 each	Per Section 4.4
Backups	None	Weekly (retention 7 days)	Per Section 8.2

> **TITAN-OPS-003 (MUST)** — Staging MUST run the same images (identical digests) later promoted to production; only configuration (env, secrets, replicas) may differ. *Acceptance:* The production deploy job refuses any image digest not previously deployed to staging (verified via the release manifest).

2.2 Local — Docker Compose

`docker-compose.yml` at the repo root defines: `frontend` (Next.js 15 dev server), `api` (FastAPI + Uvicorn, hot reload), `orchestrator` (agent scheduler), `worker` (agent executor, scalable via `--scale worker=N`), `postgres:16`, `redis:7`, `qdrant`, `ollama`, plus an observability profile (`--profile obs`) with `prometheus`, `grafana`, `loki`, `promtail`, and `otel-collector`. All services share one bridge network; volumes persist Postgres, Qdrant, and Ollama model data.

2.3 Staging — Kubernetes

Staging is a scaled-down replica of production in the `titan-staging` namespace (or a separate small/k3s cluster for cost). Every merge to `main` auto-deploys here. Staging runs the full E2E suite nightly and is the mandatory soak environment before production promotion (minimum 2 h with synthetic agent workload).

2.4 Production — Kubernetes

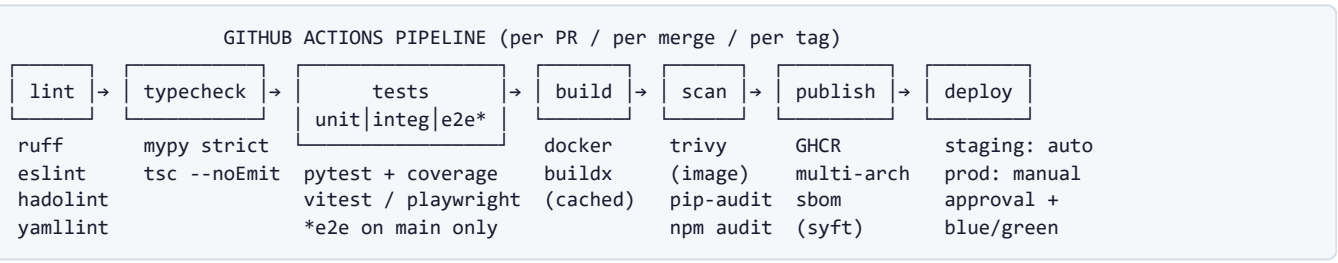
Production runs in the `titan-prod` namespace with the topology of Section 4. Access is restricted per Volume 8 RBAC; humans get read-only `kubectl` by default, with break-glass write access audited and time-boxed.

---

3. CI/CD Pipeline Specification

3.1 Pipeline Stages

The canonical pipeline is GitHub Actions. Stage order is fixed: **lint** → **typecheck** → **tests** → **build** → **scan** → **publish images** → **deploy**.



Stage	Tools	Fail condition
lint	`ruff check`, `ruff format --check`, `eslint`, `prettier --check`, `hadolint`, `yamllint`, `kubeconform` on manifests	Any violation
typecheck	`mypy --strict` (backend), `tsc --noEmit` (frontend)	Any type error
tests	`pytest` (unit + integration w/ Postgres+Redis service containers), `vitest`; Playwright E2E on `main` merges and release tags	Failure, or backend coverage < 80% / frontend < 70%
build	`docker buildx` multi-stage builds for `api`, `orchestrator`, `worker`, `frontend`, `migrate`; layer cache via GHA cache	Build error
scan	`trivy image` (CRITICAL/HIGH block), `pip-audit`, `npm audit --audit-level=high`, `gitleaks` (secrets)	Any blocking finding (see Volume 8 for triage/waiver process)
publish	Push to GHCR `ghcr.io/titan-platform/<service>` tagged `sha-<short>` and `vX.Y.Z` on tags; SBOM attached via `syft`; images signed with `cosign` keyless	Push/sign failure
deploy	`kubectl apply -k` via OIDC-federated service account (no long-lived cloud keys)	Rollout not healthy in 10 min

> **TITAN-OPS-004 (MUST)** — Every PR MUST pass lint, typecheck, unit + integration tests, and dependency/secret scans before merge; branch protection enforces this with no admin bypass.

> *Acceptance:* GitHub branch protection on `main` requires the named checks; a PR with a failing check cannot be merged.
>
> **TITAN-OPS-005 (MUST)** — Container images MUST be immutable, digest-addressed, signed (cosign), and carry an SBOM; deploy manifests MUST pin digests, never floating tags. *Acceptance:* Kyverno/Gatekeeper policy in-cluster rejects unsigned images and `:latest` tags; `kubectl get deploy -o yaml` shows `@sha256:` digests only.
>
> **TITAN-OPS-006 (MUST)** — Merge to `main` MUST auto-deploy to staging; production deploys MUST require a signed git tag `vX.Y.Z` plus manual approval in a GitHub Environment with required reviewers. *Acceptance:* Audit log shows approver identity for every production deploy; untagged commits cannot reach production.

3.2 Workflow Definitions

Workflows live in `.github/workflows/`:

- `ci.yml` — PR pipeline (lint → typecheck → tests → build → scan; no publish).
- `main.yml` — on merge to `main`: full pipeline, publish `sha-*` images, deploy staging, run smoke tests, run nightly-tier E2E.
- `release.yml` — on tag `v*`: retag staged digests, generate changelog (conventional commits), create GitHub Release, await approval, execute blue/green production deploy (Section 5.1).
- `nightly.yml` — full E2E on staging, load test (k6, 15-minute profile), Trivy re-scan of *deployed* digests (catches newly disclosed CVEs), backup verification job (Section 8.3).

Monorepo path filtering (`dorny/paths-filter`) skips unaffected jobs (frontend-only PRs skip backend tests) but the merge queue always runs the full suite.

3.3 Image Build & Publish

Five images, one `Dockerfile` per service under `infra/docker/`: `titan-frontend` (Next.js standalone output, node:22-slim, non-root), `titan-api`, `titan-orchestrator`, `titan-worker` (all three from a shared python:3.12-slim base stage with uv-installed dependencies, non-root, read-only root FS), and `titan-migrate` (Alembic + migration scripts only). Multi-arch: `linux/amd64,linux/arm64`.

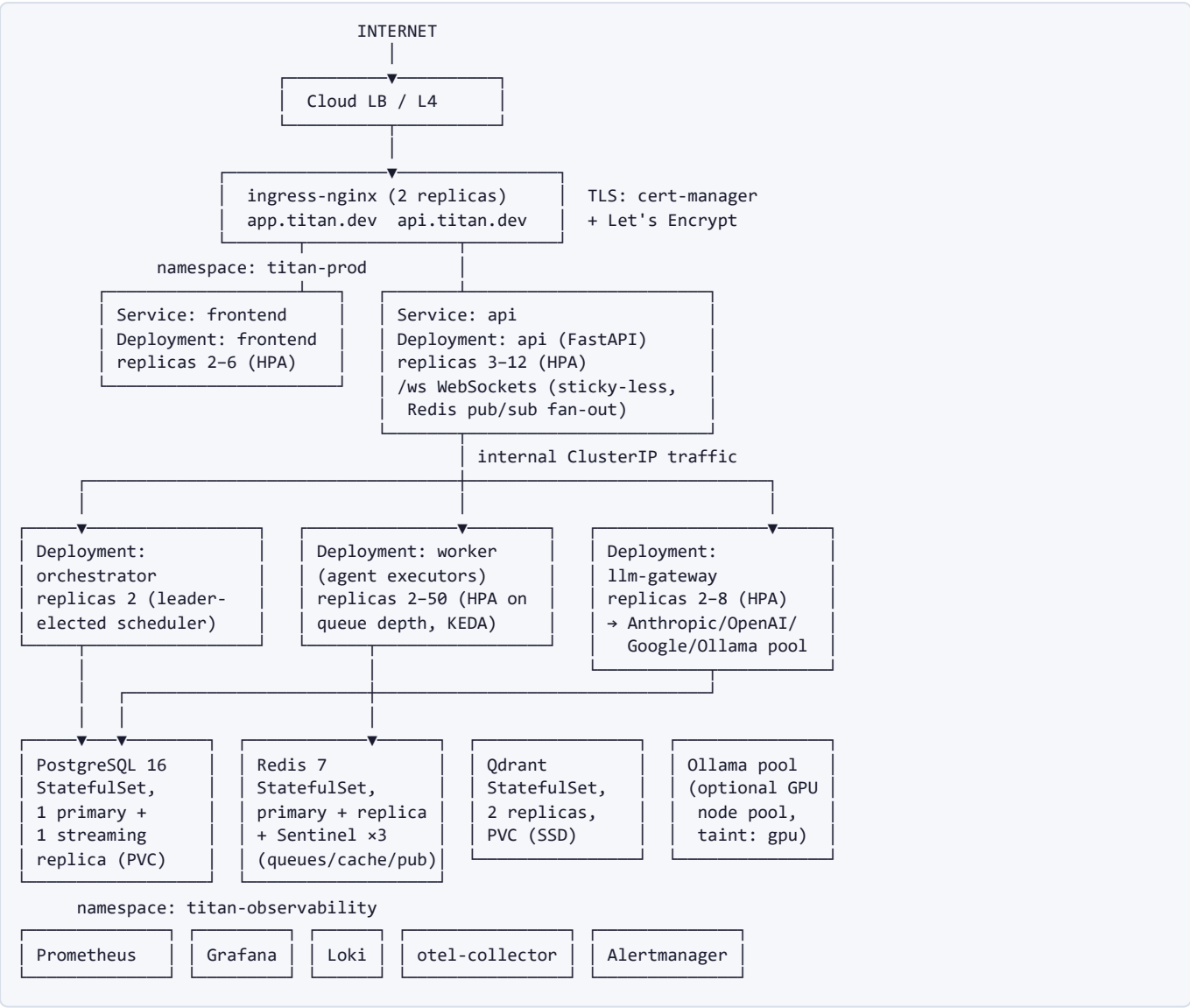
3.4 Deployment Gates

Production promotion requires, in order: (1) digest served ≥ 2 h in staging; (2) staging error-rate SLO not breached during soak; (3) E2E suite green on that digest; (4) no open SEV-1/SEV-2 incident; (5) human approval. Gates 1–3 are enforced mechanically by the `release.yml` pre-deploy job querying Prometheus and the CI API; gates 4–5 are the approval step.

---

4. Kubernetes Deployment Architecture

4.1 Cluster Topology (ASCII)



4.2 Workloads, Services & Ingress

Workload	Kind	Purpose	Service	Exposure
frontend	Deployment	Next.js 15 UI	ClusterIP	Ingress `app.titan.dev`
api	Deployment	FastAPI REST + WebSockets	ClusterIP	Ingress `api.titan.dev` (`/ws` with extended timeouts)
orchestrator	Deployment	Task graph scheduling, agent lifecycle	ClusterIP (metrics only)	Internal
worker	Deployment	Executes agent tasks from Redis Streams	Headless (metrics)	Internal
llm-gateway	Deployment	Provider-agnostic LLM routing, budgets, fallback	ClusterIP	Internal only

Workload	Kind	Purpose	Service	Exposure
postgres	StatefulSet	System of record	ClusterIP	Internal; no Ingress ever
redis	StatefulSet ×3 + Sentinel	Queues, cache, pub/sub	ClusterIP	Internal
qdrant	StatefulSet	Vector memory	ClusterIP	Internal
titan-migrate	Job (Helm hook / pre-deploy)	Alembic migrations	n/a	n/a

WebSocket connections terminate on any `api` pod; cross-pod event fan-out uses Redis pub/sub, so no session affinity is required (Volume 3 owns the eventing contract). NetworkPolicies default-deny; only the arrows in Section 4.1 are permitted flows (details in Volume 8).

> **TITAN-OPS-007 (MUST)** — All stateful services (PostgreSQL, Redis, Qdrant) MUST run as StatefulSets with PVCs on SSD storage classes and MUST NOT be exposed outside the cluster. *Acceptance:* NetworkPolicy tests (part of nightly suite) confirm datastore ports unreachable from Ingress and from outside their allowed client set.
>
> **TITAN-OPS-008 (MUST)** — Every pod MUST define liveness, readiness, and startup probes; API/worker readiness MUST gate on dependency health (DB, Redis) so rollouts halt on broken configuration. *Acceptance:* Deploying an image with an invalid DB DSN to staging results in a stalled rollout (old pods still serving), not an outage.

4.3 Autoscaling (HPA)

Workload	Min	Max	Signal	Target
frontend	2	6	CPU	70%
api	3	12	CPU + `http_requests_in_flight` (custom)	70% / 50 in-flight per pod
worker	2	50	KEDA: Redis Streams pending entries (`XPENDING` lag per consumer group)	10 pending tasks per replica
llm-gateway	2	8	CPU + in-flight LLM calls	70% / 20 per pod

Workers scale on **queue depth, not CPU** — agent tasks are I/O-bound (waiting on LLM responses). KEDA's Redis Streams scaler reads consumer-group lag directly. Scale-down stabilization is 300 s to avoid flapping; workers drain gracefully (finish current task, ack, exit) within a 120 s `terminationGracePeriodSeconds`.

> **TITAN-OPS-009 (MUST)** — Worker autoscaling MUST be driven by Redis Streams consumer-group lag via KEDA, with graceful drain on scale-down (no task lost or double-executed beyond at-least-once semantics). *Acceptance:* Load test injecting 500 tasks scales workers to plateau and back down to min within 10 min of drain, with zero unacked-lost tasks (verified via task ledger in Postgres).

4.4 Resource Requests & Limits

Workload	CPU req / lim	Mem req / lim	Notes
frontend	100m / 500m	256Mi / 512Mi	
api	250m / 1000m	512Mi / 1Gi	Uvicorn, 1 process/pod; scale via replicas
orchestrator	250m / 1000m	512Mi / 1Gi	
worker	250m / 1000m	512Mi / 2Gi	Memory head-room for large contexts
llm-gateway	100m / 500m	256Mi / 512Mi	Mostly I/O wait
postgres	1000m / 2000m	2Gi / 4Gi	Tune `shared_buffers` = 1Gi
redis	250m / 1000m	1Gi / 2Gi	`maxmemory 1536mb`, `noeviction` on queue DB
qdrant	500m / 2000m	1Gi / 4Gi	

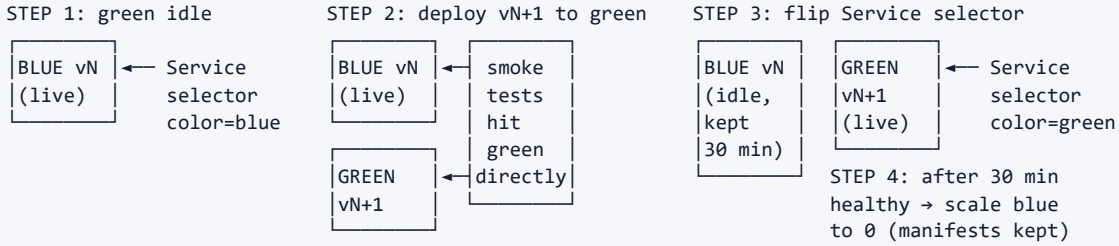
Namespace `ResourceQuota` caps `titan-prod` totals; a `LimitRange` injects defaults so no pod ships without requests. Memory limits are hard (OOMKill is a signal, alerted on); CPU limits are set generously to avoid throttling latency-sensitive paths.

---

5. Release Engineering

5.1 Blue/Green Deployment

Stateless tiers (frontend, api, orchestrator, worker, llm-gateway) deploy blue/green; datastores are shared across colors (hence the migration discipline in 5.4).



Mechanics: each Deployment exists twice with a `color` label; the stable Service selects `app=<svc>,color=<active>`. The release job (1) applies vN+1 to the inactive color, (2) waits for rollout and runs smoke tests against the color-specific preview Service, (3) patches the stable Service selector — an atomic, sub-second flip; existing WebSockets drain within the 120 s grace window and clients auto-reconnect per Volume 4's contract, (4) watches SLO burn for 30 minutes, then scales the old color to zero. Workers flip by switching which color's consumer group is scaled up; in-flight tasks on the old color drain to completion.

> **TITAN-OPS-010 (MUST)** — Production deploys MUST use blue/green with automated smoke tests against the idle color before traffic flip, and a 30-minute observation window before decommissioning the previous color. *Acceptance:* Release pipeline logs show smoke-test pass on the preview endpoint, timestamped flip, and old color retained ≥ 30 min for every production release.

5.2 Rollback Strategy

Rollback is the selector flip in reverse and completes in under one minute while the old color is warm. Triggers: automatic when the post-flip watch detects SLO fast-burn (Section 7) within the 30-minute window; manual anytime via `ops/scripts/rollback.sh <service|all>`. After the old color is decommissioned, rollback = redeploy of the previous digest through the same blue/green flow (target < 15 min). Database migrations are **never** rolled back in production (see 5.4); code must roll back cleanly against the newer schema.

> **TITAN-OPS-011 (MUST)** — Automated rollback MUST trigger on fast-burn SLO breach during the 30-minute post-flip window; manual rollback MUST complete in < 5 minutes while the previous color is warm. *Acceptance:* Chaos drill (mandatory per release-candidate cycle, staged in staging): deploy a build with injected 5xx faults; system auto-reverts without human action and pages the on-call.

5.3 Feature Flags

TITAN self-hosts flags (local-first; no SaaS dependency): a `feature_flags` table in PostgreSQL (flag key, description, type: boolean/percentage/allowlist, scope: global/org/project, state, owner, expiry date), cached in Redis with a 10 s TTL and busted on write via pub/sub. The API exposes flags to the frontend via a `/v1/flags` endpoint and to agents via the orchestrator context. Admin UI lives in the dashboard's Settings area (Volume 4).

Flag classes: **release flags** (new features ship dark, flipped after deploy — decoupling deploy from release), **ops flags** (kill switches: `agents.pause_all`, `llm.provider_override`, `intake.read_only`), and **experiment flags** (percentage rollouts). Every flag has an owner and an expiry; CI warns on flags older than 90 days.

> **TITAN-OPS-012 (MUST)** — Risky changes (new agent behaviors, new external integrations, schema-adjacent logic) MUST ship behind release flags defaulting OFF; global kill switches `agents.pause_all` and `llm.provider_override` MUST exist and take effect within 15 seconds. *Acceptance:* Flipping `agents.pause_all` in the admin UI halts new task dispatch across all workers within 15 s (verified by dispatch-rate metric dropping to zero) without pod restarts.

5.4 Database Migrations in Deployment

Migrations are Alembic scripts run by the `titan-migrate` Job **before** the new color receives traffic, under the **expand → migrate → contract** discipline:

1. **Expand (release N):** additive-only DDL — new tables, nullable columns, new indexes built `CONCURRENTLY`. Old and new code both work.
2. **Migrate (release N or N+1):** backfill data in batches (≤ 5,000 rows per transaction, sleep between batches); dual-write behind a flag if needed.
3. **Contract (release N+2 at the earliest):** drop old columns/tables only after no deployed code references them and a full backup cycle has passed.

Rules: the migrate Job takes a Postgres advisory lock (one migrator at a time); any statement needing `ACCESS EXCLUSIVE` for more than 2 s is forbidden in standard releases and requires a maintenance-window change record; `alembic downgrade` scripts must exist and be tested in CI (staging), but production recovery uses roll-forward or restore, never downgrade.

> **TITAN-OPS-013 (MUST)** — All production schema changes MUST follow expand→migrate→contract; each release's migrations MUST be backward-compatible with the previous release's code (blue/green

> requirement, since both colors share one database). *Acceptance:* CI job boots previous-release images against post-migration schema in staging and runs the smoke suite green before any production migration is approved.

---

6. Monitoring & Observability

All services emit OpenTelemetry (traces, metrics, logs) to a namespace-local `otel-collector`, which routes metrics to Prometheus (via the Prometheus exporter / remote-write), traces to Tempo (local/staging) or a configured OTLP backend, and enriches logs picked up by Promtail into Loki. In Docker Compose, the identical collector config runs as a container — full local parity.

6.1 Prometheus Metrics (Platform & Agent-Level)

Standard RED/USE metrics come from FastAPI middleware, `redis_exporter`, `postgres_exporter`, and `kube-state-metrics`. TITAN adds first-class **agent-level metrics** (all labeled `{agent_role, project_id, provider, model}` — cardinality guarded: project_id capped to active projects, never per-task labels):

Metric	Type	Meaning
`titan_agent_tasks_total{status}`	counter	Tasks completed/failed/retried per agent role
`titan_agent_task_duration_seconds`	histogram	End-to-end task latency per role
`titan_agent_tokens_total{direction}`	counter	Prompt/completion tokens per role, provider, model
`titan_llm_cost_usd_total`	counter	Accrued spend (gateway computes from provider price table)
`titan_llm_request_duration_seconds`	histogram	Gateway-observed LLM latency per provider/model
`titan_llm_errors_total{kind}`	counter	Rate-limit / timeout / refusal / 5xx per provider
`titan_queue_depth{stream, group}`	gauge	Redis Streams pending entries per consumer group
`titan_queue_oldest_pending_seconds`	gauge	Age of oldest unacked task (backpressure signal)
`titan_project_budget_remaining_usd`	gauge	Per-project remaining LLM budget
`titan_agent_review_rejections_total`	counter	Code Reviewer Agent rejections (quality signal, feeds Volume 10)

> **TITAN-OPS-014 (MUST)** — Every LLM call MUST increment token and cost counters at the gateway (single choke point), and every agent task MUST record duration and outcome; no agent may call a provider except through the gateway. *Acceptance:* Sum of `titan_llm_cost_usd_total` over a test window matches the gateway's Postgres cost ledger within 1%, and NetworkPolicy blocks worker egress to provider APIs directly.

6.2 Grafana Dashboards

Dashboards are provisioned as JSON in `ops/grafana/dashboards/` (no click-ops):

Dashboard	Panels (minimum)
Platform Overview	Request rate/error rate/latency (p50/p95/p99) per service; pod restarts; HPA replica counts; deploy annotations from CI

Dashboard	Panels (minimum)
Agent Operations	Task throughput by role; task duration heatmap; queue depth & oldest-pending age; retry/failure rates; active agents by role
LLM Spend & Providers	Cost per hour/day by provider+model+project; token burn rate; budget-remaining table (thresholds green/amber/red); provider error & latency comparison; local-vs-cloud routing ratio
Data Stores	Postgres connections/replication lag/slow queries; Redis memory/ops/evictions (must be 0 on queue DB); Qdrant RAM & search latency
SLO / Error Budget	Burn-rate gauges per SLO (Section 7), 28-day budget remaining

> **TITAN-OPS-015 (MUST)** — The five dashboards above MUST be provisioned as code and identical (same JSON) in local, staging, and production Grafana. *Acceptance:* `docker compose --profile obs up` shows all five dashboards populated; a diff between local and prod provisioned JSON is empty.

6.3 Loki Log Aggregation

All services log structured JSON to stdout: `timestamp, level, service, trace_id, span_id, project_id, agent_role, task_id, message, attrs`. Promtail (DaemonSet) ships to Loki with labels `{namespace, service, level}` only (low cardinality; everything else is queried from the JSON body). Retention: 14 days local/staging, 30 days production hot + 90 days object-storage archive. Secrets/token values are redacted at the logging library level (Volume 8 owns the redaction list). Agent *conversation* transcripts are business data stored in PostgreSQL (Volume 5), not in Loki — logs carry only task/message IDs linking to them.

6.4 OpenTelemetry Tracing Across Agent Pipelines

A single trace follows a user request across the entire agent pipeline: `POST /v1/projects/{id}/requests` → orchestrator planning span → per-task spans (queued → assigned → executing) → nested LLM-call spans at the gateway → review/QA spans → deploy action spans. Context propagates via W3C `traceparent` headers on HTTP and an embedded `traceparent` field in every Redis Streams message envelope (workers resume the context on consume). LLM spans record `gen_ai.*` semantic-convention attributes (model, provider, token counts) but **never** prompt/completion bodies (PII/IP risk — bodies live in the Volume 5 memory store). Sampling: 100% local/staging; production tail-sampling keeps all error traces + 10% of successes.

> **TITAN-OPS-016 (MUST)** — Trace context MUST propagate through Redis Streams so one trace covers a request end-to-end (API → orchestrator → workers → LLM gateway); traces MUST NOT contain prompt or completion text. *Acceptance:* For a staged "build a TODO app" request, Tempo shows a single trace containing ≥ 1 orchestrator span, ≥ 3 agent task spans, and nested LLM spans, with zero prompt-body attributes.

---

7. SLOs & Alerting

SLOs are measured over rolling 28 days; error budgets drive release pace (budget exhausted → feature deploys pause, only fixes ship).

SLO ID	Service objective	Target	SLI source
SLO-API-AVAIL	API success rate (non-5xx)	99.9%	`http_requests_total`
SLO-API-LAT	API p95 latency (non-LLM endpoints)	< 500 ms	request histogram
SLO-WS	WebSocket event delivery within 2 s	99.5%	synthetic probe
SLO-TASK	Agent task pickup ≤ 60 s of enqueue (at normal load)	99%	`titan_queue_oldest_pending_seconds`
SLO-GW	LLM gateway availability (incl. fallback success)	99.5%	gateway metrics
SLO-DUR	Durability: zero acknowledged-task loss	100%	ledger reconciliation job

Alertmanager routes: `severity=page` → PagerDuty/ntfy (self-hosted option) to on-call; `severity=ticket` → GitHub issue via webhook. Core rules (full set in `ops/prometheus/rules/`):

Alert	Condition	Severity
APIErrorBudgetFastBurn	14.4× burn rate over 1 h AND 5 m (multiwindow)	page
APIErrorBudgetSlowBurn	3× burn over 6 h AND 30 m	ticket
QueueBackpressure	`titan_queue_oldest_pending_seconds > 300` for 10 m	page
WorkerCrashLoop	pod restarts > 3 in 15 m	page
PostgresReplicationLag	lag > 30 s for 5 m	page
RedisQueueMemoryHigh	queue DB memory > 80% for 10 m	page
LLMProviderDegraded	provider error rate > 20% over 10 m	ticket (auto-fallback handles)
ProjectBudgetNearExhaustion	`titan_project_budget_remaining_usd / budget < 0.1`	ticket + in-app notification
GlobalSpendAnomaly	hourly spend > 3× trailing 7-day hourly median for 2 h	page
BackupFailed	backup job non-zero exit or missed schedule	page
CertExpirySoon	TLS cert < 14 days to expiry	ticket

> **TITAN-OPS-017 (MUST)** — SLO alerts MUST use multiwindow multi-burn-rate rules (not static thresholds) for availability/latency SLOs; every page MUST link to its runbook section. *Acceptance:* Alert definitions carry `runbook_url` annotations; synthetic fault injection in staging fires fast-burn within 5 minutes and the page contains the link.

---

8. High Availability & Disaster Recovery

8.1 HA Design

Every stateless tier runs ≥ 2 replicas with `podAntiAffinity` across nodes and `topologySpreadConstraints` across zones (where the cluster is multi-zone); `PodDisruptionBudgets` keep `minAvailable: 1` (api: 2) during node drains. PostgreSQL runs one primary + one streaming replica with automated failover via the operator (CloudNativePG); Redis runs primary/replica with 3 Sentinels; Qdrant runs 2 replicas with collection replication

factor 2. The orchestrator is active/passive via Postgres advisory-lock leader election — two replicas, one scheduling. Kubernetes reschedules failed pods; the single-node local environment explicitly accepts no HA.

8.2 Backup Schedule

Data	Method	Frequency	Retention	Encrypted
PostgreSQL	WAL archiving (continuous, to S3-compatible object storage — MinIO acceptable) + `pg_basebackup`	WAL: continuous; base: daily 02:00 UTC	Base: 30 d; WAL: 7 d (PITR window)	AES-256 at rest (Volume 8 KMS)
Qdrant	Collection snapshots to object storage	Every 6 h	14 d	Yes
Redis	RDB snapshot to object storage	Hourly	48 h	Yes
Object storage (artifacts, workspaces)	Cross-bucket replication	Continuous	30 d versioning	Yes
Cluster state	`infra/` in git is source of truth; Velero for PVC/state safety net	Velero: daily	14 d	Yes
Secrets	Sealed/external-secrets source of truth in KMS (Volume 8)	On change	n/a	Yes

Redis queue contents are deliberately **not** the durability boundary: every task is recorded in the PostgreSQL task ledger before enqueue, so queue loss is recoverable by re-enqueueing non-terminal ledger rows (`ops/scripts/requeue-from-ledger.py`).

8.3 Restore Procedure & RPO/RTO

Targets: PostgreSQL RPO ≤ 5 min (WAL PITR), RTO ≤ 1 h. Qdrant RPO ≤ 6 h, RTO ≤ 2 h (degraded-but-operational sooner: agents run with reduced memory recall; Volume 5 defines re-embedding from Postgres source data as the true recovery of record). Redis RPO ≤ 1 h nominal / zero task-loss via ledger; RTO ≤ 15 min. **Full-platform disaster (cluster loss): RTO ≤ 4 h, RPO ≤ 5 min** for system-of-record data.

Restore order (full DR): (1) provision cluster from `infra/` (Terraform + kustomize); (2) restore secrets from KMS; (3) restore Postgres via PITR to latest WAL; (4) start Redis empty, run ledger re-enqueue; (5) restore Qdrant snapshots; (6) deploy last-known-good digests from the release manifest; (7) run smoke suite; (8) flip DNS. The detailed step-by-step lives in RUNBOOK-04 (Section 12).

> **TITAN-OPS-018 (MUST)** — Backups MUST run per the Section 8.2 schedule, be encrypted, and be **verified by automated restore**: nightly CI restores the latest Postgres base+WAL into a scratch database and runs integrity checks. *Acceptance:* Nightly `backup-verify` job green; a BackupFailed page fires if the job is red or a scheduled backup is missing.
>
> **TITAN-OPS-019 (MUST)** — A full DR drill (cluster-loss simulation into a fresh staging cluster) MUST be executed quarterly and MUST meet RTO ≤ 4 h / RPO ≤ 5 min. *Acceptance:* Quarterly drill report in `ops/dr-drills/` records measured RTO/RPO; a miss opens a SEV-2 remediation issue.

---

9. Scaling Model & Backpressure

What scales horizontally: frontend, api, llm-gateway (per traffic); workers (per queue lag — the primary scaling dimension, since one "build me an app" request fans out into dozens of parallel agent tasks); Ollama pool (per GPU-node availability). **What scales vertically first:** PostgreSQL (then read replicas for read-heavy dashboard queries), Redis (then functional sharding: separate queue vs. cache instances), Qdrant (then sharded collections).

Backpressure chain (must degrade gracefully, never collapse):

```
client → API admission → Postgres ledger → Redis Streams → worker pool → LLM gateway
           │                                  │                            │
           │ 429 + Retry-After when           │ XADD MAXLEN ~1M cap;       │ per-provider token-
           │ project concurrent-task          │ KEDA scales workers on     │ bucket rate limits;
           │ quota exceeded                   │ XPENDING lag; alert at     │ bounded retries w/
           │                                  │ oldest-pending > 300 s      │ jitter; circuit
           │                                  │                            │ breaker → fallback
           └── intake keeps accepting & ledgering (durable) even when execution is saturated;
               tasks wait in "queued" state visible in the dashboard with honest ETAs
```

Rules: admission control happens at the API (per-project concurrency quotas, default 25 concurrent tasks); the ledger accepts writes even under execution saturation so no work is refused-and-lost; workers pull (never pushed), so a slow worker only slows itself; the gateway enforces per-provider rate limits with a circuit breaker that trips to the fallback chain (Section 10). Scale-out beyond ~5,000 tasks/min on Redis Streams triggers the documented NATS JetStream migration path (ADR-OPS-4).

> **TITAN-OPS-020 (MUST)** — The platform MUST remain available for intake (accept + ledger + honest status) even when the execution tier is saturated; saturation manifests as queue wait, never as data loss or cascading failure. *Acceptance:* k6 test at 5× worker capacity for 30 min: zero 5xx on intake endpoints, zero lost tasks, `QueueBackpressure` alert fires, and the system fully drains after load stops.

---

10. Cost Optimization & LLM Spend Controls

LLM tokens are TITAN's dominant marginal cost; controls are enforced **in the gateway** and configured per org/project in PostgreSQL.

Budget hierarchy: global monthly ceiling → per-organization budget → per-project budget → per-task cap (default $2.00, configurable). The gateway checks remaining budget pre-call (prompt tokens × price table) and settles post-call with actual usage. At 80% of a project budget: in-app + ticket notification. At 100%: hard stop — new LLM calls are rejected, tasks pause in a resumable `budget_hold` state, and the owner is prompted to raise the budget or switch to local models.

Routing & fallback ladder (configuration, not code — per the multi-LLM gateway contract in Volume 7):

1. **Tiered model routing:** each agent role declares a capability tier; the gateway maps tiers to models via config. Planning/architecture roles get frontier models; mechanical tasks (formatting, summarizing progress reports, doc stubs) route to small/cheap or local models. Default config ships with a cost-optimized mapping.
2. **Local-model fallback:** on provider outage, rate-limit exhaustion, or budget hold, the gateway falls back per its chain, e.g. `anthropic → openai → ollama`. Falling back to Ollama is always legal (free tier is a

platform guarantee); results are tagged `degraded_model=true` so the Code Reviewer Agent applies stricter review (Volume 6).

3. **Caching:** gateway-level exact + normalized prompt cache in Redis (TTL 24 h, keyed on model+prompt hash, opt-out per call) — deduplicates repeated agent boilerplate; provider-native prompt caching enabled where supported.
4. **Context discipline:** Volume 5's context-compression pipeline is the biggest cost lever; the gateway enforces per-role max-context budgets as a backstop and emits `titan_llm_context_truncated_total` when it clips.

Infra cost controls: worker scale-to-min off-hours (KEDA min replicas schedule); staging scale-to-zero nights/weekends (except nightly-job windows); GPU node pool for Ollama is optional and tainted so nothing else lands there; Loki/Tempo retention tiers to object storage. A monthly cost report (infra + LLM by project) is generated by a CronJob into the dashboard's admin area.

> **TITAN-OPS-021 (MUST)** — Per-project LLM budgets MUST be enforced pre-call at the gateway with hard stop at 100% and notification at 80%; paused tasks MUST resume without loss when budget is restored. *Acceptance:* Integration test: set a $0.50 project budget, run a task exceeding it → task enters `budget_hold` , no provider call exceeds the ceiling, raising the budget resumes and completes the task.
>
> **TITAN-OPS-022 (MUST)** — The gateway MUST support automatic fallback to local models (Ollama) on provider failure or budget exhaustion, controlled purely by configuration. *Acceptance:* With cloud provider keys revoked in staging, a submitted project still completes end-to-end on Ollama, with `degraded_model=true` recorded on affected tasks.
>
> **TITAN-OPS-023 (SHOULD)** — Tiered model routing per agent role SHOULD reduce blended cost ≥ 40% versus routing everything to the frontier model, measured on the standard benchmark project set (Volume 10 owns the benchmark). *Acceptance:* Benchmark report comparing single-model vs. tiered routing shows ≥ 40% cost reduction at equal-or-better QA pass rate.

---

11. Production Readiness Checklist

Release captains complete this checklist (tracked as a GitHub issue template, `ops/templates/prod-readiness.md`) before the **first** production launch and re-verify the delta items each release:

Build & supply chain: [ ] All images digest-pinned, cosign-signed, SBOM published; [ ] CVE scan clean or waivered per Volume 8; [ ] reproducible build verified for release tag. **Deploy:** [ ] Blue/green flip and rollback exercised in staging this cycle; [ ] migration follows expand/contract and passed the previous-version compatibility job; [ ] feature flags for all new risky paths default OFF, owners assigned. **Resilience:** [ ] Probes defined and validated (kill-pod drill); [ ] PDBs, anti-affinity, HPA/KEDA configured and load-tested; [ ] backpressure test (TITAN-OPS-020) green. **Data:** [ ] Backups running and restore-verified (TITAN-OPS-018); [ ] PITR tested within 90 days; [ ] task-ledger reconciliation job green. **Observability:** [ ] Five dashboards live; [ ] all Section 7 alerts loaded and test-fired; [ ] runbook links resolve; [ ] trace continuity verified (TITAN-OPS-016). **Security (verify with Volume 8):** [ ] NetworkPolicies default-deny verified; [ ] secrets from external secret store, none in env-committed files; [ ] TLS everywhere; [ ] RBAC audit complete. **Cost:** [ ] Budgets configured for every active org/project; [ ] spend-anomaly alert live; [ ] Ollama fallback verified (TITAN-OPS-022). **Process:** [ ] On-call rotation staffed (min 2 people or documented solo-operator mode for self-hosters); [ ] SEV definitions and escalation path published; [ ] DR drill within last quarter.

> **TITAN-OPS-024 (MUST)** — No first production launch without 100% checklist completion signed by two maintainers; per-release delta items MUST be re-verified each release. *Acceptance:* The release issue shows every box checked with links to evidence (CI runs, drill reports) and two maintainer approvals.

---

12. Operational Runbooks (Outline)

Runbooks live in `ops/runbooks/`, one file each, linked from alerts, all following one skeleton: *Preconditions → Steps (copy-pasteable commands) → Verification → Escalation → Post-actions*.

RUNBOOK-01 — Deploy (`deploy.md`): preconditions (gates of §3.4 green, checklist delta done); steps: trigger `release.yml` from tag → watch green rollout → confirm smoke pass on preview URL → approve flip → watch SLO burn panel 30 min → confirm old-color scale-down; verification: deploy annotation on Platform Overview, error rate flat; escalation: any fast-burn → RUNBOOK-02; post: update release notes, close release issue.

RUNBOOK-02 — Rollback (`rollback.md`): decision matrix (auto vs. manual, warm-color vs. redeploy path); steps: `ops/scripts/rollback.sh all` → verify selector flip → verify SLO recovery → re-enable paused flags only after cause known; verification: error budget burn returns < 1×; escalation: rollback ineffective → declare incident (RUNBOOK-03), consider `agents.pause_all`; post: mandatory blameless post-mortem within 48 h, previous digest quarantined from re-promotion.

RUNBOOK-03 — Incident (`incident.md`): SEV definitions (SEV-1: platform down or data loss; SEV-2: SLO breach or major feature down; SEV-3: degraded); roles (commander, comms, operator); first 10 minutes: acknowledge page → open incident channel + status page → check Platform Overview and recent deploys (deploy? → RUNBOOK-02; provider outage? → verify gateway fallback; backpressure? → check KEDA scaling and quotas); kill-switch inventory (`agents.pause_all`, `intake.read_only`, `llm.provider_override`); status updates every 30 min for SEV-1; post: timeline from Loki/Tempo, post-mortem with action items as issues.

RUNBOOK-04 — Backup & Restore (`backup-restore.md`): verify-backup steps (check nightly job, list object-store artifacts); single-table recovery (PITR clone → dump table → surgical restore); full Postgres PITR (stop writers via `intake.read_only` → restore base → replay WAL to target time → validate ledger consistency → reopen); Qdrant snapshot restore + re-embedding fallback; Redis rebuild via ledger re-enqueue; full-DR sequence (§8.3) with expected timings per step; verification: smoke suite + reconciliation job; post: drill report or incident annex.

> **TITAN-OPS-025 (MUST)** — The four runbooks MUST exist as executable documents (tested command blocks), be linked from every paging alert, and be re-validated by drill at least quarterly. *Acceptance:* Each runbook has a `last-drilled` date within 90 days; alert annotations resolve to the correct runbook anchor.

---

13. Decision Records

ADR-OPS-1 — Blue/green over rolling and canary for production releases. *Context:* TITAN needs sub-minute rollback and safe schema evolution with long-lived WebSockets and queue consumers. *Decision:* Blue/green with Service-selector flip, 30-min observation, shared datastores under expand/contract migrations. *Alternatives:* Rolling updates (simplest, but rollback is a full re-rollout with long mixed-version windows); canary via service mesh (finer risk control, but Istio/Linkerd is unjustified operational weight and hostile to the local-first goal); Argo Rollouts (deferred — see Open Questions). *Trade-offs:* accepted 2×

capacity during deploys (idle color scales to zero between releases) and stricter migration discipline, in exchange for atomic flips and < 5-min rollback.

ADR-OPS-2 — GitHub Actions as the sole CI/CD system. *Context:* Open-source monorepo on GitHub; contributors need free CI; TITAN's own DevOps Agent must be able to read and modify pipelines programmatically. *Decision:* GitHub Actions with OIDC-federated deploys, environments with required reviewers. *Alternatives:* GitLab CI (would force repo migration); Jenkins (self-hosted burden, plugin security surface); Argo CD pull-based GitOps (strong candidate, adds a controller to operate — revisit post-MVP; the manifests-in-git structure is already GitOps-compatible). *Trade-offs:* vendor coupling to GitHub accepted; mitigated by keeping all logic in repo-local scripts (`ops/scripts/`) callable from any runner, so workflows are thin wrappers.

ADR-OPS-3 — Self-hosted kube-prometheus-stack + Loki over managed observability. *Context:* Local-first parity and zero-cost self-hosting are hard requirements; agent metrics (tokens, cost) are custom anyway. *Decision:* Prometheus + Grafana + Loki + otel-collector everywhere, provisioned as code, identical configs across environments. *Alternatives:* Datadog/New Relic (excellent UX; violates free/local requirement and per-host pricing punishes worker autoscaling); Grafana Cloud free tier (acceptable *optional* remote-write target, documented, not required). *Trade-offs:* we own storage sizing and Prometheus operations; retention beyond 30 d goes to object storage rather than a vendor.

ADR-OPS-4 — Redis Streams at MVP with a documented NATS JetStream escape hatch. *Context:* Canonical Volume 1 decision; operations must define when and how it changes. *Decision:* Operate Redis Streams (with Sentinel, `noeviction`, ledger-backed durability) until any of: sustained > 5,000 tasks/min, consumer-group count > 200, or cross-region delivery is required; then execute the migration plan in Volume 3 (queue interface is abstracted, so workers swap consumers without task-model changes). *Alternatives:* NATS from day one (better semantics at scale, but one more stateful system for a laptop install); Kafka (operational overkill). *Trade-offs:* accepted at-least-once semantics with idempotent task handlers (dedup key = task ID) as the universal contract so the swap stays invisible.

ADR-OPS-5 — Self-hosted Postgres-backed feature flags over a flag SaaS. *Context:* Flags gate releases, agent behavior, and kill switches; they must work offline on a laptop. *Decision:* `feature_flags` table + Redis cache + pub/sub invalidation, admin UI in the dashboard. *Alternatives:* LaunchDarkly (best-in-class, paid, network-dependent); Unleash self-hosted (viable, adds a service + DB; our needs are a strict subset). *Trade-offs:* no advanced experimentation analytics at MVP; percentage rollouts are deterministic hashes on org/project ID, which is sufficient for §5.3's flag classes.

---

14. Risks & Open Questions

Risks

#	Risk	Likelihood	Impact	Mitigation
R1	LLM provider regional outage stalls all agent work	Medium	High	Gateway fallback ladder incl. Ollama (TITAN-OPS-022); degraded-model tagging; SLO-GW measured on fallback-inclusive success
R2	Runaway agent loop burns budget rapidly	Medium	High	Per-task cost caps, per-project budgets, GlobalSpendAnomaly page, `agents.pause_all` kill switch
R3	Blue/green schema discipline erodes (someone ships a breaking migration)	Medium	High	CI previous-version compatibility job (TITAN-OPS-013) is a hard gate, not a convention

#	Risk	Likelihood	Impact	Mitigation
R4	Metric cardinality explosion from per-project labels	Medium	Medium	Label allowlist in otel-collector; recording rules; cap active-project label values; alert on series count
R5	Self-hosters skip backups/DR entirely	High	High (for them)	Compose ships backup sidecar defaults ON; readiness checklist has a documented solo-operator mode; docs make restore a 1-script action
R6	Single-cluster production is a zonal/regional single point of failure	Low	High	Multi-zone spread at MVP; documented DR rebuild ≤ 4 h; multi-region is explicitly out of scope until post-MVP (Open Question Q2)

Open Questions

- **Q1:** Adopt Argo Rollouts to mechanize the blue/green flip and burn-rate analysis (replacing our release-job scripting), at the cost of another controller? Decide by end of MVP hardening; the Service-selector design is compatible either way.
- **Q2:** Multi-region active/passive for managed-cloud TITAN: at what user threshold does RTO ≤ 4 h become unacceptable? Requires pricing data from Volume 10's enterprise track.
- **Q3:** Per-agent-role K8s workload separation (dedicated Deployments per role for isolation/priority) vs. the current homogeneous worker pool with role dispatch — revisit when role-specific resource profiles diverge (e.g., browser-automation agents needing more memory).
- **Q4:** Should the provider price table (used for cost metrics) be fetched from a maintained community source or vendored and reviewed per release? Currently vendored in `config/llm-prices.yaml`; staleness risk is accepted and surfaced in the monthly cost report.
- **Q5:** GPU scheduling for the Ollama pool: time-slicing vs. MIG vs. whole-GPU nodes — depends on the model mix chosen in Volume 5's local-model policy.

---

15. Cross-References

- **Volume 1 — Foundation:** owns and justifies all canonical stack decisions applied here (Kubernetes, Postgres 16, Redis 7, Redis Streams, observability stack); NFR targets that Section 7 SLOs operationalize.
- **Volume 2 — Multi-Agent Intelligence:** the DevOps Agent's role spec — it executes the pipelines and runbooks defined in this volume; agent KPIs feed the Agent Operations dashboard.
- **Volume 3 — Backend Engineering:** service internals, the queue/message envelope (including the `traceparent` field), task ledger schema, WebSocket eventing, and the NATS migration plan referenced by ADR-OPS-4.
- **Volume 4 — Frontend & Dashboard:** the Deployment Dashboard, Monitoring views, budget notifications, and flag admin UI surfaced from this volume's data; WebSocket reconnect contract used during blue/green flips.
- **Volume 5 — Memory, Knowledge & RAG:** Qdrant recovery-by-re-embedding, context compression as the primary LLM cost lever, storage of prompt/completion bodies outside logs and traces.
- **Volume 6 — Engineering Workflow:** how deploy/rollback events feed back into agent task states; stricter review policy for `degraded_model=true` outputs.
- **Volume 7 — Tool Ecosystem & Integrations:** the LLM gateway's provider abstraction and configuration contract that Sections 6.1 and 10 instrument and govern.

- **Volume 8 — Security:** secrets management, KMS, NetworkPolicy details, image-signing policy enforcement, log redaction lists, RBAC for cluster access, CVE waiver process.
- **Volume 10 — Autonomous Evolution:** benchmark suite used by TITAN-OPS-023; agent performance metrics (review rejections, throughput) consumed by the self-improvement loop.

End of Volume 9 — TITAN-VOL-09, Draft v1.0.

PROJECT TITAN — Autonomous Software Engineering Platform

Volume 10 — Future Roadmap & Autonomous Evolution

Field	Value
Series	PROJECT TITAN — Autonomous Software Engineering Platform Specification
Volume	10 of 10 — Future Roadmap & Autonomous Evolution
Doc ID	TITAN-VOL-10
Requirement Prefix	TITAN-EVO-NNN
Status	Draft v1.0
Date	2026-07-04

Abstract. This volume specifies how TITAN improves itself over time without compromising safety or human control. It defines the closed-loop evolution system: per-task agent scorecards and KPI aggregation, post-task and post-project reflection loops written to organization memory, a failure taxonomy with root-cause records that feed prompt and policy updates, an A/B evaluation harness and benchmark suite for continuous optimization of prompts and model routing, multi-model debate and judge protocols, and an automated pipeline for proposing, sandbox-trialing, and promoting entirely new agents — always behind a human approval gate and full version control. It further specifies the plugin marketplace vision, enterprise capabilities (SSO, multi-tenancy, compliance, private model hosting), fine-tuning on organization memory, a phased long-term roadmap, and the known limits and open research questions of autonomous software engineering. Every mechanism in this volume is implementable on the canonical stack fixed in Volume 1 (FastAPI + PostgreSQL 16 + Redis 7 + Qdrant, provider-agnostic LLM gateway, Docker Compose locally, Kubernetes in production) and preserves TITAN's local-first, free-tier-friendly, open-source posture.

Table of Contents

- 1. Introduction & Scope
- 2. The Evolution Safety Model
- 3. Agent Self-Evaluation: Scorecards & KPIs
- 4. Reflection Loops
- 5. Learning from Failures
- 6. Continuous Optimization: Prompts, Routing & Benchmarks
- 7. AI Debates & Multi-Model Collaboration
- 8. Automated Creation of New Agents
- 9. Plugin Marketplace Vision
- 10. Enterprise Features
- 11. Model Fine-Tuning on Organization Memory
- 12. Long-Term Roadmap
- 13. Risks, Known Limits & Open Research Questions

- 14. Cross-References

1. Introduction & Scope

1.1 Purpose

Volumes 1–9 specify a platform that builds software. This volume specifies the platform that builds a *better platform*: the feedback loops, evaluation infrastructure, and governance that let TITAN's agents measurably improve week over week. The core thesis is that self-improvement is an engineering discipline, not an emergent property. Improvement only happens when (a) performance is measured per task, (b) failures are classified and traced to causes, (c) candidate changes (prompts, routing policies, new agents) are evaluated against fixed benchmarks before deployment, and (d) every change is versioned, reviewable, and reversible.

1.2 Design Principles

1. **Measured, not vibes.** No prompt, policy, or agent change ships without benchmark evidence (§6).
2. **Human-gated self-modification.** TITAN may *propose* changes to itself; only a human with the `evolution:approve` RBAC permission may *apply* them (§2).
3. **Everything is versioned.** Prompts, routing policies, agent specifications, and fine-tuned adapters live in the monorepo under Git; runtime state references immutable versions.
4. **Local-first.** Every mechanism in this volume — including the evaluation harness and fine-tuning — runs on a single laptop via Docker Compose using free/local tools (Ollama for local models). Cloud/scale paths are documented, never required.
5. **Fail toward the status quo.** If any evolution component is down or ambiguous, agents run the last approved configuration.

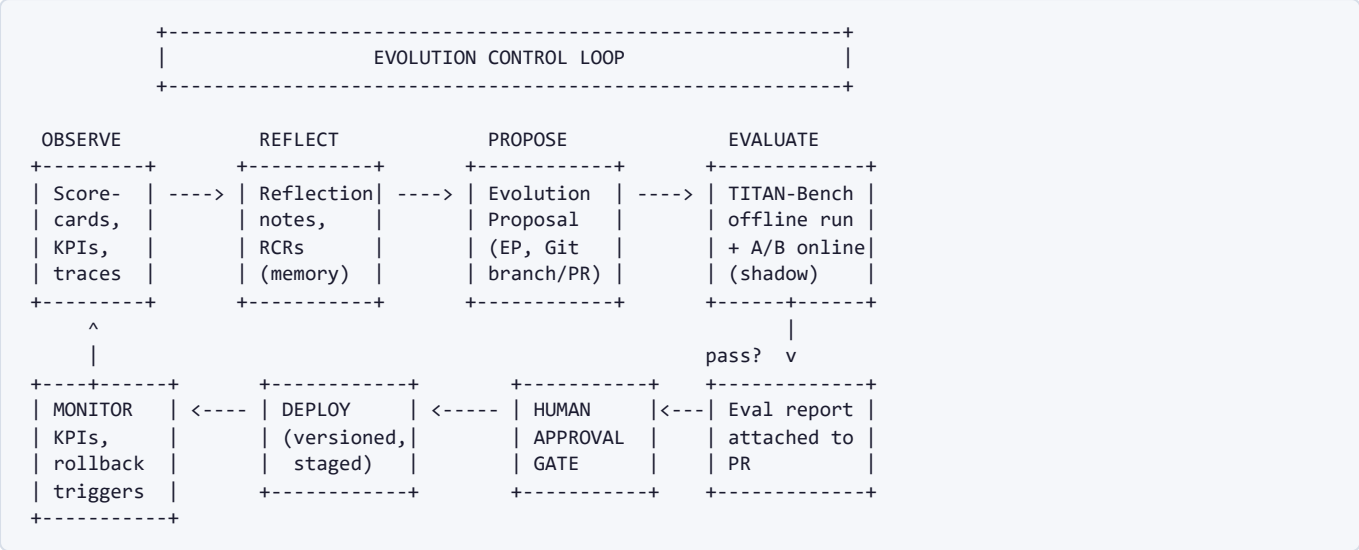
1.3 Terminology

Term	Definition
Scorecard	Structured per-task evaluation record produced when a task closes.
Reflection	Agent-authored retrospective note persisted to memory (Volume 5).
RCR	Root-Cause Record — a structured analysis of a classified failure.
Evolution Proposal (EP)	A versioned, machine-readable proposal to change a prompt, policy, routing rule, or agent roster.
TITAN-Bench	The fixed benchmark suite used to evaluate agent/prompt/model changes.
Champion / Challenger	The currently deployed configuration vs. a candidate under A/B evaluation.

2. The Evolution Safety Model

All capabilities in this volume are constrained by a single safety model, specified first because every later section depends on it.

2.1 The Evolution Control Loop



Every loop iteration produces an auditable trail: telemetry → memory artifact → EP (a Git pull request in the monorepo) → benchmark report → human approval → staged deployment → post-deploy monitoring with automatic rollback triggers.

2.2 Requirements

TITAN-EVO-001 (MUST) — Human approval gate for all self-modification. No change to a prompt template, routing policy, agent specification, guardrail, or fine-tuned model may take effect in any environment other than an isolated sandbox without explicit approval by a human holding the `evolution:approve` RBAC permission (Volume 8 RBAC model). *Acceptance criteria:* (a) The orchestrator refuses to load any evolution artifact whose EP lacks an `approved_by` user ID and signed approval timestamp; (b) an integration test proves that an unapproved prompt version is never selected at runtime; (c) approvals are recorded in the audit log with actor, artifact hash, and diff link.

TITAN-EVO-002 (MUST) — Version control for all evolution artifacts. Prompts, routing policies, agent specs, benchmark definitions, and EPs are files in the monorepo (`/evolution/**`), changed only via pull request. The PostgreSQL `evolution_artifacts` table stores only immutable references (Git commit SHA + path + content hash). *Acceptance criteria:* (a) Runtime loads artifacts by pinned SHA; (b) `git revert` of an evolution commit followed by redeploy restores prior behavior byte-for-byte; (c) no mutable prompt text exists in the database.

TITAN-EVO-003 (MUST) — Automatic rollback triggers. Every deployed evolution change carries a monitoring contract: KPI floors (e.g., task success rate, review acceptance rate) and an observation window. Breaching a floor automatically reverts to the champion configuration and opens an incident RCR. *Acceptance criteria:* (a) Rollback executes without human action within 5 minutes of a floor breach; (b) rollback emits a Prometheus alert and audit event; (c) a chaos test demonstrates rollback on an artificially degraded challenger.

2.3 ADR-EVO-01 — Self-Modification via Git PRs with Human Approval

- **Context:** TITAN must improve its own prompts, policies, and agent roster, but uncontrolled self-modification risks silent capability drift, reward hacking, and unauditable behavior.
- **Decision:** All self-modification is expressed as pull requests against `/evolution/**` in the monorepo. Agents may open, benchmark, and comment on these PRs; only humans may merge. Runtime

configuration pins Git SHAs.

- **Alternatives considered:** (1) *Database-stored mutable prompts with an admin UI* — faster iteration but no diff review, weak audit, easy to bypass; (2) *fully autonomous self-modification with statistical safeguards only* — maximizes learning speed but unacceptable for an open-source platform run on customer code; (3) *no self-modification, manual tuning only* — safe but abandons the volume's purpose.
- **Trade-offs:** Human gating slows the improvement loop (hours–days instead of minutes) and requires maintainer attention. We accept this: TITAN's evolution cadence is weekly, not per-request, and the Git-based flow reuses the review tooling TITAN already has (Volume 6 workflow, Volume 7 GitHub integration) at zero extra infrastructure cost.

---

3. Agent Self-Evaluation: Scorecards & KPIs

3.1 Task Scorecards

Every task that reaches a terminal state (`done`, `rejected`, `abandoned`) triggers scorecard generation by the orchestrator, combining objective telemetry with a rubric-based LLM evaluation performed by the Code Reviewer Agent (for code tasks) or the Project Manager Agent (for non-code tasks) — never by the agent that did the work.

Dimension	Source	Scale	Example signal
Correctness	QA Agent results, CI	0–100	% acceptance criteria passing
First-pass quality	Review cycle count	0–100	100 if merged with ≤1 review round
Rework ratio	Diff churn	0–100	Lines rewritten after first submission / total
Efficiency	OpenTelemetry spans	0–100	Tokens + wall-clock vs. task-class baseline
Cost	LLM gateway metering	USD + 0–100	Normalized against task-class p50 cost
Standards compliance	Linters, Volume 8 rules	0–100	Violations per KLoC
Security	Security Agent scan	0–100	Findings weighted by severity
Collaboration	Message bus analysis	0–100	Blocked-time caused, handoff completeness

TITAN-EVO-004 (MUST) — Scorecard per terminal task. *Acceptance criteria:* (a) A row exists in `agent_scorecards` for 100% of terminal tasks within 60 seconds of closure; (b) each scorecard links the task ID, agent ID, prompt version SHA, model ID, and OpenTelemetry trace ID; (c) rubric-based dimensions are produced by a different agent than the task executor.

TITAN-EVO-005 (MUST) — Scorecard schema.

```
CREATE TABLE agent_scorecards (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  task_id         UUID NOT NULL REFERENCES tasks(id),
  agent_id        TEXT NOT NULL,             -- e.g. 'backend-engineer'
  agent_spec_sha  TEXT NOT NULL,             -- pinned Git SHA of agent spec
  prompt_sha      TEXT NOT NULL,
  model_id        TEXT NOT NULL,             -- e.g. 'anthropic/claude-*', 'ollama/llama3'
  task_class      TEXT NOT NULL,             -- taxonomy from Volume 6
  dimensions      JSONB NOT NULL,            -- {correctness: 92, rework: 71, ...}
  composite_score NUMERIC(5,2) NOT NULL,     -- weighted per task_class
  cost_usd        NUMERIC(10,4) NOT NULL DEFAULT 0,
  tokens_in       BIGINT NOT NULL,
  tokens_out      BIGINT NOT NULL,
  wall_clock_s    INTEGER NOT NULL,
  trace_id        TEXT NOT NULL,
  created_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX ON agent_scorecards (agent_id, task_class, created_at);
```

Acceptance criteria: Migration exists; composite weights per task class are defined in `/evolution/scoring/weights.yaml` and versioned per TITAN-EVO-002.

3.2 KPI Aggregation — Agent Report Cards

Scorecards roll up into the per-agent KPIs defined in Volume 2 (each agent's specification names its KPIs; e.g., the Backend Engineer Agent's task success rate, first-pass review acceptance, and defect escape rate). A nightly job (Redis Streams consumer `evolution.kpi-rollup`) computes 7/30/90-day windows and publishes them to Grafana dashboards and to the frontend Agent Status view (Volume 4).

TITAN-EVO-006 (MUST) — KPI rollups. *Acceptance criteria:* (a) Per-agent, per-task-class KPI aggregates exist for 7/30/90-day windows; (b) exposed as Prometheus metrics (`titan_agent_kpi{agent,kpi,window}`) and a FastAPI endpoint `GET /api/v1/evolution/agents/{agent_id}/kpis`; (c) values reconcile with raw scorecards within ±0.5%.

TITAN-EVO-007 (SHOULD) — Degradation alerts. A sustained KPI drop (>10% vs. trailing 30-day mean for 3 consecutive days) opens an `evolution.investigation` task assigned to the Memory Agent and notifies maintainers via configured channels (Volume 7 integrations). *Acceptance criteria:* Alert rule in Prometheus; auto-created investigation task carries the KPI series snapshot.

---

4. Reflection Loops

4.1 Post-Task Reflection

After scorecard generation, the executing agent runs a bounded reflection step (one LLM call, capped tokens) answering four questions: *What was the goal? What worked? What failed or was inefficient? What would I do differently?* The output is a structured note written to episodic memory (Qdrant collection `reflections`, metadata in PostgreSQL) per Volume 5's memory write path.

```
{
  "kind": "post_task_reflection",
  "task_id": "…", "agent_id": "backend-engineer",
  "scorecard_id": "…",
  "went_well": ["Reused the pagination pattern from memory item mem_8121"],
  "went_poorly": ["Missed the tenant-scoping requirement; caught in review round 2"],
  "lesson": "For multi-tenant projects, verify every query against the tenancy checklist before submitting for
review.",
  "proposed_change": {"type": "prompt_note", "target": "backend-engineer", "text": "…"},
  "confidence": 0.7
}
```

TITAN-EVO-008 (MUST) — Post-task reflections persisted to memory. *Acceptance criteria:* (a) Reflection exists for ≥95% of terminal tasks (bounded retries; failure logged, never blocks task closure); (b) reflections are embedded and retrievable via the Volume 5 retrieval pipeline so future similar tasks surface relevant lessons; (c) reflection cost is capped at 2% of the task's LLM cost.

4.2 Post-Project Retrospectives

When a project reaches `released` or `cancelled`, the Project Manager Agent convenes a structured retrospective: it gathers all scorecards, reflections, incidents, and decision logs, and produces a retrospective document (stored in the project repo under `/docs/retrospectives/` and summarized into organization memory). The CEO Agent reviews and countersigns it; disagreements are recorded, not overwritten.

TITAN-EVO-009 (MUST) — Post-project retrospective. *Acceptance criteria:* (a) Retrospective generated within 24h of project terminal state; (b) contains: KPI summary vs. targets, top 5 lessons, failure inventory with RCR links, reuse candidates (patterns/snippets promoted to organization memory), and ≥0 Evolution Proposals; (c) summarized embedding stored in Qdrant `org_memory` with `kind=retrospective`.

TITAN-EVO-010 (SHOULD) — Lessons influence future planning. Sprint planning (Volume 6) retrieves the top-k relevant retrospective lessons for the project's domain and includes them in planner context. *Acceptance criteria:* Planner traces show retrieved lesson IDs; an eval task in TITAN-Bench verifies a known lesson changes a plan.

---

5. Learning from Failures

5.1 Failure Taxonomy

All failures — task rejections, CI failures attributable to agents, incidents, rollbacks, guardrail trips — are classified into a fixed, versioned taxonomy (`/evolution/taxonomy/failures.yaml`):

Code	Class	Examples
F-REQ	Requirement misunderstanding	Built wrong feature; missed acceptance criterion
F-CTX	Context/memory failure	Ignored existing pattern; lost mid-task context
F-GEN	Generation defect	Logic bug, hallucinated API, type error
F-STD	Standards violation	Broke Volume 8 conventions; skipped tests
F-SEC	Security defect	Injection risk, secret in code, missing authz
F-ORCH	Orchestration failure	Deadlocked dependency, wrong agent assigned, dropped handoff
F-TOOL	Tool/integration failure	Git conflict mishandled, sandbox timeout, plugin error

Code	Class	Examples
F-LLM	Model/provider failure	Refusal, truncation, rate-limit cascade
F-HUM	Human-input gap	Ambiguous client request that should have triggered clarification

TITAN-EVO-011 (MUST) — Every failure classified. *Acceptance criteria:* (a) Failure events carry exactly one primary code (+ optional secondaries); (b) classification is performed by the Memory Agent with a rubric prompt and is human-overridable in the dashboard; (c) unclassifiable events land in `F-UNK` and page for taxonomy review — sustained `F-UNK` >5% triggers a taxonomy-update EP.

5.2 Root-Cause Records (RCR)

Failures above a severity threshold (or any recurring failure signature, ≥3 occurrences in 30 days) get an RCR: a structured five-whys analysis authored by the Memory Agent, reviewed by the agent owning the failing domain.

```
CREATE TABLE root_cause_records (
  id             UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  failure_code   TEXT NOT NULL,
  severity       TEXT NOT NULL CHECK (severity IN ('low','medium','high','critical')),
  task_ids       UUID[] NOT NULL,
  summary        TEXT NOT NULL,
  five_whys      JSONB NOT NULL,
  contributing   JSONB NOT NULL,      -- prompt_sha, model_id, routing rule, tool versions
  remediation    JSONB NOT NULL,      -- proposed EPs, memory writes, checklist additions
  status         TEXT NOT NULL DEFAULT 'open',  -- open|remediated|accepted_risk
  created_at     TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

TITAN-EVO-012 (MUST) — RCRs for high-severity and recurring failures. *Acceptance criteria:* (a) RCR created within 24h of qualifying failure; (b) every RCR links contributing artifact versions (prompt SHA, model, routing rule); (c) RCRs are searchable in the Memory Explorer (Volume 4).

5.3 From Failure to Prompt/Policy Update

Remediations that require behavior change become Evolution Proposals: a prompt patch, a policy/checklist addition, a routing change, or (rarely) an agent-spec change. Each EP references its motivating RCR(s), must pass TITAN-Bench (§6), and merges only through the human gate (§2). Remediations that are pure knowledge ("lesson learned") are written to organization memory instead — no approval needed, because memory influences but does not define behavior, and memory writes are already governed by Volume 5's provenance and pruning rules.

TITAN-EVO-013 (MUST) — EP linkage and closure. *Acceptance criteria:* (a) Every high/critical RCR reaches `remediated` or `accepted_risk` within 30 days, tracked on a Grafana panel; (b) merged EPs automatically annotate their RCRs; (c) recurrence of the same failure signature after remediation reopens the RCR and flags the EP as ineffective.

TITAN-EVO-014 (SHOULD) — Regression tests from failures. Each remediated F-GEN/F-STD/F-SEC failure adds a corresponding case to TITAN-Bench so the failure class cannot silently return. *Acceptance criteria:* Bench case references the RCR ID; suite growth is reported monthly.

---

6. Continuous Optimization: Prompts, Routing & Benchmarks

6.1 Prompt Registry & Versioning

All agent prompts live in `/evolution/prompts/<agent>/<template>.md` with YAML front-matter (id, semver, variables, model constraints, eval requirements). The LLM gateway (canonical provider-agnostic layer) resolves `agent → prompt SHA` from the pinned runtime manifest `/evolution/manifest.yaml`.

TITAN-EVO-015 (MUST) — Single source of truth for prompts. *Acceptance criteria:* (a) No prompt text is constructed outside the registry except for dynamic context injection through declared variables; (b) every LLM call logs its prompt SHA into the OpenTelemetry span; (c) the dashboard shows which prompt version served any given task.

6.2 TITAN-Bench — Benchmark Suite

TITAN-Bench is a fixed, versioned suite of agent-performance tasks executed in disposable Docker sandboxes. It runs entirely locally (small tasks against Ollama models on a laptop; full suite in CI on Kubernetes).

Track	Contents	Graded by
B-CODE	60+ scoped implementation tasks (API endpoint, schema migration, UI component) with hidden tests	Deterministic test harness
B-REVIEW	Seeded-defect PRs; measure defect detection precision/recall	Ground-truth defect labels
B-PLAN	Requirements → task graphs; measure dependency correctness, coverage	Rubric + graph checks
B-ARCH	Design questions with expert-written rubrics	LLM judge panel (§7) + rubric
B-SEC	Vulnerable code samples; detection + fix quality	Known CVE/CWE labels
B-REG	Regression cases generated from RCRs (TITAN-EVO-014)	Case-specific assertions

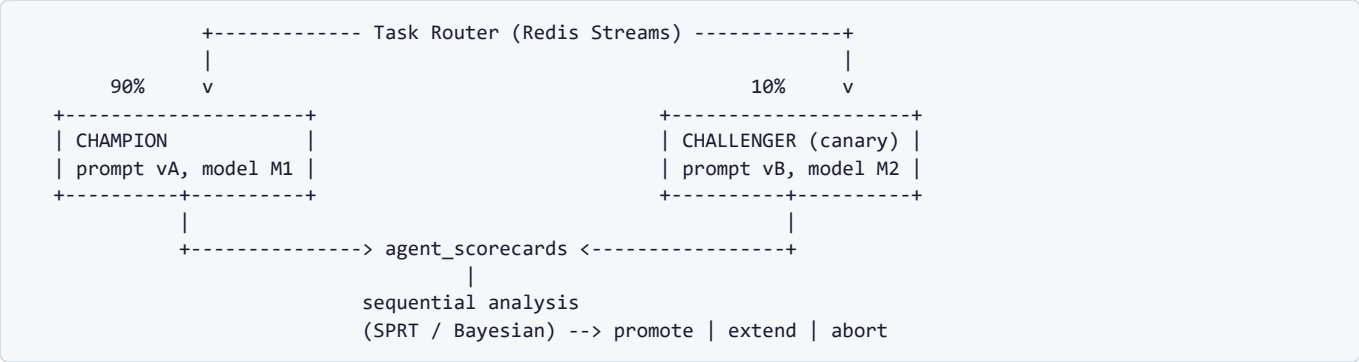
TITAN-EVO-016 (MUST) — Benchmark suite exists and gates evolution. *Acceptance criteria:* (a) `titan bench run --track all --manifest <sha>` produces a scored report (JSON + Markdown) attached to EP pull requests by CI; (b) suite content is versioned; score comparisons are only valid within a suite version; (c) a "smoke" subset (≤15 min, laptop-runnable on local models) and the full suite (CI) both exist.

TITAN-EVO-017 (MUST) — Offline-first evaluation gate. No EP may enter online A/B testing until it beats or matches the champion on the relevant TITAN-Bench tracks (non-inferiority margin defined per track in `/evolution/bench/gates.yaml`). *Acceptance criteria:* CI blocks the EP PR merge checklist until bench results are attached and pass gates.

6.3 A/B Evaluation Harness (Online)

After offline pass and human approval for trial, challengers run in one of two modes:

1. **Shadow mode (default):** the challenger receives copies of real task inputs, produces outputs that are scored but *never used*. Zero user-facing risk; costs are metered against the evolution budget.
2. **Canary mode:** the challenger serves a small percentage (default 10%) of eligible low-risk tasks (task classes whitelisted in the EP), with rollback triggers per TITAN-EVO-003.



TITAN-EVO-018 (MUST) — A/B harness. *Acceptance criteria:* (a) Assignment is deterministic per task (hash-based) and recorded on the scorecard; (b) analysis uses a sequential test with pre-registered success metrics from the EP; (c) promote/abort recommendations are generated automatically but executed only via the human gate; (d) shadow mode works on Docker Compose with a single worker.

6.4 Routing Optimization

The LLM gateway routes each (agent, task_class) pair to a model per `/evolution/routing/policy.yaml` (e.g., architecture debates → strongest available model; lint-fix tasks → cheap local model). Scorecard data continuously estimates per-(task_class, model) quality/cost frontiers; the Memory Agent proposes routing EPs when a cheaper model is statistically non-inferior or a task class underperforms its floor.

TITAN-EVO-019 (MUST) — Data-driven routing policy. *Acceptance criteria:* (a) Routing decisions are logged with policy version; (b) a monthly cost/quality frontier report is generated; (c) switching any route is a configuration-only change (canonical decision), validated by tests.

TITAN-EVO-020 (SHOULD) — Cost budget enforcement for evolution. All evolution activity (reflections, bench runs, shadows, debates) is metered under a distinct budget with per-day caps configurable per deployment; local-model-only mode must be supported for zero-cost operation. *Acceptance criteria:* Budget exhaustion degrades gracefully (skips optional reflection/shadow work, never blocks product tasks); Grafana panel tracks evolution spend.

6.5 ADR-EVO-02 — Offline Bench Gate Before Online A/B

- **Context:** Online experiments on real projects are slow, costly, and risk user-visible regressions; offline benchmarks are fast but can diverge from production distribution.
- **Decision:** Two-stage evaluation: TITAN-Bench (offline, mandatory, blocking) → shadow/canary A/B (online, human-approved) → promotion.
- **Alternatives considered:** (1) *Online-only* — realistic but slow and risky, and unusable for laptop-scale deployments with few tasks; (2) *offline-only* — fast but overfits the bench and misses distribution shift; (3) *interleaved evaluation on every request* — statistically efficient but doubles cost permanently.
- **Trade-offs:** Two stages add latency to the improvement loop and require bench maintenance (mitigated by TITAN-EVO-014 auto-growing the suite from real failures). Accepted: safety and reproducibility outrank iteration speed.

---

7. AI Debates & Multi-Model Collaboration

7.1 When to Convene a Debate

Debates are an *escalation mechanism*, not a default, because they multiply cost. The orchestrator convenes a debate when any trigger fires:

Trigger	Example
High-stakes decision class	Architecture selection, data-model design, security-sensitive design (always debated)
Agent disagreement	Reviewer rejects the same artifact twice; Architect and Backend Engineer deadlock
Low confidence	Executing agent self-reports confidence < 0.5 on a critical task
High blast radius	Change touches ≥ N modules or a public API contract
Bench ambiguity	Challenger and champion within the non-inferiority margin on B-ARCH

TITAN-EVO-021 (MUST) — Debate triggers are policy-defined. *Acceptance criteria:* Triggers live in `/evolution/debate/policy.yaml`; every convened debate logs its trigger; debates never exceed the configured per-project daily cap.

7.2 Debate Protocol

```
Round 0: Framing      — Orchestrator states the question, constraints, and
                        decision criteria (from Volume 1 principles).
Round 1: Positions    — 2–3 debaters (different models via the LLM gateway,
                        e.g., Anthropic vs. OpenAI vs. local) independently
                        produce a position paper (no visibility of others).
Round 2: Rebuttals    — Each debater receives the other positions and must
                        concede, rebut, or refine. Max 2 rebuttal rounds.
Round 3: Judgment     — Judge protocol (§7.3) selects/synthesizes a decision.
Output:  Decision Record (ADR format) + dissent appendix, stored in project
         decision log and organization memory.
```

TITAN-EVO-022 (MUST) — Structured debate with persisted outcome. *Acceptance criteria:* (a) Debaters use distinct model IDs when ≥2 providers/models are configured (single-model deployments fall back to distinct personas with temperature variation, flagged as such); (b) output is an ADR with explicit dissent section; (c) full transcript stored and viewable in the dashboard's Live Conversations view (Volume 4).

7.3 Judge Protocols

Three judge protocols, selected by decision stakes:

1. **Single judge (default):** the strongest configured model, given a rubric (criteria weights from Round 0), scores each position and writes the synthesis. Cheap; used for medium-stakes debates.
2. **Panel majority:** 3 judges (distinct models where available) score independently; majority on the decision, judge disagreement recorded. Used for architecture and security decisions.
3. **Human-in-the-loop:** panel produces a recommendation; a human decides. Mandatory when the decision alters the platform itself (any EP debate) or when the client flagged the area as sensitive.

TITAN-EVO-023 (MUST) — Judges are independent of debaters. *Acceptance criteria:* A judge's model+prompt configuration never equals any debater's in the same debate (verified at convening time); judges receive positions anonymized (position A/B/C) to reduce provider bias.

TITAN-EVO-024 (SHOULD) — Debate efficacy tracking. Decisions produced by debates are tagged; their downstream outcomes (rework, incidents linked to the decision) feed a quarterly report on whether debates outperform single-agent decisions per decision class. *Acceptance criteria:* Report generated; debate triggers tightened/loosened only via EP.

7.4 ADR-EVO-03 — Debate as Escalation, Not Default

- **Context:** Multi-model deliberation measurably improves hard decisions but costs 5–15x a single call.
- **Decision:** Convene debates only on policy triggers (§7.1); default path remains single-agent with review.
- **Alternatives considered:** (1) *Debate everything* — quality ceiling but cost-prohibitive and slow, hostile to the free-tier requirement; (2) *never debate* — cheap but repeats known single-model failure modes on high-stakes design; (3) *self-consistency sampling (same model, n samples)* — cheaper middle ground, retained as a MAY within the single-judge path, but does not diversify provider blind spots.
- **Trade-offs:** Trigger tuning becomes an ongoing responsibility (handled by TITAN-EVO-024's efficacy loop). Accepted.

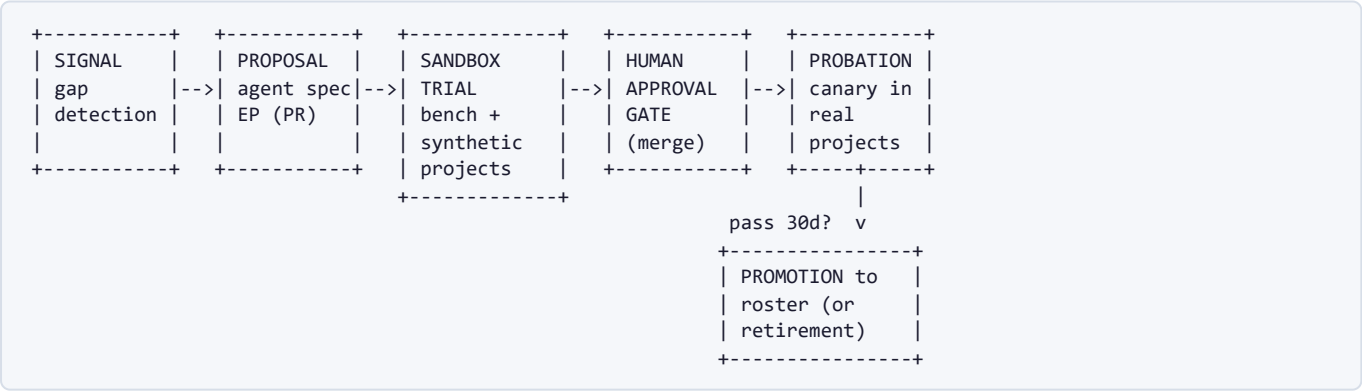
---

8. Automated Creation of New Agents

8.1 Concept

TITAN's roster (Volume 2) is not final. The platform detects capability gaps — recurring failure clusters, task classes with chronically low scores, or workload patterns where one agent is a bottleneck — and proposes *new agents*. A new agent is not new code: it is a new **agent specification** (role, responsibilities, prompt set, tool grants, memory scopes, KPIs, escalation rules) executed by the existing agent runtime. This makes agent genesis a data/configuration problem with tractable safety properties.

8.2 Genesis Pipeline



TITAN-EVO-025 (MUST) — Gap detection produces agent proposals. The Memory Agent runs a weekly analysis over scorecards, RCRs, and queue telemetry; when a gap matches proposal heuristics (e.g., task class with composite score < 60 for 30 days and no owning agent), it drafts an agent-spec EP including: charter, non-goals, prompt templates, minimal tool grants (least privilege per Volume 8), KPIs with targets, and estimated cost impact. *Acceptance criteria:* Proposal EP validates against the agent-spec JSON Schema (`/evolution/agents/schema.json`); includes the motivating evidence dossier.

TITAN-EVO-026 (MUST) — Sandbox trial before any human decision. The candidate agent runs in an isolated sandbox (Docker, no external network, synthetic Git remotes) against: relevant TITAN-Bench tracks,

and ≥2 synthetic end-to-end projects (Volume 6 workflow executed against fixture requirements). *Acceptance criteria:* (a) Trial report quantifies score deltas vs. the status quo roster on the target task classes; (b) sandbox provably cannot reach production systems (network policy test); (c) trial is reproducible from the EP SHA.

TITAN-EVO-027 (MUST) — Human approval gate for roster changes. Adding, materially modifying, or retiring an agent requires EP merge by a human with `evolution:approve` (TITAN-EVO-001 applies). *Acceptance criteria:* Roster loaded strictly from merged manifest; audit log records roster transitions.

TITAN-EVO-028 (MUST) — Probation with automatic demotion. Newly promoted agents run 30 days in canary scope (whitelisted task classes, mandatory review of all outputs by an established agent). KPI floors breached → automatic suspension (tasks rerouted to previous owner) + RCR. *Acceptance criteria:* Suspension path tested; probation status visible in the Agent Status dashboard.

TITAN-EVO-029 (SHOULD) — Agent retirement. Agents whose task classes disappear or whose KPIs are dominated by another agent for 90 days are proposed for retirement via EP; their memory scopes are archived, not deleted. *Acceptance criteria:* Retirement EP template exists; archived scopes remain queryable by the Memory Agent.

8.3 ADR-EVO-04 — Agents as Specifications, Not Arbitrary Code

- **Context:** "TITAN creates new agents" could mean generating new Python services — powerful but a severe security and maintenance hazard.
- **Decision:** New agents are declarative specifications executed by the shared, human-written agent runtime (Volume 2/3). Capability changes that require new *tools* go through the plugin system (Volume 7 / §9) with its own review path.
- **Alternatives considered:** (1) *Code-generating agent factory* — maximum flexibility, unacceptable attack surface, unbounded review burden; (2) *fixed roster forever* — safe but the platform cannot adapt to new domains; (3) *human-only agent authoring* — status quo tooling; retained as the fallback, but loses the systematic gap-detection signal.
- **Trade-offs:** Spec-driven agents are bounded by the runtime's existing capabilities; genuinely novel capabilities still need human-developed plugins/runtime features. Accepted: this is exactly the safety boundary we want.

---

9. Plugin Marketplace Vision

The plugin architecture (Volume 7) enables a community marketplace. This section specifies its governance; implementation lands in roadmap Phase 3 (§12).

9.1 Submission & Review

TITAN-EVO-030 (MUST) — Marketplace submission pipeline. Plugins are submitted as Git repositories with a signed manifest (`titan-plugin.yaml`: identity, declared permission scopes, API surface, compatibility range). Automated review runs: static analysis, dependency/supply-chain scan (Volume 8), permission-scope lint (deny-by-default), sandboxed conformance tests. TITAN's own Security and Code Reviewer Agents produce an advisory review; a human marketplace maintainer approves initial listing. *Acceptance criteria:* (a) Unsigned or scope-escalating updates are auto-rejected; (b) review verdicts and reports are public on the listing; (c) the entire pipeline is open-source so third parties can self-host a private registry.

TITAN-EVO-031 (MUST) — Trust tiers and ratings.

Tier	Meaning	Requirements
Experimental	Listed, unvetted beyond automated scan	Automated pipeline pass
Community	Human-reviewed once	Maintainer approval + 30 days, ≥10 installs, no security reports
Verified	Publisher identity verified, re-reviewed each release	Signed releases, SLA on vulnerability response
Core	Maintained in the TITAN monorepo	Full Volume 8 security bar

Ratings combine user stars with objective telemetry (opt-in): install count, crash rate, permission prompts triggered. *Acceptance criteria:* Tier displayed at install time; installing below-Community tier requires explicit admin confirmation; telemetry is opt-in and anonymized.

TITAN-EVO-032 (MAY) — Revenue model options. The marketplace remains optional and must never paywall core functionality (open-source, local-first canon). Documented options, decided by the governance body in Phase 3: (a) free-only listings (default at launch); (b) paid listings with a revenue share (e.g., 85/15 developer/platform) processed by a pluggable payment provider; (c) organization-level subscriptions bundling Verified-tier support; (d) donations/sponsorship links. Self-hosted registries never owe fees. *Acceptance criteria:* Payment integration is a plugin itself; the platform runs fully with payments disabled.

9.2 ADR-EVO-05 — Open Registry Protocol over Closed Store

- **Context:** A single central store maximizes quality control but conflicts with the open-source, local-first mandate and creates a single point of governance failure.
- **Decision:** Define an open registry protocol (static-JSON index + signed artifacts, OCI-registry-compatible distribution); the official TITAN registry is one instance, and enterprises/communities can host private registries with identical tooling.
- **Alternatives considered:** (1) *Closed official store only* — better curation, unacceptable centralization; (2) *no registry, Git URLs only* — maximally open but no trust signals, poor discovery; (3) *npm/PyPI piggybacking* — reuses infrastructure but cannot carry TITAN's permission-scope and review metadata natively.
- **Trade-offs:** Federated registries fragment ratings and duplicate review effort; mitigated by making review reports portable signed artifacts. Accepted.

---

10. Enterprise Features

Enterprise capabilities extend — never replace — the open-source core (open-core boundaries are governance decisions recorded here; all security mechanisms build on Volume 8).

TITAN-EVO-033 (MUST) — SSO. Enterprise deployments support SAML 2.0 and OIDC identity providers (Okta, Entra ID, Google Workspace, Keycloak) on top of the canonical OAuth2/OIDC + JWT auth. SCIM 2.0 provisioning maps IdP groups to TITAN RBAC roles. *Acceptance criteria:* (a) Keycloak-based local test proves login, group→role mapping, and deprovisioning within 15 minutes of IdP removal; (b) break-glass local admin remains available and audited.

TITAN-EVO-034 (MUST) — Multi-tenancy. Tenancy model: shared control plane, per-tenant isolation of data and execution. PostgreSQL row-level security keyed on `tenant_id` for all tenant tables; per-tenant Qdrant collections and Redis key prefixes; per-tenant Kubernetes namespaces (or Compose project isolation

locally) for agent sandboxes; per-tenant LLM gateway credentials and budgets. *Acceptance criteria:* (a) Automated cross-tenant access test suite passes (attempts to read another tenant's tasks/memory/artifacts all fail at the database layer, not just the API layer); (b) noisy-neighbor controls: per-tenant queue quotas on Redis Streams consumer groups; (c) tenant export/delete implements data portability and right-to-erasure.

TITAN-EVO-035 (MUST) — Compliance foundations. Immutable audit log (append-only PostgreSQL table + object-storage archive) covering auth events, approvals (all §2 gates), data access, and agent actions with trace IDs; configurable data-retention policies; data-residency pinning (region-scoped storage classes); SOC 2 Type II and ISO 27001 control mappings maintained as documentation in-repo; GDPR data-processing documentation for memory contents. *Acceptance criteria:* (a) Audit events are hash-chained and verifiable; (b) retention jobs provably purge expired data including vector embeddings; (c) a compliance-controls matrix maps each control to implementing requirements across volumes.

TITAN-EVO-036 (MUST) — Private model hosting. Enterprises can route all LLM traffic to self-hosted models (Ollama or vLLM/TGI behind the same gateway interface) with a hard-egress mode: a network policy plus gateway configuration that guarantees no prompt/completion leaves the deployment boundary. *Acceptance criteria:* (a) Hard-egress mode verified by an automated test that fails if any non-allowlisted egress occurs during a full project run; (b) switching a deployment from cloud providers to private hosting is configuration-only (canonical decision); (c) TITAN-Bench publishes reference quality/cost numbers for popular open-weight models so enterprises can set expectations.

TITAN-EVO-037 (SHOULD) — Enterprise governance controls. Policy-as-code hooks: organizations can require, e.g., "human review on all F-SEC-class changes," "no canary A/B on production-tagged projects," "debates mandatory for data-model changes." Policies are versioned files evaluated by the orchestrator. *Acceptance criteria:* Policy violations block the action and are audited; policy files validate against a published schema.

---

11. Model Fine-Tuning on Organization Memory

11.1 Purpose and Boundaries

After months of operation, an organization's memory (Volume 5) contains thousands of high-quality (context → action → outcome) triples. Fine-tuning small local models on this corpus can raise quality on organization-specific conventions and cut cost/latency for routine task classes. Fine-tuning is strictly optional, off by default, and never a substitute for the memory/RAG pipeline — it complements it for style, conventions, and routing-cheap task classes.

11.2 Data Pipeline

```
org memory + scorecards
       |
       v
[1] SELECT   — only samples from tasks with composite_score >= 85,
               no open RCRs, review-approved artifacts
       |
[2] SCRUB    — secret scanning, PII removal, license filter,
               tenant-consent check (per-project data-use flags)
       |
[3] FORMAT   — instruction-tuning pairs / preference pairs
               (chosen = merged artifact, rejected = pre-review draft)
       |
[4] DEDUPE   — near-duplicate removal, per-task-class balancing
       |
[5] FREEZE   — dataset manifest (content hashes) committed via EP
       |
       v
LoRA fine-tune (local GPU or rented) --> adapter artifact (versioned)
       |
       v
TITAN-Bench full run + safety eval  --> human approval --> gateway route
```

TITAN-EVO-038 (MUST) — Consent-scoped, scrubbed training data. *Acceptance criteria:* (a) Samples originate only from projects whose data-use policy allows training (per-tenant, per-project flag, default OFF); (b) secret/PII scrubbing runs with a published detector list and a quarantine queue for uncertain samples; (c) the frozen dataset manifest makes any training run reproducible and auditable; (d) client code marked proprietary is excluded regardless of score.

TITAN-EVO-039 (MUST) — Evaluation and safety gates for tuned models. A tuned adapter ships only if it: matches or beats its base model on the full TITAN-Bench; shows no regression on the safety eval set (guardrail-adherence probes, injection-resistance probes from Volume 8); and passes a memorization audit (canary-string extraction and near-verbatim training-sample regurgitation checks). *Acceptance criteria:* All three gates automated in the tuning pipeline; results attached to the model's EP; deployment via gateway routing EP (human gate applies).

TITAN-EVO-040 (MUST) — Tuned models are routed, versioned artifacts. Adapters are content-addressed artifacts referenced by the routing policy; rollback is a routing change (TITAN-EVO-003 applies). *Acceptance criteria:* `ollama`-servable adapter format supported for local use; per-adapter lineage (base model, dataset manifest, training config) queryable via API.

TITAN-EVO-041 (MAY) — Provider-hosted fine-tuning. Where a cloud provider offers fine-tuning APIs, the same pipeline MAY target them, subject to the tenant's data-residency and egress policies (TITAN-EVO-036 hard-egress mode forbids it). *Acceptance criteria:* Provider tuning is a gateway-plugin capability; disabled by default.

11.3 ADR-EVO-06 — LoRA on Local Open-Weight Models as the Default Tuning Path

- **Context:** Fine-tuning must be affordable, private, and reproducible for a local-first open-source project; frontier-model tuning APIs are provider-locked and move data off-premises.
- **Decision:** Default path is parameter-efficient LoRA/QLoRA tuning of open-weight models (7–14B class), producing adapters servable via Ollama through the existing gateway. Provider-hosted tuning is optional (TITAN-EVO-041).
- **Alternatives considered:** (1) *Full fine-tunes* — better ceiling, but cost/VRAM-prohibitive for most users and slower to iterate; (2) *provider tuning APIs as default* — simplest ops but breaks local-first and hard-

egress guarantees; (3) *no tuning, RAG-only* — simplest and remains the default *runtime* behavior, but leaves measurable cost/latency wins for routine tasks unrealized.

- **Trade-offs:** Small tuned models will not out-reason frontier models; they are targeted at conventions and routine task classes selected by the routing frontier data (§6.4). Accepted.

---

12. Long-Term Roadmap

Phases assume Volumes 1–9 deliver the MVP platform ("Phase 0"). Dates are relative to MVP GA; each phase's exit criteria are the listed requirement IDs reaching "implemented + verified."

Phase	Horizon	Theme	Milestones (exit criteria)
1 — Instrumented	GA + 3 months	Measure everything	Scorecards & KPIs live (EVO-004..007); reflections persisted (EVO-008..010); failure taxonomy + RCRs (EVO-011..013); prompt registry pinned (EVO-015); TITAN-Bench smoke track in CI (EVO-016 partial)
2 — Optimizing	GA + 6 months	Close the loop	Full TITAN-Bench + gates (EVO-016, 017); A/B harness shadow+canary (EVO-018); routing frontier reports (EVO-019, 020); debate protocol for architecture/security decisions (EVO-021..024); regression-from-failure pipeline (EVO-014)
3 — Extensible	GA + 12 months	Ecosystem	Agent genesis pipeline end-to-end with first community-proposed agent promoted (EVO-025..029); marketplace registry protocol + Experimental/Community tiers (EVO-030, 031); revenue-model decision by governance (EVO-032)
4 — Enterprise	GA + 18 months	Trust at scale	SSO/SCIM (EVO-033); hardened multi-tenancy with cross-tenant test suite (EVO-034); audit/compliance matrix, SOC 2 readiness (EVO-035); private model hosting hard-egress mode (EVO-036); policy-as-code (EVO-037)
5 — Learning	GA + 24 months	Organization-tuned intelligence	Fine-tuning pipeline with all safety gates (EVO-038..041); tuned local models serving ≥30% of routine task volume in reference deployments; documented cost reduction ≥40% on those classes at non-inferior quality
6 — Frontier	GA + 24 months +	Research program	Items from §13 pursued as public RFCs: cross-organization federated pattern learning, long-horizon multi-project planning, formal verification of agent guardrails

TITAN-EVO-042 (SHOULD) — Public roadmap tracking. The roadmap is maintained as a living document in-repo with per-milestone status, linked issues, and quarterly public review notes. *Acceptance criteria:* Each phase milestone maps to tracked issues; status auto-generated from requirement verification records.

---

13. Risks, Known Limits & Open Research Questions

13.1 Risks

#	Risk	Likelihood	Impact	Mitigation
R1	**Metric gaming / reward hacking** — agents optimize scorecard dimensions (e.g., minimizing	Medium	High	Cross-agent scoring (never self-scored), hidden bench tests, periodic human audit of scorecard–

#	Risk	Likelihood	Impact	Mitigation
	review rounds by under-reporting uncertainty) rather than true quality			outcome correlation, KPI definitions changeable only via EP
R2	**Benchmark overfitting** — prompts evolve to ace TITAN-Bench while degrading on real work	Medium	High	Online A/B stage is mandatory (ADR-EVO-02); bench refresh from real RCRs (EVO-014); held-out private test split rotated quarterly
R3	**Evolution loop instability** — oscillating prompt/routing changes	Low	Medium	Sequential-test promotion thresholds, cool-down windows between EPs touching the same artifact, rollback floors (EVO-003)
R4	**Approval fatigue** — humans rubber-stamp EPs, hollowing out the safety gate	Medium	High	EPs carry auto-generated risk summaries and diffs; batch low-risk changes; governance requires two approvers for roster/model changes
R5	**Training-data leakage** — secrets or proprietary code in tuned models	Low	Critical	Default-OFF consent flags, scrubbing + quarantine, memorization audit gate (EVO-039), hard-egress mode for enterprises
R6	**Marketplace supply-chain attack** — malicious plugin exfiltrates code	Medium	Critical	Signed manifests, deny-by-default scopes, sandboxed execution (Volume 7), trust tiers with human review (EVO-030, 031)
R7	**Cost blow-up from evolution overhead** — reflections, debates, shadows consuming budget	Medium	Medium	Dedicated evolution budget with graceful degradation (EVO-020); local-model-only mode
R8	**Debate theater** — multi-model debates converge on confident consensus that is still wrong	Medium	Medium	Dissent is mandatory output; efficacy tracking (EVO-024) can demote debate from decision classes where it adds no measured value

13.2 Known Limits

1. **TITAN cannot verify its own alignment.** Guardrail adherence is tested empirically (probes, audits), not proven. Formal verification of agent policies is a research goal, not a shipped capability.
2. **Small-sample statistics.** Laptop-scale deployments generate too few tasks for online A/B significance; they rely on offline bench results plus community-shared (opt-in, anonymized) evaluation data.
3. **Taxonomy lag.** Genuinely novel failure modes will first appear as `F-UNK`; the taxonomy update loop takes days, during which classification quality drops.
4. **Spec-bounded agent genesis.** New agents cannot exceed the runtime's tool surface (ADR-EVO-04); capability breakthroughs still require human engineering.
5. **Tuned-model scope.** Fine-tuned local models improve conventions and routine work; they do not close the reasoning gap with frontier models on hard design tasks.

13.3 Open Research Questions

1. **Credit assignment across agents:** when a project fails, how should blame/credit propagate through a multi-agent task graph? Current per-task scorecards approximate this poorly for emergent, cross-task failures.
2. **Federated organizational learning:** can organizations share learned patterns (not data) — e.g., differentially private prompt deltas or adapter merging — without leaking proprietary information?

3. **Long-horizon planning evaluation:** TITAN-Bench measures task- and project-scale quality; no accepted benchmark exists for multi-month, multi-project portfolio behavior.
4. **Judge reliability:** LLM judges exhibit position, verbosity, and self-preference biases; anonymization (EVO-023) mitigates but does not eliminate them. What audit rate of human re-judging is sufficient?
5. **Safe autonomy dial:** under what measured conditions (if any) could specific low-risk EP classes graduate to automated approval — and how would that decision itself be validated and reversed? (Position of this spec: not before Phase 6, and only via public RFC.)

---

14. Cross-References

Volume	Relationship to this volume
Volume 1 — Foundation	Owns the canonical stack and design principles referenced throughout; success metrics that evolution KPIs roll up into
Volume 2 — Multi-Agent Intelligence	Defines each agent's KPIs consumed by §3; agent specification format extended by the genesis pipeline (§8); Memory Agent responsibilities
Volume 3 — Backend Engineering	FastAPI services, Redis Streams consumers, and PostgreSQL schemas that host scorecards, RCRs, and the A/B router
Volume 4 — Frontend & Dashboard	Agent Status, Memory Explorer, and Live Conversations views that surface scorecards, RCRs, debates, and EP status
Volume 5 — Memory, Knowledge & RAG	Write/retrieval paths for reflections and retrospectives (§4); provenance and pruning rules governing lesson memory; source corpus for fine-tuning (§11)
Volume 6 — Engineering Workflow	Task lifecycle emitting scorecard triggers; planning stage consuming retrospective lessons; synthetic-project fixtures used in sandbox trials
Volume 7 — Tool Ecosystem & Integrations	Plugin architecture and sandboxing that the marketplace (§9) governs; GitHub integration carrying EP pull requests
Volume 8 — Security	RBAC permissions (`evolution:approve`), audit logging, supply-chain scanning, secret/PII detection used by §2, §9, §10, §11
Volume 9 — Deployment & Production	Prometheus/Grafana/OpenTelemetry infrastructure behind KPI rollups and rollback triggers; canary deployment machinery reused by the A/B harness

---

End of Volume 10 — Future Roadmap & Autonomous Evolution (TITAN-VOL-10, Draft v1.0, 2026-07-04).
